

CONSTITUTION

HelixCode Constitution

HelixCode Project Constitution

Version: 1.0.0 **Effective Date:** 2026-04-30 **Scope:** This Constitution applies to HelixCode and ALL its submodules **Authority:** Cascaded from HelixAgent root governance with HelixCode-specific addenda

INHERITED FROM constitution/Constitution.md (HelixConstitution submodule)

All universal rules in constitution/Constitution.md apply unconditionally to HelixCode. This project Constitution **extends** those universal rules with HelixCode-specific addenda below — it does NOT and MAY NOT weaken any universal clause. When this Constitution disagrees with the constitution submodule, the constitution submodule wins.

The constitution submodule is at ./constitution/ (added 2026-05-14 per user mandate, pinned at the upstream HelixDevelopment/HelixConstitution HEAD). To locate it from nested submodules at arbitrary depth, source constitution/find_constitution.sh from the parent project tree.

Project remote-policy scope (revised 2026-06-03) — Per operator decision 2026-06-03, HelixCode aligns with the universal Constitution's multi-upstream Git-push allowance: GitHub (vasic-digital/*, HelixDevelopment/*), GitLab (same), GitFlic, and GitVerse are ALL permitted as Git remotes, and pushes target every configured upstream. This **SUPERSEDES** the earlier CLAUDE.md §6.W tightening that forbade GitFlic/GitVerse — that remote-provider restriction is **LIFTED**. Project tightenings unrelated to remote providers (host power management, anti-bluff, secret-leak, etc.) remain in force.

Preamble

HelixCode is an enterprise-grade distributed AI development platform. This Constitution establishes the non-negotiable rules that govern all development, testing, deployment, and maintenance activities within the project. Every

contributor, agent, and automated process MUST adhere to these rules. No exceptions.

CONST-001: No CI/CD Pipelines (Permanent)

No `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, `.travis.yml`, `.circleci/`, or any automated pipeline. No Git hooks. All builds and tests run manually or via `Makefile`/script targets.

Rationale: Manual execution ensures human oversight and prevents automated propagation of bluffs.

CONST-002: No Mocks in Production (Permanent)

CONST-002a: Production Code

Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. All production code is fully functional with real integrations.

CONST-002b: Test Code

Mocks/stubs/fakes MAY be used ONLY in unit tests (files ending `_test.go` run under `go test -short`).

Rationale: Production bluffs have repeatedly been discovered where features appeared implemented but were non-functional.

CONST-003: No HTTPS for Git (Permanent)

SSH URLs only (`git@github.com:...`, `git@gitlab.com:...`, etc.) for clones, fetches, pushes, and submodule updates. SSH keys are configured on every service.

CONST-004: No Manual Container Commands (Permanent)

Container orchestration is owned by the project's binary/orchestrator (e.g., `make build` → `./bin/<app>`). Direct `docker/podman start|stop|rm` and `docker-compose up|down` are prohibited as workflows.

CONST-005: 100% Real Data for Non-Unit Tests

Beyond unit tests, all components MUST use actual API calls, real databases, live services. No simulated success. Fallback chains tested with actual failures.

Verification: Every integration/E2E test MUST connect to real services or skip (not fail) if unavailable.

CONST-006: Challenge Coverage (Permanent)

Every component MUST have Challenge scripts (`./challenges/scripts/`) validating real-life use cases. No false success — validate actual behavior, not return codes.

CONST-007: Health & Observability

Every service MUST expose health endpoints. Circuit breakers for all external dependencies. Prometheus / OpenTelemetry integration where applicable.

CONST-008: Documentation & Quality

Update `CLAUDE.md`, `AGENTS.md`, and relevant docs alongside code changes. Pass language-appropriate format/lint/security gates. Conventional Commits: `<type>(<scope>): <description>`.

CONST-009: Validation Before Release

Pass the project's full validation suite (`make ci-validate-all-equivalent`) plus all challenges (`./challenges/scripts/run_all_challenges.sh`).

CONST-010: Comprehensive Verification

Every fix MUST be verified from all angles: runtime testing (actual HTTP requests / real CLI invocations), compile verification, code structure checks, dependency existence checks, backward compatibility, and no false positives. Grep-only validation is NEVER sufficient.

CONST-011: Resource Limits for Tests & Challenges

ALL test and challenge execution MUST be strictly limited to 30-40% of host system resources. Use `GOMAXPROCS=2, nice -n 19, ionice -c 3, -p 1` for go test. Container limits required.

CONST-012: Bugfix Documentation

All bug fixes MUST be documented in `docs/issues/fixed/BUGFIXES.md` with root cause analysis, affected files, fix description, and a link to the verification test/challenge.

CONST-013: Real Infrastructure for All Non-Unit Tests

Mocks/fakes/stubs/placeholders MAY be used ONLY in unit tests. ALL other test types — integration, E2E, functional, security, stress, chaos, challenge, benchmark, runtime verification — MUST execute against REAL running systems with REAL containers, REAL databases, REAL services, and REAL HTTP calls.

CONST-014: Reproduction-Before-Fix (Mandatory)

Every reported error, defect, or unexpected behavior MUST be reproduced by a Challenge script BEFORE any fix is attempted. Sequence: 1. Write the Challenge first 2. Run it; confirm fail (it reproduces the bug) 3. Then write the fix 4. Re-run; confirm pass 5. Commit Challenge + fix together

The Challenge becomes the regression guard for that bug forever.

CONST-015: Concurrent-Safe Containers

Any struct field that is a mutable collection (map, slice) accessed concurrently MUST use thread-safe primitives. Bare `sync.Mutex` + `map/slice` combinations are prohibited for new code.

CONST-016: Definition of Done (Universal)

A change is NOT done because code compiles and tests pass. “Done” requires pasted terminal output from a real run.

- **No self-certification:** Words like *verified*, *tested*, *working*, *complete*, *fixed*, *passing* are forbidden in commits/PRs/replies unless accompanied by pasted output from a command that ran in that session.
 - **Demo before code:** Every task begins by writing the runnable acceptance demo
 - **Real system, every time:** Demos run against real artifacts
 - **Skips are loud:** `t.Skip` without a trailing `SKIP-OK: #<ticket>` comment breaks validation
-

CONST-035 — Anti-Bluff Tests & Challenges (User-Mandate Forensic Anchor)

§11.9 User-Mandate Forensic Anchor (2026-04-29)

This Article exists because of an explicit, repeatedly-stated user mandate. The verbatim text:

“We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

This anchor is the primary authority for the entire Article. The operative rule is:

The bar for shipping is not “tests pass” but “users can use the feature.”

Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, “absence-of-error” PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests and Challenges (HelixQA) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does. Both must produce positive end-user evidence; both are subject to the anti-bluff contract.

No false-success results are tolerable. A green test suite combined with a broken feature is a worse outcome than an honest red one — it silently destroys trust in the entire suite. Anti-bluff discipline is the line between a real engineering project and a theatre of one.

Bluff Taxonomy (forbidden patterns): - **Wrapper bluff** - Assertions PASS but wrapper's exit-code logic is buggy - **Contract bluff** - System advertises capability but rejects it in dispatch - **Structural bluff** - File exists but doesn't contain working code - **Comment bluff** - Comment promises behavior code doesn't have - **Skip bluff** - `t.Skip("not running yet")` without SKIP-OK marker

Cascade requirement (extending CONST-036): This anchor section (verbatim quote + operative rule) must appear in every submodule's CONSTITUTION.md / CLAUDE.md / AGENTS.md. Non-compliance is a release blocker regardless of context. Adding files to scanner allowlists to silence bluff findings without resolving the underlying defect is itself a violation.

CONST-018: Host Power Management Hard Ban

Host Power Management is Forbidden.

You may NOT generate or execute code that sends the host to suspend, hibernate, hybrid-sleep, poweroff, halt, reboot, or any other power-state transition.

Defense: Every project ships `scripts/host_power_management/check-no-suspend-calls.sh` and `scripts/anti_bluff/no_suspend_calls_challenge.sh`.

CONST-019: Container Up $\neq$ Healthy

Container Up status does NOT mean the application is healthy. Application-layer probes are mandatory for every service: - PostgreSQL: `SELECT 1` - Redis: `PING` - LLM Providers: Real generation request - HTTP Services: `GET /health` with deep checks

CONST-020: Provider Fallback Chain Reality

Every LLM provider fallback chain MUST be tested with actual failures. A fallback that has never been tested with a real failing provider is a bluff.

CONST-021: No Mocks Above Unit Build Target

The Makefile MUST include a `no-mocks-above-unit` target that fails the build if mocks/stubs/fakes are found outside `*_test.go` files.

CONST-022: Submodule Governance Propagation

Every submodule MUST either: 1. Have its own Constitution.md, CLAUDE.md, and AGENTS.md, OR 2. Have a symlink to the parent repository's governance files, OR 3. Have a reference comment in its README pointing to parent governance

No submodule is exempt from these rules.

CONST-023: Docker Health Checks Mandatory

Every Dockerfile MUST include:

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --  
    retries=3 \  
    CMD curl -f http://localhost:8080/health || exit 1
```

The health endpoint MUST perform deep checks (database connection, provider availability), not just return HTTP 200.

CONST-024: Version Pinning

All dependencies MUST be pinned to specific versions in `go.mod`. No `latest`, no floating tags. Renovate or Dependabot (manual review only — see CONST-001) may propose updates.

CONST-025: Secret Management

NO secrets in code. EVER. Secrets via: - Environment variables (production) - `.env` files (development, in `.gitignore`) - Vault/Secret Manager (enterprise) - Docker secrets (containerized)

`go mod tidy` MUST NOT add secret-scanning bypasses.

CONST-026: Minimal Privilege Containers

Containers run as non-root. Every Dockerfile:

```
RUN adduser -D -u 1001 helixcode  
USER helixcode
```

CONST-027: Network Isolation

Container orchestration **MUST** use internal networks. Services communicate via named hosts, not exposed ports where possible.

CONST-028: Backup Before Destructive Operations

Every file editing tool **MUST** create backups before modification. The backup **MUST** be restorable.

CONST-029: Input Validation at All Boundaries

Every public function **MUST** validate inputs. No trust of caller-provided data. SQL injection, path traversal, command injection **MUST** be impossible by design.

CONST-030: Graceful Degradation

When external services are unavailable, the system **MUST** degrade gracefully: - Return partial results where possible - Queue operations for retry - Inform user of degraded state - NEVER crash or hang indefinitely

CONST-031: Audit Trail

Every significant operation **MUST** be logged with: - Timestamp - User identity - Operation type - Success/failure status - Resource affected

Log retention: 90 days minimum.

CONST-032: Emergency Stop

Every long-running or distributed operation **MUST** support cancellation via `context.Context`. Users **MUST** be able to interrupt any operation.

CONST-033: Data Integrity

Database writes **MUST** be transactional. Partial writes **MUST** be rolled back. Consistency checks **MUST** run periodically.

CONST-034: API Stability

Public APIs maintain backward compatibility within major versions. Deprecation requires: - 6-month notice - Migration guide - Compatibility shim

CONST-035: End-User Usability Mandate (2026-04-29 Strengthening)

A test or Challenge that **PASSES** is a **CLAIM** that the tested behavior **works for the end user of the product**.

The HelixAgent project has repeatedly hit the failure mode where every test ran green **AND** every Challenge reported **PASS**, yet most product features did not actually work. This **MUST NOT** recur in HelixCode.

Every **PASS** result **MUST** guarantee: a. **Quality** - correct behavior under real inputs, edge cases, concurrency b. **Completion** - wired end-to-end with no stub/placeholder gaps c. **Full usability** - a user following documented request shapes **SUCCEEDS**

A passing test that doesn't certify all three is a **bluff** and **MUST** be tightened.

Bluff taxonomy (each pattern observed and now forbidden): - **Wrapper bluff** - assertions **PASS** but wrapper's exit-code logic is buggy - **Contract bluff** - system advertises capability but rejects it in dispatch - **Structural bluff** - `check_file_exists` passes but doesn't run the test - **Comment bluff** - comment promises behavior code doesn't actually have - **Skip bluff** - `t.Skip("not running yet")` without `SKIP-OK: #<ticket>` marker

Full background: `docs/HOST_POWER_MANAGEMENT.md` and this Constitution (CONST-035).

CONST-036: Propagation to Submodules

This Constitution, along with CLAUDE.md and AGENTS.md, MUST be propagated to ALL submodules. Each submodule's governance MUST reference this parent Constitution. Changes to this Constitution MUST trigger review of all submodule governance files.

CONST-037: LLMsVerifier Single Source of Truth Mandate

Rule: LLMsVerifier SHALL BE the sole authoritative source for: 1. All model metadata (names, IDs, context windows, capabilities) 2. All provider metadata (endpoints, auth types, supported models) 3. All verification status (verified, partial, failed, pending) 4. All scoring data (overall scores, capability scores, tier rankings) 5. All rate-limit and cooldown state

Prohibition: NO hardcoded model lists, NO hardcoded provider lists, NO simulated model discovery. Any code path that presents a model or provider listing to a user MUST fetch that listing from the LLMsVerifier subsystem or its cached replica.

Anti-Bluff Verification: - The challenge script `scripts/anti_bluff/verifier_hardcode_check.sh` MUST scan all Go source files for hardcoded model arrays. - Any `[]string{"gpt-4", "claude-3"}` or equivalent literal in production code is a constitutional violation. - The only permitted hardcoded data is the LLMsVerifier service endpoint URL and the list of verification test types.

Enforcement: `make test-complete` MUST include a test that asserts `ModelManager.GetAvailableModels()` returns at least as many models as the verifier's database contains for configured providers. A test that passes while the CLI shows a hardcoded list is a TEST BLUFF and violates CONST-035.

CONST-038: Model Provider Anti-Bluff Guarantee

Rule: Every model displayed to an end user MUST have been verified by LLMsVerifier within the last `verification_timeout` period (default: 24h). Models older than this MUST display a "stale" indicator and be deprioritized.

Prohibition Against Test Bluffing: - A unit test that mocks the verifier client and asserts `GetAvailableModels()` returns 3 models DOES NOT satisfy this rule. - An integration test that starts the verifier server, performs real provider discovery,

and confirms the model count matches the actual provider API response DOES satisfy this rule. - The Makefile target `make test-verifier-integration` MUST exist and MUST run without mocks.

The “Tests Pass But Features Don’t Work” Guarantee:

NO TEST MAY PASS UNLESS THE FEATURE IT TESTS IS DEMONSTRABLY USABLE BY AN END USER IN THE SAME BUILD.

- If `TestModelList` passes but `helixcode --list-models` shows hardcoded data, the test is a BLUFF.
- If `TestProviderHealth` passes but the health endpoint returns `200 OK` for a provider that is actually down, the test is a BLUFF.
- If `TestLLMGeneration` passes but `--prompt "hello"` returns a simulated string, the test is a BLUFF.
- Bluff tests MUST be rewritten or deleted. There is no “grandfather” exception.

Evidence Standard: Every test that claims to verify model/provider functionality MUST: 1. Call a real API endpoint or a real verifier database 2. Assert on response content that could only come from that real source 3. Include a test that runs the CLI binary with `--list-models` and checks output against verifier data

CONST-039: Real-Time Model Status Accuracy

Rule: Model status (available, rate-limited, cooldown, offline, deprecated) displayed to users MUST reflect the actual state as known by `LLMsVerifier` within `max_staleness` seconds (default: 60s).

Polling vs. Push: - If WebSocket/SSE push is unavailable, the system MUST poll `LLMsVerifier` at most every `status_poll_interval` (default: 30s). - The TUI MUST display a “last updated” timestamp with every model listing. - Models in “cooldown” or “rate-limited” state MUST show the estimated recovery time if known.

Accuracy Verification: - Challenge script `challenges/scripts/model_status_accuracy_challenge.sh` MUST: 1. Artificially rate-limit a provider by exhausting its quota 2. Wait for the status to propagate to the verifier 3. Check that `helixcode --list-models` shows the rate-limited status within 60s 4. Check that `SelectOptimalModel()` no longer selects the rate-limited model

Prohibition: Status indicators that are “always green” or that lag >60s behind reality violate this rule.

CONST-040: All Providers and Models Integration Mandate

Rule: HelixCode MUST integrate with ALL providers and models that LLMsVerifier supports, subject only to: 1. The provider being explicitly disabled in configuration (enabled: false) 2. The API key being absent and the provider requiring one 3. The provider being marked deprecated in the verifier database

Minimum Provider Set (SHALL NOT be reduced without constitutional amendment):

Provider	Auth Type	Required Env Var
OpenAI	API Key	OPENAI_API_KEY
Anthropic	API Key / OAuth	ANTHROPIC_API_KEY
Gemini	API Key	GEMINI_API_KEY
DeepSeek	API Key	DEEPSEEK_API_KEY
Groq	API Key	GROQ_API_KEY
Mistral	API Key	MISTRAL_API_KEY
xAI	API Key	XAI_API_KEY
OpenRouter	API Key	OPENROUTER_API_KEY
Ollama	Local	None (auto-detect)
Llama.cpp	Local	None (auto-detect)

Integration Requirement: For every provider in the minimum set: - There MUST be a provider adapter file in internal/llm/ or internal/verifier/adapters/ - There MUST be a *_test.go file with real API tests (skipped only if HELIX_SKIP_LIVE_PROVIDER_TESTS is set) - There MUST be a challenge script in challenges/scripts/ - The model listing MUST include models from this provider when the provider is enabled

CONST-041: MCP / LSP / ACP / Embedding / RAG / Skills / Plugins Integration Mandate

Rule: LLMsVerifier integration SHALL extend beyond basic model listing to cover ALL capability dimensions:

- MCP (Model Context Protocol):** The verifier MUST report which models support MCP tool calling. HelixCode's MCP subsystem MUST consult verifier capability flags before selecting a model for tool-use tasks.
- LSP (Language Server Protocol):** The verifier MUST report code-analysis capabilities. Models without code_analysis capability MUST NOT be selected for refactoring or debugging tasks.
- ACP (Agent Capability Protocol):** The verifier MUST report multi-agent coordination support. Models with supports_parallel_tool_use MUST be preferred for ACP workflows.

4. **Embedding:** The verifier MUST report supports_embeddings for each model. The CognitionConfig embedding model selection MUST be verifier-aware.
5. **RAG (Retrieval-Augmented Generation):** The verifier MUST report context-window sizes. RAG chunking strategies MUST adapt to the selected model's context_window_tokens as reported by the verifier.
6. **Skills / Plugins:** The verifier MUST track plugin compatibility. Models flagged plugin_compatible MUST be used when skill/plugin execution is required.

Capability Checklist (MUST be verified by challenge): - [] MCP tool calling verified for at least 3 providers - [] LSP code-analysis verified for at least 3 providers - [] ACP parallel tool use verified for at least 2 providers - [] Embedding generation verified for at least 2 providers - [] RAG context-window adaptation verified - [] Skills/plugin execution verified for at least 2 providers

Prohibition: Capability flags MUST NOT be hardcoded. The Provider.GetCapabilities() method MUST return data sourced from the verifier's VerificationResult fields.

CONST-046: No Hardcoded Content — All User-Facing Text Must Be Dynamic

Rule: NO user-facing text, prompt template, question text, error message, label, helper text, or explanatory content may be hardcoded as a static literal string in any source file, test file, configuration template, script, or governance document. All such content MUST be either: 1. Generated dynamically by an LLM at runtime based on context (user's language, prompt content, session state), OR 2. Loaded from a resource file (.yaml, .json, .toml) that supports i18n and can be overridden per locale, OR 3. Composed programmatically from verifier-provided metadata or configuration data

Rationale: Hardcoded English-only text creates a system that silently breaks for non-English users. A clarification question hardcoded as "Which file has the bug?" could be asked identically to a Japanese, Serbian, or Spanish user who specified their request in that language — producing an incoherent, unusable experience. Every piece of user-facing text MUST adapt to the user's language and context. A static string array is a constitutional violation.

Prohibition: Static literal arrays/slices of question text, fixed prompt templates with hardcoded English phrases, hardcoded error messages in anything other than Go standard library format (which uses English identifiers but is not user-

facing), and hardcoded label/display text in TUI, CLI, desktop, or mobile interfaces.

Examples of violations: - `[]string{"Which file has the bug?", "What is the expected behavior?"}` — hardcoded question bank - `fmt.Sprintf("The build failed with: %s", err)` — user-visible English template - `textView.NewTextView().SetText("Press Enter to continue")` — hardcoded UI label - `return "feature not implemented"` — user-visible static string

Examples of compliant patterns: - Questions generated by `llm.Generate(ctx, prompt)` and parsed into structured output - Labels loaded from `locales/en.yaml` with fallback to verifier metadata - Error wrapping: `fmt.Errorf("build: %w", err)` — identifier-based, not user-facing prose - Dynamic formatting: `s.formatMessage(ctx, key, args...)` — dispatches through i18n/LLM layer

Enforcement: make `lint` MUST include a check that scans for hardcoded human-readable strings exceeding a length threshold. The anti-bluff sweep command (`grep -rn "simulated\|placeholder\|stub\|TODO"`) MUST also flag obvious hardcoded-text patterns.

Cascade requirement: This rule (verbatim or by CONST-046 ID reference) MUST appear in every owned-by-us submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md.

CONST-045: No Hardcoded Distribution Hosts — All Distribution Via `containers/.env`

Rule: ALL container distribution targets SHALL be configured exclusively through the `CONTAINERS_REMOTE_HOST_N_*` environment variables in `containers/.env` ($N=1..100$; iteration stops at first absent `_NAME`). The containers module's `pkg/envconfig/parser.go` is the authoritative loader.

Prohibition: NO distribution host (hostname, IP address, SSH user, SSH key path, runtime, labels) may be hardcoded in ANY HelixCode-owned source file, test file, challenge, configuration template, script, documentation, governance document (CONSTITUTION.md, CLAUDE.md, AGENTS.md), or any other committed artefact.

The sole source of truth for host enrolment is `containers/.env` (gitignored, mode 0600). `containers/.env.example` documents the format but contains NO operative host entries. Adding, removing, or modifying a distribution host MUST be done by editing `containers/.env` ONLY; no code change is required or permitted.

Audit command: `grep -rn 'CONTAINERS_REMOTE_HOST_' containers/.env`. The configured set at any point is whatever `.env` declares. At rule introduction time (2026-05-07), the configured hosts are `thinker.local`, but the rule applies to whatever set is in `.env` at any future point.

Testing: Non-unit tests and Challenges that require remote distribution SHALL read `containers/.env` at runtime. They SHALL skip (with `SKIP-OK: #P{X} no remote hosts configured`) when `CONTAINERS_REMOTE_ENABLED` is `false` or unset, and SHALL use whatever hosts are configured when enabled. No test may hardcode a host name.

Cascade requirement: This rule (verbatim or by `CONST-045` ID reference) MUST appear in every owned-by-us submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`.

Article XII — Repository Safety

§12.1 (CONST-042) — No-Secret-Leak

No API key, token, password, certificate, or other credential may be committed to any repository owned by HelixDevelopment or vasic-digital, transitively or otherwise. All secrets live in `.env` files (mode 0600) listed in `.gitignore`. Any leak — to git, logs, build artefacts, screenshots, or external services — is a release blocker until rotated and post-mortemed.

Operational requirements: - Every repo must have `.env`, `.env.local`, `.env.*` (with `!.env.example` exception), `*.pem`, `*.key`, `*.crt`, `id_rsa*` in `.gitignore`. - `scripts/scan-secrets.sh` (or equivalent) must run before every push; failing it blocks the push. - API keys for development are sourced from the canonical `../helix_agent/.env` (mode 0600, never under git) and copied — never symlinked, never committed — into per-repo `.env` files.

Cascade requirement: This article must appear verbatim in every owned-by-us repository's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Owned-by-us repos are listed in `scripts/owned-repos.txt` (or, until that file exists, the meta-repo `propagate-governance.sh` script's submodule walk excluding third-party trees).

§12.2 (CONST-043) — No-Force-Push

No force push, force-with-lease push, history rewrite, branch deletion of `main/master`, or upstream-overwriting operation may be performed without explicit, in-conversation user approval given for that specific operation. Authorization for one push does not extend to subsequent pushes. Bypassing hooks (`--no-verify`),

signature verification (`--no-gpg-sign`), or protected-branch rules also requires explicit approval. This applies to every repository in the HelixDevelopment / vasic-digital stack.

Operational requirements: - Local pre-push hook at `scripts/git_hooks/pre-push` (installed by `scripts/install-git-hooks.sh`) must reject `--force` / `--force-with-lease` unless `HELIX_FORCE_PUSH_APPROVED=1` is set. - The hook is a courtesy gate; this constitutional clause is the actual contract. - Regular non-force pushes of new commits to existing branches on already-configured remotes are PERMITTED without per-push approval, scoped to a programme/conversation in which the user has authorised the cadence.

Cascade requirement: Same as §12.1 — verbatim, every owned-by-us repo's three governance files.

Article XIII — Programme Continuity

§13.1 (CONST-044) — Continuation Document Maintenance Mandate

The `docs/CONTINUATION.md` document MUST be maintained in sync with the actual state of the CLI-Agent Fusion programme at all times. It is the authoritative resumption record for any CLI agent or LLM picking up the work from any session, at any time.

Mandate: Every commit that advances programme state — feature task completion, feature close-out, push to remotes, known-issue discovery, deferred-item resolution, phase transition, addition or removal of submodules / remotes — MUST update `docs/CONTINUATION.md` to reflect the new state in the same commit (or in an immediately-following commit if the state-changing commit is small and topical).

Definition of “out-of-sync”: - Last updated SHA in `CONTINUATION` $\neq$ `git rev-parse HEAD` on main. - Active feature in flight in `CONTINUATION` $\neq$ feature in `docs/improvements/PROGRESS.md` “Current focus”. - Tasks marked done in `CONTINUATION` $\neq$ ticked tasks in `PROGRESS.md`. - Known-issue list missing a documented failure that exists in evidence files (`docs/improvements/0[5-9]_phase_*_evidence.md`). - Repository state table missing or stale-SHA for any submodule listed in `.gitmodules`.

Severity: Out-of-sync `CONTINUATION.md` is a **CRITICAL DEFECT**. Same severity as a false-success test result under CONST-035 / Article XI §11.9. A green build with an out-of-sync continuation document is unshippable.

Verification: `scripts/verify_continuation_sync.sh` (TBD; planned for Phase 3) compares CONTINUATION fields against PROGRESS.md, `git rev-parse HEAD`, evidence files, and `.gitmodules`. Non-zero exit = sync violation → blocking pre-push.

Cascade requirement: This article must appear verbatim — or be referenced by CONST-044 ID with a pointer back to this anchor — in every owned-by-us repository's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Submodules without their own docs/CONTINUATION.md reference the meta-repo's CONTINUATION as the authoritative document; submodules that maintain their own CONTINUATION must pin its SHA and update on every state-changing commit identically.

CONST-047: Recursive Submodule Application Mandate

Verbatim user mandate (2026-05-14): *“Make sure all work we do is applied ALWAYS to all Submodules we control under our organizations (vasic-digital and HelixDevelopment) fully recursively everywhere with full bluff-proofing and comprehensive documentation, user manuals and guides and full tests and Challenges coverage!”*

Operative rule: Every engineering deliverable produced for HelixCode MUST be applied — fully and recursively — to every owned submodule under the vasic-digital and HelixDevelopment GitHub organizations. “Owned” means any submodule whose remote URL is `git@github.com:vasic-digital/*.git` or `git@github.com:HelixDevelopment/*.git` (including the `HelixDevelopment-s-Code/*` brand variant). Each such submodule MUST receive in lockstep:

1. **Anti-bluff posture (CONST-035 / Article XI §11.9):** every test and Challenge in the submodule must guarantee real, end-user-usable functionality. Metadata-only / absence-of-error / fake-PASS results are critical defects in submodules just as in the meta-repo.
2. **Comprehensive documentation:** README plus user manuals plus guides that match the submodule's actual current capabilities. Documentation that advertises features the code doesn't deliver is the same class of defect as a fake-PASS test.
3. **Full tests AND Challenges coverage:** unit tests where applicable, Challenges that exercise real workflows with captured runtime evidence (per CONST-035). 100% of user-reachable surfaces must be covered.

4. **Recursive propagation:** when an owned submodule itself depends on sub-submodules under vasic-digital or HelixDevelopment, the mandate cascades. A submodule that owns child submodules under our organizations is responsible for cascading the mandate to them.
5. **Synchronized commits:** when a meta-repo commit advances state that touches a submodule's surface, the corresponding submodule commit MUST land in the same engineering session and be pushed to all configured remotes.

Owned submodule baseline (direct children of the meta-repo as of 2026-05-14): vasic-digital/Containers, vasic-digital/Security, vasic-digital/Challenges, vasic-digital/LLMsVerifier, vasic-digital/Models, HelixDevelopment/HelixQA, HelixDevelopment/DocProcessor, HelixDevelopment/LLMOrchestrator, HelixDevelopment/LLMProvider, HelixDevelopment/VisionEngine, HelixDevelopment/HelixAgent, HelixDevelopment-s-Code/Website. Plus any nested submodule under these orgs (full recursion).

Cascade requirement: This anchor (verbatim or by CONST-047 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Non-compliance is a release blocker — same severity as a CONST-035 bluff or a CONST-044 out-of-sync CONTINUATION.

Verification: scripts/verify-governance-cascade.sh MUST be extended to confirm CONST-047 anchor presence in every owned submodule (in addition to its existing CONST-035 checks). A submodule missing the anchor is treated identically to one missing the anti-bluff anchor: blocked from merge.

CONST-048: Full-Automation-Coverage Mandate (cascaded from constitution submodule §11.4.25)

Verbatim user mandate (2026-05-15): “Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!”

No feature / functionality / flow / use case / edge case / service / application on any supported platform of HelixCode may be considered deliverable until covered by

automation tests proving six invariants: (1) anti-bluff posture (CONST-035) with captured runtime evidence; (2) proof of working capability end-to-end on target topology (no mocks beyond unit tests — see CONST-050); (3) implementation matches documented promise; (4) no open issues/bugs surfaced — cross-checked against §11.4.15 / §11.4.16 trackers; (5) full documentation in sync per §11.4.12; (6) four-layer test floor per §1 (pre-build + post-build + runtime + paired mutation).

Consuming projects MUST publish a coverage ledger (feature × platform × invariant-1..6 × status) regenerated as part of the release-gate sweep. Gaps tracked per §11.4.15 (UNCONFIRMED: / PENDING_FORENSICS: / OPERATOR-BLOCKED: with §11.4.21 audit) — rows that quietly omit a platform are CONST-048 violations.

Cascade requirement: This anchor (verbatim or by CONST-048 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. See constitution submodule Constitution.md §11.4.25 for the full mandate.

CONST-049: Constitution-Submodule Update Workflow Mandate (cascaded from constitution submodule §11.4.26)

Verbatim user mandate (2026-05-15): “Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE committed and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!”

Before ANY modification to constitution/Constitution.md, constitution/CLAUDE.md, or constitution/AGENTS.md, the agent or operator MUST execute the following 7-step pipeline in order:

1. **Fetch + pull first** inside the constitution submodule worktree — every configured remote fetched, then `git pull --ff-only` (or `--rebase` if non-FF; NEVER `--strategy=ours` / `--allow-unrelated-histories` without explicit authorization).
2. **Apply the change** with §11.4.17 classification + verbatim mandate quote.
3. **Validate before commit** — `meta_test_inheritance.sh` (or equivalent), no merge-conflict markers, cross-file consistency.

4. **Commit + push to ALL upstreams** — governance files only (NEVER `git add -A`); push to every configured remote. One-upstream commit = CONST-049 violation (also CONST-038/§6.W and §2.1).
5. **Conflict resolution** preserving union of governance content. Force-push to bypass conflicts is FORBIDDEN (CONST-043 / §9.2).
6. **Post-merge validation** — `git submodule update --remote --init + re-run cascade verifier` (CONST-047) confirming the new clause reaches every owned submodule.
7. **Bump consuming project pointer** — `.gitmodules`-tracked submodule pointer advanced to the new constitution HEAD in the SAME commit as cascade work.

Cascade requirement: This anchor (verbatim or by CONST - 049 ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a force-push without CONST-043 / §9.2 authorization. No escape hatch. See constitution submodule `Constitution.md` §11.4.26 for the full mandate.

CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from constitution submodule §11.4.27)

Verbatim user mandate (2026-05-15): “Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interact with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — <https://github.com/vasic-digital/Challenges>). EVERYTHING MUST BE tested using helix_qa (fully incorporating helix_qa Submodule — <https://github.com/HelixDevelopment/HelixQA>). helix_qa MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full helix_qa QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!).”

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, TODO, FIXME, “for now”, “in production this would”, or empty-implementation patterns are PERMITTED only in unit-test sources (`*_test.go` files invoked without the integration build tag; `helix_code/tests/unit/`; etc.). Every other test type —

integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, helix_qa — MUST exercise the real, fully implemented HelixCode system against real infrastructure (real PostgreSQL, real Redis, real LLM endpoints, real containers, real captured devices). Production code (anything under helix_code/cmd/, helix_code/applications/, helix_code/internal/<pkg>/<file>.go not ending _test.go) MUST NOT import from helix_code/internal/mocks/.

(B) 100% test-type coverage. HelixCode's codebase MUST be covered by every supported test type the domain warrants: - **Unit** — fast, isolated, mocks permitted per (A). - **Integration** — multi-component, no mocks, real backing services. - **End-to-end (E2E)** — full user-flow exercise on target topology. - **Full automation** — orchestrated suites exercising every feature × platform combination (CONST-048 coverage ledger). - **Security** — authn/authz boundaries, CONST-042 secret-leak scans, input-fuzzing, dependency-CVE scanning, threat-model verification. - **DDoS** — request-flood resilience at advertised throughput tier. - **Scaling** — horizontal + vertical scale behaviour under linear load growth. - **Chaos** — controlled failure injection (network partition, process kill, disk full, clock skew). - **Stress** — sustained load above advertised tier. - **Performance** — latency / throughput / tail-latency invariants vs SLO baselines. - **Benchmarking** — micro + macro suites with historical p95-drift detection. - **UI** — visual-regression + DOM-state + interaction-flow coverage on every target platform's UI surface. - **UX** — flow-correctness + accessibility + i18n + visual-cue ordering (§11.4.23 composition). - **Challenges** — vasic-digital/Challenges submodule (at ./challenges/) fully incorporated; per-feature Challenge scripts with captured runtime evidence. - **HelixQA** — HelixDevelopment/HelixQA submodule (at ./helix_qa/) fully incorporated; ALL written test banks executed; full autonomous QA sessions run as part of release gates with captured wire evidence per check.

Required dependency submodules (recursive per CONST-047): - Challenges — git@github.com:vasic-digital/Challenges.git — incorporated at ./challenges/. - helix_qa — git@github.com:HelixDevelopment/HelixQA.git — incorporated at ./helix_qa/. - Any additional functionality submodules under vasic-digital/* / HelixDevelopment/* orgs that HelixCode depends on — incorporate rather than duplicate work the orgs already maintain.

Submodule pointers MUST be bumped to upstream HEAD in the SAME commit as any dependent cascade work (CONST-049 step 7). Pointer drift = CONST-050 violation.

Cascade requirement: This anchor (verbatim or by CONST - 050 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. See constitution submodule Constitution.md §11.4.27 for the full mandate.

CONST-051: Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from constitution submodule §11.4.28)

Verbatim user mandate (2026-05-15): “All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory - we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project’s codebase! We MUST work on that code as much as we do with main project’s codebase! All on equal basis! Equally important! We MUST take it into the account, analyze it, extend it, create missing tests, do full testing of it, fill the gaps (if any), fix any issues that we discover or they pop-up, write and extend the documentation, user guides, manulas, diagrams, graphs, SQL definitions, Website(s) and all other relevant materials! We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE accessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project - directly from project’s root project_name/submodule_name or some more proper structure project_name/submodules/submodule_name!”

Three cooperating invariants apply to every HelixCode-owned submodule (those whose upstream origin lives under vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory, or any subsequently authorised org):

(A) Equal-codebase. Every owned-by-us submodule is an **equal part** of HelixCode’s codebase. The same engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, graphs, SQL definitions, website pages, all materials) — applies to each owned submodule on equal basis. A round of work that improves only HelixCode’s main while leaving an owned-submodule deficiency unaddressed is a CONST-051 violation, severity-equivalent to a §11.4 PASS-bluff at the project-scope layer. The §11.4.25 / CONST-048 coverage ledger MUST list every owned submodule as an in-scope target.

(B) Decoupling / reusability. Owned submodules MUST remain fully decoupled from HelixCode (and any other consuming project). No HelixCode-specific context, hardcoded paths, hostnames, asset names, or runtime assumptions may be introduced into an owned submodule’s source tree. When a submodule needs

information from HelixCode, the honest path is configuration injection (env var, config file, constructor parameter) — never a hardcoded reach into the parent’s tree. Every owned submodule MUST be project-not-aware, fully reusable, modular, and completely testable as a standalone repository.

(C) Dependency-layout. Every dependency that an owned submodule consumes MUST be accessible from HelixCode’s root at one of two canonical paths: - `<repo_root>/<submodule_name>/` (flat layout — current HelixCode layout for Challenges, HelixQA, Containers, Security, etc.) - `<repo_root>/submodules/<submodule_name>/` (grouped layout — alternate)

Nested own-org submodule chains are FORBIDDEN. A submodule MUST NOT have its own `.gitmodules` entries pulling in further owned-by-us repos. Every dependency required by submodule X is added to HelixCode’s root at the canonical path; X reaches it via documented import / SDK path / runtime resolver — never via its own nested submodule pointer. Third-party submodules (not under our orgs) are exempt — they MAY appear at any depth.

The owned-org list is dynamically discoverable at any time via `gh org list / glab CLIs` or the orgs’ public APIs.

Cascade requirement: This anchor (verbatim or by CONST-051 ID reference) MUST appear in every owned submodule’s `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the codebase-completeness layer. No escape hatch. See constitution submodule `Constitution.md` §11.4.28 for the full mandate (audit gates, mutation pairs, workflow integration).

Amendment Process

Constitution amendments require: 1. Written proposal with rationale 2. Challenge demonstrating the need 3. 72-hour review period 4. Approval by project architect 5. Update to all submodule governance files

This Constitution is the supreme law of the HelixCode project. No code, test, or process may contradict it.

CONST-052: Lowercase-Snake_Case-Naming Mandate (cascaded from constitution submodule §11.4.29)

Verbatim user mandate (2026-05-15): *`“naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MUST HAVE lowercase names with space separator between the words of ‘_’ character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! However, since this will most likely break some of the functionalities renaming we do MUST BE applied to all references to particular Submodule or directory! ... There MUST BE reasonable exceptions for this rules - source code for programming languages or Submodules which apply different naming convention - Android, Java, Kotlin and others. ... Upstreams directory which all of our projects and Submodules have MUST BE renamed to the lowercase letters too, however root project containing the install_upstreams system command (it is exported in our paths in our .bashrc or .zshrc) MUST BE updated to fully work with both Upstreams and upstreams directory. ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!”`*

Every directory, submodule, and file in HelixCode MUST use lowercase snake_case names. Existing non-compliant names (helix_code/, challenges/, containers/, helix_agent/, helix_qa/, security/, github_pages_website/, upstreams/, dependencies/, etc.) MUST be renamed as part of the phased migration opened by this clause. Every reference (configs, docs, links, source-code imports, governance files) MUST be updated atomically with the rename — reference drift after a rename is a CONST-052 violation of equal severity to the rename itself.

Common-sense exceptions (technology-preserving): language-mandated case for Java/Kotlin/Android/Apple/C#/Swift INSIDE the language root (submodule root follows our convention; subtree follows language convention); vendor/upstream third-party submodules keep upstream names; build artefacts (node_modules, __pycache__, .git, target, build, bin) keep tool-mandated names. The test “does renaming break the technology?” trumps the rule.

upstreams/ → upstreams/ transition: the constitution submodule’s `install_upstreams.sh` (exported via `.bashrc/.zshrc`) supports BOTH `upstreams/` and `upstreams/` directory layouts (commit 45d3678 of the constitution submodule); lowercase wins when both present.

Test coverage of renames (per CONST-050(B)): every rename batch ships with (i) regression test verifying every reference now resolves, (ii) full test-type matrix run post-rename, (iii) anti-bluff wire-evidence captured.

Phased execution per the operator's explicit instruction: comprehensive brainstorming → phase-divided plan → fine-grained tasks/subtasks → every change covered by every applicable test type. §11.4.20 subagent delegation for cross-cutting rename sweeps.

Cascade requirement: This anchor (verbatim or by CONST-052 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the reference-integrity layer. No escape hatch beyond the common-sense exceptions enumerated above. See constitution submodule Constitution.md §11.4.29 for the full mandate.

CONST-053: .gitignore + No-Versioned-Build-Artifacts Mandate (cascaded from constitution submodule §11.4.30)

Verbatim user mandate (2026-05-15): “every project module, every Submodule, every service and application MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivative which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be committed every time we are preparing to execute commit! If any violation is detected it MUST be fixed before commit is executed!”

Every project module, owned-by-us submodule, service, and application MUST ship a proper .gitignore. Forbidden-from-version-control classes:

1. **Build artefacts:** /bin/, /build/, /dist/, /out/, target/, *.exe, *.dll, *.so, *.dylib, *.a, *.o, *.class, *.pyc, generator-produced files when the generator is committed.
2. **Cache files:** __pycache__/, .pytest_cache/, .mypy_cache/, .ruff_cache/, node_modules/, .next/, .cache/, .gradle/, .terraform/, language-server caches.
3. **Temp files:** *.tmp, *.swp, *~, .DS_Store, Thumbs.db, *.orig, *.rej.
4. **Sensitive-data files:** .env, .env.* (allow .env.example placeholder only — no real secrets even as examples), *.pem, *.key, *.crt, id_rsa*, id_ed25519*, .netrc, secrets/, api_keys.sh.
5. **Generated reports/logs:** *.log, coverage.out, htmlcov/, runtime captures unless reference assets.

6. **OS/IDE personal state:** `.idea/`, `.history/`, `.vscode/` (except shared settings).

Anti-bluff invariant: `.gitignore` line alone is not sufficient — no file matching the forbidden patterns may be CURRENTLY TRACKED. A tracked `*.log` despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged + git status` BEFORE executing the commit. Forbidden-class hits abort the commit until fixed (un-stage, add to `.gitignore`, scrub if already-tracked). Gate CM-GITIGNORE-PRECOMMIT-AUDIT + paired mutation.

Secret-leak intersection (CONST-042 / §11.4.10): a `.env` leak is BOTH a CONST-053 and a CONST-042 violation; rotation + post-mortem required.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it is a build derivative and MUST be ignored. The committed sources MUST include the generator.

Cascade requirement: This anchor (verbatim or by CONST-053 ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. See constitution submodule `Constitution.md` §11.4.30 for the full mandate.

CONST-054: Submodule-Dependency-Manifest Mandate (cascaded from constitution submodule §11.4.31)

Verbatim user mandate (2026-05-15): “We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file with expected Submodules Git ssh urls perhaps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to everyone.”

Every owned-by-us submodule MUST ship `helix-deps.yaml` at its root declaring its own-org dependencies. Schema: `schema_version`, `deps: [{name, ssh_url, ref, why, layout: flat|grouped}], transitive_handling: {recursive, conflict_resolution}, language_specific_subtree`. Tooling: `incorporate-submodule <ssh-url>` adds the submodule at the parent project's canonical path (CONST-051(C)), reads `helix-deps.yaml`, recurses for each

declared dep, aborts on conflicting refs, emits <root>/ .helix-manifest.yaml audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches the manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a CONST-054 violation.

§11.4.31 / CONST-054 is the **operational complement** of CONST-051(C): nested own-org submodule chains are FORBIDDEN — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Cascade requirement: This anchor (verbatim or by CONST - 054 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to §11.4 PASS-bluff at the dependency-graph layer. See constitution submodule Constitution.md §11.4.31 for the full mandate.

CONST-055: Post-Constitution-Pull Validation Mandate (cascaded from constitution submodule §11.4.32)

Verbatim user mandate (2026-05-15): *“Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!”*

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run `scripts/verify-all-constitution-rules.sh` BEFORE the new constitution HEAD is treated as canonical for any other work. The sweep re-runs the governance-cascade verifier AND every implementable rule gate (CONST-053 .gitignore audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke, etc.) against the post-pull tree. Failures populate the project's Issues tracker per §11.4.15 (Status: Reopened, Type: Bug); closure requires positive-evidence per §11.4.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook OR commit-wrapper invocation). Operator-explicit manual invocation also available.

Anti-bluff: the sweep's own meta-test (paired mutation per §1.1) plants a known violation of each enforced gate and asserts the sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a CONST-055 violation.

CONST-055 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced.

Cascade requirement: This anchor (verbatim or by CONST-055 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to §11.4 PASS-bluff at the constitutional-enforcement layer. See constitution submodule Constitution.md §11.4.32 for the full mandate.

CONST-056: Mandatory install_upstreams on clone/ add Mandate (cascaded from constitution submodule §11.4.36)

Verbatim user mandate (2026-05-15): “Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zshrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!”

Every clone / add of a Git repository under HelixCode MUST be followed by install_upstreams invocation from the repository's root IF its tree contains upstreams/ (or legacy upstreams/ per CONST-052 transition) populated with *.sh recipe files. The utility (installed on operator's PATH via .bashrc/.zshrc; implementation in the constitution submodule's install_upstreams.sh — already supports BOTH directory names since constitution commit 45d3678) reads the recipe files, configures every declared upstream as a named git remote, and fans out origin push URLs.

Skipping the invocation when upstreams/ is present silently breaks §2.1 (multi-upstream push is the norm) — the next push lands on only one upstream. Gate CM-INSTALL-UPSTREAMS-ON-CLONE + paired mutation. Automation: the future incorporate-submodule per CONST-054 auto-invokes; manual invocation supported. Pre-commit check: `git remote -v | grep -c push` reports expected count.

Cascade requirement: This anchor (verbatim or by CONST-056 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and

AGENTS.md. See constitution submodule Constitution.md §11.4.36 for the full mandate.

CONST-057: Type-aware Closure-Status Vocabulary (cascaded from constitution submodule §11.4.33)

Every project tracking work items by Type per §11.4.16 MUST close them with the Type-appropriate terminal **Status** value, drawn from this 3-element closed map:

Item Type	Closure Status value
Bug	Fixed (→ Fixed.md)
Feature	Implemented (→ Fixed.md)
Task	Completed (→ Fixed.md)

The (→ Fixed.md) suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (in HelixCode: the §11.4.93 workable-items export DB exporter — which SUPERSEDES the markdown-derived generate_issues_summary.sh / generate_fixed_summary.sh, both of which now REFUSE to run — and the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all “closed, positive evidence captured”) while preserving the literal in the emitted document.

Closing a Feature with Fixed (→ Fixed.md) or a Task with Implemented (→ Fixed.md) is a CONST-057 violation. Gate CM-CLOSURE-VOCAB-TYPE-AWARE walks every Fixed.md heading + every Issues.md heading whose **Status** is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23.

Cascade requirement: This anchor (verbatim or by CONST-057 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.33 for the full mandate.

CONST-058: Reopened-Source Attribution Mandate (cascaded from constitution submodule §11.4.34)

Every Issues.md (or equivalent project tracker) heading whose **Status:** is Reopened MUST carry, within 8 non-blank lines of the heading, a **Reopened-Details:** line capturing four sub-facts:

- **By:** AI or User (source-of-truth observer who flipped the status). AI covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). User covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (YYYY-MM-DD).
- **Reason:** one-line cause classification — chosen from the closed vocabulary { test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }. Other values permitted with explicit Reason: <free text> annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations (demotion from Fixed requires captured evidence under the conditions that re-exposed the defect).

The Issues_Summary.md Status column MUST distinguish the four Reopened sub-states by source so a sweep query for “reopens by AI in the last 30 days” is mechanically possible. Suggested column rendering: Reopened (AI: test-failed) vs Reopened (User: manual-testing). Gate CM-ITEM-REOPENED-DETAILS mirrors CM-ITEM-OPERATOR-BLOCKED-DETAILS (§11.4.21 walk pattern). Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21.

Cascade requirement: This anchor (verbatim or by CONST-058 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.34 for the full mandate.

CONST-059: Canonical-Root Inheritance Clarity (cascaded from constitution submodule §11.4.35)

The **constitution submodule’s** three files (constitution/Constitution.md, constitution/CLAUDE.md, constitution/AGENTS.md) ARE the **canonical root** (also called the **parent** files). They contain only universal rules per §11.4.17.

The consuming project’s **repository-root files** (<project-root>/CLAUDE.md, <project-root>/AGENTS.md, optionally <project-root>/Constitution.md)

are **consumer extensions**. They MUST start with the inheritance pointer (either the Claude-Code native @constitution/CLAUDE.md import or the portable ## INHERITED FROM constitution/CLAUDE.md heading). They contain only project-specific rules per §11.4.17.

When in doubt about which file to edit: universal rule → constitution submodule's file; project-specific rule → consumer's file. Default consumer-side when uncertain (§11.4.17 — narrower scope is cheap to widen).

Terminology: “the parent CLAUDE.md” / “the root Constitution” → constitution-submodule file at constitution/<filename>; “the project CLAUDE.md” / “this project's AGENTS.md” → consumer-side file at <project-root>/<filename>.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — git mv of a section if it's a clean clone, or explicit Lifted from <project> to constitution per §11.4.35 / Demoted from constitution to <project> per §11.4.35 commit-message annotation.

Gate CM-CANONICAL-ROOT-CLARITY verifies (a) consumer's CLAUDE.md opens with the inheritance pointer, (b) constitution submodule's three files are present at the expected path, (c) no ## INHERITED FROM block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17.

Cascade requirement: This anchor (verbatim or by CONST-059 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.35 for the full mandate.

CONST-060: Fetch-before-edit Mandate (cascaded from constitution submodule §11.4.37)

Verbatim user mandate (2026-05-15): *“Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them! Follow the constitution Submodule documentation for details.”*

The FIRST git-touching action of every session, on every consuming project (owned or third-party), MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}
git submodule foreach --recursive 'git fetch --all --prune --quiet'
```

If HEAD . .@{u} is non-empty, integrate the upstream changes BEFORE any local edit. Acting on stale local state produces three failure modes documented in the originating §11.4.37 incident (multi-agent / parallel-session work): (1) **redundant work** — the agent re-does what a parallel session already finished, (2) **false confidence** — completion reports for already-done work, (3) **divergent history** — duplicate sibling commits that double the conflict surface on next push.

Anti-bluff invariant: the fetch+log check MUST produce captured evidence — the actual HEAD . .@{u} output, even if empty. Skipping the check on the basis of “I just fetched” or “nothing could have changed in the last N minutes” is a §11.4.6 (no-guessing) violation: the remote state is not knowable without a fetch.

Cascade requirement: This anchor (verbatim or by CONST-060 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to §11.4 PASS-bluff at the parallel-session-coordination layer. See constitution submodule Constitution.md §11.4.37 for the full mandate.

§11.4.68 — Positive Sink-Side / Downstream Evidence Mandate (cascaded from constitution submodule §11.4.68)

Verbatim user mandate (2026-05-20): *“We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! This MUST BE investigated and fixed! How come we passed the tests with Arvus validation? What were values for the Codec In Use field? Empty means nothing! This is not working! It MUST BE FIXED, TESTED AND VERIFIED WITH FULL AUTOMATION TESTING ASAP!!!”*

A test that asserts audio or video routing PASS MUST capture and verify **positive sink-side or downstream evidence** — never config-only, never metadata-only, never PCM-open-state-only. At least one of the closed enumeration MUST be captured for every audio/video routing PASS: (1) sink-side codec-state with non-empty Codec-In-Use matching the expected codec regex; (2) strictly-positive PCM frames-written delta from /proc/asound/. . . /status hw_ptr; (3) ALSA ELD/EDID-Like-Data showing negotiated channel count + format; (4) ffprobe-on-captured-mp4 with non-zero frame count + expected codec/resolution/fps; (5) recording-analyzer event match per §11.4.2/§11.4.5; (6) tinycap RMS amplitude above the line-level floor. Empty / <unreachable> / <N.E.> / <None> placeholders are NOT positive evidence; a missing-but-required sink is OPERATOR-BLOCKED (release-blocker), never SKIP, never PASS. No escape hatch — no --skip-sink-evidence, --allow-empty-codec, --sink-unreachable-is-pass, --metadata-only-suffices flag exists.

Cascade requirement: This anchor (verbatim or by §11.4.68 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule Constitution.md §11.4.68 for the full mandate.

§11.4.70 — Subagent-Driven Execution Is The Default (cascaded from constitution submodule §11.4.70)

Verbatim user mandate (2026-05-20): *“Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!”*

When executing implementation plans (or any task-decomposed execution flow), the **default execution model is subagent-driven** per superpowers : subagent-driven-development. Inline execution is permitted ONLY when (a) the task is trivial AND fits a single sub-300-line edit, OR (b) the operator explicitly requests inline at brainstorm-handoff time. Subagent-driven is the default because it gives isolated context per task, naturally enforces two-stage review, is parallel-PWU compatible (§11.4.58), creates an anti-bluff seam (§11.4), and survives operator absence. No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff.

Cascade requirement: This anchor (verbatim or by §11.4.70 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the execution-model layer. **Canonical authority:** constitution submodule Constitution.md §11.4.70 for the full mandate.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (cascaded from constitution submodule §11.4.71)

Verbatim user mandate (2026-05-20): *“before pushing changes to any upstream for any repository - main repo or Submodule, we MUST fetch and pull all changes. Once these are obtained WE MUST investigate what is different compared to head position we were on last time before fetching and pulling new changes! We MUST understand what is done*

and for what purpose, especially how that does affect our project and our System in general! Any mandatory changes or improvements required by fresh changes we just have brought in MUST BE incorporated, covered with all supported types of the tests which will produce as a result of its success execution REAL PROOFS of working for all componetns and functionalities covered and work fully in anti-bluff manner!”

The everyday-push variant of §11.4.41. EVERY push (every repository — main + every submodule) MUST follow the 5-step cycle: (1) fetch all remotes (`git fetch --all --prune --tags`, capture stdout); (2) pull all upstream branches whose tip differs, resolving conflicts per consumer judgment (never auto--ours/--theirs); (3) investigate the diff vs OUR previous HEAD — read EVERY foreign commit’s body, understand what/why/how-it-affects-our-system; (4) integrate mandatory changes with §11.4.4(b) four-layer coverage + §11.4.43 TDD-fix discipline, every PASS carrying §11.4.5 captured-evidence (REAL PROOFS, not metadata-only); (5) only then push, verifying with `git ls-remote post-push`. No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag.

Cascade requirement: This anchor (verbatim or by §11.4.71 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the push-discipline layer. **Canonical authority:** constitution submodule Constitution.md §11.4.71 for the full mandate.

§11.4.72 — Audio Top-Priority Mandate (cascaded from constitution submodule §11.4.72)

Verbatim user mandate (2026-05-20): *“Make sure all fixes for audio are always top priority in main working stream!”*

The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. Parallel BACKGROUND subagents (research, refactors, infrastructure documentation) MAY run concurrently with audio work but do NOT preempt audio on the main-stream serial dispatch queue. No escape hatch — there is no “but this non-audio task is faster” or “but this research is more interesting” override; audio-stack regressions are user-perceptible and high-impact while research and refactors can wait.

Cascade requirement: This anchor (verbatim or by §11.4.72 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a process violation at the dispatch-priority layer. **Canonical authority:** constitution submodule Constitution.md §11.4.72 for the full mandate.

§11.4.73 — Main-Specification Document Versioning + Revision Discipline (cascaded from constitution submodule §11.4.73)

Verbatim user mandate (2026-05-20): “Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Add this into root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md as mandatory rule / constraint applicable ONLY IF we have something like the main specification document or we do recognize something like the main specification document. Document MUST BE updated ALWAYS to follow the versioning rules we are applying here + revision and other properties we have!”

Applies **only when a project recognises a main specification document**. When it does: (1) every additive operator requirement, refinement, or accepted recommendation MUST be applied to the spec before or as part of the implementing work; (2) spec versioning has two axes — *primary* (V1/V2/V3, bumped for major rewrites by explicit operator decision, old versions archived) and *secondary* (the §11.4.61 metadata-table Revision integer, bumped for every other change); (3) the metadata table MUST stay current (Revision, Last modified, Status summary, Fixed); (4) propagated copies of the rule MUST reference the active specification.V<primary>.md, not a stale archive; (5) on primary bump the old file moves to <spec-dir>/archive/ with Status: superseded. Classification: universal, applicable conditionally per the scope condition.

Cascade requirement: This anchor (verbatim or by §11.4.73 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a release blocker when a project has a main spec and lets it drift. **Canonical authority:** constitution submodule Constitution.md §11.4.73 for the full mandate.

§11.4.74 — Submodule-Catalogue-First Discovery + Extend-Don't-Reimplement (cascaded from constitution submodule §11.4.74)

Verbatim user mandate (2026-05-20): *“We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times if they are already done as some of existing universal, reusable general development purpose Submodules! For any missing features that some Submodules we incorporate may be missing we MUST IMPLEMENT the properly and extend those Submodules furter! We do control all of the and we CAN and MUST maintain and extend the regularly! All development cycle rules we have MUST BE applied to them and fully respected!”*

Before scaffolding ANY new module, package, helper, or utility, the contributor (human or AI agent) MUST: (1) survey the canonical Submodule catalogue — vasic-digital and HelixDevelopment on both GitHub AND GitLab; (2) inventory existing Submodules; (3) reuse before reimplement — if a Submodule provides the functionality (or 80%+ of it), add it as a Git submodule rather than write fresh; (4) extend in-place when 80%+ matches but features are missing — add the missing features TO THAT SUBMODULE (PR upstream + bump pointer), never as a duplicating consuming-project helper; (5) apply all development-cycle rules to those Submodules; (6) document the survey result in the feature's tracker entry with a Catalogue-Check: field (reuse <org/repo>@<sha> / extend <org/repo>@<sha> / no-match <date>). Classification: universal.

Cascade requirement: This anchor (verbatim or by §11.4.74 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a process violation; duplicate implementations landed without catalogue check are release blockers. **Canonical authority:** constitution submodule Constitution.md §11.4.74 for the full mandate.

§11.4.69 — Universal Sink-Side Positive-Evidence Taxonomy + Mechanical Enforcement (cascaded from constitution submodule §11.4.69)

Verbatim user mandate (2026-05-20): *“THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for*

every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! ... Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! ... Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!”

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a2dp, bluetooth_pair, touch_input, sensor, gpu_render, storage_read, storage_write, mediacodec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render — open to additions, never contraction). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape. New helper contracts (additive during grace, mandatory after 2026-06-19): ab_pass_with_evidence <description> <evidence_path> (verifies path exists + non-empty), ab_skip_with_reason <description> <closed-set-reason> (reasons: geo_restricted, operator_attended, hardware_not_present, topology_unsupported, network_unreachable_external, feature_disabled_by_config; forbids network_unreachable_external for any taxonomy feature with a sink-side probe); bare ab_pass deprecated (WARN pre-grace, FAIL post-grace). Three pre-build gates + paired §1.1 mutations: CM-SINK-EVIDENCE-PER-FEATURE, CM-NO-FAIL-OPEN-SKIP, CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE. No escape hatch — no --skip-evidence, --config-only-pass, --allow-fail-open-skip, --legacy-ab-pass-permitted flag.

Cascade requirement: This anchor (verbatim or by §11.4.69 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-69-PROPAGATION enforces the anchor literal across the consumer fleet; paired mutation strips the literal → gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer.

Canonical authority: constitution submodule Constitution.md §11.4.69 for the full mandate.

§11.4.75 — Mechanical Enforcement Without Exception (cascaded from constitution submodule §11.4.75)

Verbatim user mandate (2026-05-20): “Why do these violations still happen!? This is a serious problem! We cannot rely on stability nor consistency if we cannot respect our Constitution, mandatory rules and constraints! Is there a way to make this always respected, followed and applied without exception fully and unconditionally!? WE MUST HAVE THIS WORKING FLAWLESSLY!!! Do investigate the root causes of such problems! Once all problems are identified WE MUST apply proper mechanisms for this not to happen NEVER EVER AGAIN!”

The §11.4 covenant historically relied on agent + operator vigilance; three 2026-05-19→20 forensic incidents proved that late-binding enforcement fires hours-to-days after the violator commit reaches every remote. §11.4.75 closes the gap with FIVE independent mechanical enforcement layers — bypassing any single layer does not bypass the discipline: (1) local pre-commit git hook (refuses staged .md lacking sibling .html+.pdf); (2) commit_all.sh integration (_constitution_sibling_check + auto-sync_all_markdown_exports.sh self-repair); (3) local pre-push git hook (re-runs siblings + propagation-gate subset); (4) post-commit auto-repair hook (auto-generates orphan-.md siblings, idempotent + recursion-guarded); (5) local-only final-gate ritual (remote CI DISABLED per User mandate — operator runs pre_build_verification.sh + meta-test before every tag per §11.4.40). Helper contracts: scripts/install_git_hooks.sh, scripts/git_hooks/{pre-commit,pre-push,post-commit,commit-msg}, _constitution_sibling_check. The commit-msg hook enforces a Bypass-rationale: <reason> footer when --no-verify is detected; docs/audit/bypass_events.md accumulates the audit trail. Five gates with paired §1.1 mutations: CM-COVENANT-114-75-PROPAGATION, CM-GIT-HOOKS-INSTALL-SCRIPT, CM-GIT-HOOKS-SOURCE-DIR, CM-COMMIT-ALL-SIBLING-CHECK, CM-CI-WORKFLOW-PRESENT. No escape hatch — no --skip-hooks, --bypass-enforcement, --allow-orphan-md, --ci-not-applicable, --mechanical-enforcement-not-needed flag.

Cascade requirement: This anchor (verbatim or by §11.4.75 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-75-PROPAGATION; paired mutation strips the literal → gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the enforcement layer. **Canonical authority:** constitution submodule Constitution.md §11.4.75 for the full mandate.

§11.4.76 — Containers-Submodule Mandate (cascaded from constitution submodule §11.4.76)

Verbatim user mandate (2026-05-20): “For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemu / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: <https://github.com/vasic-digital/containers> (git@github.com:vasic-digital/containers.git). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!”

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), every consuming project MUST: (1) install vasic-digital/containers (digital.vasic.containers) as a Git submodule; (2) consume via replace directive during development + pinned commit SHAs in production; (3) boot infra on-demand via pkg/boot + pkg/compose + pkg/health so operators are never required to start podman machine / docker compose up manually — the boot is part of the test entry point (the on-demand-infra invariant); (4) extend the Submodule (PR upstream) for missing runtimes / lifecycle primitives — never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components MUST actually boot them via the Submodule — short-circuit fakes that bypass boot are a §11.4 violation. Tracker rows touching containerization MUST record Catalogue-Check: extend vasic-digital/containers@<sha> (or reuse). Planned gate CM-CONTAINERS-USED scans container-touching PRs for digital.vasic.containers/... imports; paired mutation strips the import + asserts FAIL.

Cascade requirement: This anchor (verbatim or by §11.4.76 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-76-PROPAGATION; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule Constitution.md §11.4.76 for the full mandate.

§11.4.77 — Regeneration-Mechanism-Required Mandate (cascaded from constitution submodule §11.4.77)

Verbatim user mandate (2026-05-20): *“We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content!”*

Every `.gitignore` entry excluding (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project MUST carry a documented + automated mechanism to either re-obtain (download from authoritative source: vendor tarball, SDK installer, npm/pip/cargo/go-mod/container registry, dedicated git submodule, S3/GCS) OR re-generate (run from tracked source via build pipeline, code-gen, asset render, captured-evidence replay, container build). Required artefacts per qualifying entry: (1) `.gitignore-meta/<entry-slug>.yaml` declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source + integrity hash + requires-network + requires-credentials; (2) a non-interactive entry in `scripts/setup.sh` post-clone bootstrap; (3) a pre-build gate verifying regenerated content present OR a recent `.gitignore-meta/.regenerated/<slug>.ok` stamp; (4) `README + docs/guides/*.md` describing the mechanism + manual fallback + time/disk budget + §11.4.10 credentials. Bare `.gitignore` additions without the mechanism are a §11.4 PASS-bluff variant — codebase appears complete but a fresh clone cannot build/run. No escape hatch — no `--skip-regen-mechanism`, `--gitignore-is-enough`, `--operator-already-has-content` flag. Planned gate `CM-GITIGNORE-REGEN-MECHANISM` + paired §1.1 mutation (strip a required YAML key → gate FAILs).

Cascade requirement: This anchor (verbatim or by §11.4.77 reference) MUST appear in every owned submodule’s `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-77-PROPAGATION`; paired mutation strips the literal → gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.77 for the full mandate.

§11.4.78 — CodeGraph Code-Intelligence Mandate (cascaded from constitution submodule §11.4.78)

Verbatim user mandate (2026-05-20): *“Make codegraph MANDATORY CHOICE for this purpose for all of our project ... All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!”*

Every consuming project worked on by AI coding agents MUST install, initialize, and use **CodeGraph** (<https://github.com/colbymchenry/codegraph>, npm @colbymchenry/codegraph) — a local SQLite semantic code-knowledge-graph exposed to agents over MCP (100% local, no cloud). (1) Install globally via npm with a user-writable npm prefix (no sudo). (2) `codegraph init` + `codegraph index`: `.codegraph/config.json` is tracked, `.codegraph/codegraph.db` is gitignored with `codegraph index` as its §11.4.77 regeneration mechanism; the `config.json` exclude list MUST exclude every credential/secret path per §11.4.10. (3) Wire `codegraph serve --mcp` into every CLI agent (Claude Code `.mcp.json`, OpenCode `opencode.json`, Qwen Code `.qwen/settings.json`, Crush `.crush.json`, host-local otherwise) referencing the bare `codegraph` command on PATH (no hardcoded host path). (4) Cover the integration with an anti-bluff suite whose per-agent end-to-end layer uses an unforgeable challenge (a fact obtainable only by calling a CodeGraph MCP tool, e.g. `index node count` via `codegraph_status`); a genuinely un-drivable agent is a documented SKIP per §11.4.3, never a faked PASS. (5) Document in `docs/CODEGRAPH.md`, kept in sync per §11.4.12 / §11.4.65. CodeGraph is consumed as the published npm package (§11.4.74) — not a git submodule, adds no Git remote. Planned gate CM-CODEGRAPH-WIRED + paired §1.1 mutation (strip a secret-exclusion → gate FAILs).

Cascade requirement: This anchor (verbatim or by §11.4.78 reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate CM-COVENANT-114-78-PROPAGATION; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule `Constitution.md` §11.4.78 for the full mandate.

§11.4.79 — Own-Org Submodules MUST Be Included in the CodeGraph Index (cascaded from constitution submodule §11.4.79)

Verbatim user mandate (2026-05-21): “All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!”

Refines §11.4.78's exclude-list with a per-submodule-ownership split: (a) own-org submodules (full write access via the project's CLIs — canonical orgs `vasic-digital` + `HelixDevelopment`) MUST be INCLUDED in the index; (b) third-party submodules (the §11.4.74 no-match → vendor path) MUST be EXCLUDED. Operational steps: (1) `git submodule update --remote --merge` to pull latest before re-indexing, respecting load-bearing pins on third-party submodules; (2) adjust `.codegraph/config.json` exclude list to keep own-org paths in scope; (3) re-index via `scripts/codegraph_setup.sh`; (4) verify via `scripts/`

codegraph_validate.sh with ≥ 1 probe resolving a symbol living ONLY inside an own-org submodule; (5) paired §1.1 mutation — temporarily add the own-org submodule to exclude → validate MUST FAIL on the cross-submodule probe → restore. An index that lies about reachable symbols is a PASS-bluff against AI agents. Own-org submodules silently excluded without an audit trail in .codegraph/config.json comments is a release blocker.

Cascade requirement: This anchor (verbatim or by §11.4.79 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-79-PROPAGATION; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule Constitution.md §11.4.79 for the full mandate.

§11.4.80 — CodeGraph Regular-Update + Sync Automation Mandate (cascaded from constitution submodule §11.4.80)

Verbatim user mandate (2026-05-21): “We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed on the host machine! ... Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule ... Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents ... and regularly export them like all other Status docs into the PDF and HTML!”

Three deliverables (all living in the constitution submodule, inherited by reference per §3 — consuming projects invoke at $\${CONST_DIR}/scripts/codegraph_*.sh$, never copy): (1) scripts/codegraph_update.sh — npm-installs latest @colbymchenry/codegraph after a registry version check; appends old/new version to docs/codegraph/Status.md; anti-bluff verifies codegraph --version reflects the new version after install (npm exit 0 ≠/working binary). (2) scripts/codegraph_sync.sh — after a successful update runs codegraph status → codegraph sync . → codegraph status → the project's scripts/codegraph_validate.sh; appends every step's output to BOTH the project's and the constitution's docs/codegraph/Status.md. (3) docs/codegraph/Status.md + Status_Summary.md append-only ledgers, exported to .html + .pdf per §11.4.65. Cadence: weekly floor (per §11.4.45). A consuming project that has not run codegraph_update.sh in >2 weeks AND has open AI-agent work is a release blocker. Paired §1.1 mutation: downgrade installed version → script detects drift → restore.

Cascade requirement: This anchor (verbatim or by §11.4.80 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-80-PROPAGATION; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule Constitution.md §11.4.80 for the full mandate.

§11.4.81 — Cross-Platform-Parity Mandate (cascaded from constitution submodule §11.4.81)

Verbatim user mandate (2026-05-21): *“Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!”*

Every consuming project whose supported-platforms manifest lists more than one OS MUST, for every feature/test/gate/challenge/mutation depending on platform-specific primitives, ship a per-OS-equivalent implementation chosen at runtime via `uname -s` (or equivalent detection). Three sub-mandates: **(A) Per-OS implementation REQUIRED** — Linux `cgroup/systemd//proc` primitives MUST have documented per-OS equivalents (POSIX `setrlimit/ulimit`, macOS `launchd`, BSD `rctl`, Windows Job Object) chosen via runtime dispatch. **(B) Per-OS tests REQUIRED** — every platform-dependent gate test MUST have case `"$(uname -s)"` in branches with positive captured evidence per §11.4.2 + §11.4.5 in each branch; SKIP-with-reason acceptable ONLY when the platform genuinely cannot enforce the invariant. **(C) Honest kernel-gap citation + adjacent equivalent test REQUIRED** — where a Linux primitive has NO equivalent due to a documented kernel limitation (canonical: XNU does not enforce `RLIMIT_AS` for unprivileged processes), the test MUST detect the gap at runtime, SKIP with exact kernel reason + reproducer + honest-gap-doc link, AND provide an ADJACENT test exercising the closest invariant the platform CAN enforce (e.g. `RLIMIT_CPU+SIGXCPU` as the macOS proxy), itself anti-bluff with a paired §1.1 mutation. Gate CM-CROSS-PLATFORM-PARITY scans for case `"$(uname -s)"` blocks asserting a non-SKIP branch (or honest-gap citation) per platform in the manifest; paired mutation strips a Darwin branch → gate FAILs. No escape hatch.

Cascade requirement: This anchor (verbatim or by §11.4.81 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-81-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker on multi-platform projects. **Canonical authority:** constitution submodule Constitution.md §11.4.81 for the full mandate.

§11.4.82 — Iteration-Speedup Discipline Mandate (cascaded from constitution submodule §11.4.82)

Verbatim user mandate (2026-05-22): *“How can we speed-up this whole development and fixing process? ... Do not forget to all speed optimizations critical rules and mandatory constraints MUST BE all added into our root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md and QWEN.md and all other relevant constitution Submodules files!”*

Iteration cycle time is a first-order quality enabler. Every consuming project’s build / test / commit / debug pipeline MUST adopt these speedup disciplines AS MANDATORY (each independently enforceable): (A) Phase-1 forensic (superpowers:systematic-debugging) before any speculative source patch — speculative patches without FACT-grade root cause are §11.4.6 + §11.4.82 violations; (B) Live-ADB-First (or live-equivalent) before any rebuild — strengthens §11.4.51 to a release-blocker mandate; (C) 30-second pre-flight before launching rebuild orchestrators (device/sink reachability, host memory/disk, no stale locks, no orphan processes); (D) persistent build caches outside containers (ccache/sccache/Gradle daemon bind-mounted to host); (E) module-only rebuild for loadable-module-only changes; (F) parallel multi-device testing with separate qa-results/<TS>/<device-tag>/ outputs; (G) subagent scope discipline + worktree isolation (≤30 min budget, single-responsibility, isolation: "worktree" default); (H) lock-file + stale-process hygiene (clean .git/index.lock, disable auto git-gc in concurrent repos); (I) cycle telemetry per §11.4.24 (commit hash, per-phase wall-clock, speedup-flag set, outcome — aggregated weekly). Gate CM- ITERATION - SPEEDUP - DISCIPLINE audits recent cycles for telemetry citing which of (A)-(I) applied; paired §1.1 mutation strips the speedup-flag column → gate FAILs. No escape hatch — no --skip-phase1-forensic, --no-pre-flight, --rebuild-everything-always, --unlimited-subagent-scope, --ignore-locks, --no-telemetry flag.

Cascade requirement: This anchor (verbatim or by §11.4.82 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-82-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.82 for the full mandate.

§11.4.83 — docs/qa/ End-User Evidence Mandate (cascaded from constitution submodule §11.4.83)

Verbatim user mandate (2026-05-22): “every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under docs/qa/<run-id>/ (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof.”

Every feature that ships MUST carry a recorded end-to-end communication transcript plus any attached materials (screenshots, request/response payloads, audio, file uploads) committed under docs/qa/<run-id>/ — one directory per feature run. Operative rule: (1) every consuming project MUST maintain a docs/qa/ tree, each new feature under docs/qa/<run-id>/ where <run-id> is monotonic + greppable (timestamp / ATM-NNN / other workable-item ID per §11.4.54); (2) transcripts MUST be full bidirectional — every prompt/command sent + every response received (one-sided is not a transcript); (3) attached materials MUST be committed in-repo (no external-only links — that is a §11.4.13 sink-side violation); (4) bot-driven / agent-driven QA automation MUST preserve the full conversation thread as the proof artefact; (5) release gates MUST refuse to tag a version that has any feature-shipping commit without its matching docs/qa/<run-id>/ directory. A feature with no QA transcript is a §11.4 / §107 PASS-bluff. Composes with §11.4.2 / §11.4.5 / §11.4.13 / §11.4.65 / §11.4.69 / §1.1.

Cascade requirement: This anchor (verbatim or by §11.4.83 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-83-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — no --qa-evidence-optional escape hatch. **Canonical authority:** constitution submodule Constitution.md §11.4.83 for the full mandate.

§11.4.84 — Working-Tree Quiescence Rule for Subagent Commits (cascaded from constitution submodule §11.4.84)

Verbatim user mandate (2026-05-22): “no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before git add, the committing agent MUST grep its own working tree for mutation markers (MUTATED for paired, // always pass, return json.Marshal shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT.”

No subagent (or main-thread) commit may proceed while any concurrent mutation gate, paired-mutation experiment, or other in-flight mutation is live in the same checkout. Before `git add`, the committing agent **MUST** grep its own working tree for mutation markers (`MUTATED` for paired, `// always pass`, `return json.Marshal` shortcut paths, `// MUTATION / # MUTATION` annotations, `_mutated_*` filename suffixes, etc.) and explicitly account for every modified file in the staging area; any unexplained file → **ABORT**. (Forensic case: a logo-fix subagent's `git add` swept an `// always pass` JWT-verify mutation residue into an unrelated commit pushed to all four mirrors — a real security-defect window.) Operative rule: (1) pre-`git add` greps for mutation markers + cross-checks `git status --porcelain` against the subagent's declared scope; unaccounted entries → **ABORT**; (2) any active mutation gate **MUST** be serialised (mutate → assert **FAIL** → restore → assert **PASS**) and the working tree verifiably clean before any unrelated commit; (3) concurrent subagents in the **SAME** checkout **MUST** coordinate through a lockfile (`.git/MUTATION_IN_PROGRESS`) — cleaner solution is `git worktree add` per subagent (composes with §11.4.20/§11.4.70); (4) post-commit mutation-residue-scanner **MUST** run before push — any commit containing a mutation marker → push **BLOCKED**.

Cascade requirement: This anchor (verbatim or by §11.4.84 reference) **MUST** appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-84-PROPAGATION`; paired mutation strips the literal → gate **FAILs**. A mutation marker that lands in a tagged commit is a critical defect regardless of how briefly it persisted. **Canonical authority:** constitution submodule `Constitution.md` §11.4.84 for the full mandate.

§11.4.85 — Stress + Chaos Test Mandate (cascaded from constitution submodule §11.4.85)

Verbatim user mandate (2026-05-24): *“Every fix or improvement you do **MUST BE** covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!”*

Every fix or improvement landed **MUST** ship with full-automation **stress** AND **chaos** test suites exercising edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a §11.4 / §107 **PASS-bluff** at the resilience layer. **Stress** (closed-set): sustained load ($N \geq 100$ iterations OR ≥ 30 s wall-clock, p50/p95/p99 latency recorded) + concurrent contention ($N \geq 10$ parallel invocations, no deadlock/leak) + boundary conditions (empty/max/off-by-one, each categorised). **Chaos** (closed-set, per fix-class appropriateness): process-

death injection + network-fault injection (drop/delay/reorder) + input-corruption injection + resource-exhaustion injection (disk full, OOM, FD exhaustion — refuse cleanly OR degrade, NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write). Every stress + chaos PASS MUST cite a captured-evidence artefact path per §11.4.5 + §11.4.69. Helper library `stress_chaos.sh` provides `ab_stress_run`, `ab_stress_concurrent`, `ab_chaos_kill_pid_during`, `ab_chaos_drop_network_during`, `ab_chaos_corrupt_file_during`, `ab_chaos_oom_pressure_during`, `ab_chaos_disk_full_during`, each composing with `ab_pass_with_evidence` / `ab_skip_with_reason`. Cleanup non-negotiable in `trap '... ' EXIT` (cleanup failure = §11.4.14 violation). Four-layer coverage per §11.4.4(b) + paired §1.1 mutation (strip chaos-injection or evidence-capture → gate FAILs). No escape hatch — no `--skip-stress`, `--no-chaos`, `--happy-path-suffices`, `--stress-test-later` flag.

Cascade requirement: This anchor (verbatim or by §11.4.85 reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-85-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.85 for the full mandate.

§11.4.86 — Roster/Corpus-Backed Status-Doc Auto-Sync Mandate (cascaded from constitution submodule §11.4.86)

Verbatim user mandate (2026-05-25): *“Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs — any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!”*

Some Status docs (§11.4.45) are backed by a tracked roster (installed apps/components) or a tracked asset corpus (test/media asset directory) rather than narrative alone. Their freshness MUST NOT depend on operator vigilance — the moment a roster/corpus member changes (app added/removed/renamed; asset added/modified/removed) the Status doc + Status_Summary + HTML + PDF MUST resync out of the box, mechanically. Mechanism (all must hold): (1) drift-proof fingerprint — sha256 of the sorted member list (NOT mtime), persisted in a sidecar beside the Status doc; (2) a sync helper that regenerates the fingerprint + re-exports HTML+PDF via the §11.4.65 exporter, wired so sync is automatic; (3) a pre-build gate that FAILs when the live fingerprint differs from the persisted one (mirrors §11.4.12 `CM-ISSUES-SUMMARY-SYNC` + §11.4.45 `sync_integration_status`); (4) a paired §1.1 mutation corrupting the fingerprint and asserting the gate FAILs. Classification: universal — the

consuming project supplies the specific docs, roster/corpus sources, helper, and gate name per §11.4.35.

Cascade requirement: This anchor (verbatim or by §11.4.86 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-86-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — no --skip-roster-sync, --allow-status-drift, --roster-sync-not-applicable flag.

Canonical authority: constitution submodule Constitution.md §11.4.86 for the full mandate.

§11.4.87 — Endless-Loop Autonomous Work + Zero-Idle Agent Dispatch + Anti-Bluff Testing Mandate (cascaded from constitution submodule §11.4.87)

Verbatim user mandate (2026-05-26): *“continue in endless loop fully autonomously”* (and any semantically-equivalent phrasing).

When the operator instructs an AI agent to continue in an endless autonomous loop, the agent MUST treat it as a HARD-CONTRACT covenant: (A) continue working until docs/Issues.md Status-column has zero non-terminal entries AND docs/CONTINUATION.md §3 Active work is empty AND no background subagent is mid-execution AND no external dependency is in-flight; (B) dispatch background subagents for parallelisable work — main + every subagent operate concurrently, “waiting for results” is the ONLY acceptable idle reason; (C) every closure lands four-layer test coverage per §11.4.4(b) with captured-evidence (audio/video/network/UI/sysfs physical proofs); (D) the §11.4 anti-bluff covenant family (§11.4.1 / §11.4.2 / §11.4.6 / §11.4.7 / §11.4.27 / §11.4.50 / §11.4.52 / §11.4.68 / §11.4.69 / §11.4.83) is the operative truth-discipline — tests AND HelixQA Challenges bound equally; (E) the loop terminates ONLY on all-conditions-met, explicit operator STOP, host-session-safety demand, or scheduled wake on a known-future-actionable signal. No escape hatch — no --idle-OK, --skip-endless-loop, --bluff-permitted-for-this-task, --metadata-only-test-suffices, --no-physical-proof-required flag.

Cascade requirement: This anchor (verbatim or by §11.4.87 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-87-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.87 for the full mandate.

§11.4.88 — Background-Push Mandate: Commit-Lock Release Immediately After Commit, Push Runs Detached (cascaded from constitution submodule §11.4.88)

Forensic anchor (2026-05-26): a single `commit_all.sh` held its flock ~5 hours because `do_push` ran synchronously after the commit landed — every subsequent commit blocked on a slow mirror push irrelevant to the local commit's durability. Implementation seam for §11.4.87(B) zero-idle. The mandate: (A) `.git/.commit_all.lock` MUST be released IMMEDIATELY after `git commit` returns 0 — the commit is durable on local disk regardless of remote push outcome; (B) push runs detached via `nohup ./push_all.sh ... > <log> 2>&1 & + disown` — the orchestrator's exit code reports COMMIT success, NOT push success; (C) `push_all.sh` acquires per-remote flock `.git/.push.<remote>.lock` so concurrent invocations targeting the same remote serialize but different-remote invocations run in parallel; (D) backgrounded push failures land in `qa-results/push_failures/` `<ts>_<remote>.log` — the next autonomous-loop tick checks per §11.4.87(A) “no external dependency in-flight” gate; (E) synchronous-push escape: explicit `--sync-push` CLI flag preserves legacy behaviour for §11.4.41 force-push merge-first audit paths. Gates CM-COVENANT-114-88-PROPAGATION + CM-BACKGROUND-PUSH-WIRED + paired §1.1 mutations. Synchronous push (without `--sync-push`) = §11.4 PASS-bluff at the execution layer.

Cascade requirement: This anchor (verbatim or by §11.4.88 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-88-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — no escape hatch beyond `--sync-push` for force-push events. **Canonical authority:** constitution submodule Constitution.md §11.4.88 for the full mandate.

§11.4.89 — Background Test Execution Mandate (cascaded from constitution submodule §11.4.89)

Verbatim user mandate (2026-05-27): *“Any tests we are executing, especially long test cycles, MUST BE performed in background in parallel with main work stream! This MUST NOT block our capabilities to work on queued workable items. Main work stream can be blocked or sit iddle only if absolutely needed and if it depends hard on results of some background execution.”*

Symmetric anchor to §11.4.88 (background push) at the test-execution layer. Mandate: (A) long-running tests (>30 s expected: pre_build, meta_test, test_all_fixes, recent_work_validate, HelixQA banks, 4-phase cycles, full-suite retests, audio supervisors, dual-display recorders) MUST run via nohup ... > <log> 2>&1 & + disown with the log under a known dir (qa-results/<test_id>_<ts>.log); (B) the main stream proceeds to the §11.4.42 priority queue immediately; (C) hard-dependency gating — poll an exit-status file or pgrep -af <test> before steps that need the exit code, surfacing as §11.4.66 interactive options if the test is still running; (D) failures land in <log> files, the next loop tick checks; (E) foreground execution permitted ONLY for <30 s tests OR explicit operator authorisation; (F) per-script flock serialises same-script invocations, different-script invocations parallel. Gates CM-COVENANT-114-89-PROPAGATION + CM-BACKGROUND-TEST-EXECUTION-WIRED + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.89 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-89-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — no escape hatch beyond explicit per-invocation operator authorisation. **Canonical authority:** constitution submodule Constitution.md §11.4.89 for the full mandate.

§11.4.90 — Obsolete Status + Per-Item Obsolescence Audit (cascaded from constitution submodule §11.4.90)

Verbatim user mandate (2026-05-27): *“Bug No 6 ... seems obsolete after latest request for new behavior ... mark obsolete tickets with some light gray background ... text - the description to be strikethrough styled ... review all existing open or resolved workable items if they are obsolete - not valid any more ... There MUST NOT be any mistake! No bluff is allowed of any kind!”*

The §11.4.15 Status closed-set is extended with a terminal Obsolete (→ Fixed.md) value (orthogonal to Type per §11.4.16). Obsolescence reasons (closed vocabulary): superseded-by-design-change | superseded-by-later-mandate | feature-removed | duplicate-of | unsupported-topology. Every Obsolete heading MUST carry an ****Obsolete-Details:**** line (Since + Reason + Superseding-item + Triple-check evidence) within 8 non-blank lines. The §11.4.23 colorizer adds a cell-status-obsolete class — light-gray #E0E0E0 background + strikethrough description. Audit cadence: every release-gate sweep per §11.4.40 + §11.4.42; triple-check is non-negotiable per the operator mandate. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.21 / §11.4.23 / §11.4.33 /

§11.4.34 / §11.4.40 / §11.4.42 / §11.4.66 / §11.4.71. Gates CM-COVENANT-114-90-PROPAGATION + CM-ITEM-OBSOLETE-DETAILS + CM-OBSOLETE-COLORIZER-WIRED + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.90 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-90-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.90 for the full mandate.

§11.4.91 — Summary-Doc Clarity Mandate (cascaded from constitution submodule §11.4.91)

Verbatim user mandate (2026-05-27): “Summary docs - Issues_Summary some not clear one line descriptions - like ‘Composes with’ ... For each workable item we MUST HAVE clearly understandable meaning ... every team member can clearly understand what that particular workable item is exactly about! There cannot be misunderstanding or unclarity of any kind and no bluff allowed!”

Every summary entry (Issues_Summary, Fixed_Summary, README doc-link, Status_Summary pages 1+2, all one-liners) MUST contain a self-contained meaningful description ≥ 6 words OR ≥ 40 chars naming SUBJECT + PROBLEM/GOAL. Forbidden one-liner anti-patterns: section labels (Composes with, Closure criteria, Fix direction, etc.); bare metadata fragments (Critical, Bug, In progress, etc.); section-marker echoes; a §-letter alone. Generators (in HelixCode: the §11.4.93 workable-items export DB exporter — which renders BOTH tracker summaries from the SQLite single-source-of-truth docs/workable_items.db, and which SUPERSEDES the markdown-derived generate_issues_summary.sh / generate_fixed_summary.sh, both of which now REFUSE to run; plus update_readme_doc_links.sh / generate_status_summary.sh) MUST extract from the H1/H2 heading line per the §11.4.54 ATM-NNN convention, NEVER from arbitrary downstream text, and MUST refuse anti-pattern rows — emitting a (MISSING DESCRIPTION — fix source heading) placeholder with visual highlight. Gate CM-SUMMARY-CLARITY-DESCRIPTIONS scans every summary; an anti-pattern match = FAIL. Audit cadence: every §11.4.40 + §11.4.42 sweep.

Cascade requirement: This anchor (verbatim or by §11.4.91 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-91-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.91 for the full mandate.

§11.4.92 — Multi-Pass Change-Evaluation Discipline (cascaded from constitution submodule §11.4.92)

Verbatim user mandate (2026-05-27): *“Every change to the project or codebase we do MUST BE evaluated in several passes and in in-depth analysis for potential new issues or problems it can introduce! ... no bluff of any kind! After we do change or set of changes this mandatory steps MUST BE taken!”*

Every non-trivial change MUST pass a 5-pass evaluation BEFORE it is commit-ready: **(Pass 1)** main-task verification — change achieves the stated goal, captured-evidence per §11.4.5/§11.4.69; **(Pass 2)** regression-blast-radius analysis — enumerate every direct dependency, demonstrate no contract break; **(Pass 3)** cross-feature interaction analysis — audit parallel features sharing state/timing/hardware/shell environment; **(Pass 4)** deep-research validation per §11.4.8 — external precedent OR “NO external solution found — original work” + CodeGraph queries per §11.4.78/§11.4.79; **(Pass 5)** anti-bluff confirmation per §11.4 / §11.4.1 / §11.4.6 / §11.4.27 / §11.4.50 / §11.4.52 / §11.4.69 / §11.4.83 — no new bluff surface introduced. Each pass is documented (commit footers OR docs/entries OR qa- results/ evidence). Only after all 5 passes complete may commit/push/test/release proceed. Trivial exemption: typo / revision-bump / MD-export-regen IF zero source touched AND the commit message cites the exemption explicitly. Gates CM- COVENANT - 114 - 92 - PROPAGATION + CM- MULTI - PASS - EVALUATION - EVIDENCE + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.92 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM- COVENANT - 114 - 92 - PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.92 for the full mandate.

§11.4.93 — SQLite-Backed Single-Source-of-Truth for Workable Items (cascaded from constitution submodule §11.4.93)

Verbatim user mandate (2026-05-27): *“There MUST be single source of truth for all of our workable items - SQLite database ... proper scripts (we recommend Go programs) ... reduce a chance for sync to be broken ... generate always all docs from DB or to re-generate Db from all docs we have in opposite direction”*

The text-based Issues/Fixed/Summary/CONTINUATION constellation is converted to a SQLite-DB-backed single source of truth. Schema mandatory tables: items (atm_id PK + Type + Status incl. Obsolete + Severity + title + description ≥40 chars + created/modified + composes_with JSON + current_location); item_history (append-only audit per §11.4.34 By/Reason/Evidence); obsolete_details (§11.4.90); operator_block_details (§11.4.21); firebase_metadata (§11.4.47); meta (schema version + last sync + integrity hash). A Go binary at cmd/workable-items/ provides sync md-to-db / db-to-md / diff / validate / add / close; bidirectional regen is byte-identical round-trip (closed-set whitespace/section-order tolerance). commit_all.sh refuses on non-empty diff; sync_issues_docs.sh invokes the Go binary; pre-build runs workable-items validate. Anti-bluff: unit + integration + stress (1000-row insert + 10 concurrent writers) + chaos (mid-write SIGKILL + corrupt-DB recovery + disk-full) + paired §1.1 mutation + HelixQA Challenge CME-WORKABLE-ITEMS-001. The Go binary lives in the constitution submodule (constitution/scripts/workable-items/) per §11.4.74. Gates CM-COVENANT-114-93-PROPAGATION + CM-WORKABLE-ITEMS-DB-PRESENT + CM-WORKABLE-ITEMS-MD-DB-IN-SYNC + paired §1.1 mutations. (NOTE: the DB tracking rule is AMENDED by §11.4.95 — DB is TRACKED, not gitignored.)

Cascade requirement: This anchor (verbatim or by §11.4.93 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-93-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — text-based-only trackers are a §11.4 PASS-bluff at the data-architecture layer. **Canonical authority:** constitution submodule Constitution.md §11.4.93 for the full mandate.

§11.4.94 — Zero-Idle Priority-First Parallel-By-Default Operating Mode (cascaded from constitution submodule §11.4.94)

Verbatim user mandate (2026-05-27): *“We MUST NEVER sit iddle / wait or sleep if there is possibility for us to work on something ... Always check if there is a possibility to work on something while we are not working actively on something! Pick always by priority - most critical workable items and other tasks MUST BE done first! ... Stay still / iddle if nothing is left to be done at all or waiting for something that is blocking us / you!!!”*

§11.4.94 binds §11.4.20 + §11.4.42 + §11.4.58 + §11.4.70 + §11.4.72 + §11.4.82 + §11.4.87 + §11.4.88 + §11.4.89 into a single always-on enforcement: (A) idle ONLY when every queued item is genuinely blocked on an external dependency

(hardware / network upstream / build/test completion the conductor cannot accelerate) OR operator STOP OR §12 host-safety — “don’t see what to do” is NEVER valid; (B) before ANY wake/sleep the conductor MUST survey parallel-work feasibility per §11.4.42 + §11.4.72 + §11.4.87, identify non-contending items, and dispatch in parallel per §11.4.20/§11.4.70 (subagent) + §11.4.58 (PWU disjoint scope) + §11.4.89 (background long tests); (C) priority order MANDATORY — pick highest-severity + §11.4.72 audio-first the conductor can autonomously progress; (D) subagent-driven default for non-trivial; (E) background default for >30 s wall-clock work via nohup+disown; (F) stability-preserving (composes with §11.4.92 multi-pass + §11.4.84 quiescence + §12.6–§12.9 host safety); (G) progress updates surfaced at milestone boundaries. Gates CM-COVENANT-114-94-PROPAGATION + CM-PARALLEL-WORK-AUDIT + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.94 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-94-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.94 for the full mandate.

§11.4.95 — Workable-Items SQLite DB Is TRACKED in Git, NEVER Gitignored (cascaded from constitution submodule §11.4.95)

Verbatim user mandate (2026-05-27): *“We shall not Git ignore our workable items SQLite DB since it is our single source of truth ... workable items SQLite DB regularly committed and pushed to all upstreams!”*

§11.4.93’s earlier “gitignored per §11.4.30” clause is AMENDED — the DB at docs/workable_items.db is TRACKED in git, NEVER gitignored. It IS authoritative source data, NOT a build artefact. Every workable-items sync md-to-db that mutates state MUST stage + commit + push the DB alongside the MD regen per §11.4.19 atomic-move + §2.1 multi-upstream push. A WAL-checkpoint (PRAGMA wal_checkpoint(TRUNCATE)) is required before commit-stage so the transient .db-wal + .db-shm sidecars (gitignored per §11.4.30) are safely discardable. The §11.4.77 regeneration mechanism does NOT apply — the DB IS the source. Destructive DB ops require §9.2 hardlinked-backup + operator authorization; §11.4.41 force-push merge-first applies if DB history ever needs rewrite. Gates CM-COVENANT-114-95-PROPAGATION + CM-WORKABLE-ITEMS-DB-TRACKED + paired §1.1 mutation.

Cascade requirement: This anchor (verbatim or by §11.4.95 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-95-PROPAGATION; paired

mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.95 for the full mandate.

§11.4.96 — Safe-Parallel-Work-With-Long-Build Catalogue + Mandate (cascaded from constitution submodule §11.4.96)

Verbatim user mandate (2026-05-27): *“Are there except AOSP build process any other active jobs being done at the moment? Can we work on something in parallel while build is in progress so we slowly cleanup our slate? ... do as much as possible work in background in parallel with main work stream and oreferably using subagents-driven approach!”*

An operational catalogue for the canonical long-running workload (multi-hour containerised build per §12.9). **SAFE during build:** (A) MD/docs work; (B) generator/helper script work under scripts/; (C) pre-build + meta-test gate authoring + paired §1.1 mutations; (D) on-device test scripts; (E) constitution submodule edits + push; (F) any submodule commit + push per §11.4.88; (G) read-only live-ADB probes (dumpsys/getprop/cat /proc/.../screencap/logcat); (H) subagent dispatch per §11.4.20/§11.4.70 + §11.4.84 quiescence; (I) web research + external API queries with §11.4.10 credentials; (J) workable-items DB ops per §11.4.93+§11.4.95; (K) backgrounded pre-build + meta-test execution per §11.4.89. **UNSAFE during build:** (α) git checkout/reset --hard/clean -df on the source tree (use git worktree); (β) mass file deletes/renames under built source trees; (γ) submodule pointer updates affecting built artefacts; (δ) out/ mutations; (ε) make clean/m clobber/rm -rf out/; (ζ) container destruction; (η) disk-filling breaching §12.9 free-space minimum; (θ) §12 host-session-safety breaches. Conductor responsibility: before EVERY pause point during a long build, consult the catalogue, identify (A)-(K) queue items per §11.4.42+§11.4.72, and dispatch ≥1 per §11.4.20/§11.4.70 subagent default + §11.4.89 background. “Build running, nothing else to do” is NEVER true per §11.4.94+§11.4.96. Gates CM-COVENANT-114-96-PROPAGATION + CM-PARALLEL-WORK-DURING-BUILD-AUDIT + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.96 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-96-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.96 for the full mandate.

§11.4.97 — Maximum-Use-of-Idle-Time + Progress-Update Cadence (cascaded from constitution submodule §11.4.97)

Verbatim user mandate (2026-05-27): *“keep it working, we should do as much as possible, if not it all but as much as we can as long as there is iddle time! it MUST be used! ... keep us updated about all progress and all phisycal proofs and gathered data as you progress through all open workable items!”*

Operating-mode capstone strengthening §11.4.87 + §11.4.94 + §11.4.96: (A) every minute of conductor idle time during which work could autonomously progress AND is not genuinely blocked = a §11.4.97 violation; “as much as possible, if not it all but as much as we can” is operative — dispatch CONTINUOUSLY through the entire idle window, not just at scheduled wakes; (B) progress-update cadence — emit an operator-facing 1-line update at every commit landed / subagent return / constitutional anchor / captured evidence / milestone closure, no operator prompt required; (C) continuous physical-proof gathering per §11.4.5 + §11.4.6 + §11.4.69 — every autonomous closure cites captured-evidence (evidence path goes into the §11.4.93 `item_history.evidence_path` when the DB lands); (D) composes with §11.4.5/6/13/20/27/42/50/52/69/70/72/83/85/87/88/89/94/96; (E) the idle-only-when-blocked closed-set is unchanged from §11.4.94(A). Gates CM-COVENANT-114-97-PROPAGATION + CM-IDLE-TIME-AUDIT + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.97 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-97-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.97 for the full mandate.

§11.4.98 — Full-Automation Anti-Bluff Mandate (cascaded from constitution submodule §11.4.98)

Verbatim user mandate (2026-05-28): *“Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! Everything MUST BE fully automatic and autonomous! These tests MUST BE able to rerun endless times when needed! ... Make sure there is no false positives in testing! Every test and its results MUST obtain real proofs of everything working! No bluff is allowed!”*

Closes the manual-intervention gap (§11.4 / §11.4.2 / §11.4.5 / §11.4.50 / §11.4.85 / §11.4.87 / §11.4.89 / §11.4.94 did not explicitly forbid it). A live/integration/e2e/Challenge test that requires a human action during execution (typing a message, clicking UI, hand-triggering a webhook, attaching a file — anything beyond startup) is by definition a §11.4 PASS-bluff at the automation layer. (A) Every governed test — unit/integration/e2e/Challenge/stress/chaos/live — MUST be fully self-driving end-to-end, reporting PASS/FAIL/SKIP-with-reason without any further human action after startup. (B) Single permissible exception: one-time credential bootstrap performed OUTSIDE test execution (.env from vault, shell exports, OAuth at first install, MTProto session activation) — configuration, not test driving. (C) Live messenger/channel/agent tests: no “operator must type” prompts (drive programmatically via second account / webhook fixture / loopback); no hard-coded session UUIDs that collide with the active dev session (Herald 2026-05-28 claude - - resume silent exit -1 lesson); no 60 s human-response windows (§11.4.50 determinism violation); re-runnability proof — PASS at -count=3 consecutive automated invocations with self-cleaning state; §11.4.98 obsolescence audit classifies every existing test COMPLIANT vs NON-COMPLIANT; no silent-skip-reported-as-PASS or stale-evidence-as-fresh. (D) With §11.4.85 + §11.4.89 + §11.4.87 + §11.4.94 forms a continuously-validated, non-flake, anti-bluff regime. (F) Manual-dependency tests not rewritten within 30 days graduate to §11.4.90 Obsolete citing §11.4.98.

Cascade requirement: This anchor (verbatim or by §11.4.98 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-98-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.98 for the full mandate.

§11.4.99 — Latest-Source Documentation Cross-Reference Mandate (cascaded from constitution submodule §11.4.99)

Verbatim user mandate (2026-05-28): “Make sure we ALWAYS check against latest versions of services we use web / online docs before creating instructions! This situation is illustration of how we can misguide ourselves or get banned! ... These are mandatory rules / constraints and the result is consistency and safety of created instructions, guides and manuals!”

Misguidance-by-stale-docs is the same severity class as a §11.4 PASS-bluff at the documentation layer (Herald 2026-05-28 case: a first-draft MTProto guide recommended VoIP fallback numbers and omitted the recover@telegram.org pre-login email — both contradicted Telegram’s official docs + the gotd/td

maintainer guide and could have caused a permanent account ban). Closes the gap §11.4.92 Pass 4 alludes to but does not mandate. (A) Before committing any operator-facing instruction/guide/manual/troubleshooting/setup doc, the author MUST: (1) fetch the LATEST official online documentation of the documented service/library via WebFetch / MCP / direct browsing — NEVER training data, memory, or prior committed docs; (2) cross-reference every instruction step against that source; (3) seek secondary authoritative sources (maintainer SUPPORT.md, official changelogs, vetted community FAQs) when the official source is sparse/silent; (4) cite source URLs + date in a `## Sources verified` footer in the doc; (5) cite a `Sources verified <date>: <urls>` footer in the commit message. (B) Negative findings (gaps/silences/contradictions) MUST be documented explicitly. (C) Docs older than 6 months are STALE — re-verify before citing as operator authority, at every vN.0.0 release boundary, on service breaking-change announcements, or on operator error reports. (D) Risk-classified services (messengers, cloud APIs, payment systems, AI/LLM providers, code-hosting, package managers) carry a 90-day max staleness + explicit safety warnings. (E) Composes with but is INDEPENDENT of §11.4.92 Pass 4. (G) Commit missing either footer is BLOCKED at release-gate; stale-beyond-grace docs graduate to §11.4.90 Obsolete (Reason=stale-documentation).

Cascade requirement: This anchor (verbatim or by §11.4.99 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-99-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.99 for the full mandate.

§11.4.101 — Autonomous-Decision-Over-Blocking Mandate (cascaded from constitution submodule §11.4.101)

Verbatim user mandate (2026-05-28): “when working in endless working loop fully autonomously try to decide most properly about points which would block execution and wait for us. If we haven’t answered now work would be blocked whole night! If possible and if that will not cause any issues make proper and most reliable and safe decision so we achieve maximal efficiency and work gets fully done!”

In autonomous / endless-loop mode (per §11.4.87), the agent MUST minimize operator-blocking and make the safe, reliable, reversible decision itself so work is not stalled (e.g. overnight) waiting for input — §11.4.87 says keep working, §11.4.101 says HOW to clear the decision points. **Proceed-autonomously (closed-set, ALL must hold):** (a) the action is reversible OR has a captured pre-op backup per §9.2; (b) the safe choice is determinable from captured evidence per §11.4.6

(no guessing — LIKELY/probably/seems is NOT a determination); (c) a wrong choice's blast radius is bounded AND recoverable; (d) it composes with anti-bluff §11.4, host-safety §12, data-safety §9. **Block-only-when (BLOCK via the §11.4.66 interactive mechanism ONLY when ALL hold):** the action is irreversible AND high-blast-radius AND the safe choice cannot be determined from evidence — e.g. external-account state the agent cannot inspect, hardware it cannot access, destructive ops without backup, force-push (also §9.2 + §11.4.41), spending money or sending data to third parties. Operator-blocked per §11.4.21 is reached only after this rule fires AND the self-resolution-exhaustion audit completes. An unavoidable block parks one work unit — it does NOT pause the loop; the agent keeps progressing every non-blocked item in parallel per §11.4.87 + §11.4.94 (posing the question then going idle is a §11.4.94 + §11.4.97 violation).
Classification: universal (§11.4.17).

Cascade requirement: This anchor (verbatim or by §11.4.101 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-101-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.101 for the full mandate.

§11.4.102 — Mandatory systematic-debugging activation + always-loaded skill-discovery + plugin-dependency availability (cascaded from constitution submodule §11.4.102)

Verbatim user mandate (2026-05-29): **“Make sure that we ALWAYS trigger / start the”/superpowers:systematic-debugging” skills when any issues happen! If this is possible to activate and use in this situations out of the box when we spot problems / issues / bugs / misalignments / inconsistencies we MUST activate the skill(s) and make strongest efforts in full in depth analysis / debugging and determine root causes of all problem or obtain relevant data and information we need! ... we MUST make sure that “/using-superpowers” skill is ALWAYS loaded, applied and used! All dependencies (plugins) that Claude Code or other market places are offering MUST BE installed if these are not already available for loading and use!”*

Three cooperating invariants — the difference between guess-and-retry and investigate-to-root-cause-first. **(A) Mandatory systematic-debugging activation.** On ANY spotted issue / bug / test failure / gate failure / regression / misalignment / inconsistency / unexpected behaviour, the agent MUST activate `superpowers:systematic-debugging` (or the platform-equivalent structured-

debugging discipline) **BEFORE proposing, writing, or applying any fix** — the **Iron Law: NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST**. Full four-phase arc: root-cause → pattern → hypothesis → implementation. Guess-and-retry, symptom-patching, and re-running a failed test hoping it passes (“probably transient / flaky”) **WITHOUT** a completed investigation are §11.4.102 violations; calling a failure transient/flaky/intermittent/probably-timing without captured forensic evidence is simultaneously a §11.4.6 and §11.4.7 violation. **(B) Mandatory always-loaded using-superpowers.** superpowers:using-superpowers (or platform-equivalent skill-discovery discipline) **MUST** be loaded and applied at session start and consulted before any task; if ANY skill could apply — even at 1% relevance — it **MUST** be invoked rather than improvised from memory. **(C) Mandatory plugin / dependency availability.** Every skill plugin / marketplace package / capability dependency the project relies on **MUST** be installed + loadable **BEFORE** the dependent work proceeds; a missing plugin that blocks a mandated skill is a release-blocker until installed + confirmed loadable (install exit 0 ≠ skill loadable — confirm by observing the skill in the live capability list). Composes with §11.4.4 / §11.4.6 / §11.4.7 / §11.4.8 / §11.4.43 / §11.4.70 / §11.4.82(A) / §11.4.92. Classification: universal (§11.4.17). No escape hatch — no --skip-systematic-debugging, --guess-and-retry-OK, --symptom-patch-permitted, --skip-skill-discovery, --plugin-optional, --missing-plugin-is-warning flag.

Cascade requirement: This anchor (verbatim or by §11.4.102 reference) **MUST** appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-102-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.102 for the full mandate.

§11.4.103 — Continuous parallel-stream working routine (cascaded from constitution submodule §11.4.103)

Cascade reference — see constitution submodule Constitution.md §11.4.103 for the full mandate. The main work stream **MUST** always stay FREE. ALL commit AND push operations run detached. At least three parallel background subagent-driven streams **MUST** run alongside the main stream whenever three or more non-contending actionable items exist; the moment any one stream finishes a new stream **MUST** immediately start. Most-critical and most-visible first; audio always top per §11.4.72. Propagation gate CM-COVENANT-114-103-PROPAGATION (literal 11.4.103). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.103.

§11.4.104 — Participant identity, attribution and notification-tagging (cascaded from constitution submodule §11.4.104)

Cascade reference — see constitution submodule `Constitution.md` §11.4.104 for the full mandate. Every messenger MUST relate messages to a Participant (logical Subscriber/User); the same logical person may have a different username per messenger. Workable items MUST carry `created_by` and `assigned_to` (canonical handles). Notifications MUST @-tag the right participant — never the operator and never the system agent. Propagation gate CM-COVENANT-114-104-PROPAGATION (literal 11.4.104). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.104.

§11.4.105 — Natural-language intent recognition and clarification (cascaded from constitution submodule §11.4.105)

Cascade reference — see constitution submodule `Constitution.md` §11.4.105 for the full mandate. Users MUST NOT need to know command syntax. The System determines intent via three-tier resolution: recognise existing commands, infer exact intent, or reply-tag-and-ask with a precise clarifying question. Never guess, never drop a message silently. Propagation gate CM-COVENANT-114-105-PROPAGATION (literal 11.4.105). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.105.

§11.4.106 — Docs Chain — mechanical documentation/DB sync engine (cascaded from constitution submodule §11.4.106)

Cascade reference — see constitution submodule `Constitution.md` §11.4.106 for the full mandate. Docs Chain (`vasic-digital/docs_chain`) is the canonical mechanical enforcer of documentation-sync mandates. Consumers MUST use the engine instead of ad-hoc sync scripts, register chains via per-context YAML, and never accept a faked transform. Propagation gate CM-COVENANT-114-106-PROPAGATION (literal 11.4.106). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.106.

§11.4.107 — Anti-bluff AV/test-validation techniques mandate (cascaded from constitution submodule §11.4.107)

Cascade reference — see constitution submodule `Constitution.md` §11.4.107 for the full mandate. A single captured frame is NOT proof of playback. Every test asserting audio/video output MUST prove liveness via freeze-detection oracle AND an independent frame-advance counter from platform telemetry. No-stale-from-previous cross-check, measured FPS, no-flash-on-wrong-output, drive through realistic UI path, metamorphic relations, full-reference quality metrics, mutation-test every analyzer, per-channel audio RMS/loudness, calibrated thresholds. Propagation gate CM-COVENANT-114-107-PROPAGATION (literal 11.4.107). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.107.

§11.4.108 — Four-layer fix-verification + runtime-signature-as-definition-of-done mandate (cascaded from constitution submodule §11.4.108)

Cascade reference — see constitution submodule `Constitution.md` §11.4.108 for the full mandate. A fix crosses four distinct layers: SOURCE, ARTIFACT, RUNTIME-ON-CLEAN-TARGET, USER-VISIBLE. Green at layer 1 says nothing about layers 2–4. Every fix declares one machine-checkable runtime signature verified on a clean deployment. Gates span all four layers. Deployment MUST yield a clean state. Three or more “fixed-but-not-working” discoveries signal an architectural verification flaw — stop patching symptoms, fix the verification pipeline. Propagation gate CM-COVENANT-114-108-PROPAGATION (literal 11.4.108). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.108.

§11.4.109 — Mandatory Anti-Forgetting Enforcement: PreToolUse Guard Hook + Subagent Constitutional Preamble + Orchestrator Pre-Action Checklist (cascaded from constitution submodule §11.4.109)

Cascade reference — see constitution submodule `Constitution.md` §11.4.109 for the full mandate. Every consumer MUST wire `constitution/scripts/hooks/guard-forbidden-commands.sh` as a `PreToolUse` hook, maintain `docs/AGENT_GUARDRAILS.md` with the subagent constitutional preamble and orchestrator pre-action checklist, and provide a hermetic hook test suite. The hook is inherited by reference, never copied. Propagation gate `CM-COVENANT-114-109-PROPAGATION` (literal `11.4.109`). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.109.

§11.4.110 — Pre-build build-readiness verdict + change-impact clash detection mandate (cascaded from constitution submodule §11.4.110)

Cascade reference — see constitution submodule `Constitution.md` §11.4.110 for the full mandate. A single deterministic `READY-FOR-BUILD` verdict gates every rebuild. A diff-driven change-impact and clash detector cross-checks every newly-introduced second-artifact dependency. Coverage-completeness is a gate. Two-speed honesty separates `grep-speed` always-on gates from heavy `REQUIRES_BUILD` gates. Every gate is anti-bluff by paired §1.1 mutation. Propagation gate `CM-COVENANT-114-110-PROPAGATION` (literal `11.4.110`). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.110.

§11.4.111 — Resolve-by-stable-name-not-by- enumeration-index mandate (cascaded from constitution submodule §11.4.111)

Cascade reference — see constitution submodule `Constitution.md` §11.4.111 for the full mandate. Any binding to a hardware device / resource handle /

enumerated entity MUST resolve by a stable identifier (name / UUID / serial / label) and MUST NOT bind by enumeration index / ordinal / slot unless the platform documents that ordinal as deterministically pinned and the pin is captured + asserted. Propagation gate CM-COVENANT-114-111-PROPAGATION (literal 11.4.111). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.111.

§11.4.112 — Structural-impossibility won't-fix classification mandate (cascaded from constitution submodule §11.4.112)

Cascade reference — see constitution submodule Constitution.md §11.4.112 for the full mandate. When deep research proves a goal is structurally impossible on the target platform (forbidden by platform design / hardware-protocol constraint / documented kernel-or-API limitation), the goal MUST be classified Won't-fix, documented with impossibility evidence, and NOT re-attempted without new evidence the platform constraint changed. Propagation gate CM-COVENANT-114-112-PROPAGATION (literal 11.4.112). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.112.

§11.4.113 — Absolute no-force-push + merge-onto-latest-main mandate (cascaded from constitution submodule §11.4.113)

Cascade reference — see constitution submodule Constitution.md §11.4.113 for the full mandate. Force-push is STRICTLY FORBIDDEN with NO exception. Every push MUST follow the 6-step integration procedure: fetch all remotes, set base to latest main/master, carefully merge all changes on top, resolve every conflict, commit, push to ALL upstreams as a fast-forward. The force-push escape hatch from §11.4.41/§11.4.71 is REMOVED. Propagation gate CM-COVENANT-114-113-PROPAGATION (literal 11.4.113). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.113.

§11.4.114 — Last-known-good-tag regression isolation mandate (cascaded from constitution submodule §11.4.114)

Cascade reference — see constitution submodule `Constitution.md` §11.4.114 for the full mandate. When a previously-working feature is observed broken, the FIRST diagnostic action MUST be to identify the last release tag at which it was known-good and diff/bisect against it before any open-ended root-cause hunt or speculative fix. Default to surgical forward-fix over wholesale revert unless the operator prefers otherwise. Propagation gate CM-COVENANT-114-114-PROPAGATION (literal 11.4.114). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.114.

§11.4.115 — RED-baseline-on-the-broken-artifact + polarity-switch mandate (cascaded from constitution submodule §11.4.115)

Cascade reference — see constitution submodule `Constitution.md` §11.4.115 for the full mandate. Every RED test MUST be authored to reproduce the defect on the CURRENT pre-fix artifact. The same test source MUST carry a polarity switch (RED_MODE, default 1 = reproduce-and-assert-defect-present; 0 = standing GREEN regression-guard). RED-on-broken-artifact then GREEN-on-fixed-artifact on a clean target MUST both be captured. Propagation gate CM-COVENANT-114-115-PROPAGATION (literal 11.4.115). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.115.

§11.4.116 — Real-time conductor↔autonomous-test-framework sync channel mandate (cascaded from constitution submodule §11.4.116)

Cascade reference — see constitution submodule `Constitution.md` §11.4.116 for the full mandate. Any autonomous long-running test/QA/validation framework MUST expose a structured append-only event stream and an atomically-rewritten status snapshot. Every verdict event MUST carry the evidence path that backs it. A PASS event with no evidence path is a channel-layer PASS-bluff. Propagation gate CM-COVENANT-114-116-PROPAGATION (literal 11.4.116). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.116.

§11.4.117 — Computer-vision / OCR pixel-oracle fallback for non-introspectable UIs mandate (cascaded from constitution submodule §11.4.117)

Cascade reference — see constitution submodule Constitution.md §11.4.117 for the full mandate. Any test that needs to drive a UI control OR assert on-screen content MUST NOT assume the accessibility/semantic/DOM hierarchy is the source of truth. When the hierarchy is blank/partial/known-unreliable, the test MUST fall back to a pixel oracle: drive input by computer-vision template-match; assert content by ROI OCR with a per-word confidence floor. The CV/OCR analyzer MUST be self-validated with golden-good/golden-bad fixtures. Propagation gate CM-COVENANT-114-117-PROPAGATION (literal 11.4.117). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.117.

§11.4.118 — Discovery-pressure to confirm known-issue-set completeness mandate (cascaded from constitution submodule §11.4.118)

Cascade reference — see constitution submodule Constitution.md §11.4.118 for the full mandate. A remediation/release cycle MUST NOT treat “every reported defect is fixed” as “the build is good.” After/alongside fixing the reported set, the cycle MUST run a discovery + stress pass across ALL target devices/environments, producing an enumerated list of subsystems/user-journeys/stress scenarios actually exercised with outcomes. “We found no other issues” without a coverage enumeration is a §11.4 bluff. Propagation gate CM-COVENANT-114-118-PROPAGATION (literal 11.4.118). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.118.

§11.4.119 — Single-resource-owner partitioning for parallel hardware testing mandate (cascaded from constitution submodule §11.4.119)

Cascade reference — see constitution submodule `Constitution.md` §11.4.119 for the full mandate. When multiple parallel streams exercise shared hardware or any exclusive-access resource, exactly ONE stream MUST own each such resource at a time. Every other concurrent stream targeting the same resource MUST be READ-ONLY. Parallelism is partitioned by resource; distinct devices/sinks/handles run fully concurrent. Ownership MUST be enforced by an advisory lock/token. Propagation gate CM-COVENANT-114-119-PROPAGATION (literal 11.4.119). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.119.

§11.4.120 — Fix-breaks-its-own-gate reconciliation mandate (cascaded from constitution submodule §11.4.120)

Cascade reference — see constitution submodule `Constitution.md` §11.4.120 for the full mandate. When a correct fix causes a pre-existing gate/test to FAIL because it asserted old behaviour, the gate MUST be reconciled (rewritten to assert the new mechanism + paired §1.1 mutation updated) — never fake-passed, weakened to a tautology, or deleted; and the correct fix MUST NOT be reverted to satisfy the stale gate. Propagation gate CM-COVENANT-114-120-PROPAGATION (literal 11.4.120). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.120.

§11.4.121 — No-commit-while-build-writes-tracked-artifacts mandate (cascaded from constitution submodule §11.4.121)

Cascade reference — see constitution submodule `Constitution.md` §11.4.121 for the full mandate. A commit MUST NOT run while a build/packaging/generation step is actively writing artifacts into tracked (version-controlled) directories. The commit MUST be deferred until the writing step completes so the tree is quiescent and committed artifacts are the fresh, whole outputs. Propagation gate CM-

COVENANT - 114 - 121 - PROPAGATION (literal 11 . 4 . 121). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.121.

§11.4.122 — No-silent-removal-of-existing-components-without-operator-confirmation mandate (cascaded from constitution submodule §11.4.122)

Cascade reference — see constitution submodule Constitution.md §11.4.122 for the full mandate. No application, system component, service, package, feature, driver, module, library, or prebuilt asset — any already-existing end-user capability — may be removed WITHOUT FIRST interactively asking the operator and receiving an EXPLICIT keep-or-remove decision. A silent removal is a release blocker regardless of rationale. Propagation gate CM-COVENANT - 114 - 122 - PROPAGATION (literal 11 . 4 . 122). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.122.

§11.4.123 — Rock-solid-proof-or-deep-research mandate (cascaded from constitution submodule §11.4.123)

Cascade reference — see constitution submodule Constitution.md §11.4.123 for the full mandate. Every reported issue, fix, and claimed completion MUST be fully validated with rock-solid captured proof before closure. When the validation method is unclear, deep web research MUST be performed to discover or build an evidence-producing method. Declaring something untestable without exhausting the research path is a §11.4.123 violation. Propagation gate CM-COVENANT - 114 - 123 - PROPAGATION (literal 11 . 4 . 123). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.123.

§11.4.124 — Dead/unwired-code investigate-before-remove mandate (cascaded from constitution submodule §11.4.124)

Cascade reference — see constitution submodule Constitution.md §11.4.124 for the full mandate. Before removing any seemingly-dead (zero-importer / unwired) codebase, investigate via git history where/how it was originally used and how it became dead. Removal is permitted ONLY with captured proof it is genuinely no longer needed — in its own separate commit. If no such proof exists, investigate where/how to wire it in properly and add missing tests. Propagation gate CM-

COVENANT - 114 - 124 - PROPAGATION (literal 11 . 4 . 124). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.124.

§11.4.125 — Code-review-agent gate before pre-build + main build (cascaded from constitution submodule §11.4.125)

Cascade reference — see constitution submodule Constitution.md §11.4.125 for the full mandate. After all fixes/changes/implementations in a batch are done, and BEFORE running pre-build tests and the main build, the agent MUST dispatch dedicated code-review agent(s) performing a multi-layer review. Any finding MUST be fixed, polished, improved, and covered with additional tests before the build proceeds. The review iterates until no blocking findings remain.

Propagation gate CM-COVENANT - 114 - 125 - PROPAGATION (literal 11 . 4 . 125). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.125.

§11.4.126 — Default autonomous-loop working mode from first prompt (cascaded from constitution submodule §11.4.126)

Cascade reference — see constitution submodule Constitution.md §11.4.126 for the full mandate. The endless fully-autonomous loop is the DEFAULT working mode, engaged automatically the moment the operator sends the FIRST request/prompt of a session. The loop continues until a new fully-validated-and-verified version tag is created and published across all owned submodules and the main repo, OR the main-stream goal is fully completed with the queue empty. Stops only on explicit operator STOP, empty queue, or §12 host-safety. Mimicking this behavior or producing false results is absolutely forbidden. Propagation gate CM-COVENANT - 114 - 126 - PROPAGATION (literal 11 . 4 . 126). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.126.

§11.4.127 — Session-handoff resumption-prompt mandate (cascaded from constitution submodule §11.4.127)

Cascade reference — see constitution submodule Constitution.md §11.4.127 for the full mandate. When a fresh session is needed, the agent MUST ALWAYS prepare and proactively provide a ready-to-paste resumption prompt valid for that exact moment and project phase — self-contained enough that pasting it into

a fresh session resumes work with zero loss. Both SHORT and FULL variants. A missing, stale, or generic prompt is a §11.4.127 violation. Propagation gate CM-COVENANT-114-127-PROPAGATION (literal 11.4.127). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.127.

§11.4.128 — Always-on device-recording mandate (cascaded from constitution submodule §11.4.128)

Cascade reference — see constitution submodule Constitution.md §11.4.128 for the full mandate. For every test/debug device and every device under known manual testing, across every reachable transport, the project MUST ALWAYS live-record all analysable data. Extra-careful, side-effect-free, non-invasive read-only probes only. Background + parallel + subagent-driven. Raw data is git-ignored and code-intelligence-excluded; only curated evidence is committed at release prep. Propagation gate CM-COVENANT-114-128-PROPAGATION (literal 11.4.128). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.128.

§11.4.129 — Huge-blocker release protocol (cascaded from constitution submodule §11.4.129)

Cascade reference — see constitution submodule Constitution.md §11.4.129 for the full mandate. On discovery of a huge blocker during release validation: STOP all testing; fix ALL discovered issues; process all recorded data; land rock-solid fixes; author NEW validation+verification tests of ALL supported test types; rebuild (full) + reflash to a CLEAN target; RESTART the full validation+verification from the last release tag to now on all devices in parallel, recorded, with real physical captured proofs. Propagation gate CM-COVENANT-114-129-PROPAGATION (literal 11.4.129). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.129.

§11.4.130 — Post-remediation validate-the-fix-FIRST-after-redeploy (cascaded from constitution submodule §11.4.130)

Cascade reference — see constitution submodule Constitution.md §11.4.130 for the full mandate. When a blocker is fixed and a new artifact produced + target reflashed, FIRST re-test the specific last-failing features and validate the just-incorporated fixes with real captured evidence — BEFORE broader/full-suite

validation. Only after the targeted fix is CONFIRMED working proceed to the full retest. Propagation gate CM-COVENANT - 114 - 130 - PROPAGATION (literal 11.4.130). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.130.

§11.4.131 — Standing session-resumption file mandate (cascaded from constitution submodule §11.4.131)

Cascade reference — see constitution submodule Constitution.md §11.4.131 for the full mandate. Every project MUST maintain a SINGLE canonical, always-current session-resumption file at a fixed project-declared standard path. Creating a new session requires ONLY pointing the new agent at this one file. The file MUST be rewritten whenever a fresh session is needed or live state materially changes. A stale resumption file is a §11.4.131 violation. Propagation gate CM-COVENANT - 114 - 131 - PROPAGATION (literal 11.4.131). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.131.

§11.4.132 — Risk-ordered validation priority mandate (cascaded from constitution submodule §11.4.132)

Cascade reference — see constitution submodule Constitution.md §11.4.132 for the full mandate. Tests/validations/verifications MUST run in RISK-DESCENDING order: most-recently-worked first, then historically most-problematic, then highest crash/break/regress likelihood, then most-reopened. The highest-risk set MUST pass with real captured evidence before the remainder of the suite runs. Running in arbitrary order is a §11.4.132 violation. Propagation gate CM-COVENANT - 114 - 132 - PROPAGATION (literal 11.4.132). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.132.

§11.4.133 — Target-System + hardware safety mandate (cascaded from constitution submodule §11.4.133)

Cascade reference — see constitution submodule Constitution.md §11.4.133 for the full mandate. Every change to the target system MUST ALWAYS be safe for both the target System itself (MUST NOT brick, boot-loop, corrupt data, or render the device unrecoverable) AND the hardware it runs on (MUST NOT exceed safe electrical/thermal/voltage/clock limits). Reversible-first; no unverified hardware-

control writes; thermal/perf changes respect cooling design; flashing uses the sanctioned tool + integrity-verified image. Propagation gate CM-COVENANT - 114 - 133 - PROPAGATION (literal 11 . 4 . 133). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.133.

§11.4.134 — Code-review iterate-until-GO + rock-solid-evidence mandate (cascaded from constitution submodule §11.4.134)

Cascade reference — see constitution submodule Constitution.md §11.4.134 for the full mandate. When a code review returns ANY finding, and the author fixes/improves the batch, the code review MUST BE RE-RUN and MUST KEEP being re-run after each remediation round, until it returns a clean GO with ZERO new issues AND ZERO warnings of any kind. Every round's verdict AND every fix's validation MUST carry rock-solid physical captured evidence. A reported GO unbacked by captured physical evidence is a §11.4 PASS-bluff. Propagation gate CM-COVENANT - 114 - 134 - PROPAGATION (literal 11 . 4 . 134). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.134.

§11.4.135 — Standing regression-guard suite + every-fixed-defect-gets-a-permanent-regression-test (cascaded from constitution submodule §11.4.135).

Every project MUST maintain a standing regression-guard suite that runs on every build+deploy and blocks the release tag on any failure. Every closed defect MUST, in the SAME commit as its fix, register a permanent §11.4.115 RED-on-broken-artifact regression test into the suite. A closure without a registered guard is a §11.4.123 violation. Propagation gate CM-COVENANT - 114 - 135 - PROPAGATION (literal 11 . 4 . 135). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.135.

§11.4.136 — Real-content end-to-end playback-test mandate (cascaded from constitution submodule §11.4.136). Any test asserting media playback works MUST drive REAL content through the user's path and assert it genuinely plays via the §11.4.107 liveness battery PLUS a decoder-health census. Metadata-only/launch-only/registration-only/single-frame/config-only PASS is forbidden. Propagation gate CM-COVENANT - 114 - 136 - PROPAGATION (literal 11 . 4 . 136). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.136.

§11.4.137 — Subtitle/caption content-correctness oracle + secure-display-proxy-honesty mandate (cascaded from constitution submodule §11.4.137). A subtitle-correctness test MUST classify the cue's content class — a present cue is

NOT a correct cue. CHROME (FAIL) if a known control/menu label, time/numeric chrome, not prose, outside the lower safe-title band, or static across the window. DIALOGUE (PASS) only when prose + not-denied + not-chrome + position-ok + cadence ≥ 2 or fuzzy-matches the source-extracted cue. The oracle MUST be self-validated with golden-good/golden-bad fixtures. Propagation gate CM-COVENANT-114-137-PROPAGATION (literal 11.4.137). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.137.

§11.4.138 — Operator-escape => mandatory bluff-audit + permanent guard (cascaded from constitution submodule §11.4.138). When the operator finds a defect that the GREEN test suite passed, this MUST trigger before the fix is closed: a §11.4.102 systematic-debugging pass; a bluff-audit identifying the exact assertion that should have caught it but didn't; a permanent §11.4.135 regression guard registered in the SAME commit as the fix; the bluff-audit committed under docs/research/<scope>/<defect>_bluff_audit/. Propagation gate CM-COVENANT-114-138-PROPAGATION (literal 11.4.138). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.138.

§11.4.139 — Fresh-process clean-artifact runtime-signature mandate (cascaded from constitution submodule §11.4.139). Before any post-deploy validation, the harness MUST assert running-artifact == built-artifact: the deploy yielded a clean target OR a pre-validation check proves no stale overlay shadows the deployed code. A stale shadow makes the proxy report on code that was never deployed — any PASS is a §11.4 PASS-bluff. Propagation gate CM-COVENANT-114-139-PROPAGATION (literal 11.4.139). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.139.

§11.4.140 & §11.4.141 — action-prefix system + token-efficiency (cascaded from constitution submodule §11.4.140/§11.4.141)

§11.4.140 — Universal action-prefix system (ACTION_NAME ::): When a user prompt's FIRST non-blank line starts with a recognised action prefix, look it up in the action registry (constitution/actions/registry.yaml), REPLACE the prefix with that action's expansion text and apply its rules. Four equivalent forms: ACTION_NAME :: <rest>, PREFIX::ACTION_NAME :: <rest>, /ACTION_NAME <rest>, /PREFIX::ACTION_NAME <rest>. An unknown token that matches the grammar shape but is NOT registered is NEVER silently expanded — ask which registered action was meant or treat as a literal prompt. The registered action BACKGROUND expands to executing the prompt in background in parallel with all

main work streams using subagents-driven development with rock-solid physical evidence. Propagation gate CM-COVENANT-114-140-PROPAGATION (literal 11.4.140). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.140.

§11.4.141 — Token-efficiency mandate (cascaded from constitution submodule §11.4.141). Every project worked on by AI coding agents MUST cut token spend toward 30–40% of current (a 60–70% reduction) WITHOUT degrading quality/performance/safety or breaking any existing mechanism, via: prompt-cache the static governance prefix; subagent model-tiering + output-to-file; thin always-loaded INDEX + on-demand detail; CodeGraph/retrieval-first; output-token reduction; tool-call batching + no re-reads; compaction/context-editing for long sessions. A mandatory measured proof harness verifies the reduction from the authoritative usage object. Propagation gate CM-COVENANT-114-141-PROPAGATION (literal 11.4.141). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.141.

§11.4.142 — Universal code-review mandate (cascaded from constitution submodule §11.4.142)

§11.4.142 — EVERY change made to ANY governed repository — without exception (source, fixes, tests, gates, docs, build scripts, governance files, conductor edits, sub-agent output, single-line edits) — MUST pass through an INDEPENDENT code-review step BEFORE it is accepted, committed, or built. The ABSOLUTE form of §11.4.125: no “just a doc edit” / “just a one-liner” / “self-review suffices” / “trivial change” carve-out. The reviewer MUST be structurally separated from the author (dedicated code-review agent §11.4.70/§11.4.20 or distinct human), the review is itself anti-bluff (§11.4/§11.4.1, no rubber-stamp), and iterates to a clean GO per §11.4.134. Propagation gate CM-COVENANT-114-142-PROPAGATION (literal 11.4.142) + gate CM-EVERY-CHANGE-REVIEWED. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.142.

§11.4.143 — Real-user-journey mandate for video-streaming-app tests (cascaded from constitution submodule §11.4.143)

§11.4.143 — Any full-automation test asserting a video player / streaming app plays content MUST drive the REAL end-user journey through the app’s OWN UI (launch → browse catalog → choose a SPECIFIC title → press real Play/Resume → confirm THAT content plays with correct subtitles on the intended target). A sample/demo/loop-clip, deep-link/am start/intent shortcut, or any path bypassing

browse-select-play is a §11.4 PASS-bluff (validates routing, leaves “the show I picked actually plays” unproven). Chosen-content confirmed via §11.4.107 liveness on the §11.4.136 real-content path + §11.4.137 subtitle oracle; non-introspectable UIs use the §11.4.117 CV/OCR pixel oracle; genuinely infeasible ⇒ honest §11.4.52/§11.4.3 SKIP, never a faked PASS. Propagation gate CM-COVENANT - 114 - 143 - PROPAGATION (literal 11 . 4 . 143) + gate CM-VIDEO-REAL - JOURNEY - TEST. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.143.

§11.4.144 — Tracked/recorded-device availability-following mandate (cascaded from constitution submodule §11.4.144)

§11.4.144 — Every device the project tracks/records (any reachable transport — USB/wireless-ADB/SSH/serial/network) **MUST** be availability-FOLLOWED: connection state continuously monitored, any drop handled, never silently abandoned nor presented as continuous recording. On a drop the system **MUST** automatically (a) DETECT + log an honest offline event into the corpus (a silent gap presented as continuous capture is a fabricated-continuity §11.4 bluff), (b) WAIT using the project’s already-defined reconnection timings (never invented, §11.4.6), (c) RE-ATTACH + log resume on return, (d) ESCALATE to the sanctioned authorization-gated recovery path (§11.4.21/§11.4.101 — no autonomous destructive recovery) on permanent absence. The agent-side analogue is §11.4.147. Propagation gate CM-COVENANT - 114 - 144 - PROPAGATION (literal 11 . 4 . 144) + gate CM-DEVICE-AVAILABILITY-FOLLOWED. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.144.

§11.4.145 — Independent multi-angle impact-research per change (cascaded from constitution submodule §11.4.145)

§11.4.145 — For EVERY fix/change/new feature, INDEPENDENT impact-research agents (subagent-driven §11.4.70/§11.4.20, adversarial “refute-the-change” stance, separate from author) **MUST** research the change + its connected features across a CLOSED SET OF EIGHT ANGLES: (1) correctness/logic, (2) regression (call-graph impact, every direct+transitive caller mapped to a test), (3) latent/dangerous-code (runtime-only/interface-ABI/concurrency/resource-lifecycle), (4) security, (5) performance, (6) host/data/target-hardware safety (§12/§9/§11.4.133), (7) cross-feature interaction, (8) business-logic conformance. Each angle names tools + obtains required depth + emits captured-evidence (“looks fine” forbidden); N/A

angles recorded NOT_APPLICABLE: <reason>. Output = a per-change impact-research report + single GO/NO-GO blocking acceptance on any unmitigated risk, iterating to a clean GO before the §11.4.125 review gate. Propagation gate CM-COVENANT-114-145-PROPAGATION (literal 11.4.145) + gate CM-IMPACT-RESEARCH-PER-CHANGE. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.145.

§11.4.146 — Reproduce-first + same-test-confirms-fix + extend-to-all-cases (cascaded from constitution submodule §11.4.146)

§11.4.146 — Every reported problem handled by a named three-step workflow: STEP 1 REPRODUCE-FIRST + INVESTIGATE — author the §11.4.43 RED test as a §11.4.115 RED-baseline-on-the-broken-artifact capturing defect-present evidence AND used as a §11.4.102 investigation instrument gathering forensic characterisation; STEP 2 SAME-TEST-CONFIRMS-FIX — the SAME test source (§11.4.115 polarity RED_MODE=1→0) confirms the defect ABSENT on a clean target (§11.4.108/§11.4.139), validated-first-after-redeploy (§11.4.130), no separate happy-path substitute; STEP 3 EXTEND-TO-ALL-CASES (mandatory per-fix) — fan out across the full case-space (valid+invalid+boundary+concurrent+chaos+topology per §11.4.85/§11.4.3) with enumerated provable coverage (§11.4.118), each user-visible case registered into the §11.4.135 suite. All steps anti-bluff with captured physical evidence (§11.4.123). Propagation gate CM-COVENANT-114-146-PROPAGATION (literal 11.4.146) + gate CM-REPRODUCE-FIRST-THEN-EXTEND. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.146.

§11.4.147 — Crashed-agent respawn-until-complete + no-work-loss registry (cascaded from constitution submodule §11.4.147)

§11.4.147 — Every dispatched agent/subagent MUST be tracked through its full lifecycle so a crash NEVER loses/forgets/corrupts its work (a crashed agent is NOT a completed agent; “it probably finished before dying” is a forbidden §11.4.6 guess). ALL hold: (a) durable append-only agent REGISTRY (stable id, task + §11.4.58 file-scope, output/evidence paths, status from {dispatched|in-flight|crashed|respawned|complete}, reusing the §11.4.116 sync substrate) = single source of truth for “is this work owed?”; (b) mechanical crash-detection + respawn-until-complete (any abnormal termination flips to crashed + keeps the unit OPEN, respawned with same id+scope until an agent reaches complete, transient causes use already-defined backoff, non-transient terminal errors

investigated §11.4.102 first); (c) PARTIAL-STATE preserve → §11.4.84-quiescence-check → resume-or-clean-restart (§9.2 backup), per-agent git worktree isolation; (d) “crash ≠done” — the endless-loop done-condition (§11.4.87/§11.4.94/§11.4.126) MUST NOT read satisfied while any entry is non-complete. Propagation gate CM-COVENANT-114-147-PROPAGATION (literal 11.4.147) + gate CM-CRASHED-AGENT-RESPAWN-TRACKED. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.147.

§11.4.148 — Workable-item integrity + comprehensive description + bidirectional tracker sync + BLOCKED unblock-choices (cascaded from constitution submodule §11.4.148)

§11.4.148 — Binds + strengthens the workable-item discipline (§11.4.15/.16/.54/.21/.91/.93/.95/.106) into one integrity contract spanning DB ↔ docs ↔ external tracker: (D1) no item without a valid status + valid type + stable id on ALL three surfaces (id is the cross-surface binding key); (D2) comprehensive structured description per item (WHAT + HOW-it-manifests + HOW-to-reproduce + ACCEPTANCE-CRITERIA, never a stub); (D3) every BLOCKED/Operator-blocked item enumerates the closed list of unblock CHOICES ([A]... · [B]... · [C]... per §11.4.66); (D4) regular never-missed bidirectional DB↔docs↔tracker sync (git-tracked SQLite SSoT §11.4.93/.95, §11.4.86 fingerprint-gated, §11.4.106 docs_chain-bound); (D5) generic idempotent credential-redacted external-tracker push (statuses/types/assignee §11.4.104/sub-tasks, match-by-stable-key, sink-side created/updated/failed proof §11.4.69), machinery project-agnostic §11.4.28. Propagation gate CM-COVENANT-114-148-PROPAGATION (literal 11.4.148) + gates CM-ITEM-INTEGRITY-STATUS-TYPE-ID/CM-ITEM-COMPREHENSIVE-DESCRIPTION/CM-BLOCKED-UNBLOCK-CHOICES/CM-TRACKER-SYNC-IDEMPOTENT. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.148.

§11.4.149 — Per-workable-item testing-diary mandate (cascaded from constitution submodule §11.4.149)

§11.4.149 — Every workable item MUST carry an append-only TESTING DIARY (test EXECUTIONS, distinct from §11.4.93 lifecycle history + §11.4.55 reopens): (a) a test_diary table additive to the §11.4.93/.95 SQLite SSoT — one row per run (ISO-8601-UTC date_time, tested_by ∈ {User|Operator|AI-agent|HelixQA}, result §11.4.45, in-depth observations, action/status-change, §11.4.69 evidence_path+feature_class) with a SCHEMA CONSTRAINT making

a PASS row without a non-empty evidence path impossible; (b) an in-depth diary doc + derived at-a-glance summary view feeding §11.4.132 risk-ordering; (c) four-format per-item Diary/Diary_Summary exports (§11.4.65) §11.4.86-fingerprinted + §11.4.106 docs_chain-bound; (d) external-tracker SUB-TASK model (each run a child sub-task, idempotent §11.4.148-D5 push); (e) minimal-LLM deterministic tooling (zero LLM in the data path), 100% test-covered §11.4.27. Propagation gate CM-COVENANT-114-149-PROPAGATION (literal 11.4.149) + gates CM-TEST-DIARY-SYNC/CM-DIARY-PASS-REQUIRES-EVIDENCE. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.149.

§11.4.150 — Mandatory deep multi-angle web research per change/issue (cascaded from constitution submodule §11.4.150)

§11.4.150 — For EVERY fix/improvement/change/closure — however big/small/simple/complex — the agent MUST, in addition to systematic-debugging (§11.4.102), the fix, independent review (§11.4.125/§11.4.142), and comprehensive tests (§11.4.4(b)/§11.4.40), perform a DOCUMENTED deep multi-angle web research pass (articles, official+vendor docs, API refs, standards, issue trackers, maintainer guidance, open-source code) to BOTH (i) discover the best solution AND (ii) confirm there is NOT a more serious underlying problem. ALL hold: (A) no closure-as-fixed/structural without a documented deep-research pass (makes §11.4.8 unconditional); (B) ≥ 2 genuinely-distinct angles (no single-source confirmation-bias miss); (C) confirm-no-bigger-problem (enumerated §11.4.118); (D) latest-source + cited (§11.4.99 URLs + access date in artefact + commit footer); (E) reopen-breaking is the purpose (§11.4.34/.55); (F) ALWAYS in parallel with the main stream (§11.4.70/.103/.89, never serialising); (G) apply ASAP to every workable item across the §11.4.93/.95 SSoT. Propagation gate CM-COVENANT-114-150-PROPAGATION (literal 11.4.150) + gate CM-DEEP-RESEARCH-PER-ISSUE. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.150.

§11.4.151 — Project-prefixed release-tag/version-naming mandate (cascaded from constitution submodule §11.4.151)

§11.4.151 — Every release tag AND version name on the main repo AND every owned submodule (§11.4.28) MUST be prefixed with the project's release prefix, form <PREFIX>-<version> (e.g. helix_translate-1.0.0-dev-0.0.1), so a release is greppable across every repository (git tag -l '<PREFIX>-*'). Prefix

resolution (closed-set, deterministic §11.4.6): (1) `HELIX_RELEASE_PREFIX` from `.env` (git-ignored §11.4.30, documented in `.env.example` §11.4.77) else (2) lowercased snake_case project-root dir name (§11.4.29). The prefix MUST be IDENTICAL across the main repo and all owned submodules within one release; version codes increment monotonically. The tag is created only after the §11.4.40 GREEN retest and reaches every upstream via §11.4.113 merge-onto-latest-main (never a force-push), fanned out per §2.1. Propagation gate CM-COVENANT-114-151-PROPAGATION (literal `11.4.151`) + gate CM-RELEASE-PREFIX-NAMING. Non-compliance is a release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.151.

§11.4.152 — Crashlytics-recorded-data continuous monitoring + systematic-debug + regression-test-coverage (cascaded from constitution submodule §11.4.152)

§11.4.152 — Every project with Firebase Crashlytics enabled/wired MUST treat the Crashlytics console as a first-class captured-evidence channel from real end-user devices and continuously process every datum (an open Crashlytics issue with a green suite is a §11.4 field-telemetry PASS-bluff). Strengthens+completes §11.4.47 (which surfaces findings) with the fix lifecycle. ALL hold: (1) continuous monitoring of ALL four surfaces — fatal crashes, ANRs, performance traces, AND non-fatals (the silent class — every catch/fallback path is a real degraded UX); (2) systematic-debugging reproduce-before-fix (§11.4.102/§11.4.115, the recorded stacktrace IS the captured defect evidence); (3) a fix for every confirmed issue against its proven root cause (§11.4.9/43/108; “acknowledged in console” is not a fix); (4) a permanent §11.4.135 regression guard per closed issue (§11.4.115 polarity test) in the same commit + a closure log citing the console issue id/URL; (5) regular cadence (§11.4.47 five-trigger set); (6) no false results, no bluff. Propagation gate CM-COVENANT-114-152-PROPAGATION (literal `11.4.152`) + gate CM-CRASHLYTICS-ISSUE-FULLY-COVERED. Non-compliance is a release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.152.

§11.4.153 — Comprehensive per-feature Status + Status_Summary set with mandatory video confirmation (cascaded from constitution submodule §11.4.153)

§11.4.153 — Every project MUST maintain under `docs/features/` a comprehensive feature Status set (`Status.md` + its §11.4.56 `Status_Summary.md`)

enumerating EVERY component, EVERY client app/binary/surface (TUI/CLI/Web/desktop/mobile/API/gRPC/library/submodule/infra), and EVERY feature (incl. ported from incorporated CLI-agent/submodule catalogues §11.4.74) with NO feature left out, reconciled vs the codebase (§11.4.6/.118). Per-feature fields: Component/Feature/Category/Implementation/Wiring (§11.4.108)/Real-use/Tests-coverage (§11.4.4(b))/Validation (PASS/FAIL/SKIP/PENDING_FORENSICS/OPERATOR-BLOCKED §11.4.45)/Video-confirmation. Every user-visible “confirmed” claim is backed by a recorded real-use video (real prompts → real LLM/service responses → real results), NEVER a frozen frame (§11.4.107), NEVER a faked/bluff response (§11.4.2/.5); autonomous-infeasible ⇒ honest §11.4.3/.52 SKIP. Video-analysis remediation loop §11.4.102/.146/.134; always-in-sync §11.4.45/.106 docs_chain + §11.4.86 fingerprint; four-format export HTML+PDF+DOCX (adds DOCX to §11.4.65 for this class). Propagation gate CM-COVENANT-114-153-PROPAGATION (literal 11.4.153) + gates CM-FEATURE-STATUS-COMPLETE/CM-FEATURE-STATUS-VIDEO-CONFIRMED. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.153.

§11.4.154 — Window-scoped capture + fresh-corpus rotation for feature/QA recordings (cascaded from constitution submodule §11.4.154)

§11.4.154 — Refines §11.4.2/.5/.107/.153 recording discipline with two capture-hygiene invariants. (A) Window-scoped, NOT whole-screen — every feature/QA video captures ONLY the window/surface under test (GUI window / CLI-TUI pane / web tab-viewport / emulator frame), NEVER the whole desktop or unrelated windows (whole-desktop leaks operator-private content §11.4.10/.83, dilutes the §11.4.107 freeze oracle, breaks the §11.4.137 OCR/ROI oracle); target by stable identity (window id/title, device serial, browser context, tmux target) per §11.4.111; cannot capture below whole-screen ⇒ honest §11.4.3 SKIP. (B) Fresh-corpus rotation — when a new in-scope run begins, the agent’s OWN prior in-scope stale recordings at the raw path are removed FIRST (§11.4.107 not-stale + §11.4.86 freshness); honest boundary (§11.4.6/§9.2) — own SAME-scope recordings ONLY, never another project’s/operator-authored files (uncertain ⇒ surface, §11.4.122); committed docs/qa/<run-id>/ (§11.4.83) is durable, not rotated. Propagation gate CM-COVENANT-114-154-PROPAGATION (literal 11.4.154) + gate CM-WINDOW-SCOPED-FRESH-CORPUS-RECORDING. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.154.

§11.4.155 — Project-name-prefixed feature/QA recording filenames (cascaded from constitution submodule §11.4.155)

§11.4.155 — Every recorded video the project produces — every feature/QA real-use recording (§11.4.153), every window-scoped capture (§11.4.154), every always-on device recording (§11.4.128), every raw/curated artefact + the committed docs/qa/<run-id>/ trail (§11.4.83) — MUST have a filename STARTING WITH the PROJECT-NAME prefix, ALWAYS (an unprefixed recording makes a multi-project corpus un-greppable/un-attributable — the §11.4.151 failure on the recording-corpus axis). Prefix resolution is the closed-set IDENTICAL to §11.4.151: (1) HELIX_RELEASE_PREFIX from .env else (2) lowercased snake_case project-root dir name (§11.4.29); canonical form <PREFIX>- - -<feature-or-scope>- - -<run-id>.<ext>; MUST equal the §11.4.151-resolved release prefix (one project, one name). Honest boundary §11.4.6 — guarantees attribution+greppability, not content validity (§11.4.107/.137/.153), and does not relax §11.4.154 window-scope/rotation. Propagation gate CM-COVENANT-114-155-PROPAGATION (literal 11.4.155) + gate CM-RECORDING-PROJECT-NAME-PREFIX. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.155.

§11.4.156 — All CI/CD automation MUST be disabled (cascaded from constitution submodule §11.4.156)

§11.4.156 — Every governed repository (main repo, this constitution submodule, every owned+nested submodule we author and push) MUST ship with ALL server-side CI/CD automation DISABLED: no push to any owned upstream may trigger a GitHub Actions run, GitLab pipeline, or equivalent (Jenkins/CircleCI/Travis/Drone/Woodpecker/Bitbucket/Azure). Generalises + makes ABSOLUTE the §11.4.75 Layer-5 posture across all governed repos; enforcement migrates to the LOCAL §11.4.75 git-hook ritual + §11.4.40 pre-tag sweep. ALL hold: (A) zero active .github/workflows/*.yml|yaml / .gitlab-ci.yml / equivalent at the ROOT of any authored repo; (B) “disabled” = a push triggers ZERO runs (delete OR rename to a non-trigger name; a live-on:+if:false workflow is NOT compliant); (C) scope = repos we author+push (vendored/third-party nested configs below the root are INERT, out of scope, §11.4.29 vendor-exempt); (D) no new CI may be added; (E) pre-push verify + §11.4.109-class guard enforce mechanically. Honest boundary §11.4.6 — file-level disabling does not reach provider-side server settings (operator turns those off). Propagation gate CM-COVENANT-114-156-PROPAGATION (literal 11.4.156) + gate CM-NO-ACTIVE-CI. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.156.

§11.4.157 — GEMINI.md maintained in lockstep with CLAUDE.md / AGENTS.md / QWEN.md (cascaded from constitution submodule §11.4.157)

§11.4.157 — GEMINI.md is a FIRST-CLASS governance context carrier EQUAL to CLAUDE.md/AGENTS.md/QWEN.md, never optional/best-effort (forensic FACT 2026-06-15: GEMINI.md silently drifted to §11.4.141 while the others carried §11.4.155 — a §11.4 propagation-bluff, a Gemini-CLI agent reading a stale Constitution). ALL hold: (A) five-carrier lockstep — no governance change is complete until GEMINI.md carries it alongside the other three mirrors; GEMINI.md is added to the §11.4.26 propagation + cross-reference set explicitly; (B) no silent drift — GEMINI.md lagging the other mirrors' highest rule is a §11.4.157 violation (§11.4.65-class), back-fill required; (C) equal status — GEMINI.md restates the SAME literal 11.4.N anchors the propagation gates require (§11.4.35), the fleet count INCLUDES GEMINI.md; (D) consumer projects' own CLAUDE/AGENTS/QWEN/GEMINI bind too (§11.4.35). Propagation gate CM-COVENANT-114-157-PROPAGATION (literal 11.4.157, GEMINI.md INCLUDED) + gate CM-GEMINI-MD-LOCKSTEP. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.157.

§11.4.158 — Intensive all-feature/flow/edge-case video-recording + read-the-screen content-verification mandate (User mandate, 2026-06-16). Every project MUST be covered by intensive automated testing that exercises + RECORDS every feature/flow/use-case/edge-case (valid/invalid/boundary/concurrent/failure-injection §11.4.85), each recording showing the feature GENUINELY WORKING with REAL results (real prompts→real responses→real outputs §11.4.153) and NO false/simulated/stale/frozen result (§11.4.2/.5/.107) — a recording showing an error/bluff/non-working feature is a FINDING (→§11.4.153(4)/§11.4.4 fix→retest→re-record), never a confirmation. The testing System MUST ACTUALLY READ every shown log line / message / UI label / dialog / toast / status text and VERIFY it is a genuine working result (OCR/ROI §11.4.117/.137 + confidence floor for pixel surfaces; direct capture for terminal/log; §11.4.107(10) self-validated golden-good/golden-bad analyzer) — “a video was produced” is NOT evidence, “the System read the screen + confirmed a genuine result” is. HelixQA (§11.4.27) MUST drive this exercise→record→read→score pass, PASS only on a read-confirmed genuine result + captured artefact path (§11.4.69). Default recording save path = \$HOME/Downloads (host user's home Downloads, resolved at runtime never hardcoded) unless a project declares an override per §11.4.35; §11.4.155 project-prefix + §11.4.154 window-scope/rotation + §11.4.128 git-ignored raw corpus + §11.4.83 curated docs/qa evidence apply. Composes §11.4.2/.5/.25/.27/.52/.69/.83/.85/.107/.108/.117/.118/.128/.137/.138/.153/.154/.155/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-158-PROPAGATION (literal 11.4.158) + recommended gates CM-INTENSIVE-

RECORDING-COVERAGE + CM-RECORDING-CONTENT-READ-VERIFIED + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.158. Non-compliance is a release blocker. No escape hatch — no --skip-recording-coverage, --video-without-content-read, --happy-path-recording-suffices, --recording-path-anywhere, --unread-recording-OK, --skip-helixqa-read flag. **Project instantiation (§11.4.35):** helix_code overrides the default save path to /Volumes/T7/Downloads/Recordings (recordings are NOT moved to \$HOME/Downloads).

§11.4.142 — Universal code-review mandate — every change reviewed, always, no exception (User mandate, 2026-06-09). Verbatim operator mandate: “ALL changes we do MUST pass through the code review step!!! ALWAYS!!!” EVERY change made to ANY repository this Constitution governs — without exception — MUST pass through an INDEPENDENT code-review step BEFORE it is accepted, committed, or built. NO change class is exempt: source code, fixes, tests, gates, meta-test mutations, documentation, doc-tooling, build/CI scripts, configuration, governance files (Constitution / CLAUDE / AGENTS / QWEN), conductor main-stream edits, sub-agent output, refactors, single-line edits — if a diff exists, it gets an independent review. This is the ABSOLUTE form of §11.4.125 (code-review agent gate after a batch, before pre-build sweep + main build): §11.4.142 strips every implicit scoping §11.4.125’s “after all fixes/changes/ implementations are done” phrasing could leave open — no “just a doc edit”, no “just a one-liner”, no “the author already self-reviewed (§11.4.92)”, no “trivial change” carve-out. **Independence is load-bearing** — the reviewer MUST be structurally separated from the author (a dedicated code-review agent, subagent-driven by default per §11.4.70 / §11.4.20, or a distinct human), NEVER the author re-reading their own work; §11.4.92’s multi-pass self-evaluation is the AUTHOR-side discipline and PRECEDES (never satisfies) §11.4.142. **The review is itself anti-bluff** (§11.4 / §11.4.1) — a rubber-stamp “looks good” is not a review; it MUST genuinely analyse correctness, safety (no host §12 / data §9 / target-hardware §11.4.133 regression), will-it-really-work (no solve-A-create-B), end-user behaviour (§11.4 / §107), and test-genuineness (§1.1), its conclusions captured evidence per §11.4.5 / §11.4.69, and **it iterates to a clean GO per §11.4.134** — any finding (BLOCKING / nit / warning) re-arms the loop, acceptance only on ZERO new findings + ZERO warnings with rock-solid physical evidence. Honest boundary (§11.4.6): a passing review does NOT replace §11.4.108 four-layer runtime-signature verification nor the §11.4.40 full-suite retest — it is one of MULTIPLE STRONG LAYERS, and the FIRST one every change crosses. Composes §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.20 / §11.4.40 / §11.4.69 / §11.4.70 / §11.4.92 / §11.4.108 / §11.4.110 / §11.4.125 / §11.4.134 / §107 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its reviewer-dispatch mechanism + change-acceptance seam (commit wrapper / merge queue / PR gate) per §11.4.35. Propagation gate CM-COVENANT-114-142-PROPAGATION (literal 11.4.142) + recommended gate CM-EVERY-CHANGE-REVIEWED (every accepted change carries

a fresh independent-review-completed marker for its diff, produced by a reviewer distinct from the author, before acceptance/commit/build) + paired §1.1 mutation (strip the literal → propagation gate FAILs; accept a diff with no independent-review marker → CM- EVERY- CHANGE- REVIEWED FAILs). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.142. Non-compliance is a release blocker. No escape hatch — no --skip-review, --no-review, --trivial-change-exempt, --doc-edit-exempt, --self-review-suffices, --review-after-commit flag.

§11.4.143 — Real-user-journey mandate for video-streaming-app full-automation tests (User mandate, 2026-06-10). Verbatim operator mandate: “All video streaming apps ... require to choose some title and to press proper UI button to start or resume playing! Proper UI interaction to play exact show with proper content and subtitles is MANDATORY! Without it we just eventually play on 2nd display sample, and that’s it mostly! THIS MUST BE ADDED as MANDATORY RULE regarding testing of any video streaming app in general with full automation tests! Universal in root constitution + ATMOSphere project extensions.” Any full-automation test that asserts a video player / streaming application plays content MUST drive the REAL end-user journey through the app’s OWN UI — launch → BROWSE the actual catalog → choose a SPECIFIC title → press the real Play/Resume button → confirm THAT chosen content is genuinely playing, with its correct subtitles, on the intended routing target. A test that bypasses the journey with a sample / demo / built-in-loop clip, a deep-link / am start -a VIEW / intent shortcut, a synthetic or pre-staged stream, or any path that does NOT exercise the app’s own browse-select-play UI is a §11.4 PASS-bluff at the user-journey layer: it validates ROUTING (that *something* reaches the display) while leaving the user-visible behaviour the operator cares about — “the show I picked actually plays” — unproven (the operator’s “we just eventually play on 2nd display sample, and that’s it mostly” gap). The mandate (ALL hold): (1) **Real journey, not a shortcut** — launch → catalog browse → specific-title selection → real Play/Resume press through the app’s own UI (§11.4.48 UI-driven), NEVER a deep-link / intent / sample / loop-clip shortcut; (2) **Chosen-content confirmation** — the PASS proves the SPECIFIC selected title is playing (not merely that pixels move on the target) via the §11.4.107 liveness battery on the §11.4.136 real-content path + the §11.4.137 subtitle content-correctness oracle for the chosen title’s captions; (3) **Non-introspectable UIs use the pixel oracle** — when the app’s accessibility hierarchy is blank/unreliable (TV-Compose / leanback / canvas / GL), DRIVE input + ASSERT content via the §11.4.117 CV/OCR pixel oracle, never a hierarchy-only tool; (4) **Login via the credential single-source** (§11.4.10), never hardcoded, never logged; (5) **Honest SKIP, never a faked PASS** — where the autonomous journey is genuinely infeasible (hard human-only login / CAPTCHA, geo-block per §11.4.3, secure-surface pixel-blanking per §11.4.112) the test is operator_attended SKIP-with-reason per §11.4.52 + §11.4.3 with a tracked migration item — NEVER a metadata-only / sample-played / routing-only PASS.

Honest boundary (§11.4.6): “the routing fired so the title is playing” is a guess — only the chosen-content liveness + subtitle oracle on the real journey proves it. Composes §11.4.48 / §11.4.107 / §11.4.117 / §11.4.136 / §11.4.137 / §11.4.52 / §11.4.3 / §107 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its concrete app roster, login-credential source, routing target, and UI-driving / pixel-oracle harness per §11.4.35. Propagation gate CM-COVENANT-114-143-PROPAGATION (literal 11.4.143) + recommended gate CM-VIDEO-REAL-JOURNEY-TEST (every video-streaming-app playback test drives the real browse-select-play journey + confirms the chosen content + subtitles, or SKIPS-with-reason) + paired §1.1 mutation (replace a real-journey test’s browse-select-play path with a deep-link / sample shortcut → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.143. Non-compliance is a release blocker. No escape hatch — no --sample-playback-ok, --skip-real-journey, --deep-link-suffices, --routing-only-pass, --no-title-selection flag.

§11.4.144 — Tracked/recorded-device availability-following mandate (User mandate, 2026-06-10). Direct operator mandate: every device the project is tracking / following / recording (test / debug / manual-testing device, across every reachable transport — USB / wireless ADB / SSH / serial / network introspection API) MUST be availability-FOLLOWED — its connection state continuously monitored, any drop handled, never silently abandoned and never presented as a continuous recording. The §11.4.128 always-on recorder KNOWS a tracked device is absent (its per-device loop guards on the reachable state) but, lacking a following discipline, merely spins idle — no data captured, no offline event logged, no resume, no escalation — so the recording corpus presents a continuous timeline with a silent hole, a §11.4 PASS-bluff at the recording-integrity layer (it claims continuous capture while no data exists for the gap). On a tracked device leaving its reachable state the system MUST, automatically: (a) **DETECT** the drop and **log an honest offline event** into the recording corpus (§11.4.6 — a silent gap presented as continuous capture is a fabricated-continuity bluff; §11.4.128 — a silent recording gap defeats always-on recording); (b) **WAIT** for the device to return using the project’s ALREADY-DEFINED reconnection timings — never invented numbers (§11.4.6), the SAME grace / reconnect / poll budgets the project’s recovery path already uses; (c) **RE-ATTACH** — resume recording / tracking the moment the device returns and log an honest online / resume event; (d) **ESCALATE** to the project’s sanctioned device-recovery path (per §11.4.69 feature class `device_recovery`) if the device does not return within the defined timeout — through the sanctioned, authorization-gated recovery entry point ONLY, never bypassing its gate and never performing a destructive recovery (e.g. a power-cycle) autonomously without that authorization (§11.4.21 / §11.4.101 — high-blast-radius recovery is gated; while blocked the system keeps following the device, logging the blocked-escalation honestly). A tracked-device drop that produces a silent corpus hole, a never-resumed recording, or a never-escalated permanent

absence is the bluff this anchor forbids. Composes §11.4.128 (always-on recording — §11.4.144 closes its drop-handling gap) / §11.4.69 (device_recovery sink-side positive evidence) / §11.4.6 (honest offline / online events, reused-not-invented timings) / §11.4.14 (watchdog children reaped on stop) / §11.4.21 + §11.4.101 (gated, non-autonomous escalation). Classification: universal (§11.4.17) — the consuming project supplies its concrete tracking transport, device set, already-defined reconnection timings, and sanctioned recovery entry point per §11.4.35. Propagation gate CM-COVENANT-114-144-PROPAGATION (literal 11.4.144) + recommended gate CM-DEVICE-AVAILABILITY-FOLLOWED + paired §1.1 meta-test mutation (strip the watchdog wiring / let an absence go unlogged → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule [Constitution.md](#) §11.4.144. Non-compliance is a release blocker. No escape hatch — no --skip-availability-following, --silent-recording-gap-OK, --no-reconnect-wait, --invent-reconnect-timing, --skip-recovery-escalation, --autonomous-power-cycle-OK flag.

§11.4.145 — Independent multi-angle impact-research per change (User mandate, 2026-06-10). For EVERY fix / change / new feature, INDEPENDENT impact-research agents (subagent-driven §11.4.70/§11.4.20, structurally separate from the author — §11.4.92 self-eval PRECEDES, never satisfies — adversarial “refute-the-change” stance to defeat the documented LLM confirmation-bias failure mode) MUST research the change AND its connected/dependent features across a CLOSED SET OF EIGHT ANGLES — (1) correctness/logic, (2) regression (what existing working feature could break — via call-graph impact enumerating every direct + transitive caller and contract-dependent feature, each mapped to its test), (3) latent/dangerous-code — the recents class (runtime-only failure, interface/ABI/contract mismatch, concurrency/data-race, resource lifecycle), (4) security (taint/injection/secret/unsafe-API), (5) performance (latency/memory/thermal under load vs baseline), (6) host/data/target-hardware safety per §12/§9/§11.4.133, (7) cross-feature interaction (shared state/timing/hardware contention), (8) business-logic conformance (matches the spec AND the connected features’ contracts, not merely compiles). Forensic anchor (FACT, project-internal): the recents / 3-button-nav path shipped a latent AIDL-interface-contract mismatch that PASSED code review yet failed at RUNTIME ONLY (ANR on a recents-reaping gesture) because no review angle interrogated the interface contract / concurrency-lifecycle / silently-broken connected feature — the §11.4.108 SOURCE→ARTIFACT→RUNTIME→USER-VISIBLE gap caught only after the fact; §11.4.145 shifts that discovery LEFT. Each angle MUST name the TOOL(S) used + obtain the required DEPTH (the diff, the full changed unit, its declared contract, the spec section, every connected feature — NEVER the diff lines alone) + emit a captured-evidence conclusion per §11.4.5/§11.4.69 (“looks fine” forbidden, §11.4.1); a genuinely-N/A angle is recorded NOT_APPLICABLE: <reason> per §11.4.6, never silently skipped. Output = a per-change impact-research REPORT

(one section per angle: tool + evidence path + verdict) + a single GO/NO-GO that BLOCKS acceptance/commit/build on ANY unmitigated risk in ANY angle; the change is fixed/mitigated + affected angles re-researched, iterating to a CLEAN GO (zero unmitigated risk + zero warning) per §11.4.134 before the §11.4.125 review gate. Honest boundary (§11.4.6): a GO proves cross-angle internal-consistency + bounded blast-radius — it does NOT replace §11.4.108 runtime-signature verification nor §11.4.40 full-suite retest; it is one of MULTIPLE STRONG LAYERS, the FIRST research pass every change crosses. Composes/strengthens §11.4.92/.125/.108/.110/.134/.78/.79/.107/.85/.133/§12/§9/.99/.6/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-145-PROPAGATION (literal 11.4.145) + recommended gate CM-IMPACT-RESEARCH-PER-CHANGE + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; accept a change with a report missing an angle or an unmitigated NO-GO → CM-IMPACT-RESEARCH-PER-CHANGE FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.145. Non-compliance is a release blocker. No escape hatch — no --skip-impact-research, --single-angle-suffices, --author-researches-own-change, --diff-skim-OK, --no-go-overridable, --trivial-change-no-research flag.

§11.4.146 — Reproduce-first test + same-test-confirms-fix + mandatory extend-to-all-cases workflow (User mandate, 2026-06-10). Every reported problem MUST be handled by a NAMED three-step test workflow — §11.4.146 does NOT re-author its component disciplines, it NAMES + BINDS them into the operator's exact per-defect sequence and adds ONLY the two emphases the components leave implicit. **(STEP 1 — REPRODUCE-FIRST + INVESTIGATE)** before any fix, author the §11.4.43 RED test as a §11.4.115 RED-baseline-on-the-broken-artifact (reproduce the defect on the CURRENT pre-fix artifact, capture defect-present physical evidence per §11.4.5/§11.4.69/§11.4.107); NEW EMPHASIS (D1) that same RED test is ALSO a deliberate investigation instrument per §11.4.102(A) — it MUST gather ADDITIONAL forensic data characterising the defect (triggers, boundaries, input/topology scope, adjacent failure modes) that feeds BOTH the fix design AND the STEP-3 extend scope; a RED test used only as a binary present/absent gate with no characterisation satisfies §11.4.115 but NOT §11.4.146 STEP 1. **(STEP 2 — SAME-TEST-CONFIRMS-FIX)** after the fix the SAME test source (the §11.4.115 polarity switch flipped RED_MODE=1→0) confirms the defect ABSENT — RED-on-broken then GREEN-on-fixed, both captured, on a clean target per §11.4.108/§11.4.139, validated-first-after-redeploy per §11.4.130, deterministic per §11.4.50; no separate happy-path test substitutes (the §11.4.43/§11.4.115 PASS-bluff). **(STEP 3 — EXTEND-TO-ALL-CASES, mandatory per-fix)** NEW EMPHASIS (D2) immediately after STEP 2 the test MUST be FANNED OUT across the full case-space of the SAME functionality — all flows, valid + invalid + boundary (§11.4.85 empty/max/off-by-one) + concurrent (§11.4.85 contention) + failure-injection (§11.4.85 chaos) + topology variants (§11.4.3) — confirming no issue of any kind, with PROVABLE enumerated coverage per §11.4.118 (a listed

case-set with per-case outcome, never “no other issues found”); a REQUIRED step, NOT deferred to a release-cycle discovery sweep and NOT reduced to a single guard; each user-visible case carries rock-solid physical evidence per §11.4.123 + is registered into the §11.4.135 standing regression-guard suite; a newly-discovered fan-out defect triggers §11.4.4 test-interrupt + re-enters STEP 1. (ANTI-BLUFF, all steps) every PASS is rock-solid CAPTURED physical evidence per §11.4.123 (§11.4.5/§11.4.69/§11.4.107) — metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden (§11.4/§11.4.1); unclear validation method ⇒ deep-research-before-declaring-untestable per §11.4.123/§11.4.8/§11.4.99; an operator-found defect the green suite missed triggers §11.4.138. Honest boundary (§11.4.6): STEP 3 reduces the unknown-unknown surface but does not prove zero remaining defects (§11.4.118) — un-exercised cases stated as honest gaps, never silently implied clean. Composes §11.4.43/.115/.102/.130/.108/.139/.50/.85/.118/.135/.3/.123/.5/.69/.107/.138/.4/§107/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-146-PROPAGATION (literal 11.4.146) + recommended gate CM-REPRODUCE-FIRST-THEN-EXTEND (every closed defect’s fix carries a §11.4.115 reproduce-first polarity test with captured defect-characterisation [STEP 1] + its RED_MODE=0 GREEN confirmation [STEP 2] + an enumerated per-functionality extend case-set registered into the §11.4.135 suite [STEP 3]; a closure with the reproduce→confirm pair but NO enumerated extend case-set FAILs) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; close a defect with only the reproduce→confirm pair and no extend-case-set → CM-REPRODUCE-FIRST-THEN-EXTEND FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.146. Non-compliance is a release blocker. No escape hatch — no --skip-reproduce-first, --fix-without-red, --skip-extend-to-all-cases, --reproduce-confirm-suffices, --defer-extend-to-release-sweep, --single-guard-suffices, --no-defect-characterisation flag.

§11.4.147 — Crashed-agent respawn-until-complete + no-work-loss registry mandate (User mandate, 2026-06-10). Verbatim operator intent: “any agent that crashed because of something MUST BE respawned and finish its work at some point! We MUST NOT lose any work, forget about or have it corrupted!” Forensic case study (FACT): dispatched subagents died on TRANSIENT causes (API Error: Server is temporarily limiting requests rate-limit killed 5 at once, API Error: socket connection closed unexpectedly killed 2, one left PARTIAL edits — two source files written, the dependent SystemUI sliders + doc + test NOT done), each re-dispatched by pure conductor vigilance with NO mechanical guarantee — the moment conductor context is lost / compacted / the conductor itself crashes, an in-flight-but-dead work unit is silently dropped (lost / forgotten / corrupted). Every dispatched agent/subagent MUST be tracked through its full lifecycle so a crash NEVER loses, forgets, or corrupts its work; a crashed agent is NOT a completed agent (§11.4.6 — “it probably finished before dying” is a

forbidden guess; abnormal termination is positive evidence the unit is OPEN). The mandate (ALL hold): **(a) durable agent REGISTRY** — every dispatched agent has an append-only machine-readable registry entry (stable id, task + §11.4.58 file-scope, declared output path(s) + §11.4.5/§11.4.69 evidence dir, dispatch timestamp, status from the CLOSED SET {dispatched | in-flight | crashed | respawned | complete}), reusing the §11.4.116 sync substrate (JSONL event stream + atomic status snapshot; “an entry with no dispatch-event cannot show complete”); the registry is the SINGLE SOURCE OF TRUTH for “is this work owed?” and an unregistered dispatch is itself a violation; **(b) mechanical CRASH-DETECTION + RESPAWN-until-complete** — any abnormal termination (transient rate-limit/socket-close, any non-completion exit, or a terminal error after the runtime’s own retries are exhausted) flips the entry to crashed + keeps the unit OPEN, and the unit is respawned (fresh agent claims the same id + scope + preserved state) and re-respawned until an agent reaches complete; respawn is the safe/reversible/bounded decision taken autonomously per §11.4.101 (never blocks the loop for a human), transient causes use the project’s ALREADY-DEFINED backoff budgets (never invented, §11.4.6), a deterministically-reproducible non-transient terminal error is investigated per §11.4.102 before further respawn (never a blind retry loop); **(c) PARTIAL-STATE preserve → §11.4.84-check → resume-or-clean-restart** — the crashed agent’s uncommitted edits + output doc are PRESERVED (never silently discarded → lost, never blindly committed → corruption), the respawn runs the §11.4.84 quiescence check on the preserved tree (every modified file accounted-for vs scope, no mutation// always pass/_mutated_* residue, no half-written/torn artifact), then EITHER RESUMES idempotently when it passes OR CLEAN-RESTARTS from a known-good base (§9.2 pre-op backup, reversible §11.4.101) when inconsistent — nothing lost, nothing corrupted; per-agent git worktree isolation (§11.4.58 L4 / §11.4.84) keeps the partial tree from contaminating other streams; **(d) “crash ≠done” COMPLETION criterion** — a unit is DONE only when an agent reaches complete with its required captured evidence/output landed (§11.4.5/§11.4.69 + the §11.4.116 verdict-carries-evidence-path rule); the endless-loop done-condition (§11.4.87/§11.4.94/§11.4.97/§11.4.126) MUST NOT read satisfied while any entry is dispatched/in-flight/crashed/respawned-not-yet-complete, the zero-idle survey (§11.4.94) treats every non-complete entry as an OPEN item to reclaim, and a registry showing complete without landed evidence is a §11.4 PASS-bluff at the agent-lifecycle layer. Honest boundary (§11.4.6): the registry + respawn guarantee work is not lost/corrupted, NOT that it is correct — the respawned output still crosses §11.4.108 + §11.4.125/§11.4.142 review + §11.4.40 retest; §11.4.147 is the durability layer beneath those, the agent-side analogue of §11.4.144 (same detect→wait/backoff→re-attach/respawn→escalate shape). Composes §11.4.6/.58/.84/.87/.94/.97/.101/.116/.102/.108/.125/.142/.40/.128/.144/§9.2/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT - 114 - 147 - PROPAGATION (literal 11.4.147) + recommended gate CM-CRASHED-AGENT-RESPAWN-TRACKED + paired §1.1 meta-test mutation (strip the

literal → propagation gate FAILs; mark a crashed entry as the loop's done-condition complete without landed evidence, OR drop a dispatched agent without a registry entry → CM-CRASHED-AGENT-RESPAWN-TRACKED FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.147. Non-compliance is a release blocker. No escape hatch — no --skip-agent-registry, --crash-equals-done, --no-respawn, --discard-partial-state, --blind-commit-partial, --forget-dead-agent, --loop-done-with-crashed-entries flag.

§11.4.148 — Workable-item integrity (status+type+id) + comprehensive structured description + bidirectional external-tracker sync + BLOCKED unblock-choices mandate (User mandate, 2026-06-10). BINDS + STRENGTHENS the workable-item discipline (§11.4.15 status / §11.4.16 type / §11.4.54 id / §11.4.21 Operator-blocked / §11.4.91 description-clarity / §11.4.93 + §11.4.95 SQLite SSoT / §11.4.106 docs_chain) into ONE integrity contract spanning DB ↔ docs ↔ external tracker, adding the operator's three emphases: **(D1) no item without a valid status + valid type + stable id — on ALL three surfaces** (an item missing any of the three in the DB, the rendered docs, OR the external tracker FAILs the validator — release-blocker; the id is the cross-surface binding key); **(D2) comprehensive structured description per item** — WHAT it is (§11.4.91 ≥6-word/≥40-char clear meaning) + HOW it manifests + HOW to reproduce (§11.4.115/.146) + ACCEPTANCE CRITERIA (§11.4.123/.69 captured-evidence verdict), in the §11.4.93 DB description column AND the docs AND the tracker; never a stub/§11.4.91-anti-pattern fragment; **(D3) BLOCKED items carry WHY + enumerated unblock CHOICES** — tightens §11.4.21: every Operator-blocked item (incl. Blocked/BLOCKED documented alias normalised to canonical Operator-blocked, never a silent fork §11.4.6) MUST enumerate the closed list of decisions/actions that would unblock it ([A]...·[B]...·[C]..., mirroring §11.4.66's 2–4-option shape); a BLOCKED item with no enumerated choices FAILs; **(D4) regular never-missed bidirectional DB↔docs↔tracker sync** — the §11.4.93/.95 git-tracked SQLite DB is the SSoT, docs + tracker DERIVED; full sync (db-to-md / md-to-db byte-identical round-trip §11.4.93 + tracker push) runs regularly + on every change, §11.4.86 drift-proof fingerprint (sha256 of the sorted item keyset, NOT mtime) gates freshness, §11.4.106 docs_chain-bound so drift is mechanically caught not vigilance-dependent; **(D5) generic external-tracker push** carries statuses (collapsed onto the tracker's native set per §11.4.33/.112 when fixed, precise value preserved in a header, never lost), types, assignee (§11.4.104 participant handle, UNSET defaults from a project env var, never hardcoded/logged §11.4.10), and sub-tasks, IDEMPOTENT (dry-run-then-real, match-by-stable-key [<ID>] prefix / id custom-field, present⇒UPDATE/absent⇒CREATE, rate-limited, credential-redacted, sink-side created=N updated=M failed=0 proof §11.4.69), the MACHINERY project-agnostic §11.4.28 (consumer registers its tracker/list/field-map at runtime). Anti-bluff §11.4: every sync + validator pass carries captured evidence §11.4.5/.69; honest boundary §11.4.6 — guarantees well-formed items +

agreeing surfaces, NOT that the underlying work is correct (still crosses §11.4.108/.40/.123). Composes §11.4.15/.16/.21/.33/.34/.54/.66/.86/.91/.93/.95/.104/.106/.112/.123/.10/.28/.69/.6/§1.1. Classification: universal (§11.4.17) — consumer supplies its DB path, id prefix, tracker service + list/board id + field map + default-assignee env var, docs_chain context per §11.4.35. Propagation gate CM-COVENANT-114-148-PROPAGATION (literal 11.4.148) + recommended gates CM-ITEM-INTEGRITY-STATUS-TYPE-ID / CM-ITEM-COMPREHENSIVE-DESCRIPTION / CM-BLOCKED-UNBLOCK-CHOICES / CM-TRACKER-SYNC-IDEMPOTENT + paired §1.1 mutation (gate-code = separate work item). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.148. Non-compliance is a release blocker. No escape hatch — no --item-without-status, --item-without-type, --item-without-id, --stub-description-OK, --blocked-without-choices, --skip-tracker-sync, --one-way-sync-OK, --non-idempotent-tracker-push, --hardcode-assignee flag.

§11.4.149 — Per-workable-item testing-diary mandate (User mandate, 2026-06-10). Every workable item MUST carry an append-only TESTING DIARY — a chronological record of every test run against it — distinct from §11.4.93 item_history (lifecycle STATE transitions) + §11.4.55 Reopens (reopen cycles): the diary records TEST EXECUTIONS (which may or may not change status). The mandate (ALL hold): **(a)** a test_diary table additive to the §11.4.93/.95 SQLite SSoT, one row per run soft-keyed to the item id, capturing ISO-8601-UTC date_time + tested_by (closed set User|Operator|AI-agent|HelixQA) + result (§11.4.45 PASS|FAIL|SKIP + detail) + in-depth observations (long markdown, facts or explicit UNCONFIRMED:/PENDING_FORENSICS: §11.4.6, the background a future fix needs) + action_taken+status_changed+from/to (did this run change status + WHY) + §11.4.69 evidence_path+feature_class, with a SCHEMA CONSTRAINT making a PASS row WITHOUT a non-empty evidence path impossible (a PASS-bluff rejected by the schema itself); **(b)** BOTH an in-depth diary doc + a derived at-a-glance summary VIEW (total/pass/fail/skip + last-verdict + last-run + status-changes + distinct testers/feature-classes, DERIVED never duplicated §11.4.93) feeding §11.4.132 risk-ordering; **(c)** four-format per-item Diary/Diary_Summary exports §11.4.65 + §11.4.86-drift-proof-fingerprinted (sha256 of the sorted diary keyset) + §11.4.106 docs_chain-bound so a diary row not exported/pushed FAILs a gate (never-missed); **(d)** external-tracker SUB-TASK model — the item task's description stays the item (§11.4.148 D2, NOT polluted with diary text), each run a CHILD sub-task (type task) with a {TODO|In-progress|Completed} lifecycle (collapsed per §11.4.33/.112), observations + action + an Evidence: line (path not raw artefact §11.4.10/.13) + a diary-entry idempotency key, reusing the §11.4.148 D5 idempotent rate-limited credential-redacted sink-side-proven push, mapper project-agnostic §11.4.28; **(e)** MINIMAL-LLM deterministic bash/Go tooling (zero LLM in the data path — the observations prose is authored by whoever ran the test; tooling only stores/

renders/pushes/validates), 100% test-covered §11.4.27 (unit + integration-against-the-real-tracker with an honest §11.4.3 SKIP-when-token-absent never a faked PASS + export + HelixQA Challenge + paired §1.1 mutation) so a diary PASS-bluff is mechanically impossible. Honest boundary §11.4.6 — the diary guarantees a complete auditable per-item test-history, NOT that any single run's verdict is correct (rests on its own §11.4.69/.107/.123 evidence). Composes §11.4.6/.27/.45/.50/.55/.65/.69/.86/.93/.95/.106/.107/.123/.132/.148/.10/.13/.28/.33/.112/ §1.1. Classification: universal (§11.4.17) — consumer supplies its DB path, diary doc layout, export formats, tracker sub-task field map, docs_chain context per §11.4.35. Propagation gate CM-COVENANT-114-149-PROPAGATION (literal 11.4.149) + recommended gates CM-TEST-DIARY-SYNC / CM-DIARY-PASS-REQUIRES-EVIDENCE + paired §1.1 mutation (gate-code = separate work item). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.149. Non-compliance is a release blocker. No escape hatch — no --skip-testing-diary, --diary-without-evidence, --no-diary-summary, --diary-without-tracker-subtask, --llm-in-diary-data-path, --diary-export-optional flag.

§11.4.150 — Mandatory deep multi-angle web research per change/issue, before declaring fixed or structural (User mandate, 2026-06-11). Verbatim operator mandate: “For every single issue we fix or improvement we make — besides tight systematic-debugging, fixing, code review by independent agents, comprehensive tests — we MUST ALWAYS do deep web research from various angles! No matter how big/small/simple/complex, dig deep the internet for articles, technical documentation, APIs, open-source code — EVERYTHING that can help make the best possible solution OR confirm we don't have a more serious problem we're unaware of! We MUST do everything possible to STOP the constant issue-reopening and finally start closing items as fixed+working! Do this ALWAYS in parallel with the main work stream!” For EVERY fix / improvement / change / closure — no matter how big, small, simple, or complex — the agent MUST, IN ADDITION to tight systematic-debugging (§11.4.102), the fix, independent-agent code review (§11.4.125 / §11.4.142), and comprehensive multi-layer tests (§11.4.4(b) / §11.4.40), perform a DOCUMENTED **deep multi-angle web research pass** digging the internet from VARIOUS angles — articles, official + vendor technical documentation, API references, standards, issue trackers, maintainer guidance, reusable open-source code — to BOTH (i) discover the best possible solution AND (ii) confirm there is NOT a more serious underlying problem the team is unaware of. Forensic case study (FACT, 2026-06-11): a subtitle-on-secondary-display goal verdicted Won't-fix: structurally-impossible (§11.4.112 secure-surface pixel-blanking) was OVERTURNED by a deep multi-angle research pass that surfaced the real OCR-of-the-PRIMARY-display path — a “we can't” became a shipped capability; the canonical anti-pattern of a structural verdict reached for want of research. The mandate (ALL hold): **(A) No closure-as-fixed/structural WITHOUT a documented deep-research pass** — no item may

be marked Fixed/Implemented/Completed (§11.4.33) OR classified structurally-impossible won't-fix (§11.4.112) until a deep multi-angle pass is documented; §11.4.150 makes §11.4.8's pass UNCONDITIONAL (every item, however trivial, at the closure AND structural-verdict gates). **(B) Multiple angles, not a single lookup** — ≥ 2 genuinely-distinct angles (official-docs / standards / known-bug-trackers / alternative-approach / failure-mode / security / performance / platform-constraint / open-source-precedent) so a single-source confirmation-bias miss (§11.4.145) cannot pass; a one-link drive-by is NOT deep multi-angle. **(C) Confirm-no-bigger-problem, not just find-a-fix** — explicitly seek evidence the fix does not mask a deeper defect AND the closure/verdict is genuinely safe; “we found nothing worse” requires the enumerated search (§11.4.118). **(D) Latest-source + cited** — LATEST authoritative versions per §11.4.99 (never training-data/memory), each cited by URL + access date in the research artefact AND the closure commit footer (Deep-research <date>: <urls> OR the literal NO external solution found – original work per §11.4.8). **(E) Reopen-breaking is the PURPOSE** — STOP the constant Fixed↔Reopened churn (§11.4.34 / §11.4.55); a reopen whose root cause a deep pass would have surfaced is a §11.4.150 miss; composes §11.4.7 (demotion-evidence) + §11.4.112 (structural verdict now REQUIRES the cited-authorities pass). **(F) ALWAYS in parallel with the main stream** — background subagent-driven (§11.4.70 / §11.4.20 / §11.4.103 / §11.4.89) concurrent with main fix/build/test work, NEVER serialising it or stalling the loop (§11.4.94 / §11.4.97 / §11.4.101 / §11.4.126). **(G) Apply ASAP to every workable item** — retroactively + going forward across the §11.4.93 / §11.4.95 SSoT; items closed without a documented pass are re-audited in the §11.4.40 / §11.4.42 release-gate sweep. Honest boundary (§11.4.6): the pass reduces the unknown-unknown surface + breaks the most common reopen causes — it does NOT prove zero remaining defects (§11.4.118) and does NOT replace §11.4.108 runtime-signature verification, §11.4.125 / §11.4.142 review, or §11.4.40 retest; it is the research layer every fix/closure/structural-verdict additionally crosses, and “we probably don't have a bigger problem” without the enumerated multi-angle search is a guess (§11.4.6), never a finding. Composes §11.4.8 / §11.4.99 / §11.4.123 / §11.4.118 / §11.4.145 / §11.4.125 / §11.4.142 / §11.4.7 / §11.4.112 / §11.4.34 / §11.4.55 / §11.4.70 / §11.4.20 / §11.4.89 / §11.4.103 / §11.4.40 / §11.4.42 / §11.4.93 / §11.4.95 / §11.4.6 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its concrete research corpora, angle set, item tracker, and closure-commit-footer convention per §11.4.35. Propagation gate CM-COVENANT-114-150-PROPAGATION (literal 11.4.150) + recommended gate CM-DEEP-RESEARCH-PER-ISSUE (every closed/structural-verdicted item carries a documented multi-angle deep-research artefact + cited-source closure footer, run in parallel, before the closure/structural verdict is accepted) + paired §1.1 mutation (strip the literal → propagation gate FAILs; close an item or reach a structural verdict with no documented multi-angle research artefact / cited footer → CM-DEEP-RESEARCH-PER-ISSUE FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule

[Constitution.md](#) §11.4.150. Non-compliance is a release blocker. No escape hatch — no --skip-deep-research, --single-source-suffices, --trivial-change-no-research, --close-without-research, --structural-without-research, --serialise-research, --research-from-memory-OK flag.

§11.4.151 — Project-prefixed release-tag/version-naming mandate (User mandate, 2026-06-12). Verbatim operator mandate: “Every release tag and version name we create — on the main repository and on every Submodule we own — MUST be prefixed with the project’s release prefix, e.g. myproject-1.0.0-dev-0.0.1. The prefix MUST come from HELIX_RELEASE_PREFIX in our .env if it is set, otherwise from the lowercased project root directory name. The SAME prefix MUST be used across the main repo and all owned Submodules in one release so a release is greppable across every repository.” Every release tag AND every version name created on the main repository AND on every owned-by-us submodule (§11.4.28) MUST be prefixed with the project’s release prefix, form <PREFIX>-<version> (canonical example: myproject-1.0.0-dev-0.0.1), so a release is identifiable + greppable across every repository it spans (one `git tag -l '<PREFIX>-*'` enumerates the whole release surface). **Prefix resolution order (closed-set, deterministic — §11.4.6 no-guessing):** (1) HELIX_RELEASE_PREFIX from the project’s .env — authoritative when set; .env is git-ignored per §11.4.30 and the variable is documented in the tracked .env.example (a §11.4.77 re-obtain mechanism, never committed); (2) fallback = the lowercased snake_case form of the project root directory name (no spaces) per §11.4.29 — used whenever the env var is unset/empty, so a prefix is ALWAYS resolvable from the checkout with zero operator input. **The prefix MUST be IDENTICAL across the main repo and all owned submodules within a single release** — a release that tags the main repo <PREFIX>-1.2.0 while tagging an owned submodule with a different/unprefixed value is a §11.4.151 violation (the cross-repo grep no longer enumerates the release). Version codes increment monotonically within the prefix (<PREFIX>-...-0.0.1 → <PREFIX>-...-0.0.2 → ...), never reset, never skipped. Honest boundary (§11.4.6): the prefix guarantees a release is identifiable + uniform across every repository, NOT that its contents are correct — the tag is still created only after the §11.4.40 full-suite retest GREEN and reaches every upstream via the §11.4.113 merge-onto-latest-main path (NEVER a force-push), fanned out per §2.1. Composes §2.1 / §11.4.29 / §11.4.30 / §11.4.40 / §11.4.113 / §11.4.126 (the release-scope terminal condition is a published, prefixed tag). Classification: universal (§11.4.17) — the consuming project supplies its concrete prefix value + the HELIX_RELEASE_PREFIX env var per §11.4.35. Propagation gate CM-COVENANT-114-151-PROPAGATION (literal 11.4.151) + recommended gate CM-RELEASE-PREFIX-NAMING (every release tag/version on the main repo + every owned submodule carries the resolved <PREFIX>- prefix, identical across the release) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; create an unprefixed or differing-prefix release tag → CM-RELEASE-

PREFIX-NAMING FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.151. Non-compliance is a release blocker. No escape hatch — no --no-release-prefix, --unprefixed-tag, --prefix-optional, --differing-submodule-prefix flag.

§11.4.152 — Crashlytics-recorded-data continuous monitoring + systematic-debug + regression-test-coverage mandate (User mandate, 2026-06-13).

Verbatim operator intent: “For every project that has Firebase Crashlytics enabled / wired, we MUST continuously monitor ALL of the Crashlytics-recorded data — crashes, ANRs, performance traces, and non-fatals — systematically debug each, fix and improve, and cover everything with validation and verification tests. This MUST be checked regularly, with no false results and no bluff of any kind!” Every project that has Firebase Crashlytics enabled/wired (SDK linked into a shipping artifact, crash+non-fatal+ANR reporting active) MUST treat the Crashlytics console as a first-class captured-evidence channel from real end-user devices and continuously process every datum it records — an open Crashlytics issue with a green test suite is a §11.4 PASS-bluff at the field-telemetry layer (a real user hitting a broken feature while the suite reports green).

STRENGTHENS+COMPLETES §11.4.47: §11.4.47 owns the periodic REVIEW + dedup + Issue-creation pass that SURFACES Crashlytics/Analytics/Performance findings; §11.4.152 owns what happens to each surfaced item AFTER — the systematic-debug → fix/improve → regression-test-coverage lifecycle that drives it to a proven regression-immune closure (§11.4.47 finds it; §11.4.152 fixes it + proves it stays fixed; both mandatory, neither substitutes). The mandate (ALL hold): (1) **continuous monitoring of ALL four surfaces** — fatal crashes, ANRs, performance traces/regressions, AND non-fatals (skipping any one is a PASS-bluff; non-fatals are the silent class — every catch/fallback/recovered-error path is a real degraded UX and MUST be triaged, not ignored because the app did not crash); (2) **systematic-debugging of each (reproduce-before-fix)** per §11.4.102 Iron Law + §11.4.115 RED-baseline-on-the-broken-artifact (the recorded stacktrace/ANR-trace/non-fatal-context IS the §11.4.5/.69 captured defect-present evidence) BEFORE the fix — a fix without a prior reproducing test is a §11.4.43/.123 violation; unreproducible ⇒ deep-research-before-declaring-untestable §11.4.123/.150; (3) **a fix/improvement for every confirmed issue** against its proven root cause (§11.4.9/.43/.108), “acknowledged in console” is not a fix, muting a recurring issue without a tracked rationale is a §11.4.6/.90 violation; (4) **validation+verification regression-test coverage per closed issue** — in the SAME commit as the fix register a permanent §11.4.135 regression guard (§11.4.115 polarity test: RED_MODE=1 captures the recorded defect on the pre-fix artifact, RED_MODE=0 standing GREEN guard asserting ABSENT) exercising the same user-reachable path the stacktrace identifies, with rock-solid captured evidence §11.4.5/.69/.107/.123 + a closure log citing the console issue id/URL + root-cause + fix commit + the validation/verification test paths; a Crashlytics issue marked resolved WITHOUT a falsifiable regression test is FORBIDDEN (the silent-

recurrence vector, §11.4.138 operator-escape class applied to field telemetry); (5) **regular cadence** reusing the §11.4.47 five-trigger set (pre-build/pre-flash/pre-distribute/pre-tag blocking; daily + post-deployment-burn-in non-blocking) — a one-time sweep never repeated is a violation; (6) **no false results, no bluff** (§11.4/.1/.6 — “no new issues” requires the enumerated monitored-surfaces+window §11.4.118; “fixed” requires the RED→GREEN flip §11.4.115/.130; a guard whose §1.1 mutation does not FAIL it is a bluff gate). Honest boundary §11.4.6: processing every recorded issue breaks the silent-recurrence vector — it does NOT prove zero remaining field defects nor replace §11.4.108/.40; an issue is “closed” only when its guard is GREEN on a clean artifact, never when the console mark is flipped. Classification: universal (§11.4.17) — the consuming project supplies its Crashlytics handle, console-access credential (§11.4.10, never logged), severity table, and per-issue closure-log path per §11.4.35; the reference project-level instantiation is the Lava client’s §6.O (Crashlytics-Resolved Issue Coverage — per-issue validation test + Challenge test + .lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md closure log) + §6.AC (Comprehensive Non-Fatal Telemetry — every catch/fallback records a non-fatal with triage context).

Composes

§11.4/.1/.5/.6/.9/.34/.40/.43/.47/.69/.90/.102/.107/.108/.115/.118/.123/.130/.135/.138/.150/
§1.1. Propagation gate CM-COVENANT-114-152-PROPAGATION (literal 11.4.152) + recommended gate CM-CRASHLYTICS-ISSUE-FULLY-COVERED (every closed Crashlytics issue carries a systematic-debug root-cause record + a registered §11.4.135 regression guard + a closure-log entry citing the console issue id/URL + the validation/verification test paths; a console-resolved issue with no falsifiable regression guard FAILs) + paired §1.1 meta-test mutation (gate-code = separate work item). **Canonical authority:** constitution submodule [Constitution.md](#)
§11.4.152. Non-compliance is a release blocker. No escape hatch — no --skip-crashlytics-monitoring, --monitor-crashes-only, --skip-non-fatals, --resolve-without-regression-test, --console-mark-is-fixed, --monitor-once, --mute-without-rationale flag.

§11.4.153 — Comprehensive per-feature Status + Status_Summary document set with mandatory video-recording confirmation (User mandate, 2026-06-15).

Every project MUST maintain, under docs/features/, a comprehensive feature Status document set (Status.md + its §11.4.56 Status_Summary.md companion) enumerating EVERY system component, EVERY client app/binary/surface (TUI / CLI / Web / desktop / mobile / API / gRPC / library / submodule / infrastructure), and EVERY feature — including features ported from any incorporated CLI-agent / submodule catalogue (§11.4.74) — with NO single feature left out. (1) **Total categorized coverage** — per-feature table organized Component→Category, reconciled against the actual codebase (a code-present feature missing from the table, or a row with no code, is a §11.4.6/§11.4.118 bluff). (2) **Per-feature fields** — Component / Feature / Category / Implementation / Wiring (genuinely end-user-reachable, not merely compiled, §11.4.108) / Real-use / Tests-coverage (four-layer

§11.4.4(b)) / Validation (PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED per §11.4.45) / **Video-recording confirmation** (path to the real-use video or honest gap marker). (3) **Mandatory per-feature real-use video** — every user-visible “confirmed” claim backed by a recorded real-use video of a genuine end-user scenario (real prompts → real LLM/service responses → real results), NEVER a frozen/stale frame (§11.4.107), NEVER faked/mock/demo-loop/bluff response or LLM error passed off as success (§11.4.2/§11.4.5), stored at the project-declared recording path (§11.4.35); a confirmed row with no real video or a bluff video = §11.4 PASS-bluff; autonomous-infeasible ⇒ honest §11.4.3/§11.4.52 SKIP + tracked migration item, NEVER a faked confirmation. (4) **Video-analysis remediation loop** — every video analysed; any defect it surfaces triggers §11.4.102 systematic-debug → fix → §11.4.146 retest → re-record → clean GO per §11.4.134 before “confirmed” (a video exposing a broken feature is a §11.4.4 test-interrupt). (5) **Always-in-sync** — §11.4.45-class roster/corpus-backed, §11.4.106 docs_chain-bound + §11.4.86 drift-proof fingerprint (sha256 of sorted feature-key roster AND sorted video-artefact roster, NOT mtime), re-syncs out-of-the-box; stale = violation. (6) **Four-format export** — HTML + PDF + **DOCX** (this doc class ADDS DOCX to the §11.4.65 HTML+PDF set; other classes unchanged), in sync per §11.4.60. (7) Follows §11.4.44/.45/.56/.57/.59/.60. (8) **MP4 format REQUIRED**. All video confirmations MUST be in .mp4 format (H.264). Window-specific capture ONLY (§11.4.159(A)). Vision validation REQUIRED (§11.4.159(D)). .cast files are supplementary only. Honest boundary §11.4.6 — guarantees a complete video-confirmed always-synced ledger, NOT per-feature correctness (rests on each row’s §11.4.69/.107/.123 evidence) and NOT a §11.4.40 retest substitute. Classification: universal (§11.4.17). Composes §11.4.2/.5/.44/.45/.52/.56/.57/.59/.60/.65/.86/.102/.106/.107/.108/.118/.123/.134/.146/ §1.1. Propagation gate CM-COVENANT-114-153-PROPAGATION (literal 11.4.153) + recommended gates CM-FEATURE-STATUS-COMPLETE + CM-FEATURE-STATUS-VIDEO-CONFIRMED + paired §1.1 mutation.

Canonical authority: constitution submodule [Constitution.md](#) §11.4.153. Non-compliance is a release blocker. No escape hatch — no --skip-feature-status, --feature-without-video, --frozen-video-OK, --bluff-response-video-OK, --skip-video-analysis, --feature-ledger-incomplete-OK, --no-docx-export, --allow-feature-ledger-drift flag.

§11.4.154 — Window-scoped capture + fresh-corpus rotation for feature/QA recordings (User mandate, 2026-06-15). Verbatim: “recording you perform does record the window containing apps and services, not the whole desktop or monitor screen!” + “All old recording files MUST BE removed when new one starts!” Refines §11.4.2/.5/.107/.153 recording discipline with two capture-hygiene invariants. **(A) Window-scoped, NOT whole-screen** — every feature/QA video MUST capture ONLY the window/surface of the app/service under test (GUI window / CLI-TUI terminal pane / web tab-viewport / device-emulator-simulator frame), NEVER the whole desktop/monitor or unrelated windows; whole-desktop

capture leaks operator-private content (§11.4.10/.83), dilutes the §11.4.107 liveness/freeze oracle, and breaks the §11.4.137 OCR/ROI content oracle. Target the window/region by stable identity (window id/title, device serial, browser context, tmux target) per §11.4.111 — never a fixed full-screen device index. Platform genuinely cannot capture below whole-screen ⇒ honest §11.4.3 SKIP + tracked migration item, never a whole-screen pass-off. **(B) Fresh-corpus rotation** — when a new recording run for a scope begins, the agent's OWN prior in-scope stale recordings at the raw recording path MUST be removed FIRST so the live corpus reflects the current run (§11.4.107 not-stale + §11.4.86 roster-freshness). Honest boundary (§11.4.6 + §9.2): “remove old” = the agent's own prior recordings for the SAME scope/project ONLY — NEVER another project's/scope's/operator-authored files; uncertain ⇒ surface, don't delete (§11.4.122); committed docs/qa/<run-id>/ evidence (§11.4.83) is the durable record, NOT rotated. Classification: universal (§11.4.17). Composes §11.4.2/.5/.10/.83/.86/.107/.111/.122/.128/.137/.153/ §9.2/.6. Propagation gate CM-COVENANT-114-154-PROPAGATION (literal 11.4.154) + recommended gate CM-WINDOW-SCOPED-FRESH-CORPUS-RECORDING + paired §1.1 mutation.

Canonical authority: constitution submodule [Constitution.md](#) §11.4.154. Non-compliance is a release blocker. No escape hatch — no --whole-screen-capture-OK, --skip-window-scope, --keep-stale-recordings, --no-corpus-rotation, --full-desktop-recording flag. **(C) MP4 auto-conversion REQUIRED.** Any .cast file produced MUST be auto-converted to .mp4 via agg + ffmpeg immediately after capture. The .mp4 is the primary evidence; .cast is supplementary only.

§11.4.155 — Project-name-prefixed feature/QA recording filenames (User mandate, 2026-06-15). Verbatim: “All recorded videos MUST START with prefix: the PROJECT NAME (ALWAYS USE THE PROJECT NAME). Project name MUST be obtained according to the constitution's own project-name resolution.” Every recorded video the project produces — every feature/QA real-use recording (§11.4.153), every window-scoped capture (§11.4.154), every always-on device recording (§11.4.128), every raw or curated artefact at the project-declared recording path (§11.4.35) + the committed docs/qa/<run-id>/ trail (§11.4.83) — MUST have a filename that STARTS WITH the PROJECT-NAME prefix, ALWAYS; an unprefixed recording is a §11.4.155 violation (a multi-project/scope corpus on one host per §11.4.128/.103 becomes un-greppable + un-attributable — the §11.4.151 identify-and-grep failure on the recording-corpus axis). **Prefix resolution (closed-set, deterministic — §11.4.6, IDENTICAL to §11.4.151):** (1) HELIX_RELEASE_PREFIX from .env (authoritative, git-ignored §11.4.30, documented in tracked .env.example §11.4.77) else (2) lowercased snake_case project-root dir name §11.4.29 — ALWAYS resolvable, zero operator input. SAME prefix for EVERY recording in a checkout so ls '<PREFIX> - - - '* enumerates the corpus; canonical form <PREFIX> - - - <feature-or-scope> - - - <run-id>.<ext> (- - - delimits the prefix unambiguously); MUST equal the §11.4.151-resolved

release-tag prefix (divergence is itself a §11.4.155 violation — one project, one name). Honest boundary (§11.4.6): the prefix guarantees attribution + greppability, NOT content validity (still §11.4.107/.137/.153) and does NOT relax §11.4.154's window-scope/rotation (rotation removes the agent's OWN <PREFIX> - - - * only; foreign/operator files surfaced not deleted §11.4.122/§9.2). Classification: universal (§11.4.17). Composes §11.4.151/.128/.153/.154/.111/.83/.6/.29/.30/.35/.77/.86/§1.1. Propagation gate CM-COVENANT-114-155-PROPAGATION (literal 11.4.155) + recommended gate CM-RECORDING-PROJECT-NAME-PREFIX + paired §1.1 mutation.

Canonical authority: constitution submodule [Constitution.md](#) §11.4.155. Non-compliance is a release blocker. No escape hatch — no - -no-recording-prefix, - -recording-without-project-name, - -unprefixed-recording, - -prefix-optional-for-recording, - -differing-recording-prefix flag.

§11.4.156 — All CI/CD automation (GitHub Actions / GitLab pipelines / equivalents) MUST be disabled (User mandate, 2026-06-15). Verbatim operator mandate: “Any GitHub actions or GitLab pipelines MUST BE disabled! Add this critical mandatory rule / mandatory constraint into the root constitution Submodule, commit and fetch all its changes to all upstreams and make sure we respect and follow this rule (we do apply it) ASAP!!!” Every repository this Constitution governs — main repo, this constitution submodule, every owned + nested submodule we author and push — MUST ship with ALL server-side CI/CD automation DISABLED: no push to any owned upstream may trigger a GitHub Actions run, GitLab pipeline, or equivalent (Jenkins/CircleCI/Travis/Drone/Woodpecker/Bitbucket/Azure, any on: push/schedule/workflow_dispatch). GENERALISES + makes ABSOLUTE the §11.4.75 Layer-5 posture (remote CI DISABLED, workflow preserved at a ...disabled-local-only non-.yaml name a provider ignores) across ALL governed repos; enforcement migrates to the LOCAL §11.4.75 git-hook ritual + §11.4.40 pre-tag sweep, never a remote runner. ALL hold: **(A)** zero active .github/workflows/*.yaml|yml / .gitlab-ci.yml / .gitlab/** / equivalent at the ROOT of any governed repo/submodule (the only place a provider executes); **(B)** “disabled” = a push triggers ZERO runs — delete OR rename to a non-trigger name (§11.4.75 .disabled/.disabled-local-only); a live-on:+if:false workflow still queues runs, NOT compliant; **(C)** scope = repos we author+push — vendored/third-party nested configs below the root (AOSP external/**, prebuilts/**, vendored submodules) are INERT (a provider never runs a non-root config), OUT of scope, MUST NOT be mass-edited (§11.4.29 vendor-exempt); test = “does a push to OUR upstream trigger a run?” yes⇒disable, inert⇒document+leave (§11.4.6 verify-not-assume); **(D)** no new CI may be added (release blocker); **(E)** pre-push verify git ls-files | grep -E '^\.github/workflows/.*.ya?ml\$|^\.gitlab-ci\.yaml\$' empty for authored repos, §11.4.109-class PreToolUse guard + gate enforce mechanically. Honest boundary (§11.4.6): file-level disabling stops FILE-triggered runs, NOT provider-side server settings (org-default required workflows, branch-protection required checks,

provider scheduled exports) — the operator turns those off; the agent documents what it cannot reach, never claims unachieved completeness. Composes §11.4.75/.29/.6/.40/.42/.109/.113/§2.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-156-PROPAGATION (literal 11.4.156) + recommended gate CM-NO-ACTIVE-CI + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; add a root .github/workflows/x.yml to an authored repo → CM-NO-ACTIVE-CI FAILs). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.156. Non-compliance is a release blocker. No escape hatch — no --allow-ci, --enable-workflow, --keep-pipeline, --remote-ci-OK, --ci-exempt flag.

§11.4.157 — GEMINI.md maintained in lockstep with CLAUDE.md / AGENTS.md / QWEN.md (User mandate, 2026-06-15). Verbatim operator mandate: “Make sure with CLAUDE.md, AGENTS.md, QWEN.md we maintain GEMINI.md too! Add this mandatory fact / rule to root constitution Submodule we are inheriting / extending - CONSTITUTION.md, CLAUDE.md, QWEN.md, AGENTS.md, GEMINI.md and other related relevant files!” Forensic FACT (2026-06-15): when §11.4.156/§11.4.157 were authored, Constitution/CLAUDE/AGENTS/QWEN carried the family through §11.4.155 but GEMINI.md had silently drifted to §11.4.141 — 14 mandates (§11.4.142–155) never propagated — a §11.4 propagation-bluff (a Gemini-CLI agent reads a stale Constitution). GEMINI.md is a FIRST-CLASS governance context carrier EQUAL to CLAUDE.md/AGENTS.md/QWEN.md, never optional/best-effort. ALL hold: **(A)** five-carrier lockstep — no governance change is complete until GEMINI.md carries it alongside the other three mirrors; GEMINI.md is added to the §11.4.26 propagation + cross-reference set explicitly; **(B)** no silent drift — GEMINI.md lagging the other mirrors’ highest rule is a §11.4.157 violation (§11.4.65-class), back-fill required; **(C)** equal status — GEMINI.md restates the SAME literal 11.4.N anchors the propagation gates require (§11.4.35), fleet count INCLUDES GEMINI.md; **(D)** consumer projects’ own CLAUDE/AGENTS/QWEN/GEMINI bind too (§11.4.35). Honest boundary (§11.4.6): the §11.4.142–155 GEMINI.md back-fill is a tracked release-blocking remediation; claiming GEMINI.md “in sync” while the back-fill is incomplete is itself a §11.4.157 violation. Composes §11.4.26/.35/.17/.44/.65/.140/.156/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-157-PROPAGATION (literal 11.4.157, GEMINI.md INCLUDED) + recommended gate CM-GEMINI-MD-LOCKSTEP + paired §1.1 meta-test mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.157. Non-compliance is a release blocker. No escape hatch — no --skip-gemini-md, --gemini-optional, --gemini-lag-OK, --four-carrier-suffices flag.

§11.4.159 — Mandatory window-specific video recording + vision validation mandate (User mandate, 2026-06-20). Every feature test, validation, verification, challenge, and QA session that produces video evidence MUST comply with ALL of the following: **(A) Window-specific recording ONLY** — every video MUST record ONLY the target application window (Terminal pane, TUI app, browser tab,

emulator frame), NEVER the whole desktop/monitor screen; use macOS screencapture -l<window_id>, Linux xdotool+ffmpeg, or equivalent; whole-desktop capture leaks operator-private content (§11.4.10), dilutes the §11.4.107 liveness oracle, and breaks §11.4.137 OCR/ROI; platform genuinely cannot capture below whole-screen => honest §11.4.3 SKIP + tracked migration item. **(B) MP4 format REQUIRED** — .mp4 (H.264, movflags +faststart, pix_fmt yuv420p); .cast files are supplementary only; auto-convert via agg+ffmpeg per §11.4.154(C). **(C) Project-name prefix REQUIRED** — filename MUST start with project name in snake_case from HELIX_RELEASE_PREFIX in .env (§11.4.151) or lowercased project root dir name (§11.4.29); format <project_name>-<feature>-<timestamp>.mp4; unprefixed = violation. **(D) Mandatory vision validation** — after EVERY recording, the agent MUST read the terminal output / video content and verify: (i) feature ACTUALLY WORKS, (ii) LLM responses are REAL, (iii) all tests show PASS, (iv) no “TODO implement” / “simulate” / “for now” patterns, (v) output demonstrates end-user working feature; MUST produce a verdict (PASS/FAIL) with evidence path (§11.4.69). **(E) Terminal window cleanup** — after each recording, dismiss/close ONLY the Terminal window used for that recording (window-specific close via osascript/xdotool); MUST NOT close windows belonging to other processes. **(F) Real results ONLY** — every recording MUST show REAL working features; errors/empty output/simulated responses trigger fix→retest→re-record. **(G) Re-runnable evidence** — the command shown MUST be re-runnable to produce the same results. **(H) Fresh-corpus rotation** — remove agent’s own prior in-scope stale recordings FIRST (§11.4.154). **(I) Content verification MANDATORY — not duration-based** — the value of a recording is NOT its duration but its CONTENT; a recording MUST demonstrate the ACTUAL feature being used with REAL results; before accepting ANY recording, the agent MUST verify: expected output patterns ARE present (e.g., test PASS lines, API response data, feature-specific output), the feature ACTUALLY WORKS as demonstrated (not just “something ran”), LLM responses are REAL content (not simulated, not placeholder, not empty), every claim of “working” is backed by visible evidence; a 5-second recording showing a feature working correctly is MORE valuable than a 60-second recording of empty terminal; duration is NOT a proxy for quality. **(J) Expected-content specification REQUIRED** — before recording, the agent MUST specify what content SHOULD appear (expected patterns, expected test results, expected API responses); after recording, the agent MUST verify these patterns ARE present; if expected content is MISSING, the recording is REJECTED regardless of duration. **(K) Content-verification recording workflow** — mandated workflow: (1) SPECIFY expected content patterns, (2) RECORD the feature execution, (3) EXTRACT all text from the recording, (4) VERIFY expected patterns are present, (5) CHECK for simulated/placeholder content, (6) ACCEPT only if ALL patterns found AND zero bluffs detected, (7) REJECT and re-record if ANY pattern missing or bluff detected. **(L) Root cause analysis REQUIRED for rejected recordings** — when a recording is rejected (missing expected content, bluff detected, or empty capture), the agent

MUST investigate WHY before re-recording per §11.4.102; determine the root cause (timing issue, wrong command, tool failure, etc.) and fix it; simply re-recording without understanding WHY the first attempt failed is a §11.4.159 violation. **(M) Real-time monitoring RECOMMENDED** — for complex features, use real-time monitoring that analyzes output DURING recording (not after); this catches issues immediately and allows corrective action before the recording completes. Classification: universal (§11.4.17). Composes §11.4.2/.3/.5/.10/.29/.69/.83/.107/.111/.128/.137/.151/.153/.154/.155/.158/§1.1. Propagation gate CM-COVENANT-114-159-PROPAGATION (literal 11.4.159) + recommended gate CM-WINDOW-VIDEO-VALIDATED + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.159. Non-compliance is a release blocker. No escape hatch — no --whole-screen-ok, --cast-only, --skip-vision-validation, --no-cleanup, --simulated-recording-ok, --unprefixed-recording flag.

§11.4.160 — Vision-verified recording + HelixQA bridge mandate (User mandate, 2026-06-21). Compact summary: every video recording for feature/QA evidence MUST be processed through a vision/OCR pipeline that reads on-screen content and confirms expected results BEFORE acceptance; the recording system MUST provide a bridge feeding captured frames to HelixQA's test infrastructure (or equivalent) for automated read-the-screen verification against SPECIFY-phase expected patterns (§11.4.159(J)); the bridge MUST capture frames at ≤5s intervals, run OCR/vision analysis with a self-validated golden-good/golden-bad analyzer (§11.4.107(10)), compare extracted text against specified patterns, produce a per-frame PASS/FAIL verdict with an evidence path to the frame, and surface failures immediately so the recording can be re-done per §11.4.159(L). The capture interval, OCR confidence floor, and pattern-matching thresholds are project-configured per §11.4.35, calibrated on the project's own fixtures (§11.4.6). Honest boundary (§11.4.6): vision verification confirms the feature produced expected on-screen output — it does NOT replace §11.4.5 captured-evidence quality analysis (audio/video metrics) nor §11.4.108 runtime-signature verification; FLAG_SECURE/DRM-protected/sink-blanked surfaces document the gap per §11.4.112 and use the §11.4.117 proxy oracle instead. Classification: universal (§11.4.17). Composes §11.4.5/.27/.69/.107(10)/.112/.117/.153/.158/.159(J)/§1.1. Propagation gate CM-COVENANT-114-160-PROPAGATION + recommended gate CM-VISION-VERIFIED-RECORDING-BRIDGE + paired §1.1 mutation.

§11.4.161 — Rootless container runtime mandate (User mandate, 2026-06-21). Compact summary: every project MUST use Podman in rootless mode (or equivalent rootless container runtime) for ALL containerized workloads — Docker in rootful mode, sudo, or any escalation to root for container management is STRICTLY FORBIDDEN unless the target platform has no rootless option AND that constraint is documented per §11.4.112; the vasic-digital/containers Submodule (§11.4.76) MUST be used as the sole container orchestration layer — no ad-hoc docker/podman commands outside the Submodule's pkg/boot/pkg/

compose/pkg/health primitives; if a missing capability forces a raw command, the containers Submodule MUST be extended upstream per §11.4.74 rather than worked around; all container-related integration tests MUST boot infra on-demand via the Submodule (the on-demand-infra invariant). Honest boundary (§11.4.6): rootless execution eliminates the container-to-root privilege-escalation vector — it does NOT replace §11.4.10 credentials-handling (credentials in containers still require leak audits) nor §12.3 container hygiene (memory limits/OOM policies/restart backoff still apply). Classification: universal (§11.4.17). Composes §11.4.76/.74/.10/.112/§12.3/.6. Propagation gate CM-COVENANT-114-161-PROPAGATION + recommended gate CM-ROOTLESS-CONTAINER-RUNTIME + paired §1.1 mutation.

§11.4.162 — OpenDesign UI design system mandate (User mandate, 2026-06-21). Compact summary: every project producing user-facing interfaces (web, desktop, mobile, TUI) MUST use OpenDesign (<https://github.com/nexu-io/open-design>) as the mandatory UI design-and-refinement system — NOT ad-hoc CSS or one-off design tools; install as a project dependency and use its design tokens/themes system for: (a) all color palette definitions supporting BOTH light and dark themes from project brand assets (§11.4.35), (b) typography scale and spacing, (c) component-level design tokens; if a desired UI pattern is not supported, extend OpenDesign upstream per §11.4.74 (extend-don't-reimplement); every UI component MUST ship light + dark theme variants; elements MUST NOT overlap, fonts MUST NOT collide, labels MUST NOT overlay labels — any layout regression is a §11.4.162 violation; all UI changes MUST be covered by the project's standard test types including visual regression tests (before/after screenshots with per-pixel or perceptual-diff PASS/FAIL). Honest boundary (§11.4.6): OpenDesign governs design tokens and themes — it does NOT replace functional testing (§11.4.27), WCAG accessibility assertions (§11.4.107), nor the §11.4.48/§11.4.49 UI-driven and dual-approach testing methodology. Classification: universal (§11.4.17). Composes §11.4.74/.25/.27/.4(b)/.107/.48/.49/.35/.69/§1.1. Propagation gate CM-COVENANT-114-162-PROPAGATION + recommended gate CM-OPENDESIGN-UI-SYSTEM + paired §1.1 mutation.

§11.4.163 — Universal Media Validation & Verification Mandate (User mandate, 2026-06-21). Compact summary: every recorded artifact (video MP4, audio WAV, screenshots PNG, asciinema cast, text output) MUST pass through a MEDIA VALIDATION pipeline before acceptance — OCR for video/screenshots with confidence floor (§11.4.117/.107(12)), transcription for audio, text parsing for asciicast; compare extracted content against SPECIFY-phase expected patterns (§11.4.159(J)); self-validated analyzer with golden-good/golden-bad fixture pair (§11.4.107(10)) — golden-bad MUST FAIL or the pipeline is a bluff gate; produce structured verdict (PASS/FAIL + evidence path + matched/unmatched patterns + pinpoint data on FAIL — which frame/line/timestamp, expected vs actual); triggerable as post-recording AND real-time step (§11.4.159(M)); paired §1.1 mutation ensures golden-bad fixture produces FAIL. Honest boundary: confirms

artifact content matches intended patterns — does NOT replace §11.4.5 quality analysis, §11.4.107 liveness, nor §11.4.108 runtime-signature. Classification: universal (§11.4.17). Composes

§11.4.107(10)/.112/.117/.159(J)/.159(K)/.159(M)/.83/.5/.69/.102/.134/§1.1. Propagation gate CM-COVENANT-114-163-PROPAGATION + recommended gate CM-MEDIA-VALIDATION-PIPELINE + paired §1.1 mutation.

§11.4.164 — Universal Constitution Auto-Propagation & Hook System (User mandate, 2026-06-21). Compact summary: every fetch+pull of the constitution submodule MUST trigger constitution/scripts/post_update_hook.sh (inherited by reference §11.4.28, NEVER copied) that: (a) detects which files changed (Constitution.md/CLAUDE/AGENTS/QWEN/GEMINI + scripts/ hooks/ skills/ mcp/ plugins/) via `git diff --name-only`; (b) registers new/modified skills with the agent's skill system via consumer's scripts/register_skills.sh; (c) registers new/modified MCP servers via consumer's scripts/register_mcp.sh; (d) installs new/modified hooks into `.git/hooks/`; (e) makes scripts executable + validates syntax (`sh -n/bash -n` per §11.4.67); (f) emits a summary to stdout + `.constitution/state/last_update.log`; (g) on failure logs exact file:line and continues (§11.4.6); (h) tested by §1.1 mutation (add skill → hook detects it). Consumer MUST invoke hook after every constitutional pull; failure to invoke is non-compliance. Honest boundary: installs components — does NOT guarantee consumer runtime acceptance nor replace §11.4.32 validation sweep.

Classification: universal (§11.4.17). Composes

§11.4.28/.32/.35/.67/.77/.102(B)/.122/.31/§1.1. Propagation gate CM-COVENANT-114-164-PROPAGATION + recommended gate CM-CONSTITUTION-AUTO-PROPAGATION + paired §1.1 mutation.

§11.4.165 — Universal Independent Verification Agent Mandate (User mandate, 2026-06-21). Compact summary: every code change OR recorded media artifact MUST pass an INDEPENDENT verifier structurally separate from the author (§11.4.70/.20, NEVER author self-review — §11.4.92 PRECEDES, never satisfies), producing structured findings with evidence paths and iterating to GO per §11.4.134. (a) CODE: §11.4.142 review + build+test diff scope + run paired §1.1 mutations + verify §11.4.108 runtime-signature registry; (b) MEDIA: run §11.4.163 pipeline + confirm genuine content (no mock/stub/placeholder §11.4.159(I)) + confirm format (MP4 H.264/WAV/PNG/cast+mp4 §11.4.159(B)) + cross-check evidence path; (c) DOCS: validate HTML+PDF+DOCX exports current §11.4.65 + revision header incremented §11.4.44 + §11.4.86 fingerprint match; (d) CONFIG: validate YAML/JSON/TOML syntax + schema cross-reference + §11.4.10 leak check; (e–f) structured findings, iterate to zero-finding GO; (g) verifier self-validated by §1.1 mutation (knowingly broken change MUST be caught). Honest boundary: confirms source/artifact integrity — does NOT replace §11.4.108 runtime-signature nor §11.4.40 full-suite retest. Classification: universal (§11.4.17).

Composes

§11.4.142/.163/.44/.65/.86/.108/.125/.134/.10/.40/.92/.70/.20/.102/.159(I)/.159(B)/§1.1.

Propagation gate CM-COVENANT-114-165-PROPAGATION + recommended gate CM-INDEPENDENT-VERIFICATION-AGENT + paired §1.1 mutation.

Cascaded anchor backfill — parity with canonical Constitution (2026-07-27)

This root CONSTITUTION.md is a CONSUMER extension (CONST-059); the canonical text lives in constitution/Constitution.md, which had advanced to §11.4.234 while this file still topped out at §11.4.165. The entries below restore parity as short cascade references, each carrying the literal anchor token the propagation gates match on. NOTE (§11.4.6 honest boundary): §11.4.64, §11.4.205 and §11.4.206 are CITED but never DEFINED in the canonical constitution (zero block-openers), so they are not cascadable.

NOTE (§11.4.6 honest boundary, cont.): §11.4.175, §11.4.203 and §11.4.204 have ZERO occurrences of any kind in the canonical constitution — neither defined nor cited. They are pure numbering gaps, not dangling cross-references: there is nothing to cascade here and nothing to report upstream.

§11.4.11 — File-layout discipline. Cascade reference (short form): Project files MUST be organised by purpose, not by historical accident. Source code goes under canonical project roots. Tests go under canonical test directories. Logs and forensic artifacts go under operator-controlled directories (‘~/Documents/’, ‘qa-results/’, ‘/tmp/’, OS-equivalent) — NEVER scattered at the repo root, NEVER inside working source trees, NEVER tracked unless they are reference assets explicitly required for evidence-replay. ... Propagation gate CM-COVENANT-114-11-PROPAGATION (literal 11.4.11) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.11 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.18 — Script documentation mandate (User mandate, 2026-05-14). Cascade reference (short form): Forensic anchor — direct user mandate (verbatim, 2026-05-14): “Make sure that every single script inside the scripts dir (in all subdirs in depth) is fully documented and covered with full user manuals and user guides! ... Whenever is some bash script modified we MUST document it fully and create for it complete user guide(s) and manual(s) - if already exists make sure all of it is in sync and fully updated! No documentation ever can be out of sync with its codebase!” Every Bash / shell / POSIX-sh script ANYWHERE in a project (anywhere under ‘scripts/’, ... Propagation gate CM-COVENANT-114-18-PROPAGATION (literal 11.4.18) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.18 — the full mandate text is

canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.22 — Document-sync commit discipline (User mandate, 2026-05-14).

Cascade reference (short form): Forensic anchor — direct user mandate (verbatim, 2026-05-14): “For Issues, Issues_Summary and Fixed docs (and all its exports) we MUST commit and push (only them) as soon as they are updated (synced). Continuation doc and other similar / relevant documentation MUST be committed and pushed too! Like this we will always have up to date status of working items without need to do full commit and push of everything if that is not possible to do!” Classification: §11.4.17-classified universal — every project that tracks work items through an Issues / Fixed lif ... Propagation gate CM-COVENANT - 114 - 22 - PROPAGATION (literal 11.4.22) + paired §1.1 mutation.

Canonical authority: constitution submodule [Constitution.md](#) §11.4.22 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.38 — Installable-Asset Evidence Mandate (User mandate, 2026-05-17).

Cascade reference (short form): Forensic anchor — verbatim operator report (2026-05-17): “app does not have a launcher icon anymore — the latest published app tester build(s). how come app passed anti-bluff checks?” For any user-distributable build artifact (package, bundle, installer, or container image produced by the build pipeline and distributed to end users), tests and challenges MUST open the artifact and verify each user-visible asset is present and non-degenerate. ... Propagation gate CM-COVENANT - 114 - 38 - PROPAGATION (literal 11.4.38) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.38 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.39 — Per-Feature On-Device End-User Validation Mandate (iter-76, 2026-05-17).

Cascade reference (short form): Every user-facing feature in a consuming project MUST have at least one HelixQA scenario that exercises the feature from cold-launch with positive runtime evidence: - A screenshot or screen recording captured at a named checkpoint. - At least one assertion of type ‘screenshot’, ‘accessibility_count’, ‘accessibility_node’, ‘ocr_text’, ‘pixel_histogram’, or ‘editor_state’. - Evidence files written to a discoverable output directory so every PASS is cross-verifiable after the fact. ... Propagation gate CM-COVENANT - 114 - 39 - PROPAGATION (literal 11.4.39) + paired §1.1 mutation.

Canonical authority: constitution submodule [Constitution.md](#) §11.4.39 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.46 — Validate-recent-work-before-post-flash-tests mandate (User mandate, 2026-05-18). Cascade reference (short form): Forensic anchor —

verbatim user mandate (2026-05-18): “We see that on newly flashed devices we execute all post flash tests. Make sure we do execute them all after we validate and verify all recent work from the Issues and Continuation docs first! Once all new / latest work is fully validated and verified, no new issues found, nothing is discovered broken or faulty, then we run all post-flash tests! If validation and verification of recent work fails for any reason, and we must go back to fix anything that popped up we do not have to run post-flash tests ... Propagation gate CM-COVENANT-114-46-PROPAGATION (literal 11.4.46) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.46 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.53 — Fixed_Summary parity mandate (User mandate, 2026-05-18).

Cascade reference (short form): Forensic anchor — verbatim user mandate (2026-05-18T17:55Z): “Note: Just like for Issues we have Issues_Summary, for Fixed we MUST HAVE Fixed_Summary - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML! Add this mandatory rule / constraint into the root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD.” Why this anchor exists. ... Propagation gate CM-COVENANT-114-53-PROPAGATION (literal 11.4.53) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.53 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.57 — README.md doc-link section + revision metadata (User mandate, 2026-05-19). Cascade reference (short form): Forensic anchor — verbatim user mandate (2026-05-19): “Add a doc-link section to README.md — links to Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + ALL Status docs + their exports. Each link shows revision + last-modified.” Why this anchor exists. Operators, AI agents, and external reviewers arriving at the project repo for the first time need a single discoverable entry point listing every canonical tracked-items + status document, along with each document’s freshness (revision + last-modified). ... Propagation gate CM-COVENANT-114-57-PROPAGATION (literal 11.4.57) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.57 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.59 — README always-sync mandate (User mandate, 2026-05-19). Cascade reference (short form): Forensic anchor — direct user mandate (verbatim, 2026-05-19): “fully review and update our main README document. Some points are not valid anymore, some are missing. Make sure main README is among documents we MUST ALWAYS keep updated and in Sync with the projects and other documentation! Make sure we always export it (on every update) into PDF and HTML. This mandatory rules/constraints MUST BE all added into the root

(constitution Submodule) Constitution, AGENT.MD and CLAUDE.MD!”

‘README.md’ at the project root is a §11.4.12-class always-sync document. ...

Propagation gate CM-COVENANT-114-59-PROPAGATION (literal 11.4.59) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.59 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.63 — Workable-items procedure docs as single source of truth (User mandate, 2026-05-19). Cascade reference (short form): Forensic anchor — verbatim user mandate (2026-05-19 ~10:30Z): “To make workable items tracking and creation through all of the mentioned mandatory documents we **MUST** create proper procedure document which will be named in Constitution, AGENTS.md and CLAUDE.md as single source of truth for this procedure! Make sure it is created under: docs/procedures/issues/Creation.md which will be always exported to PDF and HTML whenever it is updated. ... Propagation gate CM-COVENANT-114-63-PROPAGATION (literal 11.4.63) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.63 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.100 — RETIRED. Cascade reference (short form): Demoted to consumer project (a consuming project’s video-color/visual-quality fidelity) per §11.4.17/ §11.4.35 — project-specific (RK3588/MPV/Arvus), not universal. See the consuming project’s Constitution/CLAUDE/AGENTS/QWEN. Propagation gate CM-COVENANT-114-100-PROPAGATION (literal 11.4.100) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.100 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.166 — REPEALED (operator decision, 2026-06-22). Cascade reference (short form): The Universal Semgrep static-analysis mandate is repealed: Semgrep is NO LONGER a mandatory requirement for projects governed by this Constitution. Static analysis / security scanning remains *encouraged* and may be re-adopted per project, but is NOT mandated, NOT a release blocker, and NOT cascaded. ... Propagation gate CM-COVENANT-114-166-PROPAGATION (literal 11.4.166) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.166 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.167 — Big-work-item feature work-stream lifecycle (own copy-on-write project copy + own branch/tags + per-feature builds, no-merge-until-approved, trunk-merged-into-every-stream) (User mandate, 2026-06-23). Cascade reference (short form): Every BIG work item — a new feature OR a drastic/large fix — **MUST** be developed as its own isolated feature work-stream: its

OWN full project copy in a sibling directory, its OWN feature branch, its OWN per-feature flashable builds + feature release tags, kept SEPARATE from the trunk until manually approved after testing, with the trunk regularly merged INTO every feature stream. ... Propagation gate CM-COVENANT-114-167-PROPAGATION (literal 11.4.167) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.167 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.168 — Exported-document independent content + textual + full-visual validation mandate (User mandate, 2026-06-23). Cascade reference (short form): Forensic anchor (FACT, 2026-06-23). A “green” Mermaid export fix shipped RAW gantt-diagram source — the literal lines ‘gantt’, ‘title’, ‘dateFormat’, ‘section’, ‘:done,’ — as plain readable text into user-facing PDFs, unreadable to any reader who expected a rendered diagram. The fix’s validation grepped the HTML for ‘class=“mermaid”’ but NEVER opened the EXPORTED PDF to check whether raw diagram source had leaked into the body. The operator caught it (§11.4.138 operator-escape — the green suite missed it). ... Propagation gate CM-COVENANT-114-168-PROPAGATION (literal 11.4.168) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.168 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.169 — Mandatory comprehensive test-type coverage with anti-bluff captured evidence (User mandate, 2026-06-25). Cascade reference (short form): Forensic anchor — verbatim user mandate (2026-06-25): “MANDATORY test types EACH project MUST have with coverage aiming to 100% without false results, rock solid evidence produced covered with hard physical proofs and zero bluff: unit, integration, e2e, full automation, challenges (challenges submodule), helix qa (helix qa submodule with proper tests banks (suites) and comprehensive fully autonomous qa sessions), ddos tests, security tests, stress and chaos tests, concurrency (atomicity) tests, race conditions or deadlocks tests, memory tests, benchmarking t ... Propagation gate CM-COVENANT-114-169-PROPAGATION (literal 11.4.169) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.169 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.170 — Device-independent host-side rendered-UI visual-proof mandate (User mandate, 2026-06-25). Cascade reference (short form): Verbatim operator escalation set (2026-06-25): “what this should represent, this ugliness!?!?! UI we see does not purpos or sense at all! it is messed up with an giant button!” + “Eye candy stunning funcitonal UI and UX MUST BE prooven with full automation tests which ‘see’ everything visually with no issues, false results or bluff of any kind!!! So far, anti-bluff policy seems completely ignored!” Every change to any user-

facing UI surface (screen / component / view / widget / layout / theme on any platform — Android-Compose, iOS-SwiftUI/UIKit, web, de ... Propagation gate CM-COVENANT - 114 - 170 - PROPAGATION (literal 11.4.170) + paired §1.1 mutation.

Canonical authority: constitution submodule [Constitution.md](#) §11.4.170 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.171 — Mandatory comprehensive human-readable workable-item descriptions (User mandate, 2026-06-29). Cascade reference (short form): Every workable item (ATM-NNN ticket) in the project's workable-items database, Issues.md, Fixed.md, Issues_Summary.md, Fixed_Summary.md, and ALL derived documents MUST carry a comprehensive human-readable description that explains the item in plain, non-technical language suitable for non-developers (project managers, stakeholders, team leads, business partners). ... Propagation gate CM-COVENANT - 114 - 171 - PROPAGATION (literal 11.4.171) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.171 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.172 — Mandatory production-readiness planning with realistic timeline projection (User mandate, 2026-06-29). Cascade reference (short form): Every project under this Constitution MUST maintain a living production-readiness planning document that: (a) provides REALISTIC timeline projections for all product milestones (MVP, beta, production-ready) based on actual development velocity data (items completed per week, reopening rate, blocking dependencies); (b) identifies ALL danger zones, weaknesses, and risk areas with mitigation strategies; (c) accounts for HW delivery timelines, licensing delays, and external dependency risks; ... Propagation gate CM-COVENANT - 114 - 172 - PROPAGATION (literal 11.4.172) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.172 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.173 — Containerized + distributed build mandate (User mandate, 2026-06-29). Cascade reference (short form): EVERY build of EVERY component (source compile, artifact/package/installer/container-image production, codegen, asset render — for ANY language/platform: Go, Android/Gradle, desktop, web, native, firmware) MUST run INSIDE a specialized build container provisioned via the 'digital.vasic.containers' submodule (§11.4.76) — NEVER on the bare host. The build containers MUST be DISTRIBUTED to the designated remote build host(s) (e.g. ... Propagation gate CM-COVENANT - 114 - 173 - PROPAGATION (literal 11.4.173) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.173 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.174 — Shared-host process-ownership verification mandate (User mandate, 2026-06-29). Cascade reference (short form): Verbatim operator mandate: “Make sure that processes we are checking are always OURS! Add this as mandatory constitution rule / constraint, since on every host there may be other work being done!” On ANY host where concurrent work from other projects / users / agents may run (the normal case — a shared developer workstation), EVERY inspection of, conclusion about, or action on a process, build, daemon, port, lock-file, or system-state signal MUST FIRST positively VERIFY the target is OURS (this project’s) before drawing a diagnosis or acting on it. ... Propagation gate CM-COVENANT-114-174-PROPAGATION (literal 11.4.174) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.174 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.176 — Conflict-free multi-track work-division + exactly-once claim registry + capability-aware deadlock-proof device-lock (User mandate, 2026-07-02). Cascade reference (short form): When a project runs multiple concurrent work-STREAM tracks (§11.4.167 feature work-stream lifecycle; §11.4.58/§11.4.103 parallel streams) that share (a) a common pool of workable items and (b) a scarce pool of exclusive hardware/resources, TWO cooperating arbitration layers are MANDATORY — both conflict-free BY CONSTRUCTION, not by convention. ... Propagation gate CM-COVENANT-114-176-PROPAGATION (literal 11.4.176) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.176 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.177 — Developer-tooling project-decoupling + invocation-directory operation (User mandate, 2026-07-04). Cascade reference (short form): No project-specific script, hook, alias, or binary may be wired — copied, symlinked, PATH-exported, or aliased — into a global or shared developer-tooling PATH; shared tooling MUST be project-agnostic and operate on the INVOCATION directory (the cwd, or an explicit path/config argument), NEVER on a hardcoded project path. A project-specific helper lives in and runs from its own repo; ... Propagation gate CM-COVENANT-114-177-PROPAGATION (literal 11.4.177) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.177 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.178 — Track-qualified identity for parallel work streams (session-name collision ban) (User mandate, 2026-07-04). Cascade reference (short form): Parallel work streams that share hardware, a git backing store, a tmux/session namespace, a lock namespace, or any single-owner resource MUST be addressed

by a TRACK-QUALIFIED identity (“), NEVER a bare directory basename. Multiple checkouts/clones/worktrees sharing basename ‘atmosphere’ collapse into one session/lock/log key and cross-wire tmux targets, lock files, device claims, and log directories (the observed collision, 2026-07-04). ... Propagation gate CM-COVENANT - 114 - 178 - PROPAGATION (literal 11.4.178) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.178 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.179 — Corruption-isolated parallel git streams (own-.git independent clones, not shared-common-dir worktrees) (User mandate, 2026-07-04).

Cascade reference (short form): When corruption-isolation is a requirement, parallel git work streams MUST each be an isolated repository with its OWN ‘.git’ (own object store, own index, own lock namespace), NOT ‘git worktree’ checkouts sharing one common ‘.git’. A shared common-dir is a single point of failure: one stale ‘index.lock’ / ‘.commit_all.lock’ / ‘.push_all.lock’ freezes EVERY stream at once (the observed 9-h all-track freeze, 2026-07-04), and one corrupted object store loses every stream. ... Propagation gate CM-COVENANT - 114 - 179 - PROPAGATION (literal 11.4.179) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.179 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.180 — Commit/push (single-writer) wrappers MUST auto-reap provably-stale locks before acquiring (User mandate, 2026-07-04).

Cascade reference (short form): Every commit / push / single-writer wrapper MUST, before acquiring its own lock, auto-reap any PROVABLY-stale git lock — a lock whose recorded holder PID is DEAD (‘kill -0’ fails), OR (no PID recorded) older than a defined threshold AND with no live wrapper/git process holding that repo. The reap covers the wrapper’s own lock plus the git-internal existence-based locks (‘index.lock’, ‘HEAD.lock’, ‘refs/*lock’) that git does NOT auto-release on holder death. ... Propagation gate CM-COVENANT - 114 - 180 - PROPAGATION (literal 11.4.180) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.180 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.181 — Consistent controlled feature-branch naming: one feature/logic-group ⇒ exactly ONE canonical branch name across the main repo + all owned submodules (User mandate, 2026-07-05).

Cascade reference (short form): Forensic anchor — verbatim user mandate (2026-07-05): “System MUST pay attention not to create multiple feat. branches with different names for one same feature or group of workable items / features! Naming MUST BE consistent and carefully controlled! This MUST BE part of the constitution too and carefully risk-free handled! No mistakes can happen! There MUST NOT be any false results, mis-

alignments, inconsistency or bluff of any kind anywhere!” One feature / logical group of workable items MUST map to EXACTLY ONE canonical branch name, used IDENTICALLY ... Propagation gate CM-COVENANT-114-181-PROPAGATION (literal 11.4.181) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.181 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.182 — Track+branch work-stream identity label (T<N>/<branch>) on every agent/subagent/work-stream label and every operator-facing work-stream reference (User mandate, 2026-07-05). Cascade reference (short form): Forensic anchor — operator mandate (2026-07-05): “EVERY agent/subagent/work-stream label AND every operator-facing report reference to a work stream MUST carry a ‘(T/)’ prefix identifying the track + git branch — e.g. ‘(T1/main) ATM-312 MPV drm_prime investigation’, ‘(T2/feat/mistiq-vader) SPK-XXX ...’.” In a multi-track layout (parallel checkouts each on a distinct branch — e.g. ... Propagation gate CM-COVENANT-114-182-PROPAGATION (literal 11.4.182) + paired §1.1 mutation. **TRUNK RULE (operator mandate, 2026-07-28):** work performed on the canonical trunk (main/master) is **ALWAYS Track 1** — the label reads (T1/main - ...), NEVER (T?/main - ...), regardless of checkout path. Precedence is deliberate: the BRANCH decides first and the checkout path decides only for non-trunk branches, so a trunk checkout living outside /mnt/track<N> — or inside a differently-numbered one — is still Track 1; the identity that matters is which STREAM the work belongs to, not which directory it happens to sit in. Rationale: §11.4.182 labels are how concurrent streams are told apart, so an unnumbered T? on trunk work makes it indistinguishable from a genuinely-unknown track in every dispatch, report and progress line — and the trunk is a well-known, singular stream that is never unknown. Reference implementation: scripts/multitrack/track_branch_label.sh derives the branch BEFORE the track number and maps main|master -> 1. Labels MUST be re-derived from that script at dispatch time and kept in real-time sync with the live branch/alias/model/effort — a remembered or stale label is itself a §11.4.182 violation (§11.4.6: live identity is not knowable from memory). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.182 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.183 — Maximal multi-agent utilization per work-stream + full-constitution-application + zero-bluff mandate (User mandate, 2026-07-07). Cascade reference (short form): Forensic anchor — verbatim operator mandate (2026-07-07): “Every Track MUST USE and maximize multi-agents working approaches (subagent-driven, independent agents for various reviewing, and other forms of agents use)! We MUST maximize use of agents so every track produces as much as possible completed work with highest possible quality for the least spent time! Every Track MUST apply everything from the constitution —

all rules, mandatory constraints, guides and technology derived! Nothing can be ignored, skipped, avoided or considered less relevant! Ther ... Propagation gate CM-COVENANT-114-183-PROPAGATION (literal 11.4.183) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.183 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.184 — Mandatory SonarQube static-analysis CLI + local tooling installed and PATH-discoverable (User mandate, 2026-07-06). Cascade reference (short form): Forensic anchor — operator mandate (2026-07-06): “Make sure sonarqube is installed and configured locally same we did for Firebase, GitHub and GitLab CLIs through exported System PATH and .bashrc! Add all major points to the constitution so all projects do the same!” Every project/host that runs static analysis MUST have the SonarQube scanner CLI (‘sonar-scanner’) installed AND durably discoverable on the system PATH via the shell rc (‘.bashrc’/‘.zshrc’) — exactly like the Firebase/GitHub/GitLab CLIs — PLUS the shared ‘constitution/scripts/sonarqube/’ to ... Propagation gate CM-COVENANT-114-184-PROPAGATION (literal 11.4.184) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.184 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.185 — Manual QA-team testing as the mandatory FINAL confirmation of done (User mandate, 2026-07-07). Cascade reference (short form): Forensic anchor — verbatim operator mandate (2026-07-07): “We MUST EVERYWHERE and ALWAYS take into account that manual testing as final confirmation step for work done is MANDATORY!!! Without such confirmation by QA team of ours we cannot consider any set of features or scope of work fully completed! This MUST BE well known and mandatory part of our constitution! Fully respected, taken into account, followed and never forgotten!” No set of features / scope of work / release may be considered fully completed until it has received MANUAL TESTING confirmati ... Propagation gate CM-COVENANT-114-185-PROPAGATION (literal 11.4.185) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.185 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.186 — Anti-divergence enforcement: cross-document consistency is a mandatory-before-export/commit gate, never an after-the-fact audit (research-derived, 2026-07-08). Cascade reference (short form): Compact summary: any project maintaining more than one representation of the same tracked data (a delivery plan + its MVP brief + a workable-items DB + their summaries + ‘.md’/‘.html’/‘.pdf’/‘.docx’ exports) MUST enforce cross-document CONSISTENCY as a deterministic gate that runs BEFORE any export render, any doc/DB sync ‘verify’, and any doc-set commit — NEVER as an after-the-fact human

audit that runs only when an operator happens to ask (the forensic FACT: the same NanoKVM/phone-as-screen feature appeared in three plan clusters — one shared ticket SP ... Propagation gate CM-COVENANT - 114 - 186 - PROPAGATION (literal 11.4.186) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.186 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.187 — Automatic multi-track ruler orchestration (User mandate, 2026-07-04; landed 2026-07-09). Cascade reference (short form): Compact summary: every project incorporating this constitution MUST ship its multi-track parallel-development orchestration as a UNIVERSAL, AUTOMATIC, OUT-OF-THE-BOX, INHERITED capability — not a per-project reimplement — via a single conductor session that programmatically SPAWNS, DRIVES, RESUMES, MONITORS, and CONTROLS per-track headless workers. (1) Headless spawn primitive — workers launch via native headless ‘claude -p -output-format stream-json -verbose’; the worker’s ‘session_id’ is captured from the FIRST ‘init’ event (never invented); ... Propagation gate CM-COVENANT - 114 - 187 - PROPAGATION (literal 11.4.187) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.187 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.188 — Regular main→feature merge cadence: every track / feature-branch MUST FREQUENTLY merge canonical main into itself DURING the work, never only at the end (User mandate, 2026-07-09). Cascade reference (short form): Compact summary: every long-lived feature branch AND every parallel-development track (§11.4.58/§11.4.103 parallel streams; §11.4.167 feature work-streams; §11.4.176/§11.4.178/§11.4.179 multi-track) MUST regularly ‘git merge origin/main’ (the canonical trunk/master) INTO its own branch THROUGHOUT the work — NOT only at the end — so no branch ever drifts far from main and the eventual back-merge stays a small, low-conflict, low-risk operation. ... Propagation gate CM-COVENANT - 114 - 188 - PROPAGATION (literal 11.4.188) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.188 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.189 — Most-reopened cases get extra-depth live-testing scrutiny FIRST (User mandate, 2026-07-10). Cascade reference (short form): Verbatim operator mandate: “All live testing MUST pay extra attention to cases which have been reopened numerous times and those cases in depth retested, investigated, fully validated and verified with real physical results and no bluff of any kind anywhere!” All LIVE TESTING (on-device / on-target runtime validation, deploy-and-observe) MUST give EXTRA-DEPTH retest + in-depth investigation + FULL

validation/verification with REAL PHYSICAL captured evidence (§11.4.5/§11.4.69/ §11.4.107 — captured audio/video/sysfs/dumpsys/sink-side/runtime-signature) and ... Propagation gate CM-COVENANT-114-189-PROPAGATION (literal 11.4.189) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.189 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.190 — Website engineering-quality mandate: every project website MUST be fully responsive + completely SEO-optimized + uniquely OpenDesign-authored + bleeding-edge enterprise-quality, each PROVEN with captured evidence (User mandate, 2026-07-10). Cascade reference (short form): Verbatim operator mandate: “every website we create inside our projects MUST BE fully responsive - working on all browsers, all operating systems, all platforms and all devices and screen sizes! Every Website MUST BE completely SEO optimized! Layouts and templates done MUST BE unique and everything fully designed from OpenDesign! Make sure this applies to our project Website as well! Everything must be bleeding-edge enterprise quality, stunning visual look and feel, innovative, creative and unique! There MUST BE no false or faulty result or bluff of any ... Propagation gate CM-COVENANT-114-190-PROPAGATION (literal 11.4.190) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.190 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.191 — Work-to-track/branch binding enforcement: a logic-group’s work can NEVER be committed OR dispatched onto the wrong track/branch (research-derived, 2026-07-10). Cascade reference (short form): Compact summary: every logical group of workable items binds to EXACTLY ONE canonical ‘(track, branch)’ recorded ONCE in the single source of truth (the §11.4.93 workable-items DB ‘logic_groups.destination’ = authoritative branch + a ‘canonical_track’ column + a ‘group_paths’ file-scope manifest), and NO change belonging to a group may LAND on, nor be DISPATCHED to, a track/branch other than that group’s canonical one — the file-scope generalisation of §11.4.181/ §11.4.182 from branch-NAME consistency to work-PLACEMENT consistency (forensic FACT 2026-07-1 ... Propagation gate CM-COVENANT-114-191-PROPAGATION (literal 11.4.191) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.191 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.192 — Continuous multi-track auto-backfill: a FREE track MUST be IMMEDIATELY re-assigned its next-highest-priority domain work-set, never left idle while its domain has actionable items (User mandate, 2026-07-11). Cascade reference (short form): Verbatim operator mandate: “once a Track is free,

we shall assign next set of workable items to it ... full continuous multi-track development ... no stopping ... this is MANDATORY”. Compact summary: the moment any track’s worker COMPLETES its current work-set (its item(s) closed with anti-bluff evidence, or merged), that FREE track MUST be IMMEDIATELY assigned the NEXT-highest-priority actionable work-set from its OWN domain (its §11.4.191 logic_group / canonical branch) — a track is NEVER left idle while its domain still holds actionable workable items ... Propagation gate CM-COVENANT - 114 - 192 - PROPAGATION (literal 11.4.192) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.192 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.193 — Anti-blind-typing mandate: every UI interaction MUST be “seen” and “understood” via OCR/vision/screenshot proof, NEVER blind-typed (User mandate, 2026-07-13). Cascade reference (short form): Verbatim operator mandate: “No blind typing of any kind anywhere!!!” and “Login to apps and services MUST BE done with understanding and seeing the whole UI ALWAYS!!!” and “Add into the constitution that blind typing is STRICTLY FORBIDDEN, and instead everything MUST BE seen and understood in coordination with proper models and means we have incorporated in HelixQA!!!” and “No bluff of any kind anywhere!!!” Blind typing — sending keystrokes, text, or input events to a UI surface WITHOUT first capturing AND understanding what is on screen (OCR / vision / ... Propagation gate CM-COVENANT - 114 - 193 - PROPAGATION (literal 11.4.193) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.193 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.194 — Exhaustive all-scenario, all-angle code-review + verify-against-captured-runtime-evidence mandate (User mandate, 2026-07-14). Cascade reference (short form): Forensic anchor — verbatim user mandate (2026-07-14): “CRITICAL: Code review MUST TAKE into the accounts all scenarios, look at the problem or change from all angles, deepest possible analysis! No gaps, shortcomings or false results or bluff is allowed! Extend our root constitution Submodule and all relevant files properly so the review process is more precise and we do not have situations like the last one!” Forensic case study (genericised, 2026-07-14 — “the last one”). ... Propagation gate CM-COVENANT - 114 - 194 - PROPAGATION (literal 11.4.194) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.194 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.195 — Branch-structure governance: taxonomy + merge-after-live-QA + flavor/product non-merge (User mandate, 2026-07-14). Cascade reference (short form): Forensic anchor — verbatim operator rules (2026-07-14): 1. Branch

taxonomy: ‘feat/’, ‘product/’, ‘flavor/’ (extends §11.4.181 / §11.4.167(D)). 2. Tracks on FEATURE branches or ‘main’ MUST — after full confirmation + validation + verification + full LIVE MANUAL TESTING (§11.4.185) — MERGE their work to ‘main’ COMPLETELY (all submodules + main repo), ff-only, NEVER force-push (§11.4.113), no commit lost (§9.2) (extends §11.4.167(I) / §11.4.188). 3. ... Propagation gate CM-COVENANT-114-195-PROPAGATION (literal 11.4.195) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.195 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.196 — Native-alias-first priority + per-alias real-signal limit/ subscription tracking + auto-rebind-on-recovery + resource-detection-by-real-identity-not-substring-match (User mandate, 2026-07-14). Cascade reference (short form): Verbatim operator mandate: use ALL operational claude-NATIVE aliases FIRST — as long as ≥ 1 native alias is operational (NOT session-rate-limited, NOT weekly-limit-reached, NOT expired-subscription) — before ANY provider alias; track per alias the session rate-limit / weekly-limit / subscription expiry+renewal from REAL signals (never faked, §11.4.6) and record when each becomes operational AGAIN; as soon as a higher-priority native recovers, rebind tracks to it; ... Propagation gate CM-COVENANT-114-196-PROPAGATION (literal 11.4.196) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.196 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.197 — Research / kicked-off-work completion mandate: every research effort MUST be driven to full, wired, verified completion or explicitly closed — never left un-wired in the backlog (User mandate, 2026-07-15). Cascade reference (short form): Forensic anchor — verbatim operator mandate (2026-07-15): “all research work we start MUST BE fully finished, completely confirmed, validated and verified! It MUST NOT happen for it to stay at the backlog uncompleted and un-wired! Loss of requirements, failure to incorporate them is FORBIDDEN!!! Everything MUST BE fully completed! No exceptions!” Every research effort, kicked-off improvement, design doc, spike, prototype, proof-of-concept, or partially-landed feature that a project STARTS MUST be driven to FULL completion — OR explicitly, evidence-backed ... Propagation gate CM-COVENANT-114-197-PROPAGATION (literal 11.4.197) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.197 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.198 — Default working mechanisms: multi-alias native-first orchestration AND heavy token-optimization are ALWAYS-ON defaults for single- and multi-track work (User mandate, 2026-07-15). Cascade reference

(short form): Forensic anchor — operator mandate (2026-07-15): multiple claude-toolkit aliases (native-first) AND heavy token-optimization MUST be the DEFAULT mechanisms, always used with single- and multi-track development and with single or multiple working agents. Two proven mechanisms are promoted to STANDING, always-on defaults — engaged automatically for EVERY working configuration (single-agent OR multiple-agent; single-track OR multi-track), never a per-request opt-in: (A) Multi-alias native-first orchestration is the DEFAULT. ... Propagation gate CM-COVENANT-114-198-PROPAGATION (literal 11.4.198) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.198 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.199 — Exact-reproduction-sequence mandate: when a working reproduction exists, the investigation MUST use ITS exact sequence — a deviating repro proves NOTHING (research-derived, 2026-07-15). Cascade reference (short form): Forensic anchor (FACT, 2026-07-15). An investigation concluded that a defensive “cover” mechanism “never engages” and that the defect it guards was therefore unfixed. The conclusion was WRONG: the hand-rolled reproduction drove the target with a DIFFERENT input than the existing test did (it pressed a HOME key where the test sent a media-pause key), so the hand-rolled path never reached the PRECONDITION the mechanism keys off. Re-driven with the TEST’S EXACT sequence, the mechanism provably ENGAGED. ... Propagation gate CM-COVENANT-114-199-PROPAGATION (literal 11.4.199) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.199 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.200 — Non-targetable deploy/flash tooling MUST isolate exactly ONE eligible target and VERIFY-AFTER-WRITE on the INTENDED target (research-derived, 2026-07-15). Cascade reference (short form): Forensic anchor (FACT, 2026-07-15). A flashing tool with no device-selection argument AUTO-SELECTED a device from the bus. With a sibling board still attached, it silently wrote the freshly-built image to the WRONG board, printed a SUCCESS message, and the INTENDED board kept its OLD build. Every downstream test then validated a target that had never received the artifact: the §11.4.108 ARTIFACT → RUNTIME-ON-CLEAN-TARGET layer was silently broken while every tool and gate reported green. ... Propagation gate CM-COVENANT-114-200-PROPAGATION (literal 11.4.200) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.200 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.201 — Every guard/gate MUST assert the REAL condition: a false-positive refusal is a FAIL-bluff, a false-negative pass is a PASS-bluff (research-derived, 2026-07-15). Cascade reference (short form): Forensic anchor (FACT, 2026-07-15 — two independent instances in one cycle). (a) A host-budget guard used a bare ‘pgrep -f’ and REFUSED every worker spawn on a host with ZERO real builds, because a live worker’s own command line merely QUOTED the pattern (the §11.4.196(D) / §12.12 carrier footgun). (b) A host-safety pre-flight refused a build on a “swap full” reading that did not reflect a real memory-pressure condition, and cleared only after the operator manually cycled swap. ... Propagation gate CM-COVENANT-114-201-PROPAGATION (literal 11.4.201) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.201 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.202 — Reporting directives: a report MUST auto-create a fully-populated, fully-synced workable item — never a prose acknowledgement (User mandate, 2026-07-15). Cascade reference (short form): Verbatim operator mandate: introduce reporting directives ‘ISSUE’ / ‘BUG’ / ‘TASK’ that create a proper workable item filled with all details, with the WHOLE workable-items system automatically up to date and fully in sync — the main DB, all documents, and all external tracking systems; universal, reusable on every project, fully decoupled, recognized immediately by every project that pulls the constitution. ... Propagation gate CM-COVENANT-114-202-PROPAGATION (literal 11.4.202) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.202 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.207 — Instant multi-stream resume engine: whole-fleet continuation state is a durable, content-addressed, atomically-committed snapshot resumed in O(changed) reads, not an O(history) re-read (User mandate, 2026-07-15). Cascade reference (short form): Compact summary: the continuation mechanism (§12.10 / §11.4.127 / §11.4.131 / §11.4.205) is EXTENDED — never replaced — by a mechanical engine so a fresh session resumes ALL work (every Track, main stream, and agent) at once, INSTANTLY and at token cost O(changed streams), by reading ONE snapshot manifest + one compact blob per stream, NEVER re-reading the full handoff history. ... Propagation gate CM-COVENANT-114-207-PROPAGATION (literal 11.4.207) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.207 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.208 — Operator-request-history document: every project maintains a project-local, always-in-sync ledger of every operator request/prompt

(content + timestamp+timezone + track + alias + model + effort) (User mandate, 2026-07-15). Cascade reference (short form): Verbatim operator mandate (2026-07-15): introduce a request-history document that captures, per operator request/prompt — the full request content, the accepted timestamp with an explicit timezone, the track that processed it, the alias that took the workable item, and the model + effort used — always kept in sync, with no false data and no bluff of any kind. The operator's load-bearing correction (§11.4.35 / §11.4.17): the RULE (this universal discipline) belongs in the constitution submodule; ... Propagation gate CM-COVENANT-114-208-PROPAGATION (literal 11.4.208) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.208 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.209 — Code-review MUST run on the Fable model at xhigh effort (Opus xhigh fallback) (User mandate, 2026-07-15). Cascade reference (short form): Verbatim operator mandate (2026-07-15): the mandatory independent code-review MUST ALWAYS be performed with the Fable model at xhigh effort; if Fable is unavailable, use Opus at xhigh effort. Every mandatory independent code-review the constitution requires — the §11.4.125 code-review-agent gate before pre-build + main build, the §11.4.142 universal every-change-review (no exception), and the §11.4.194 exhaustive all-scenario/all-angle review + verify-against-captured-runtime-evidence, iterated to a zero-finding GO per §11.4.134 — MUST be dispatched on t ... Propagation gate CM-COVENANT-114-209-PROPAGATION (literal 11.4.209) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.209 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.210 — Zero-loss request/prompt intake: every operator request MUST be mechanically captured, tracked, and processed — never skipped, ignored, avoided, or lost (User mandate, 2026-07-15). Cascade reference (short form): Forensic anchor — verbatim user mandate (2026-07-15): “We MUST CHECK which requests / prompts we have given havent passed through except the accumulative prompts / requests history document !!! There may be more of it! This MUST NEVER happen! We MUST make sure that every request / prompt we issues is ALWAYS respected and taken into the account, executed and processed! This is MANDATORY RULE !!! There MUST NOT BE any avoiding, ignoring, skipping or any form of loss for requests / prompts we make! Extend properly constitution Submodule and add mandatory li ... Propagation gate CM-COVENANT-114-210-PROPAGATION (literal 11.4.210) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.210 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.211 — Merge-conflict resolution during main→feature/product/flavor merges MUST run on the Fable model at xhigh effort (Opus xhigh fallback) (User mandate, 2026-07-16). Cascade reference (short form): Verbatim operator mandate (2026-07-16): “During the merge of main branch to all feature/product/flavor branches of tracks 2 - 4, if conflicts happen they MUST BE resolved using Fable model with xhigh effort! If Fable is not available, then with Opus with xhigh! Add this MANDATORY RULE into constitution Submodule and its files CONSTITUTION.md, CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md, and other related relevant files!” Any merge-CONFLICT resolution performed while integrating the canonical trunk (‘main’/‘master’) into a feature / product / flavor branch — ... Propagation gate CM-COVENANT-114-211-PROPAGATION (literal 11.4.211) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.211 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.212 — Main README is the canonical starting-point / entry point for ALL project documentation — no doc may be an orphan unreachable from README (User mandate, 2026-07-16). Cascade reference (short form): Verbatim operator mandate (2026-07-16): “Manual tests catalog must have linking all through the main README as the starting point of this and any other Project documentation! This fact and mandatory rule MUST be added into the constitution.” The main README (‘README.md’ at the repository root) is the canonical STARTING POINT / entry point for ALL project documentation, for THIS project and EVERY project incorporating this constitution. ... Propagation gate CM-COVENANT-114-212-PROPAGATION (literal 11.4.212) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.212 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.213 — FEATURE research-scheduling directive: recognized via all §11.4.140 forms, SCHEDULES (never synchronously executes) a deep, enterprise-grade research + implementation-planning effort as a tracked workable item (operator-directed mandate, 2026-07-16). Cascade reference (short form): Mandate (2026-07-16): extend the §11.4.202 reporting-directive family with a fourth registered action, ‘FEATURE’, that turns a plain-language feature description into a SCHEDULED, deep, enterprise-grade research-and-planning effort — mirroring EXACTLY how ‘ISSUE’ / ‘BUG’ / ‘TASK’ turn a report into a tracked workable item, but for RESEARCH SCHEDULING rather than defect/task/feature reporting: the directive itself creates the tracked item and enqueues it durably; ... Propagation gate CM-COVENANT-114-213-PROPAGATION (literal 11.4.213) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.213 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.214 — Recurrence-links-not-mints: a defect that returns MUST reopen its existing item, never enter as a new id (research-derived, 2026-07-17).

Cascade reference (short form): Forensic anchor (genericised, 2026-07-17). In a consuming project, operator-reported defects that were RECURRENCES of already-tracked, already-“fixed” defects re-entered the tracker as BRAND-NEW ids rather than reopening the originals (measured: at least five distinct recurrence chains, one three deep). ... Propagation gate CM-COVENANT-114-214-PROPAGATION (literal 11.4.214) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.214 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.215 — A doc that BINDS work MUST live, tracked, in the repository where that work happens (research-derived, 2026-07-17).

Cascade reference (short form): Forensic anchor (genericised, 2026-07-17). In a consuming project, the document that DEFINED a binding acceptance gate — the gate a body of work was ordered to satisfy before proceeding — was untracked and lived in an out-of-tree scratch directory belonging to a different working checkout, i.e. it existed in NO repository and in no other agent’s or operator’s checkout. ... Propagation gate CM-COVENANT-114-215-PROPAGATION (literal 11.4.215) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.215 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.216 — Canonical machine-readable design-token source: one CSS custom-property file (:root + [data-theme="dark"]), generated bindings everywhere else (User mandate, 2026-07-22).

Cascade reference (short form): Forensic anchor — proving project (Helix Thready MVP design package, 2026-07-22). The mandate’s shape was proven end-to-end in the Helix Thready design package (‘docs/public/research/mvp/design/’): ONE machine-readable token file (‘opendesign/tokens.css’) carries every color / type / spacing / radius / elevation / motion value as CSS custom properties — light values in a ‘:root {}’ block, dark values in a ‘[data-theme=“dark”]’ override — and every value is evidence-traceable, never invented: the brand chartreuse ‘#B6E376’ is the recorded eyedrop MEAN of ... Propagation gate CM-COVENANT-114-216-PROPAGATION (literal 11.4.216) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.216 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.217 — OpenDesign brand contract: every UI-shipping product carries a 9-section DESIGN.md + tokens.css twin with disjoint color roles, AA-pinned accents, and a locked attribution footer (User mandate, 2026-07-22).

Cascade reference (short form): Forensic anchor — proving project (Helix Thready MVP design package, 2026-07-22). Helix Thready ships ‘design/opendesign/DESIGN.md’

in the OpenDesign 9-section brand-contract schema — VERIFIED against the tool's own source per §11.4.99 (spec, 'design-systems/README.md', the contributor authoring guide, and a measured 150-file corpus: two coexisting heading families, both parsed by number prefix only) — plus its machine-readable twin 'tokens.css' per the OpenDesign token schema, and an honest 'TOOLING.md' capability check. ... Propagation gate CM-COVENANT-114-217-PROPAGATION (literal 11.4.217) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.217 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.218 — Living design-library catalogue: every component × state × theme rendered self-contained + a cross-platform reusability matrix with per-cell provenance (User mandate, 2026-07-22). Cascade reference (short form): Forensic anchor — proving project (Helix Thready MVP design package, 2026-07-22). Helix Thready's 'design/library/' renders the ENTIRE component set as a living catalogue: 'components.html' — 14 groups, ~38 components, ≈ 118 component-state cells, each auditable in BOTH themes via a real toggle (≈ 236 rendered states), zero external requests, interactive behaviors REAL not mocked (theme cycling stamps 'data-theme' and persists; column sorting drives 'aria-sort'; a real 'showModal()' dialog; ... Propagation gate CM-COVENANT-114-218-PROPAGATION (literal 11.4.218) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.218 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.219 — Per-app screen catalogues: full-IA coverage per surface, per-platform chrome variants, honesty markers for unbuilt behavior (User mandate, 2026-07-22). Cascade reference (short form): Forensic anchor — proving project (Helix Thready MVP design package, 2026-07-22). Helix Thready's 'design/screens/' carries a screen catalogue PER SURFACE — 13 web screens + an interactive prototype shell walking the four key journeys, 11 device-framed 390×844 mobile artifacts with an Android/iOS chrome TOGGLE (+ HarmonyOS/Aurora port notes) covering EVERY node of the wireframes' information architecture, a desktop shell with per-OS title bars / menus / tray / file-drop mocks, ten 80-column TUI mockups covering the full state machine, and a 3-page market ... Propagation gate CM-COVENANT-114-219-PROPAGATION (literal 11.4.219) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.219 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.220 — Open-first design tooling: self-hosted open platforms are primary; proprietary tools are import/export targets only; design sources live in-repo in open formats (User mandate, 2026-07-22). Cascade reference

(short form): Forensic anchor — proving project (Helix Thready MVP design package, 2026-07-22). Helix Thready’s tooling capability check (‘opendesign/TOOLING.md’) and export capstone prove the posture: every design source lives IN-REPO in open formats (7 SVG brand masters, ‘tokens.css’, JSON manifests + 28 genuine ‘.od-figma.json’ IR captures, 6 hand-authored Lottie JSONs, self-contained HTML screens); the OpenDesign daemon/CLI status was RUNTIME-PROBED (live health endpoint, CLI round-trip, headless-vs-operator matrix), never assumed; ... Propagation gate CM-COVENANT-114-220-PROPAGATION (literal 11.4.220) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.220 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.221 — Motion discipline: token-bound durations/easings + machine-readable manifest + reduced-motion static fallbacks everywhere (User mandate, 2026-07-22). Cascade reference (short form): Forensic anchor — proving project (Helix Thready MVP design package, 2026-07-22). Helix Thready’s ‘design/motion/’ package proves the discipline: six hand-authored vector Lottie animations bound to the token motion values, each validated by parse + structural schema check + REAL render (lottie-web under jsdom with animation-progression assertions, plus headless-Chromium light+dark contact sheets) — unverified runtimes (‘lottie-compose’ / ‘lottie-ios’) honestly ‘[OPEN]’-tracked; ... Propagation gate CM-COVENANT-114-221-PROPAGATION (literal 11.4.221) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.221 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.222 — Design-completion export wave: a design is not done until token codegen, the honest-output raster matrix, both-theme renders, hand-off bundles, and doc twins land (User mandate, 2026-07-22). Cascade reference (short form): Forensic anchor — proving project (Helix Thready MVP design package, 2026-07-22). Helix Thready’s export capstone proves the wave: an icon/asset raster matrix locked 1:1 to its generator (‘generate-raster.sh’), which SELECTS its SVG renderer by PROBE — each candidate (rsvg-convert → inkscape → resvg → headless Chromium → ImageMagick) must pass a real probe render with content verification before it is used, a failing candidate falls through with a note, and if NONE passes the script exits with instructions and writes NOTHING (“No PNGs were written (output ... Propagation gate CM-COVENANT-114-222-PROPAGATION (literal 11.4.222) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.222 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.223 — Provenance-marker discipline for design documentation + central open-item registry in the area index (User mandate, 2026-07-22).

Cascade reference (short form): Forensic anchor — proving project (Helix Thready MVP design package, 2026-07-22). Every statement in the Helix Thready design area carries an inline provenance marker from a closed vocabulary ([VERIFIED], [OPERATOR], [RESEARCH], [DEFAULT — adjustable], [ASSUMED], [BUILD-NEW], [IN-HOUSE: module], [CONSTITUTION \$x], [GAP: id], [OPEN: id]); ... Propagation gate CM-COVENANT-114-223-PROPAGATION (literal 11.4.223) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.223 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.224 — Test-first (TDD) for ALL work, not only fixes + a minimum code-coverage floor that is NECESSARY-never-sufficient (User mandate, 2026-07-22). Cascade reference (short form): Forensic anchor — verbatim user mandate (2026-07-22): “Any work we do MUST START by writing the test! We MUST FULLY comply and follow the TDD - The test driven development! It MUST BE applied for any change, implementation, wiring, tests, challenges and any other form of codebase we write! Bash scripts as well! Coverage with TDD MUST BE close to 100%, not less than 85% !!!” The gap this closes. ... Propagation gate CM-COVENANT-114-224-PROPAGATION (literal 11.4.224) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.224 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.225 — Scheduler-quota burst-throttling telemetry + interactive-scope isolation from bursty fleets: progressive interactive degradation is diagnosed from throttling accounting over time, never from averages, and an interactive-latency-sensitive process never shares a fixed-quota no-burst scope with a bursty fleet (research-derived, 2026-07-23). Cascade reference (short form): Forensic anchor (genericised, 2026-07-22 — two independent instances in one day). (a) Progressive interactive sluggishness — worsening with the number of live subagents — was root-caused to a scheduler CPU quota (a cgroup ‘cpu.max’ limit) with NO burst allowance on a session scope hosting BOTH the operator’s interactive terminal-multiplexer server AND a spawning agent fleet. Measured live on a 64-CPU host: the scope’s quota was 9.6 CPU per 100 ms enforcement period; ... Propagation gate CM-COVENANT-114-225-PROPAGATION (literal 11.4.225) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.225 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.226 — Evidence-class-at-closure + standing detection pressure: machinery presence does NOT predict whether a fix holds — the EVIDENCE CLASS at closure (runtime vs source) under real DETECTION PRESSURE does; prose does not bind, seams do (research-derived, 2026-07-23). Cascade

reference (short form): Forensic anchor (genericised — the 2026-07-22 reopen/first-touch root-cause forensics; operator mandate 2026-07-22, verbatim: “do deep research on how we have for very long time so much reopened workable items!? Why the quality of delivered releases was so poor mostly? We mostly detect the same major problems on first touch when new releases are available, and this happens over and over again!”). The single strongest empirical result across four independent evidence lines (internal forensics; a 76-source external literature pass; ... Propagation gate CM-COVENANT-114-226-PROPAGATION (literal 11.4.226) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.226 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.227 — Governance-corpus self-custody: the rule corpus is bound by the same custody it imposes — every named gate is implemented-or-registered-deferral under a monotone ratchet, and propagation gates count anchor BLOCKS (exactly-once + lockstep-identical), never bare literals (research-derived, 2026-07-23). Cascade reference (short form): Forensic anchor (measured on THIS corpus, 2026-07-22 — commands and control needles preserved in the root-cause analysis). The canonical rule file held 10,735 lines and 220 distinct §11.4.N anchors; 413 named CM-* gates, of which 241 (58%) had no implementation in either canonical gate site, plus 85 literal “gate-code = separate work item” deferrals; the status-custody seam DESIGNED on 2026-07-17 was still prose on 2026-07-22 — the corpus’s highest-value remediation fell into its own gap within five days; ... Propagation gate CM-COVENANT-114-227-PROPAGATION (literal 11.4.227) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.227 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.228 — Cross-agent extension lifecycle mandate: per-platform compatibility declaration, source tracking, per-extension documentation with compatibility matrix, and auto-wiring of constitution extensions into all detected agent platforms on constitution pull (research-derived from Extensions Catalog forensics, 2026-07-24). Cascade reference (short form): Forensic anchor (FACT, 2026-07-24 — measured from the live Extensions Catalog at ‘docs/extensions/EXTENSIONS_CATALOG.md’). The project’s extension ecosystem — skills, MCP servers, ACPs, LSPs, plugins, and supporting infrastructure (34 skills, 1 MCP, 38 provider aliases, 21 CCR providers, 4 daemons, plus a separate HelixSkills repo) — exhibited four structural gaps: (G1) of 18 active Claude Code project skills, only Claude Code and OpenCode platforms are explicitly supported — HelixCode, Gemini CLI, and Qwen Code have zero recorded loading paths despite b ... Propagation gate CM-COVENANT-114-228-PROPAGATION (literal 11.4.228) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.228 — the full mandate text is

canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.229 — Live in-session task/todo tracker MUST always be up to date and fully in sync with the real work state — never stale, never showing completed work as pending nor pending as done (User mandate, 2026-07-25). Cascade reference (short form): Verbatim operator mandate (2026-07-25): “make sure it is ALWAYS up to date fully in sync! CRITICAL: This MUST BE one of mandatory rules / constraints! Add it with all relevant details into the constitution Submodule and relevant related files.” The agent’s LIVE IN-SESSION TASK/TODO TRACKER — the harness-provided task list / ‘TodoWrite’-class todo tool / any equivalent moment-to-moment execution-state surface the runtime exposes DURING a session — MUST ALWAYS be kept up to date and FULLY IN SYNC with the real live work state: never stale, never showing co ... Propagation gate CM-COVENANT - 114 - 229 - PROPAGATION (literal 11 . 4 . 229) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.229 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.230 — Parallelized-pipeline methodology: build↔validate overlap + affected-stages-only re-run + parallel multi-device testing/recording + fan-out-subagents-always (research-derived, 2026-07-26). Cascade reference (short form): Forensic motivation (genericised): the sequential pipeline shape — build FULLY completes → THEN validation begins → on ANY discovery, restart the WHOLE cycle from scratch → all devices tested one-at-a-time — serialises the operator’s wall-clock onto build latency + one-device-at-a-time evidence gathering, so a multi-hour containerised build followed by serial per-device validation and a from-scratch restart-on-discovery makes time-to-a-manual-QA-capable-release a multiple of what the genuinely-parallel work requires. ... Propagation gate CM-COVENANT - 114 - 230 - PROPAGATION (literal 11 . 4 . 230) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.230 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.231 — Nano-precision model-tier selection: lightest-capable-model-first with dynamic mid-task escalation (User mandate, 2026-07-26, CRITICAL). Cascade reference (short form): Verbatim operator mandate: every work effort MUST use the LIGHTEST CAPABLE model, escalating tier only as complexity rises — a nano-precision, per-task model-tier choice, NOT “everything on the heaviest model.” Forensic motivation (genericised, §11.4.6): dispatching every task — a trivial doc-sync, a one-line status read, a mechanical format fix — on the heaviest available model wastes capacity + token budget + wall-clock that a lighter tier would spend identically well, while the OPPOSITE failure (a hard multi-factor root-cause investigation dispatched ... Propagation gate CM-COVENANT - 114 - 231 - PROPAGATION (literal 11 . 4 . 231) + paired §1.1 mutation. **Canonical authority:**

constitution submodule [Constitution.md](#) §11.4.231 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.232 — Anti-mess long-op orchestration: every long-op is registered, single-owned-per-purpose, liveness-PROVEN, handed-off-on-stop, safely-reaped, consistency-checked, and escape-bounded — the single source of truth for what is running, never inferred (research-derived, 2026-07-26).

Cascade reference (short form): Forensic anchor (genericised): in one cycle a verification sweep produced five distinct mess states — (1) multiple concurrent same-purpose sweeps ran unintentionally (no single-owner-per-purpose claim); (2) a sweep HUNG on a consumer-blocked-on-an-unclosed-producer pipeline and was undetected because liveness was inferred from process-alive (a §11.4.201(6) FALSE-NULL: a wedged op and a progressing op returned the identical “process alive” signal); ... Propagation gate CM-COVENANT-114-232-PROPAGATION (literal 11.4.232) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.232 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.233 — Standing anti-mess control plane: a level-triggered, idempotent, invariant-catalogue reconciliation loop over the WHOLE persistent System state, orchestrating every per-domain enforcer, refusing gated transitions on un-reconciled drift (research-derived, 2026-07-26).

Cascade reference (short form): Forensic anchor (genericised): every existing consistency enforcer in a multi-track / multi-agent / multi-submodule / multi-upstream System is EDGE-TRIGGERED — a PreToolUse hook fires on an action, a check runs at a seam (lock-acquisition / merge-time / build-time / commit-time), a gate runs when a build runs — so a mess that arises WITHOUT an edge firing is un-observed until it causes harm (two live examples captured 2026-07-26: a large uncommitted-batch pile-up sat unreconciled through a session, and multiple provably-stale VCS/wrapper locks were held ... Propagation gate CM-COVENANT-114-233-PROPAGATION (literal 11.4.233) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.233 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

§11.4.234 — Dedicated hook-validation script: hook checks run as an explicit stage of a dedicated commit/push script, and the commit/push mechanism is ALWAYS unblocked (operator mandate, 2026-07-26).

Cascade reference (short form): Forensic anchor (genericised): a consumer project wired its full local CI gate as an automatic git ‘pre-push’ hook (‘core.hooksPath’ → hook → multi-minute build/test/sweep stage on EVERY push). The result, captured 2026-07-26: routine pushes took many minutes each, and when one gate stage inside the hook failed or stalled, the push was rejected outright — the commit/push mechanism itself

became blocked, with pending commits stranded locally and no operator-usable path forward short of a forbidden ‘–no-verify’ bypass. The gate logic was sound; ... Propagation gate CM-COVENANT-114-234-PROPAGATION (literal 11.4.234) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.234 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

Cascaded anchor backfill — §11.4.235 (2026-07-28)

Continues the cascade above. The canonical constitution/Constitution.md advanced past §11.4.234 after that backfill closed: §11.4.235 was minted in constitution commit fce304d and its clause (B) extended in bbbdb5 (constitution HEAD at cascade time: 32d7578). This entry restores five-carrier lockstep (§11.4.157) for the new anchor. Form and authority are unchanged from the entries above — the compact cascade reference carries the literal anchor token the propagation gates match on, and the canonical text remains the sole authority (CONST-059 / §11.4.35).

§11.4.235 — Build-and-deploy the moment source fixes are proven-correct (test-side hardening is a parallel stage, never a build gate) + post-deploy version increment (operator mandate, 2026-07-27). Cascade reference (short form): (A) the moment a batch’s SOURCE fixes are done AND proven-correct — proven-correct = the mandatory independent SOURCE-correctness review has returned a clean GO (§11.4.125/§11.4.142/§11.4.194 on the source diff, on the §11.4.209 Fable-‘xhigh’ substrate, iterated to zero-finding/zero-warning GO per §11.4.134) — the build MUST be triggered, pre-build + post-build tests EXECUTED (§11.4.108 layer-1 before, layer-2 after), and the artifact deployed; gate-instrumentation hardening + new-gate authoring (§11.4.110), paired §1.1 mutations + four-layer §11.4.4(b) coverage + HelixQA Challenges, ‘gates and checks of all kinds’ (§11.4.96), and the re-review of THAT test-instrumentation work all run IN PARALLEL with the build AND the §11.4.185 manual-QA session, NEVER gating the build (§11.4.230(A)) — the build’s GO-SIGNAL is source-correctness alone, and holding the build sequentially for the test-instrumentation buildout when the source is already proven-correct is a §11.4.235(A) violation. (B) the moment image(s)/artifact(s) deploy the version code/version is INCREMENTED so every deployed artifact carries a distinct, monotonic, greppable id (§11.4.151/§11.4.108/§11.4.200; two deployments sharing one version id defeats per-deploy artifact identity), and — extended 2026-07-28 — EVERY deployment of a release FOR MANUAL QA TESTING (§11.4.185) is a DEVELOPMENT-CYCLE BOUNDARY that CLOSES the outgoing cycle and STARTS a new one at the NEXT monotonic version, ALWAYS and by standing default, so fixes for QA findings against the deployed artifact land in the NEW cycle on the NEW version id and are never patched onto the already-QA-deployed version id. (C) STANDING DEFAULT (§11.4.126/§11.4.230/

§11.4.103), never a per-request opt-in. Honest boundary (§11.4.6): this relocates the TEST-INSTRUMENTATION re-review + gate/check hardening OFF the build critical path and weakens NO substantive gate — the SOURCE-correctness review still PRECEDES and gates the build, pre/post-build tests still run at their §11.4.108 L1-2 seams, and §11.4.185 / §11.4.129 / §11.4.40 are un-weakened; ‘test-side work is parallel, never a build gate’ is bounded to future-regression instrumentation, because a check that proves the SHIPPED artifact wrong is a source/artifact-correctness gate (§11.4.108 L1-2) and stays on the critical path. Propagation gate CM- COVENANT - 114 - 235 - PROPAGATION (literal 11 . 4 . 235) + paired §1.1 mutation. **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.235 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker.

Cascaded anchor backfill — §11.4.236 + §11.4.237 (2026-08-08)

Continues the cascade above. The canonical constitution/Constitution.md advanced past §11.4.235 after that backfill closed: §11.4.236 was minted in constitution commit d594575 and §11.4.237 in dd3a8f0 (a deliberate re-mint that resolved an anchor-number collision with the already-published §11.4.236 — §11.4.227(B) satisfied: each anchor now heads exactly ONE block, verified as a ### block-opener in the canonical file, constitution HEAD at cascade time 5eb3f11). This entry restores five-carrier lockstep (§11.4.157) for the two new anchors. Form and authority are unchanged from the entries above — the compact cascade reference carries the literal anchor token the propagation gates match on, and the canonical text remains the sole authority (CONST-059 / §11.4.35).

§11.4.236 — QA-deploy-readiness gate: no manual-QA hand-off until the mandated validation produced a candidate-fingerprinted PASS verdict; a blocker MUST bind to a seam, never prose (research-derived, 2026-07-31).

Cascade reference (short form): a release artifact MUST NOT be deployed / handed off for manual QA (§11.4.185) until the mandated autonomous validation(s) for THAT artifact produced a machine-written PASS verdict whose artifact fingerprint == the candidate (§11.4.115(F), read FROM THE TARGET at run time); ABSENCE of the verdict BLOCKS the deploy exactly as a FAIL does (§11.4.135) — a not-yet-run validation and a failed one are both refusals, never a silent pass; the refusal lives in the deploy TOOL itself (the seam), never in prose an agent is trusted to honour (§11.4.205/§11.4.226) — a source-gate asserting the validation harness merely EXISTS is necessary-not-sufficient; any requirement an operator declares a blocker (a §11.4.140 CRITICAL, or an explicit “release blocker / QA-ready blocker”) MUST bind to a mechanical seam at the point it gates, never satisfied by prose and never DOWNGRADED to a follow-up, and when the blocker’s own INSTRUMENT bugs the response is §11.4.4/§11.4.120 (STOP, fix the instrument, complete the blocker), never route around it; the seam has NO silent bypass — an override is an

explicit, logged, operator-set flag (the §11.4.234/§11.4.135 recorded-deferral pattern); the gate is SELF-VALIDATED (§11.4.107(10)/§11.4.201) with golden-good (verdict present + PASS + fingerprint-match → allow) + golden-bad (absent → refuse / not-pass → refuse / fingerprint-mismatch → refuse) so it cannot itself false-allow, and the verdict-emitter reads the fingerprint from the target at run time, never a hardcoded per-release value (§11.4.111 — a version-pinned validator is the brittleness that lets a stale verdict pass). Honest boundary (§11.4.6): the seam proves a candidate-matching PASS verdict EXISTS for the mandated autonomous validation — it does NOT replace the §11.4.185 manual-QA sufficiency gate (the human check the deploy hands off TO), and it is only as honest as the validation whose verdict it consumes (that validation's own anti-bluff stays §11.4.5/§11.4.69/§11.4.107). Classification: universal (§11.4.17) — consumers supply the mandated-validation set, the verdict path/schema, and the deploy tool per §11.4.35.

Composes

§11.4.4/.107(10)/.108/.111/.115(F)/.120/.126/.135/.146(D3)/.185/.201/.205/.226/.234.

Propagation gate CM-COVENANT-114-236-PROPAGATION (literal 11.4.236) + recommended gate CM-QA-DEPLOY-READINESS-GATE + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227).

Canonical authority: constitution submodule [Constitution.md](#) §11.4.236 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --deploy-without-verdict, --prose-blocker-suffices, --downgrade-blocker-to-followup, --skip-readiness-gate, --verdict-fingerprint-optional, --silent-override flag.

§11.4.237 — Mandatory exhaustive context-and-spirit-aware translation review: every localized artifact independently reviewed for CONTEXT (source) + SPIRIT (target-language idiom/register) + technical/non-translatable fidelity — accuracy necessary, never sufficient (operator mandate, 2026-08-05). Cascade reference (short form): every translation and every translated artifact — UI strings, pages, documents, PDFs, manuals, marketing copy, captions/subtitles, help text, any localized content — MUST undergo a HEAVY, EXHAUSTIVE, INDEPENDENT review by a reviewer (agent or human) STRUCTURALLY SEPARATE from the translator that produced it (§11.4.70/§11.4.20 independence; the translator's own §11.4.92 multi-pass self-review PRECEDES it and NEVER satisfies it, per §11.4.142's no-self-review-substitution discipline applied to localization) — a localized artifact accepted without an independent context-and-spirit review is a §11.4 PASS-bluff at the localization layer. The reviewer holds and applies THREE dimensions: (a) FULL CONTEXT OF THE SOURCE MATERIAL — the original ideas, the subject/domain, the purpose, the intended audience; the reviewer reads the source MEANING, not merely the source STRING, so intent, nuance and tone are carried across, never lost; (b) THE SPIRIT OF THE TARGET LANGUAGE — its tradition, idiom, register, tone and cultural conventions — so the result reads NATIVE, natural, fluent and

MAXIMALLY POLISHED, never literal / word-for-word / calqued, a construction correct in the source that reads foreign in the target being REWRITTEN into the natural target-language form; (c) CORRECT TECHNICAL VOCABULARY AND VERBATIM PRESERVATION OF NON-TRANSLATABLE TERMS — domain/technical terms rendered with their established target-language equivalents, and non-translatable tokens (product names, brand names, proper nouns, technology/library/API names, code identifiers, trademarks) preserved VERBATIM, never “translated”, never accidentally transliterated, never localized away. The goal is a slick, polished, professional, native-reading localization: fidelity to the source is the FLOOR (accuracy NECESSARY), never the ceiling (accuracy NEVER SUFFICIENT) — a rendering that is accurate but stilted / foreign / register-wrong / idiom-broken is a FINDING, not a pass. Anti-bluff: the review emits captured, machine-readable POSITIVE evidence per §11.4.5/§11.4.69 (feature_class = translation_review) — a per-artifact verdict carrying a PER-DIMENSION rationale across the closed dimension set {fidelity, fluency/naturalness, spirit/register, terminology, completeness, non-translatable-preservation}; a PASS with no captured per-dimension rationale is a §11.4/§11.4.1 bluff, and a FAIL on ANY dimension LOOPS BACK to re-translation/polish and re-review, iterating to a zero-finding clean verdict per §11.4.134 (a single pass that “looked fine” never closes the artifact). Honest boundary (§11.4.6): this anchor REFINES the existing localization discipline from PASS/FAIL accuracy checking to CONTEXT-AND-SPIRIT native-polish review — it does NOT replace §11.4.140’s canonical translation pipeline nor §11.4.141’s independent per-language review (a KNOWN operator-owned §11.4.54 anchor-number collision on 11.4.140/11.4.141, cited here strictly in those senses, re-mint still operator-owned, introducing no new collision), it STRENGTHENS them (accuracy → native polish); it does not make a mistranslation impossible (a miscalibrated reviewer stays §11.4.107(10) golden-good/golden-bad fixture territory), and where a target language genuinely has no established term for a source concept that gap is an honestly recorded finding (§11.4.3 SKIP-with-reason), never a silently invented word. Classification: universal (§11.4.17) — no vendor/project/language literal; consumers supply their language set, term glossary, non-translatable token list, and reviewer substrate as DATA per §11.4.35. Composes §11.4.140/.141/.6/.69/.92/.134/.142/.209/§1.1. Propagation gate CM-COVENANT-114-237-PROPAGATION (literal 11.4.237) + recommended gate CM-TRANSLATION-CONTEXT-SPIRIT-REVIEW + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.237 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --skip-spirit-review, --accuracy-only-OK, --self-review-translation-OK, --literal-translation-OK, --translate-non-translatable-OK, --skip-per-dimension-rationale flag.

§11.4.238 — Automated QA must be the DISCOVERER: manual QA finds nothing new, and anything it finds is a coverage escape (User mandate, 2026-08-08). Verbatim operator mandate: *“Helix QA full websites testing MUST find all such issues, not us! Once we do manual QA we shall not be able to find anything! THIS IS MANDATORY RULE / CONSTRAIN / GOAL to achieve with this and any other project and MUST BE part of the root constitution!”* Forensic case study (FACT, 2026-08-08): in one cycle five real defects were found — a health endpoint returning `{"status": "healthy", "components": []}` that was structurally incapable of ever reporting unhealthy; a gateway routing every completion at a port nothing served (HTTP 500 on the product’s primary capability); two submodule pins on main that upstream answers with not our ref, so no fresh clone could check them out; and two hardware-profiler defects reading a key from a file where it does not exist. **Every one was found by an agent reading source or probing by hand; not one by the automated QA system**, which reported green throughout. The mandate (ALL hold): **(A) the automated QA system is the DISCOVERER, not the confirmer** — discovery by a human, an operator’s manual pass, an agent reading code, or an end-user report is a FAILURE of the QA system; the finding is still fixed, but the QA gap it exposes is a defect of equal standing and closing only the reported defect is the violation. **(B) manual QA must find NOTHING NEW** — by the time manual QA runs (§11.4.185) the automated regime must already have surfaced everything, so manual QA is a CONFIRMATION and sufficiency gate, never the discovery mechanism; the target is explicit and measurable — a manual pass discovering zero previously-unknown defects. **(C) every escape triggers the §11.4.138 loop, widened** to EVERY out-of-band channel: systematic-debug the defect (§11.4.102), produce a **coverage-escape audit** naming WHY the regime missed it cited to the specific missing check / unexercised path / absent assertion (never “we didn’t think of it”), register a new or strengthened automated check that WOULD have caught it into the standing regime (§11.4.135) with a §11.4.115 RED capturing the escaped defect, and commit the audit as captured evidence. **(D) coverage is measured by what the regime can CATCH, not what it runs** — every surface falsifiable per §1.1 (a check whose paired mutation does not make it fail is decoration), exercising real user-reachable paths end-to-end (§11.4.98 / §11.4.143 / §11.4.136 / §11.4.158 / §11.4.159 / §11.4.160). **(E) directional and measured** — track the discovery-channel split (automated vs out-of-band) and drive out-of-band toward zero; honest boundary (§11.4.6): this does NOT claim a regime can be provably complete (§11.4.118) and does NOT license closing an escape by narrowing the definition of “defect”. Composes §11.4.27 / §11.4.138 (trigger set widened) / §11.4.185 (manual QA becomes confirmation, never discovery) / §11.4.118 / §11.4.135 / §11.4.115 / §11.4.98 / §11.4.136 / §11.4.143 / §11.4.158 / §11.4.159 / §11.4.160 / §11.4.102 / §11.4.6 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its QA system, surface inventory, and discovery-channel ledger per §11.4.35. Propagation gate CM-COVENANT-114-238-PROPAGATION (literal 11.4.238) + recommended gates CM-QA-IS-THE-

DISCOVERER / CM-MANUAL-QA-FINDS-NOTHING-NEW + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227).

Canonical authority: constitution submodule [Constitution.md](#) §11.4.238. Non-compliance is a release blocker. No escape hatch — no --manual-qa-may-discover, --skip-coverage-escape-audit, --found-by-human-is-fine, --close-escape-without-new-check, --qa-green-is-enough, --discovery-channel-not-tracked flag.

CONST-051(A) — Project instantiation: the concrete equal-codebase roster (§11.4.35)

Operator mandate (2026-09-02): *“Make sure that all of our systems, sub-systems, submodules and everything we control in our organizations repos is treated equally as part of the project — work done on them all equally as on main repo module itself! Especially Helix Agent, LLM, Translate, QA, etc. Make sure that this fact is part of the local project’s constitution so it is absolutely clear.”*

The universal rule already exists: **CONST-051(A) / §11.4.28(A) equal-codebase**. This section is its **§11.4.35 project instantiation** — the concrete roster the universal rule requires the consuming project to supply, so “every owned submodule” is never an abstraction anyone has to guess at.

The rule, stated without hedging

Every repository listed below is **an equal part of HelixCode’s codebase — not a dependency, not a vendored blob, not somebody else’s problem**. Work performed on any of them carries **exactly** the same obligations as work on the main module:

- the same anti-bluff posture — every PASS carries captured runtime evidence (§11.4, §11.4.5, §11.4.123)
- the same test-type floor (§11.4.169) and four-layer coverage (§11.4.4(b))
- the same TDD discipline and coverage floor (§11.4.224)
- the same independent code review, iterated to a zero-finding GO (§11.4.125 / §11.4.134 / §11.4.142)
- the same documentation, user manuals and guides, kept in sync (§11.4.12 / §11.4.65)
- the same Challenges + HelixQA coverage (CONST-050(B))
- the same gates and paired §1.1 mutations
- the same release discipline — prefixed tags (§11.4.151), ff-only pushes, never force (§11.4.113)

A round of work that improves the main module while leaving an owned submodule’s equivalent deficiency unaddressed is a CONST-051(A) violation, regardless of how good the main-module work was. Coverage ledgers (§11.4.48) list every repository below as in-scope.

The roster — 118 owned repositories of 131 total (measured 2026-09-02)

HelixDevelopment (13) — the Helix family the operator named explicitly:

Path	Repository
submodules/helix_agent	HelixDevelopment/HelixAgent
submodules/helix_llm	HelixDevelopment/HelixLLM
submodules/helix_qa	HelixDevelopment/HelixQA
submodules/helix_memory	HelixDevelopment/HelixMemory
submodules/helix_specifier	HelixDevelopment/ HelixSpecifier
submodules/doc_processor	HelixDevelopment/DocProcessor
submodules/llm_orchestrator	HelixDevelopment/ LLMOrchestrator
submodules/llm_provider	HelixDevelopment/LLMProvider
submodules/vision_engine	HelixDevelopment/VisionEngine
submodules/debate_orchestrator	HelixDevelopment/ DebateOrchestrator
submodules/dag_orchestrator	HelixDevelopment/ DagOrchestrator
submodules/pipeline_runtime	HelixDevelopment/ PipelineRuntime
constitution	HelixDevelopment/constitution

HelixDevelopment-Code (1): github_pages_website → HelixDevelopment-Code/Welcome

vasic-digital (104): every submodule whose URL is git@github.com:vasic-digital/* — the submodules/* capability modules (containers, security, challenges, llms_verifier, storage, auth, database, cache, rag, memory, agentic, ...) and the cli_agents/caf-* fork family. Enumerate the live list with:

```
grep -E '^\s*(path|url)' .gitmodules | paste - - | grep -E  
'vasic-digital|HelixDevelopment'
```

Not on the roster — 13 third-party submodules are READ-ONLY. Any repository whose URL owner is neither vasic-digital nor HelixDevelopment* (llama.cpp, ollama, huggingface_hub, docling, skyvern, mcp servers, the cli_agents_resources/* reference corpora, ...) is vendored: we consume it, we do not modify it, and it is pinned at an upstream revision. Local modifications to these are drift to be reverted, never work to be committed.

Honest boundary (§11.4.6)

The operator's mandate named “**Helix Agent, LLM, Translate, QA**”. Agent, LLM and QA are on the roster above and verified present. **There is no helix_translate submodule in this project** — `grep helix_translate .gitmodules` returns 0 matches, and the only occurrence of that name anywhere in the governance corpus is as an illustrative example string inside the §11.4.151 release-prefix rule (`helix_translate-1.0.0-dev-0.0.1`). It is recorded here as **NOT INCORPORATED** rather than silently listed as though it existed. If Helix Translate is a real repository that belongs in this project, adding it is a §11.4.74 catalogue-check + submodule-incorporation task, and this roster must be updated in the same commit.

Roster maintenance: this list is data, and it drifts. Any commit that adds or removes an owned submodule **MUST** update this section in the same commit — a stale roster under-states the equal-codebase surface and is itself a CONST-051(A) defect.

Cascaded anchor backfill — §11.4.239–§11.4.272 (2026-09-08)

Continues the cascade above. The canonical constitution/Constitution.md advanced past §11.4.238 while every repo-root consumer carrier (CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md, CONSTITUTION.md) still topped out at §11.4.238 — a contiguous 34-anchor drift (§11.4.239 through §11.4.272 inclusive), measured 2026-09-08 against canon by counting BOTH canonical block-opener forms (### §11.4.N — ... and **§11.4.N — ...) per §11.4.227(B); canon defines 265 anchor blocks, the five carriers carried 237 each. This section restores five-carrier lockstep (§11.4.157) for the missing 34. Form and authority are unchanged from the entries above — each compact cascade reference carries the literal anchor token the propagation gates match on, and the canonical text remains the sole authority (CONST-059 / §11.4.35); where this index and canon disagree, canon wins.

Honest boundary on derivation (§11.4.6). Every entry below was derived by reading the canonical block in constitution/Constitution.md. All 34 anchors exist there as real block-openers — none was absent, and no entry below was manufactured to reach a count. Two limits are recorded rather than papered over: (1) the canonical blocks for §11.4.241 and §11.4.242–§11.4.254 are pointer-plus-summary form (“Full anchor per Phase 3 landing brief §11.4.N”), and the draft artifact §11.4.241 names —

curriculum_anchors_241_254_full_drafts.md — is NOT present anywhere in this checkout of the constitution submodule, so the summaries below derive strictly from the substantive text present in the canonical blocks themselves, never from the un-locatable draft; (2) canon’s own §11.4.239 CITATION NOTE records that the “extraction file line N” provenance offsets carried through §11.4.239–§11.4.254 are un-resolvable against the tracked corpus — that note is canon’s, is reproduced by reference here, and is not re-derived. Anchor numbers 11.4.64, 11.4.175, 11.4.203, 11.4.204, 11.4.205 and 11.4.206 remain as already documented in the sections above and were not touched by this backfill.

§11.4.239 — Critical-invariant work-class Definition of Done mandate — money/safety/availability/integrity code MUST include failure-path scenarios as gates, not merely happy-path passes (research-derived, 2026-08-15).

Cascade reference (short form): every project’s Definition of Done (§11.4.15/ §11.4.16 lifecycle + §11.4.33 closure vocabulary) MUST identify a closed CRITICAL-INVARIANT WORK CLASS — code paths where a single incorrect outcome is unrecoverable in the window between failure and remediation — and MUST require that a change touching that class carry EXPLICIT FAILURE-PATH SCENARIOS turned into passing tests BEFORE closure; a DoD stopping at “code compiles + happy-path test green” on such code is “a machine for generating reopened tickets on the paths that move money”. ALL hold: **(A)** four universal categories, consumer-extended as DATA per §11.4.35 — MONEY (credit/debit/transfer/hold/release/refund/reconcile), SAFETY (outputs actuating hardware / medical / transport / industrial controls), AVAILABILITY (the primary user-facing capability’s critical path), INTEGRITY (mutations of a persistent record whose corruption cannot be silently repaired); a region is IN the class when at least one category applies, and the four are NOT claimed exhaustive for every domain. **(B)** failure paths are FIRST-CLASS acceptance criteria enumerated BEFORE the code is authored, each a Given/When/Then scenario whose “Then” observes a real user-observable outcome (a ledger row, a settled charge, a state delta) rather than “no error” (§11.4.69); the reference MONEY minimum is FRESH / IDEMPOTENT RETRY / SAME-KEY-DIFFERENT-PAYLOAD / CRASH-IN-THE-MIDDLE (canon: “the two criteria juniors omit are same-key-different-payload ... and the crash-in-the-middle”), with analogous consumer-supplied minima for SAFETY, AVAILABILITY and INTEGRITY; an omitted scenario is a finding of §11.4/§11.4.1 severity, never a “we’ll add it if it bites us” deferral. **(C)** the precondition binds MECHANICALLY through the existing §11.4.146(D3) status-custody chain — a done/ready/closed-

status write against an item whose diff touches the class is REFUSED unless the registry row cites coverage of the declared scenario set, each scenario mapped to a §11.4.115(F) machine-written verdict pair carrying §11.4.226 class-matched evidence; an “N/A” scenario MUST cite a §11.4.6-honest reason, a bare “N/A” is treated as omission. **(D)** composes §11.4.108 (scenarios close on runtime-class evidence, never source-green), §11.4.129 (huge-blocker protocol governs a scenario-surfaced defect during release validation), §11.4.185 (manual QA unweakened — automation NECESSARY never sufficient) and §11.4.238 (a manual-QA-discovered failure-path defect is BOTH a coverage-escape audit and an acceptance-criteria audit). **(E)** honest boundary (§11.4.6) — DIRECTIONAL: it forbids the “happy path suffices” shape and mandates a domain-appropriate minimum, it does NOT claim exhaustive enumeration of every failure mode (§11.4.118 discovery-pressure territory); each escape becomes a permanent addition to the scenario set (§11.4.135). Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-239-PROPAGATION (literal 11.4.239) + recommended gate CM-CRITICAL-INVARIANT-FAILURE-PATH-DOD + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.239 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --happy-path-suffices-for-money, --skip-failure-scenarios, --defer-failure-path-tests, --criticality-not-applicable-without-reason, --n/a-without-justification, --closure-without-scenario-map flag.

§11.4.240 — Producer ≠Verifier scope separation: the actor that produces a change/artifact/verdict cannot be the one that gates it — least privilege over cleverness (research-derived, 2026-08-15). Cascade reference (short form): every project MUST enforce STRUCTURAL SEPARATION between the actor that PRODUCES a change/artifact/verdict and the actor that GATES it — the producer cannot write, weaken, delete or edit the acceptance tests, the oracle, the gate configuration, the verdict store, the paired §1.1 mutation, or any other check whose green result certifies the producer’s own work. ALL hold: **(A)** FOUR SCOPES that MUST resolve to distinct actors — PRODUCER (writes the diff/config/migration/artifact code), ORACLE (decides “correct output” per §11.4.245), GATE (decides pass/fail — CI config, gate scripts, §11.4.146(D3) custody enforcer, §11.4.135 release-verdict machinery), VERIFIER (re-runs the gate from a CLEAN CHECKOUT and attaches receipts per §11.4.165/§11.4.209/§11.4.194); canon: “the agent that wrote the code cannot be the authority that certifies it — self-preference bias and shared context make self-review worthless ... Remove any one property and it degrades to self-certification”; a four-scope collapse is maximum-severity, any two-scope collapse involving the producer is a finding of §11.4/§11.4.1 severity. **(B)** enforcement is by CAPABILITY, not INSTRUCTION — “least privilege over cleverness ... if it can’t touch the gate, it can’t game it”: filesystem scope excluding tests//oracle//CI-config/gate/verdict paths, branch +

review protection (§11.4.248 PROTECTED-SPEC + CODEOWNERS), verdict-store isolation (§11.4.115(F) harness writes, never the producer's code path), and execution isolation (fresh context, clean checkout, no inherited dirty worktree); “we told it in the prompt not to hardcode” is canon's named shallow/red-flag answer — exhortation is refused as the enforcement layer. **(C) THREE FORBIDDEN COLLAPSES** — PRODUCER=ORACLE (tautological testing; countered by authoring the oracle from spec / §11.4.243 characterization baseline / §11.4.245 derived reference), PRODUCER=GATE (silenced gates / weakened thresholds; countered by §11.4.248-protected paths + a non-producer approver), PRODUCER=VERIFIER (self-certification; countered by §11.4.115(F) machine-written fingerprint-keyed verdicts + the §11.4.135 release seam refusing producer-asserted verdicts). **(D)** composes the existing verifier layer (§11.4.115(F)/§11.4.125/§11.4.135/§11.4.142/§11.4.146(D3)/§11.4.165/§11.4.194/§11.4.209) — §11.4.240 ARTICULATES the property those mechanisms enforce and forbids the collapses that would make them decoration. **(E)** honest boundary (§11.4.6) — separation forbids the CLASS of self-certification; it does NOT prove the verifier correct (§11.4.107(10)), nor the oracle correct (§11.4.245), nor close every gaming vector (over-fitting to seen cases stays §11.4.245 held-out/property/metamorphic territory). **(F) INDEPENDENCE-TIER VOCABULARY + TIER-C TRIGGERS, DEGRADING HONESTLY (extension, 2026-08-26)** — every accepted verdict records its achieved basis of independence from the CLOSED SET {instance, model, capability}; model diversity is PERMITTED, NEVER REQUIRED (refusing a same-model independent verdict at an ordinary seam is itself the §11.4.201 false-positive-refusal class), and this does NOT weaken §11.4.256's different-model translation-review pin nor §11.4.209's Fable-xhigh code-review pin; Tier-C strength is triggered by (1) the producer holding write access to the evidence/verdict store, or (2) the release-tag / qa-deploy / manual-qa-handoff seams (where a producer-authored verdict is REFUSED regardless); outside those two triggers a separate agent INSTANCE MUST SUFFICE and capability separation MUST NOT be demanded; on a SINGLE-UID host capability separation cannot be established by any mechanism an agent can set up, so such a host reports instance HONESTLY and never capability — the field records what was ACHIEVED, never what was ASPIRED to, and honest degradation IS the compliant path. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-240-PROPAGATION (literal 11.4.240) + recommended gates CM-PRODUCER-NOT-VERIFIER-SCOPE-SPLIT / CM-LEAST-PRIVILEGE-CAPABILITY-NOT-INSTRUCTION / CM-INDEPENDENCE-TIER-HONESTLY-RECORDED + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.240 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --producer-may-write-tests, --self-certification-OK, --exhortation-suffices-as-enforcement, --producer-authored-verdict-accepted, --capability-limit-optional, --policy-document-is-enforcement, --single-actor-for-all-scopes flag.

§11.4.241 — Illegal-state-unrepresentability preference: types before API-shape before lint before property-test before runtime-assertion (research-derived, 2026-08-15). Cascade reference (short form): every load-bearing invariant a change relies on MUST land at the STRONGEST enforcement rung the target language/toolchain supports, on the closed ordered ladder (1) TYPE SYSTEM → (2) API SHAPE → (3) LINT → (4) PROPERTY-BASED TEST → (5) RUNTIME ASSERTION; a defect PREVENTABLE at rung N but caught only at rung > N is a finding. Rung-by-language capability baseline: strongly-typed languages reach rungs 1–5; dynamically-typed with gradual typing reach rung 1 conditionally; untyped scripting reaches rungs 3–5 unconditionally. Genuinely-limited-environment clarification: a POSIX-sh surface with only rung 5 SATISFIES the anchor IFF the runtime assertion GENUINELY CATCHES the invariant AND is paired with its §1.1 meta-test mutation per §11.4.107(10) self-validation. Composes §11.4.108 (pushes enforcement LEFT of Layer 1) / §11.4.115 / §11.4.142 / §11.4.194 / §11.4.201 / §11.4.224 / §11.4.6 / §11.4.35 / §1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-241-PROPAGATION (literal 11.4.241) + recommended gate CM-ILLEGAL-STATE-LADDER-PREFERENCE + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/ §11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.241 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35); the canonical block is pointer-plus-summary form naming a curriculum_anchors_241_254_full_drafts.md draft NOT present in this checkout, so this reference derives strictly from the in-canon summary (§11.4.6). Non-compliance is a release blocker. No escape hatch — no --runtime-assert-suffices, --skip-type-preference, --dynamic-language-exempts-all-rungs, --property-test-in-lieu-of-type, --decorative-type-annotation-OK flag.

§11.4.242 — Bisection-not-blame regression cause-finding + failure modes (research-derived, 2026-08-15). Cascade reference (short form): when a previously-working feature/behaviour regresses, the FIRST-choice localisation mechanism is BISECTION (git bisect or an equivalent monotone search, O(log N)) over the ordered commit range between the last known-good tag (§11.4.114) and current HEAD — NOT blame-log inspection, NOT speculative root-cause hunting. MONOTONE-SEARCH INVARIANT: bisection is valid only when the observable is monotone across the range (broken stays broken); a non-monotone observable (flaky, environment-dependent, timing-race) MUST be stabilised first, because bisection over a flaky signal produces a WRONG “guilty” commit with the same confidence as a correct one (a §11.4/§11.4.1 bluff at the localisation layer). CLOSED 6-CLASS FAILURE-MODE CATALOGUE: (1) flaky/non-monotone observable; (2) build-broken commits mistaken for “bad” (use bisect skip, exit 125); (3) merge-commit introductions (the defect is in the conflict resolution, in neither parent); (4) external/environmental changes; (5) delayed-fuse introductions (latent commit + later trigger); (6) test-infra regressions

masquerading as product regressions. A `git bisect run` script MUST honour exit-code discipline (0=good, 1–124=bad, 125=skip, 128+=abort) — mapping every non-zero to “bad” collapses the taxonomy. The bisection RESULT is captured evidence (§11.4.5/§11.4.69) feeding §11.4.102’s root-cause phase. **EXTENSION — DELTA DEBUGGING (2026-08-20):** `git bisect` IS delta debugging over commits, not the reverse; when a failure is not a monotone commit-range regression, or when a commit-bisection returns a change too large to explain it, the investigation MUST narrow along the discriminating axis — input, config, data, dependency or feature flag — to a MINIMAL failing case that becomes the §11.4.115 RED test’s input, under “a stable, fast, unambiguous test predicate”, running the predicate multiple times per step where the failure is noisy (§11.4.50/§11.4.248); minimisation yields the smallest input that still triggers the failure, which canon says “usually is the diagnosis”, NOT the root cause (that stays §11.4.102). Honest boundary (§11.4.6): bisection localises the INTRODUCING COMMIT of a MONOTONE regression; it does NOT prove that commit’s change is the ROOT CAUSE. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-242-PROPAGATION (literal 11.4.242) + recommended gates CM-BISECTION-BEFORE-BLAME / CM-DELTA-DEBUG-MINIMAL-CASE + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.242 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no `--blame-log-suffices`, `--skip-bisection`, `--flaky-observable-bisect-anyway`, `--run-script-exit-code-collapse-OK`, `--regression-guess-without-localisation` flag.

§11.4.243 — Characterization / golden-master safety net for legacy code before behaviour-changing modification (research-derived, 2026-08-15).

Cascade reference (short form): before ANY behaviour-changing modification to legacy code lacking characterization tests, the modifier MUST FIRST author CHARACTERIZATION TESTS asserting the CURRENT observable behaviour as GROUND TRUTH (whether “correct” or not) and land them as a green baseline; only AFTER the safety net exists may the modification proceed. The pattern originates in Feathers’s WORKING EFFECTIVELY WITH LEGACY CODE and generalises GOLDEN-MASTER (record outputs across a representative input space; assert future runs match). LEGACY SCOPE (§11.4.6-honest, any one suffices): (i) lacks tests asserting observable behaviour; (ii) written spec diverges from measured actual behaviour; (iii) predates the constitution’s TDD discipline (§11.4.224); (iv) the modifier cannot describe its exact behaviour from reading source within the time budget. **AUTHORING LOOP:** identify the observable surface → generate a representative input space → run and record actual outputs as the ground-truth baseline → assert the recorded values → commit the characterization tests as a SEPARATE commit BEFORE the behaviour-changing modification. **DISTINCTION FROM CORRECTNESS:** characterization is DESCRIPTIVE not

prescriptive — it captures WHAT IS, not WHAT SHOULD BE; where current behaviour is subtly wrong the test CAPTURES that wrongness and the modifier MUST notice and ASK the operator per §11.4.66 whether to preserve or correct it, never decide silently. Composes §11.4.115 (RED-first — characterization is its legacy counterpart) / §11.4.124 (investigate-before-remove, the parallel discipline) / §11.4.118 (a sampled input space is an honest gap, not completeness) / §11.4.201 / §11.4.224. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-243-PROPAGATION (literal 11.4.243) + recommended gate CM-CHARACTERIZATION-BEFORE-LEGACY-MODIFICATION + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227).

Canonical authority: constitution submodule [Constitution.md](#) §11.4.243 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --modify-untested-legacy-directly, --skip-characterization, --claim-tests-not-needed-because-obvious, --decorative-assertion-suffices, --refactor-without-safety-net flag.

§11.4.244 — Cross-boundary contract tests + can-i-deploy gate (research-derived, 2026-08-15). Cascade reference (short form): a change modifying a boundary between independently-deployable components (service↔consumer, library↔dependents, DB schema↔readers/writers, message topic↔subscribers) MUST be covered by CONSUMER-DRIVEN CONTRACT TESTS on BOTH sides, AND a can-i-deploy gate MUST verify the proposed provider version's compatibility against every currently-deployed consumer version BEFORE the change ships. §11.4.244 deliberately MERGES the two — they are useless in isolation: contract tests without the gate produce compatibility knowledge nobody consults, and a can-i-deploy gate without contract tests has nothing to check. CDC pattern: consumers publish the assumptions they make about the provider (Pact / Spring Cloud Contract / gRPC schema-evolution rules); providers verify each consumer's contract against the actual implementation; a provider change breaking a live consumer's contract FAILS THE PROVIDER'S BUILD before the change ships — “a precise red at the source”. CAN-I-DEPLOY: at every deploy attempt the gate queries the contract broker for the compatibility matrix; a single red cell BLOCKS the deploy with a machine-readable verdict naming the incompatible pair, NEVER warning-with-continue; a MISSING contract is a first-class refusal reason per §11.4.201 (a false-positive REFUSAL is a FAIL-bluff exactly as a false-negative allowance is a PASS-bluff). SemVer becomes MEASURABLE — a PATCH bump that breaks a contract is a mis-labelled version. The backward-compatibility window is declared per §11.4.35. Composes §11.4.5 / §11.4.6 / §11.4.35 / §11.4.66 / §11.4.108 (per-side four-layer) / §11.4.135 (contract tests as permanent regression guards) / §11.4.185 (composed-system manual QA still needed) / §11.4.201 / §1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-244-PROPAGATION (literal 11.4.244) + recommended gates CM-CONTRACT-TESTS-BOTH-SIDES / CM-CAN-I-DEPLOY-GATE-WIRED + paired §1.1 mutations (gate-code

= separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.244 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --skip-contract-tests, --deploy-without-compatibility-check, --assume-compatible-when-broker-silent, --patch-may-break-contract, --contract-tests-optional-for-internal-boundaries flag.

§11.4.245 — Oracle-problem-first test authoring: identify the oracle BEFORE authoring the test (research-derived, 2026-08-15). Cascade reference (short form): every test MUST FIRST identify its ORACLE — the mechanism deciding PASS vs FAIL for the input it drives — and that oracle MUST be DIFFERENT from the code under test AND from the test’s own assertion mechanism; authoring without explicit oracle identification produces the “test agrees with the code” anti-pattern (the test asserts what the code happens to produce, so it passes as long as the code produces something and never catches wrong-output — the most common form of §11.4/§11.4.1 bluff at the test-authoring layer). §11.4.245 does not SOLVE the oracle problem (the oldest unsolved problem in software testing); it mandates HONESTLY IDENTIFYING which oracle strategy each test uses so the §11.4.194 reviewer can judge whether that oracle is genuinely independent. CLOSED ORACLE-STRATEGY SET, named explicitly in each test’s source — (1) SPECIFIED (a spec/RFC/standard names the expected output); (2) DERIVED (an independent reference implementation); (3) METAMORPHIC (relations between related inputs — §11.4.107(8) generalised); (4) GOLDEN-MASTER / CHARACTERIZATION (past captured behaviour — §11.4.243); (5) INVARIANT (a universal property that must hold — sort output sorted AND a permutation of the input); (6) STATISTICAL (distribution properties for stochastic outputs, never a specific value); (7) HUMAN (§11.4.185 manual QA). NO other classes. STRUCTURAL INDEPENDENCE: the oracle’s SOURCE of the expected value must not depend on the code being tested. §11.4.107(10) SELF-VALIDATION generalised: the paired §1.1 mutation must produce a WRONG output and the oracle must catch it. Composes §11.4.5 / §11.4.6 / §11.4.107(10) / §11.4.115 / §11.4.194 / §11.4.201 / §11.4.224 / §11.4.243 / §1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-245-PROPAGATION (literal 11.4.245) + recommended gate CM-ORACLE-STRATEGY-NAMED-AND-INDEPENDENT + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.245 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --test-agrees-with-code, --skip-oracle-identification, --oracle-is-the-code, --assertion-is-oracle, --implicit-oracle-OK flag.

§11.4.246 — Reproducible + hermetic builds + supply-chain integrity at SLSA Build Level 2 (operator decision, 2026-08-15). Cascade reference (short form): every consuming project’s build pipeline MUST produce REPRODUCIBLE,

HERMETIC artifacts and MUST meet SLSA Build Level 2 as the FLEET-WIDE MINIMUM (operator decision 2026-08-15; Level 3+ tracked as backlog per §11.4.197). REPRODUCIBLE = the same source at the same time on any conformant host produces byte-identical artifacts, or a diff-set fully accounted for by declared non-determinisms pinned to canonical values. HERMETIC = no undeclared inputs — every dependency version-pinned and lockfile-hashed, every tool version declared, no unpinned public-network reach during the build, no host-environment leakage. SLSA-L2 = signed provenance describing source URI + commit hash + builder identity + invocation + artifact hash, produced on a hosted tamper-resistant build platform (the project's ancillary posture is already satisfied by SSH-only remotes §2.1, rootless podman §11.4.161, fixed-tag pins §11.4.30/§11.4.77). Every consumer MAINTAINS docs/security/SLSA_LEVEL.md (or its §11.4.35 equivalent) recording the CURRENT level with cited evidence, the TARGET level, and tracked §11.4.197 upgrade items — a CURRENT below L2 with no tracked item is a violation. SUPPLY-CHAIN EXTENSION: dependencies are either vendored (checked in, hash-verified), sourced from a provenance-attested registry, OR mirrored through a hash-pinning local registry. THREAT MODEL IS NARROW: it covers the source→artifact chain; it does NOT cover a malicious commit landing in source (§11.4.142 review), a compromised dependency source, or a bug-free artifact (§11.4.108 four-layer + §11.4.185 manual QA). Composes §2.1 / §11.4.6 / §11.4.30 / §11.4.35 / §11.4.108 / §11.4.113 / §11.4.135 / §11.4.142 / §11.4.161 / §11.4.184 / §11.4.185 / §11.4.197 / §11.4.209 / §1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-246-PROPAGATION (literal 11.4.246) + recommended gates CM-BUILD-REPRODUCIBILITY / CM-BUILD-HERMETICITY / CM-SLSA-L2-PROVENANCE-VERIFIED / CM-SUPPLY-CHAIN-DEPS-PROVENANCE-ATTESTED + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.246 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --skip-reproducibility, --allow-network-during-build, --no-provenance, --slsa-optional, --below-l2-OK, --unpinned-dependency-OK, --host-env-may-leak-into-build flag.

§11.4.247 — Composite-output layer-move must move every layer (research-derived, 2026-08-15). Cascade reference (short form): a change MOVING a system-boundary layer (a service migrating hosts, an endpoint URL change, an API version bump, a DB schema shift, a message-topic rename, an ingress path re-route, a storage-bucket move) MUST move EVERY LAYER of the composite output atomically or via an explicit compatibility window — NEVER piecemeal. LAYER TAXONOMY (§11.4.35 baseline, consumer-supplied concrete inventory): (i) server (handler + route + OpenAPI/proto/gRPC descriptor); (ii) client (call-sites + SDK stubs + integration tests); (iii) infrastructure (load balancer + ingress + API gateway + CDN + service mesh + DNS); (iv) data stores (cached keys + DB URL rows

+ MQ subscriptions); (v) observability (metric labels + span names + log fields + dashboards + alerts + SLOs); (vi) documentation (user + dev + runbook + README + cookbook); (vii) tests (integration + e2e + contract per §11.4.244 + load + smoke). The consuming project provides a `scripts/audit/layer_move_completeness.sh`-class residual-reference report; a change proposing a move MUST show ZERO residuals OR an explicit compatibility-window plan per residual with tracked §11.4.197 items. COMPATIBILITY WINDOW: the old value keeps working alongside the new until a stated date/version/event, deprecation warnings are named per caller, the deadline is tracked, and removal at expiry is clean — never “clean up later”. Canon records a MATERIAL-THIN + CITATION-AUDIT note: the phrases “composite-output”, “surface-swap-not-recreate” and “atomic move discipline” have ZERO occurrences in the tracked corpus and are this anchor’s OWN PARAPHRASE, not quoted source text; the module-32 DOMAIN attribution is plausible and unrefuted while the PHRASE-level attribution is UNVERIFIED (§11.4.6). Composes §11.4.6 / §11.4.35 / §11.4.111 / §11.4.118 / §11.4.124 / §11.4.135 / §11.4.191 / §11.4.197 / §11.4.238 (a residual IS a coverage escape) / §11.4.244 / §1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-247-PROPAGATION (literal 11.4.247) + recommended gate CM-LAYER-MOVE-COMPLETENESS-CHECK + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/ §11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.247 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no `--partial-layer-move-OK`, `--skip-completeness-check`, `--residuals-cleaned-up-later`, `--observability-layer-optional`, `--doc-move-can-lag` flag.

§11.4.248 — Flaky-test quarantine + protected-regression-spec gate via [PROTECTED-SPEC: ATM-NNN] tag + CODEOWNERS required reviewer (operator decision, 2026-08-15). Cascade reference (short form): flaky tests are the SILENT ANTI-BLUFF DECAY vector — every flake conditions the team to ignore red until a real regression is dismissed as “probably flaky, re-run”. Two disciplines bind: **(A) QUARANTINE** — mechanical detection (a test-history aggregator flagging a two-consecutive-run-different-verdict or a rerun-after-red pattern) moves the flaky test to `tests/quarantine/` (path per §11.4.35), opens a tracked §11.4.197 stabilisation deadline, and the blocking suite runs WITHOUT the quarantined test so green means green; a deadline past due is a §11.4.66 extension or a violation. **(B) PROTECTED-REGRESSION-SPEC** — a regression test codifying a known-broken defect fix (per the §11.4.135 permanent-guard suite) MAY be tagged # `[PROTECTED-SPEC: ATM-NNN]` (or the target language’s comment equivalent) referencing the §11.4.54 workable item it guards, marking it a load-bearing guard whose modification requires elevated review; the tag is applied DELIBERATELY per test — blanket-tagging voids the discipline. CODEOWNERS-REQUIRED-REVIEWER GATE: the consuming project’s CODEOWNERS declares that

modifications to [PROTECTED-SPEC]-tagged tests or a regression/**/* path scope require a designated required-reviewer approval BEFORE merge, composing §2.1 SSH + §11.4.113 no-force-push so the gate cannot be silently bypassed, with the audit trail in the review thread and commit history. The gate CATCHES the change so a reviewer can distinguish STRENGTHEN (approve) from FIX-DEFECT-IN-TEST (approve after verifying ATM-NNN's defect is not silently re-introduced) from WEAKEN/REMOVE (refuse, or escalate per §11.4.66) — it makes modification VISIBLE, not impossible. Composes §2.1 / §11.4.6 / §11.4.35 / §11.4.50 (an empty quarantine is the ideal) / §11.4.54 / §11.4.66 / §11.4.113 / §11.4.115 / §11.4.135 / §11.4.197 / §11.4.226 / §1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-248-PROPAGATION (literal 11.4.248) + recommended gates CM-FLAKY-TEST-QUARANTINE-WIRED / CM-PROTECTED-REGRESSION-SPEC-GATE + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.248 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --rerun-until-green, --quarantine-permanent, --protected-spec-review-optional, --codeowners-skip, --blanket-protect-everything, --tag-without-atm-id flag.

§11.4.249 — Producer ≠oracle ≠gate ≠verifier + flight recorder (research-derived, 2026-08-15). Cascade reference (short form): every load-bearing quality mechanism has FOUR distinct roles that MUST be structurally separate — (1) PRODUCER (the code computing/emitting/building the output); (2) ORACLE (the mechanism deciding correctness per §11.4.245); (3) GATE (the seam consuming the oracle's verdict to allow/block/proceed); (4) VERIFIER (the mechanism auditing the gate's decision — the paired §1.1 mutation proving the gate CAN refuse a bad output). Collapsing any two produces a specific bluff class: producer=oracle (the §11.4.245 oracle-agrees-with-code failure); oracle=gate (an oracle that cannot say “cannot decide” defaults to allow — the §11.4.201(6) FALSE-NULL); gate=verifier (a self-auditing gate — the §11.4.142 self-review anti-pattern applied). FLIGHT RECORDER: a mechanism CAPTURING producer inputs + oracle verdict + gate decision + verifier audit as a durable, replayable artifact (a structured JSONL event stream per §11.4.116, content-addressed per §11.4.207) so the post-hoc question “why did this go through” has evidence rather than a shrug; the recorder is NOT the gate — a gate reading its own recorder to decide collapses gate + verifier + recorder. DETECTABLE COLLAPSES the §11.4.194 reviewer scans for: (1) producer=oracle (the test's assertion calls the code being tested); (2) oracle=gate (no “cannot decide” branch, defaults PASS); (3) gate=verifier (the meta-test IS the gate — a mutation making the gate always-pass ALSO makes the meta-test always-pass; the meta-test MUST live OUTSIDE the gate); (4) recorder=decision-substrate (the gate uses recorder contents as a decision input — precedent-based gating is not authorised). Canon records a MATERIAL-THIN note: the four-role taxonomy is generalised from the producer/verifier separation + oracle-problem discussion +

gate mechanics, and the FLIGHT RECORDER is derived from the clean-checkout re-run + attached-receipts pattern per §11.4.17. Composes §11.4.5 / §11.4.6 / §11.4.107(10) / §11.4.116 / §11.4.142 / §11.4.194 / §11.4.201 / §11.4.207 / §11.4.240 (the separation PRINCIPLE — §11.4.249 is the four-role taxonomy that instantiates it; the two mutually compose) / §11.4.245 / §1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-249-PROPAGATION (literal 11.4.249) + recommended gates CM-ROLE-SEPARATION-DECLARED / CM-FLIGHT-RECORDER-CAPTURED + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.249 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --collapse-producer-oracle, --gate-self-audits, --skip-flight-recorder, --oracle-defaults-pass, --decision-without-recorded-rationale flag.

§11.4.250 — Heuristic-tower signals a primitive defect: when N heuristic layers stack to compensate, the primitive is broken (research-derived, 2026-08-15). Cascade reference (short form): a codebase accumulating a TOWER of heuristics all compensating for the same underlying issue (retry-with-backoff + connection-pool-warmup + DNS-cache-flush + health-check-with-startup-grace, all guarding one flaky DB connection) IS the diagnostic — the PRIMITIVE is defective and every layer papers over a manifestation without fixing the cause. RECOGNITION SIGNALS: (i) each layer's justification is "we added this because sometimes X happens" — reactive; (ii) removing any single layer INCREASES failures, yet keeping all of them still shows an occasional residual; (iii) no single layer owns responsibility; (iv) the layers were added at different times by different authors per specific incidents; (v) monotone layer-count growth over time — a debugging session for the residual adds layer N+1 rather than fixing 1..N. DISTINCT FROM DEFENSE-IN-DEPTH: defense-in-depth is deliberate, each layer targets a DIFFERENT failure class, the layers are designed together rather than accreted, and removing one produces a SPECIFIC different failure rather than "the same failure more often". SYSTEMATIC-DEBUGGING RESPONSE at tower depth ≥ 2: (1) STOP adding layers; (2) INVOKE §11.4.102 systematic-debugging on the PRIMITIVE; (3) work the root-cause / pattern / hypothesis / implementation phases; (4) fix the primitive; (5) REMOVE the layers one at a time, verifying each removal with §11.4.5 captured evidence; (6) commit each removal per §11.4.124 with git-history evidence citing the primitive fix. Composes §11.4.5 / §11.4.6 / §11.4.102 / §11.4.124 / §11.4.194 / §11.4.201 (an outer layer masks the §11.4.201(6) false-null) / §11.4.226 / §1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-250-PROPAGATION (literal 11.4.250) + recommended gate CM-HEURISTIC-TOWER-TRIGGERS-PRIMITIVE-DEBUG + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.250 — the full mandate text is canonical there and is inherited by reference (CONST-059 /

§11.4.35). Non-compliance is a release blocker. No escape hatch — no --add-another-heuristic-layer, --skip-primitive-debug, --tower-is-defense-in-depth-without-justification, --reactive-mitigation-OK, --layer-per-incident flag.

§11.4.251 — Byte-identical-fork prohibition + role-as-data-pack (research-derived, 2026-08-15). Cascade reference (short form): when a project's needs would be met by TWO NEAR-IDENTICAL CODE COPIES differing only in configuration / data / role-specific parameters (prod + staging with hardcoded hostnames; reader + writer with different flag values; web + mobile with different feature flags), those copies MUST NOT be maintained as parallel forks — ONE code copy is retained and the role-specific values are extracted into a DATA PACK (role-config file / env-var set / capability descriptor / feature-flag bundle) that the one code consumes to instantiate as either role; canon's remedy phrasing: "roles become data-only packs you reference, never repos you fork". DETECTION: a reviewer or mechanical scan finds a byte-identical fork when diff shows every difference is a configuration value (hostname/port/flag/label/env/credential-ref) rather than behavioural, a common git-history ancestor exists, and byte identity exceeds the project's §11.4.35 threshold (canon's illustrative figure: > 90%). EXTRACTION: (1) IDENTIFY the configuration surface; (2) EXTRACT it to a per-role data pack; (3) REFACTOR the single code copy to consume it at instantiation (constructor injection / factory / config-loader-at-startup); (4) DELETE one fork; (5) VERIFY per §11.4.5 that both role instantiations produce the same observable behaviour their forks previously produced (§11.4.243 characterization before refactor); (6) commit the extraction per §11.4.124 with git-history evidence. LEGITIMATE FORKS: a temporary migration (tracked §11.4.197 end date), an experimental / A-B fork (bounded end date), or genuinely divergent behaviour (copies differing in algorithm / protocol / data model, not merely config). Composes §11.4.5 / §11.4.6 / §11.4.28 / §11.4.35 / §11.4.124 / §11.4.177 / §11.4.194 / §11.4.197 / §11.4.243 / §1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-251-PROPAGATION (literal 11.4.251) + recommended gate CM-NO-BYTE-IDENTICAL-FORK + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.251 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --fork-for-role-OK, --skip-data-pack-extraction, --copy-and-modify-config, --duplicate-code-per-environment, --near-identical-forks-are-fine flag.

§11.4.252 — Fail-closed-on-dangerous-combination for mutating / credential surfaces (research-derived, 2026-08-15). Cascade reference (short form): every code path COMBINING ≥ 2 dangerous capabilities MUST FAIL CLOSED — refusing the operation unless every capability's precondition is verifiably satisfied — and MUST NEVER fail open on ambiguous, missing or unresolvable input; this is the security-engineering generalisation of §11.4.201 guard-honesty applied where a

false negative's blast radius dwarfs the operator inconvenience of a false positive. DANGEROUS-COMBINATION TAXONOMY (canon records this 6-class set as a §11.4.17 closed-set formalisation of the fail-closed pattern's shape, MATERIAL-THIN rather than a literal per-class enumeration): (1) MUTATION of a shared resource (DB / filesystem / container state / config); (2) UNTRUSTED INPUT (user string / network bytes / external env var / unverified file); (3) CREDENTIAL ACCESS (secret read / authenticated API call / signing-key use); (4) EXTERNAL SIDE EFFECT (email / webhook / MQ publish / payment endpoint); (5) SHELL / EXEC (subshell / binary with user-influenced args / template eval over user values); (6) IRREVERSIBLE (delete / drop / truncate — force-push already being absolutely forbidden per §11.4.113). Any path combining ≥ 2 is a dangerous combination and MUST fail closed. THE INVARIANT: (1) VERIFY every precondition (authorization + input shape + credential presence + target identity); (2) REFUSE when ANY is unverifiable, with an ERROR EXIT naming the specific unresolved precondition; (3) EMIT captured evidence of the refusal per §11.4.5; (4) NEVER default to allow / proceed / retry-with-relaxed on ambiguity — the §11.4.101 conservative-safe default is REFUSE. FAIL-OPEN ANTI-PATTERNS refused by name: `catch { /* ignore */ }, credential = credential || default, if (!valid) { log("warn"); return proceed(input); }, target = user_input || default_target, and an unbounded retry { dangerous(); }`. Distinct from graceful degradation (a cache miss returning a cache-miss response is legitimate and is not a dangerous combination). Composes §11.4.5 / §11.4.6 / §11.4.10 (the credentials specialisation) / §11.4.66 / §11.4.101 / §11.4.113 (its maximal instantiation) / §11.4.194 / §11.4.201 / §1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-252-PROPAGATION (literal 11.4.252) + recommended gate CM-DANGEROUS-COMBINATION-FAIL-CLOSED + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.252 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no `--fail-open-on-error`, `--swallow-exception`, `--default-missing-credential`, `--proceed-on-ambiguous-input`, `--retry-until-succeeds` flag.

§11.4.253 — Idempotency under retry + DB-level durable uniqueness guard (research-derived, 2026-08-15). Cascade reference (short form): every retryable operation (network calls with auto-retry, at-least-once MQ consumers, cron jobs with overlapping schedules, HTTP endpoints with client retry, DB transactions with retry-on-serialisation-failure, deploy pipelines with rerun capability) MUST BE IDEMPOTENT — N executions produce the same observable end state as one — and idempotency MUST be ENFORCED at the LOWEST LAYER where “same end state” is durably provable, typically a DATABASE UNIQUENESS CONSTRAINT on a caller-supplied idempotency key, NEVER only an application-layer check-then-insert (which races). This prevents the DOUBLE-CHARGE / DOUBLE-EMAIL /

DOUBLE-DELETE / DOUBLE-EFFECT class. IDEMPOTENCY-KEY DISCIPLINE: every retryable operation accepts (or generates and passes forward) a unique identifier naming THIS logical operation; the persistence layer carries a UNIQUE INDEX / CONSTRAINT on the key column; a second attempt is REFUSED by the DB at the durable level rather than by application logic reading-then-writing; the caller catches that refusal as “duplicate — already succeeded” and treats it as SUCCESS. ANTI-PATTERNS: (i) check-then-insert without a unique index; (ii) an app-level dedup table read outside the spanning transaction; (iii) a key scoped to one app instance (in-memory, not shared); (iv) the effect persisted BEFORE the key (a crash between them double-effects on retry); (v) no key at all plus “the effect is ‘set X to V’ so redoing is fine” — safe for pure sets, NOT for INSERTs, MQ emissions or external-system side effects; (vi) an expiring idempotency key, which converts the operation to time-bounded-idempotent and is a §11.4.6 mis-labelling if the caller expects unbounded. CHAOS VERIFICATION per §11.4.85: (1) SIMULTANEOUS RETRIES (same key from two concurrent callers); (2) MID-OPERATION CRASHES (kill the receiver between “check exists” and “insert effect”); (3) RETRY AFTER APPARENT SUCCESS (the caller believes it failed, the receiver persisted, and the retry MUST recognise the duplicate). For external side effects the key MUST pass through to the external system; where the external system does not support one, the operation is NOT fully idempotent and MUST be labelled honestly per §11.4.6. Composes §11.4.5 / §11.4.6 / §11.4.85 / §11.4.201 / §11.4.252 / §1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-253-PROPAGATION (literal 11.4.253) + recommended gates CM-IDEMPOTENCY-DB-LEVEL-UNIQUE-GUARD / CM-IDEMPOTENCY-CHAOS-VERIFIED + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.253 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --check-then-insert-OK, --application-dedup-suffices, --skip-idempotency-chaos-test, --claim-idempotent-without-key, --expire-idempotency-marker-silently flag.

§11.4.254 — Boot-time invariant assertion + capability matrix (research-derived, 2026-08-15). Cascade reference (short form): every service / process / long-running executable MUST assert its BOOT-TIME INVARIANTS at startup — required env vars present and valid; required config files parseable with a matching declared schema version; required dependencies reachable; required credentials present and unexpired; required schema migrations applied; required feature-flag states declared; required host resources present (disk / memory / thread limits per §12.12 / port availability) — and MUST FAIL FAST AND LOUD if any is violated, BEFORE accepting a request or performing work. INVARIANT CLASSES: (i) environment, (ii) configuration, (iii) dependencies, (iv) credentials, (v) schema, (vi) capabilities, (vii) host. FAIL-FAST-LOUD: (1) EMIT a structured error to stderr plus a captured log naming the specific failed invariant; (2) EXIT non-

zero (traditionally 78 EX_CONFIG for configuration; the consumer supplies its code per §11.4.35); (3) DO NOT accept requests, begin the main event loop, or bind the listening port. A service that boots partially and serves requests-that-will-fail is WORSE than one refusing to start — a partial boot masquerades as healthy long enough for a load balancer to route traffic to it; this is the dual of §11.4.252's fail-closed-on-dangerous-combination. CAPABILITY MATRIX: the consumer's docs/services/CAPABILITY_MATRIX.md (per §11.4.35) declares, per service, (i) its boot-time invariants, (ii) the runtime capabilities it OFFERS, (iii) the runtime capabilities it REQUIRES from other services; it is regenerated and verified on every commit per §11.4.106 docs-chain + §11.4.86 fingerprint. An undeclared endpoint emitted is a violation, and a silent upstream dependency not declared is a violation (it is invisible to §11.4.244 contract testing). Composes §11.4.5 / §11.4.6 / §11.4.35 / §11.4.86 / §11.4.106 / §11.4.108 Layer 3 (boot IS the runtime-on-clean-target check) / §11.4.128 / §11.4.244 / §11.4.252 / §12.12 / §1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-254-PROPAGATION (literal 11.4.254) + recommended gates CM-BOOT-TIME-INVARIANT-ASSERTED / CM-CAPABILITY-MATRIX-PRESENT-AND-IN-SYNC + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.254 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --skip-boot-time-assertion, --boot-partial-OK, --default-missing-env-silently, --capability-matrix-optional, --service-with-undeclared-dependency-OK flag.

§11.4.255 — Mandatory HelixTranslate canonical translation pipeline (User mandate, 2026-06-25). Cascade reference (short form): ALL translations of ANY content — UI strings, documents, articles, CVs, READMEs, marketing copy, in-app messages, release notes, legal text, ANY human-language string rendered in a non-source language — MUST be produced by **HelixTranslate** (git@github.com:HelixDevelopment/HelixTranslate.git) driven through the canonical universal translation pipeline shipped in the constitution submodule at scripts/translation/ (translate-pipeline.sh + render-articles.sh); a hard, non-negotiable constraint. FORBIDDEN and constituting a violation: (a) HAND-TRANSLATION — no human-authored, manually edited or “good enough” hand-written translation of any content, ever; the engine is the single source of every translated string; (b) OTHER ENGINES — no Google Translate, DeepL, raw ChatGPT/Claude prompting, OS translation APIs, library i18n auto-translators, or any engine other than HelixTranslate's unified-translator; there is exactly one engine; (c) SILENT FALLBACK — no quiet substitution of the source string, no empty/partial output passed off as a translation, no swallowed engine error, no absence-of-error PASS (§11.4/§11.4.1); on exhaustion of all configured providers the pipeline MUST FAIL LOUD (non-zero exit, destination NOT written), a missing translation being a hard error never a

degraded-but-shipped string, while provider-to-provider failover WITHIN HelixTranslate (with logged, evidenced retries) is NOT a silent fallback and is permitted. MANDATORY VALIDATION: every generated translation is validated for accuracy with real captured evidence per §11.4.5/§11.4.69/§11.4.107 — the per-file run log (provider used, byte counts, retries, success/failure) IS the evidence artifact, and an unvalidated or unevidenced translation is treated as ABSENT. EXPLICIT PROVIDER ROUTING: routing MUST be explicit and recorded, never implicit — the canonical routing is provider groq / model llama-3.3-70b-versatile with documented failover to provider mistral / model mistral-large-latest, the actual producing provider recorded per file, any deviation stated explicitly in configuration and logged. The engine binary is supplied by the consuming project via HELIX_TRANSLATE_BIN. Composes §11.4 / §11.4.1 / §11.4.5 / §11.4.6 / §11.4.7 / §11.4.69 / §11.4.107. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-255-PROPAGATION (literal 11.4.255) + recommended gate CM-MANDATORY-HELIXTRANSLATE-PIPELINE + paired §1.1 meta-test mutation (gate-code = separate work item). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.255 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --allow-hand-translation, --other-engine-OK, --skip-translation-validation, --silent-fallback flag.

§11.4.256 — Mandatory independent per-language translation review (User mandate, 2026-06-25). Cascade reference (short form): every translated artifact produced under §11.4.255 MUST, before it is shipped / committed / deployed, be INDEPENDENTLY REVIEWED for quality by a per-language review agent — a hard, non-negotiable gate that composes with and NEVER replaces the §11.4.255 generation mandate. Requirements: (a) INDEPENDENCE — the reviewer MUST be a different model/provider than the one that produced the translation (e.g. translator mistral/mistral-large-latest → reviewer groq/llama-3.3-70b-versatile, or any distinct verified model); a model reviewing its own output is NOT an independent review and is forbidden; (b) PER-LANGUAGE SPECIALIZATION — the reviewer acts as a NATIVE speaker / linguistic reviewer of the SPECIFIC target language, evaluating that language's grammar, orthography and correct script (Cyrillic for ru/be/kk, Arabic for ar/fa, Han/Kana/Hangul/Devanagari for zh/ja/ko/hi, Latin for sr/de/es/fr/tr, etc.); (c) FULL CRITERIA — each review judges accuracy/fidelity, fluency/naturalness, completeness (nothing omitted or invented), correct script and orthography, and the absence of untranslated source leftovers (proper nouns / URLs / code excepted); (d) STRICT MACHINE-READABLE VERDICT + EVIDENCE — a structured PASS/FAIL verdict with per-criterion scores and an issues list, persisted as a physical evidence artifact (§11.4.5/§11.4.69/§11.4.107); a FAIL — or an ERROR / unavailable-reviewer — BLOCKS the content and only a real PASS may ship, with no metadata-only / absence-of-error PASS (§11.4/§11.4.1) and no bluff (§11.4.6); (e) REMEDIATION

LOOP — a FAIL routes the artifact back through the §11.4.255 pipeline (re-translate / multipass-polish) and is re-reviewed, iterating until PASS, NEVER hand-patched to pass. Canonical tooling: `scripts/translation/review-translations.sh` + `scripts/translation/review_translation.py`, reusable verbatim, with the consuming project supplying its source/translation file pairs and a reviewer provider DISTINCT from its translator. Composes §11.4.255 (the translation it gates) / §11.4 / §11.4.1 / §11.4.5 / §11.4.6 / §11.4.69 / §11.4.107 / §11.4.134. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-256-PROPAGATION (literal 11.4.256) + recommended gate CM-MANDATORY-TRANSLATION-REVIEW + paired §1.1 meta-test mutation (gate-code = separate work item). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.256 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no `--skip-translation-review`, `--self-review-OK`, `--accept-unreviewed-translation` flag.

§11.4.257 — Comprehensive user-manual + guide + FAQ coverage: every component / service / feature / user-visible workflow ships an operator-usable manual, task-oriented guide, and FAQ, always in sync, no orphan capability (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, 2026-08-15). Cascade reference (short form): every consuming project MUST cover the WHOLE project (every component, service, subsystem, binary, container, plugin, feature and user-visible workflow) with three cooperating documentation classes — (a) a complete USER MANUAL per component/service/product, (b) task-oriented GUIDES for every user-facing workflow, (c) properly-created FAQs derived from real operator/QA/end-user questions — always in sync with the shipped code, always exported per §11.4.65, always reachable from README per §11.4.212, always regenerated via Docs-Chain per §11.4.106, and tracked as §11.4.197 workable items when incomplete; a capability that ships without its trio is a violation of §11.4 PASS-bluff severity at the documentation layer (the feature ships to a user who cannot use it because nothing tells them how). ALL hold: **(A)** no orphan capability — every shipped surface maps to ≥ 1 manual, ≥ 1 guide per workflow it exposes, and one FAQ entry per real recurring question; an undocumented shipped capability is UNSHIPABLE regardless of how well the code works. **(B)** THREE DISTINCT CLASSES, none substituting another — a MANUAL is the operator reference (what it is, inputs/outputs/env/config/ports/dependencies, start/stop/status, logs/metrics/health, failure modes and remediation, complete enough to run + supervise + debug without reading source); a GUIDE is workflow-shaped (prerequisites, step-by-step commands, expected output per step, verification checkpoints, rollback path); an FAQ is built from real recurring signal (operator questions, QA reports, end-user tickets, agent-found-defect narratives), NEVER a fabricated marketing FAQ, each entry citing its question-source class + resolution + a link to the owning manual/guide section. **(C)** ALWAYS IN SYNC — §11.4.44 revision header, Docs-Chain bound at the commit /

build / constitution-pull write-seams so an edited component with stale docs REFUSES the commit, §11.4.65 export, §11.4.212 reachability; FAQ entries describing now-fixed defects are RETAINED with a “resolved in vN.N.N” note, never silently deleted. **(D)** missing docs are §11.4.197 items, with per-surface docs-status rows in the §11.4.93/§11.4.95 workable-items DB so “what is documented” is a live query. **(E)** HONESTY — a genuine in-flight gap is marked [DOC - PENDING: <tracker-id>] per §11.4.223 at both the README entry point and the component Status doc, NEVER silently omitted and NEVER faked with a placeholder paragraph that says nothing (a fake doc is a §11.4 PASS-bluff — the operator reads it, learns nothing, and believes they read the doc). Honest boundary (§11.4.6): the anchor mandates the docs EXIST, are IN SYNC and are REACHABLE; it does NOT verify semantic correctness (that stays §11.4.99 / §11.4.194 / §11.4.238), and an internal implementation detail with no user-facing exposure carries no user-manual obligation. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-257-PROPAGATION (literal 11.4.257) + recommended gates CM-USER-MANUAL-PER-COMPONENT / CM-GUIDE-PER-USER-WORKFLOW / CM-FAQ-FROM-REAL-QUESTIONS + paired §1.1 meta-test mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.257 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --skip-user-manual, --guide-later, --faq-optional, --capability-without-docs-OK, --fake-doc-placeholder-OK, --source-of-truth-is-code-comments flag.

§11.4.258 — Architectural + data-flow + state-machine + sequence-diagram coverage: every project ships accurate, exported, machine-derivable-or-honestly-authored diagrams embedded into the docs that describe the system (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, 2026-08-15). Cascade reference (short form): every consuming project MUST cover the WHOLE project with properly created graphs / diagrams / schemes across FOUR canonical classes — (a) SYSTEM ARCHITECTURE (services, containers, ports, dependencies, deployment topology, kept true to the actual compose file / deployment manifest / running system); (b) DATA FLOW (how a user request, event, credential, artifact or evidence traverses source → transformation → sink, each edge annotated with format + protocol + trust boundary); (c) STATE MACHINE (the lifecycle of every stateful entity — workable items §11.4.15, long-ops §11.4.232, orchestrator sessions §11.4.187, agents §11.4.147, deploy artifacts §11.4.108, builds — with transitions labelled by triggering event and gating guard); (d) SEQUENCE (one per critical user workflow and per critical cross-component protocol, with actors + messages + return paths + FAILURE branches, not only the happy path). ALL hold: **(B)** INCORPORATION, never orphan — each diagram lives at its point of use (architecture in README + docs/ARCHITECTURE.md; per-service data-flow embedded in that service’s §11.4.257

manual; state-machine embedded in the doc that owns the state; sequence in the guide for the workflow it depicts); a docs/diagrams/ gallery may exist as an INDEX, but the load-bearing appearance is EMBEDDED where the reader needs it (§11.4.212). **(C) OPEN + DIFFABLE** — sources in diffable text (Mermaid / PlantUML / D2 / diffable SVG), NEVER a proprietary-tool-only master (§11.4.220); rendered outputs committed alongside sources; renderers §11.4.201-probed before use, an absent tool yielding an honest §11.4.3 SKIP-with-reason and never a fake output (§11.4.222). **(D) ACCURACY VERIFICATION** — every diagram rendered non-blank, non-degenerate and accurate, with a probe render (a Mermaid parse-check, an SVG geometry check, a §11.4.107(10) golden-good vs golden-bad-with-drift fixture for derived diagrams); a diagram whose depicted service list, port map or state set diverges from its SSoT is caught by a §11.4.86 fingerprint diff plus a stale-diagram audit at pre-build, and Docs-Chain (§11.4.106) refuses a commit that changes an SSoT without the mapped diagram refresh. **(E) DERIVED-WHERE-POSSIBLE** — prefer generated diagrams (compose-file-driven architecture, schema-driven state machine, route-table-driven sequence); hand-drawn diagrams are LABELED as such with an authored date and reconciled against the SSoT each release round. A rendered blank / mislabelled / truncated diagram is a §11.4/§11.4.1 bluff at the visual-doc layer. Honest boundary (§11.4.6): the anchor does NOT claim every class applies to every project — an internal library with no runtime deployment ships its module-dependency diagram in the ARCHITECTURE class's place. §11.4.258 STRENGTHENS §11.4.218 and §11.4.219 from design-side coverage into whole-project documentation-side diagram coverage. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-258-PROPAGATION (literal 11.4.258) + recommended gates CM-ARCHITECTURE-DIAGRAM-PRESENT-ACCURATE / CM-DATAFLOW-DIAGRAM-PER-SERVICE / CM-STATEMACHINE-DIAGRAM-PER-STATEFUL-ENTITY / CM-SEQUENCE-DIAGRAM-PER-CRITICAL-WORKFLOW / CM-DIAGRAM-EMBEDDED-NOT-ORPHAN + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.258 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --skip-architecture-diagram, --diagram-in-orphan-directory, --proprietary-master-only, --stale-diagram-OK, --dataflow-implied-from-code, --sequence-later flag.

§11.4.259 — README quality-status badge row: at the top of every project README, a comprehensive row of quality badges with a closed green→amber→red vocabulary, each backed by machine-derived source data, illustrating quality and production-readiness in one glance (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, 2026-08-15). Cascade reference (short form): every consuming project MUST render, at the TOP of the main README (immediately below the H1, above the introduction — the reader's first visual signal), a comprehensive BADGE ROW carrying every applicable quality

signal with a CLOSED colour vocabulary, plus a production-readiness gauge summarising how close the project is to full production deployment, with EVERY badge backed by MACHINE-DERIVED source data; a README lacking the row, or carrying badges whose colours are fabricated / stale / not backed by real source data, is a violation of §11.4 PASS-bluff severity at the reader-first-impression layer. ALL hold: **(A) CLOSED COLOUR VOCABULARY** — GREEN (healthy at target), AMBER (attention: on-target but degrading, off-target within an acceptable band, or blocked on a tracked follow-up), RED (off-target beyond the band, a §11.4.108 gate failing, a §11.4.135 uncovered guard, an open release-blocker), optional GRAY (N/A, with an inline reason); NO other colours or verdicts. **(B) BADGE-CLASS FLOOR** (additions welcome as §11.4.35 DATA, subtraction is not) — build status; test-breadth per §11.4.27/§11.4.169; code coverage per §11.4.224; security posture per §11.4.184 + supply-chain per §11.4.246; documentation completeness per §11.4.257; diagram completeness per §11.4.258; live health; open defects per §11.4.15 with most-reopened highlighted per §11.4.189; supply-chain integrity level per §11.4.246; zero-shortcomings status per §11.4.261; machine-evidence coverage per §11.4.262; and the production-readiness gauge. **(C) MACHINE-DERIVED** — every badge's colour and value is COMPUTED from a live source of truth, never hand-typed, with the exact source path/query/build id embedded in the tooltip or a companion docs/BADGES.md provenance table; a badge whose provenance is hand - typed fails. **(D) PRODUCTION-READINESS GAUGE** — GREEN only when every clause-(B) badge is GREEN AND no §11.4.185 manual-QA blocker is open AND no §11.4.260 criterion is unmet; AMBER on any amber without red or blocker; RED on any red or open blocker; its colour is BINDING on the release decision. **(E) ALWAYS IN SYNC** — badges regenerate on every state change via §11.4.106 at the same write-seams as the rest of the doc surface; a stale badge is a §11.4.229 in-sync violation. **(F) SELF-VALIDATED** — the badge-computer is a guard (§11.4.201): golden-GREEN renders GREEN, golden-RED renders RED, golden-AMBER renders AMBER; a computer that PASSES its golden-RED fixture is itself the bluff and a release blocker. Honest boundary (§11.4.6): green badges are necessary-not-sufficient and do not prove the project bug-free; an HONESTLY red row is compliant; a genuinely inapplicable class is GRAY-labelled with a reason, never omitted (silence-as-badge is a §11.4.201(6) FALSE-NULL). Classification: universal (§11.4.17) — palette, at-top placement, closed vocabulary, machine-derivation and the gauge are universal; the specific per-project badge palette is consumer §11.4.35 DATA. Propagation gate CM-COVENANT-114-259-PROPAGATION (literal 11.4.259) + recommended gates CM-README-BADGE-ROW-AT-TOP / CM-BADGE-CLOSED-COLOR-VOCABULARY / CM-BADGE-MACHINE-DERIVED-SOURCE / CM-PRODUCTION-READINESS-GAUGE / CM-BADGE-SELF-VALIDATED + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.259 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --badges-optional, --hand-type-color, --skip-production-gauge, --

badge-not-derived-from-source, --silent-omit-red-badge, --badge-in-footer-OK flag.

§11.4.260 — Cutting-edge enterprise quality + production-deployment readiness: every work product is built for production deployment, maximal stability, zero nasty surprises, and enterprise-grade robustness — never for “good enough” or “will do for now” (BACKGROUND :: REMINDER ::

IMPORTANT operator mandate, 2026-08-15). Cascade reference (short form):

every work product a consuming project ships **MUST** be built for CUTTING-EDGE ENTERPRISE QUALITY targeted at PRODUCTION DEPLOYMENT + MAXIMAL STABILITY + ZERO NASTY SURPRISES — the standing default posture engaged automatically for every batch, subagent, feature, fix and change from the first prompt of the session (§11.4.126 + §11.4.198), never a per-request opt-in and never downgraded to “good-enough” / “will-do-for-now” / “MVP-first”. ALL hold: **(A)** CUTTING-EDGE ≠ NOVELTY-FOR-ITS-OWN-SAKE — it means the current best-known engineering approach for the problem class, sourced via §11.4.99 latest-source verification (never memory) + §11.4.150 deep multi-angle research + §11.4.8 external precedent, applied fully; never a shortcut, stub, placeholder-with-a-TODO, “for now” (§11.4.124), duct-tape workaround or “will fix later” (§11.4.197); a genuinely unavailable best-known approach is **HONESTLY** tracked (§11.4.197 item + [OPEN] marker per §11.4.223), never absorbed as “we did our best”. **(B)** the PRODUCTION-DEPLOYMENT-READINESS INVARIANT SET — every shipped product satisfies ALL of: (1) FUNCTIONAL COMPLETENESS (every stated capability wired + verified on a clean target per §11.4.108, no unwired feature, no dead-code path §11.4.124); (2) OBSERVABILITY (health endpoint, structured logs, per-operation metrics, distributed-trace-ready surface, evidence capture at every gate — §11.4.85 + §11.4.128); (3) RESILIENCE (real failure-injection per §11.4.85, recovery VERIFIED not assumed); (4) SECURITY (§11.4.184 scan, no leaked credentials §11.4.10, supply-chain at the fleet minimum §11.4.246); (5) PERFORMANCE (measured under sustained load per §11.4.85 within declared SLOs); (6) MAINTAINABILITY (code review passed at Fable-xhigh §11.4.209 with zero findings and zero warnings, iterated to GO per §11.4.134); (7) DEPLOYABILITY (rootless container §11.4.161, deterministic reproducible artifact §11.4.108 layer 2, deployment verified via §11.4.200 target-isolation + §11.4.108 layer 3 runtime signature); (8) DOCUMENTABILITY (§11.4.257 + §11.4.258 + §11.4.259 all GREEN); (9) TESTABILITY (§11.4.27/§11.4.169 breadth + §11.4.224 coverage floor + a paired \$1.1 mutation per gate + permanent guards §11.4.135); (10) OPERATIONAL RUNBOOK (a documented remediation path per non-trivial failure mode, §11.4.257 guide class). **(C)** ZERO NASTY SURPRISES — no known-broken behaviour ships (§11.4.238), no failing gate silenced (§11.4.120/§11.4.201), no test disabled without a tracked follow-up (§11.4.197), no closed-with-workaround defect masquerading as fixed (§11.4.34), no “fails sometimes but rarely” tolerance (§11.4.50 + §11.4.85), no try/catch/swallow silent-failure paths. **(D)** STANDING DEFAULT (§11.4.126/§11.4.198/§11.4.183/§11.4.231), engaged

from the first prompt, never negotiated. **(E) docs/PRODUCTION_READINESS.md** — a per-component × invariant × evidence-source tracker (§11.4.153 sibling) with a §11.4.44 revision header, auto-synced via §11.4.106, reachable from README per §11.4.212, and the source of truth for the §11.4.259 gauge. **EXTENSION to (B)(2) OBSERVABILITY (2026-08-20)** — naming a property is not a mechanism: (a) THREE COMPLEMENTARY SIGNALS each answering a different question — structured logs (queryable key-value JSON events, not prose strings), metrics (numeric time series), traces (a tree of timed spans) — plus crash/ANR analytics as a fourth signal class (canon explicitly does NOT use the labels “observability triad” or “three pillars”, and consumers MUST NOT attribute them to it); (b) a CORRELATION ID threading all signals that MUST SURVIVE ASYNC BOUNDARIES (“I make sure the correlation ID survives the queue ... or I lose the thread exactly where the hard bugs live”), pivoting metric → trace → log under “narrow before you theorize”; (c) CRASH-FREE USERS/SESSIONS as a release gate, with a crash signature NEW at a release boundary blocking regardless of volume (“Volume tells me priority; novelty at a release boundary tells me causation”) — binding §11.4.152 and §11.4.265; THRESHOLDS ARE CONSUMER DATA — canon states no numeric crash-free threshold and cautions such figures change by version, so the gate’s existence is mandated and no value is. Honest boundary (§11.4.6): the anchor mandates the POSTURE + INVARIANT SET + TRACKER; it does NOT claim any project is provably bug-free (§11.4.118 remains, §11.4.238 escapes trigger a coverage audit), does NOT license shipping with unmet invariants (an unmet invariant is honestly red-badged and blocks release), and canon does NOT assert “a system you cannot observe in production cannot be verified in production” — what it licenses is narrower: observability makes an unreproducible defect DIAGNOSABLE, it is not itself a verification of correctness (that stays §11.4.108 / §11.4.185 / §11.4.262). Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-260-PROPAGATION (literal 11.4.260) + recommended gates CM-PRODUCTION-READINESS-TRACKER-PRESENT / CM-CUTTING-EDGE-POSTURE-ALWAYS-ON / CM-ZERO-NASTY-SURPRISES-AUDIT / CM-CORRELATION-ID-THREADED-ACROSS-BOUNDARIES / CM-CRASH-FREE-RELEASE-GATE + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/ §11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.260 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --mvp-quality-OK, --good-enough, --will-fix-later, --skip-production-readiness-tracker, --production-readiness-optional, --acceptable-flakiness, --for-now-implementation flag.

§11.4.261 — Zero-shortcomings / zero-gaps / zero-weak-spots / zero-danger-zones invariant with a mechanical audit ratchet: at all times the project holds exactly zero of these; every finding is closed OR tracked-with-honest-mitigation, and the audit-run count of open findings is MONOTONE-DECREASING (BACKGROUND :: REMINDER :: IMPORTANT operator mandate,

2026-08-15). Cascade reference (short form): every consuming project MUST hold, at ALL TIMES, exactly ZERO shortcomings, gaps, weak spots or danger zones across every code path, service, component, gate, doc and configuration — a standing invariant, not a per-release checklist — enforced by a MECHANICAL AUDIT SCRIPT that inventories every finding class from a closed vocabulary, runs at every pre-build + release + constitution-pull seam, produces a machine-readable ledger, and is bound to a MONOTONE-DECREASING ratchet (§11.4.135 pattern). ALL hold: **(A)** CLOSED FINDING VOCABULARY (extensible via named consumer additions per §11.4.35, never subtractable): (1) SHORTCOMINGS (implementation gaps behind a working contract); (2) GAPS (missing capabilities the docs/schema/API promise); (3) WEAK SPOTS (paths without adequate guards / error handling / input validation); (4) DANGER ZONES (patterns known to cause serious harm under conditions); (5) TODOs / FIXMEs / “for now” / “temporary” / “will do later” / “placeholder” / NotImplementedError / stub-returns / dead code (§11.4.124); (6) SKIPPED tests / disabled gates / commented-out assertions without a tracked # SKIP-OK: <ticket> per §11.4.3; (7) BLUFFS (any §11.4/§11.4.1 covenant violation surface); (8) UNRESOLVED §11.4.197 items past their stated completion window; (9) DIVERGENT / STALE / ORPHAN artifacts per §11.4.233; (10) UN-CATALOGUED anti-patterns found in review that lack a §1.1 mutation. **(B)** MECHANICAL AUDIT SWEEP — a single executable script iterating the vocabulary and emitting a ledger {finding_id, class, file, line, description, status ∈ {open|tracked|closed}, tracker_ref, mitigation_evidence}, append-only + snapshot per §11.4.116, git-tracked per §11.4.95 so the ledger IS the SSoT; each class ships golden-good/golden-bad fixtures per §11.4.107(10) so the audit itself cannot bluff. **(C)** MONOTONE-DECREASING RATCHET — a brownfield snapshot records the accepted open-finding count per class and total (operator §11.4.66 decision per the §11.4.224(E) adoption fence, never an invented ratchet); every later run MUST be ≤ the snapshot per class; a run that increases any class REFUSES the seam; the ratchet may DECREASE and MUST NEVER INCREASE. **(D)** EVERY FINDING IS CLOSED OR TRACKED, never silently absorbed — CLOSED requires captured evidence at the defect layer per §11.4.226 with the finding disappearing from the sweep; TRACKED requires a §11.4.197 item with a §11.4.148 comprehensive description, an honest [OPEN: ATM-NNN] §11.4.223 marker at the finding site, a §11.4.101 reversible-safe mitigation, the item’s completion on the standing autonomous-loop queue, and a §11.4.135 permanent guard where the exposure could cause harm before closure. **(E)** HONEST “ZERO” — it means the closed-vocabulary audit finds zero unclosed-untracked entries; it does NOT claim omniscience (§11.4.118), which is exactly why the vocabulary is EXTENSIBLE; a §11.4.238 escape triggers a class extension plus a ratchet snapshot bump DOWN (never up), and narrowing the vocabulary to declare “zero” is a §11.4.6 fabrication and a release blocker. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-261-PROPAGATION (literal 11.4.261) + recommended gates CM-ZERO-FINDINGS-AUDIT-SWEEP / CM-ZERO-FINDINGS-MONOTONE-RATCHET / CM-EVERY-FINDING-

CLOSED-OR-TRACKED + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.261 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --skip-findings-audit, --allow-todo-without-tracker, --ratchet-may-increase, --silently-absorb-finding, --narrow-vocabulary-to-declare-zero, --for-now-in-production flag.

§11.4.262 — Machine-created evidence at every gate: every claim of “works” / “passes” / “verified” cites a captured, machine-derived, machine-verifiable evidence artifact produced by the gate — no operator eyeballing, no narrative-only PASS (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, 2026-08-15). Cascade reference (short form): every claim by the System that something WORKS / PASSES / IS VERIFIED / IS PRODUCTION-READY MUST be backed by MACHINE-CREATED, MACHINE-VERIFIABLE evidence — a file / verdict / measurement produced by the gate itself, cited by path + content-hash + timestamp, with a schema an independent verifier can parse; operator eyeballing, human narrative, “I checked it works”, “looks fine”, “no error was reported”, and every other prediction / guess / assumption about success is FORBIDDEN at the evidence layer (§11.4/§11.4.1/§11.4.6). ALL hold: **(A) THREE-GATE COVERAGE**, each producing machine-created evidence — (1) CONFIRMATION (the change lands correctly: source review, static analysis, format/lint, dependency-clash check per §11.4.110); (2) VALIDATION (the change behaves as designed at each of the §11.4.108 four layers, each with its own verdict + artifact); (3) VERIFICATION (the change satisfies its acceptance contract — §11.4.185 manual QA is the human sufficiency gate, §11.4.236 QA-deploy-readiness the machine one, each with its own captured evidence). **(B) MACHINE-CREATED — FOUR PROPERTIES:** (i) PRODUCED BY THE GATE, not authored by the tester (a screenshot the harness took, not one the operator narrated); (ii) MACHINE-VERIFIABLE (a schema-parseable JSON verdict, JSONL event stream, structured log, an image with an OCR/vision oracle per §11.4.107(11)/§11.4.117, audio with RMS/loudness analysis) an INDEPENDENT verifier can re-check without asking the tester; (iii) CONTENT-ADDRESSED (path + sha256 + timestamp per §11.4.207 so tampering is detectable); (iv) SEMANTIC-CLASS-MATCHED (the evidence class matches the claim layer — a source-grep transcript can never satisfy a “pixel renders green” claim, §11.4.226). **(C) NO NARRATIVE-ONLY PASS** — a PASS whose evidence is a paragraph of natural language authored by an operator or subagent is a violation; the narrative may ACCOMPANY the machine evidence but MUST NEVER substitute for it, and §11.4.185 manual QA’s OWN evidence is likewise machine-created (a §11.4.128 session recording, a checklist file with per-item PASS/FAIL + evidence path, a §11.4.238 escape log). **(D) SELF-VALIDATED EVIDENCE ANALYZERS** — every parser that reads an artifact and emits a verdict is golden-good + golden-bad + negative-control validated per §11.4.107(10); an analyzer that PASSES its golden-

bad fixture is the bluff and a release blocker. **(E) FOUR-LAYER BINDING WITH §11.4.108** — each layer’s PASS cites evidence PRODUCED AT THAT LAYER (source: static-analysis JSON + review artifact; artifact: build log + fingerprint checksum + a byte-check that the change’s bytes landed; runtime: on-target verdict + captured runtime signature + telemetry; user-visible: OCR/vision oracle output or a sink-side probe report per §11.4.13/§11.4.69/§11.4.107); a higher-layer PASS citing only lower-layer evidence is a §11.4.226 wrong-layer violation — the chain runs THROUGH the layers, never SKIPS. **(F) STANDING DEFAULT (§11.4.126/§11.4.198)** from the first prompt, with every dispatch / review / test run / gate emitting its artifact into the §11.4.207 content-addressed evidence store; a status report predicting “should pass” without citing a run’s evidence is a bluff at the reporting layer. Honest boundary (§11.4.6): machine evidence does not prove human satisfaction (§11.4.185 remains), and does not license shipping when the evidence disagrees with reality (a missed bug is a §11.4.238 escape triggering a coverage audit and a §11.4.261 vocabulary extension, not a “the machine said green” defence). Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-262-PROPAGATION (literal 11.4.262) + recommended gates CM-MACHINE-EVIDENCE-AT-EVERY-GATE / CM-EVIDENCE-ANALYZER-SELF-VALIDATED / CM-NO-NARRATIVE-ONLY-PASS / CM-EVIDENCE-LAYER-MATCH-108 + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.262 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --narrative-pass-OK, --operator-eyeball, --should-pass-suffices, --skip-evidence-analyzer-validation, --evidence-optional, --pass-without-machine-artifact, --predict-success flag.

§11.4.263 — Process-group signal-safety mandate: NEVER signal pgid ≤ 1, NEVER trust a mock-derived pid/pgid — validate as int > 1 before every killpg / kill(-pid, sig) / pkill / killall call (BOB-126 forensic anchor, 2026-08-19). Cascade reference (short form): SEVEN forced-logout incidents on the operator’s workstation over ~48h were traced by kernel audit trail to a single Python test — an AsyncMock().pid left unset returned the MagicMock.__int__ default of 1, so os.getpgid(1) == 1 and os.killpg(1, SIGKILL) reached glibc as kill(-1, SIGKILL), SIGKILLing every UID-1000 process (the systemd user@1000 manager, the desktop shell, tmux, ssh, browsers, the agent session) — with contextlib.suppress(Exception) swallowing nothing, because the syscall SUCCEEDED. ALL hold: **(A)** NEVER signal a pgid ≤ 1 in any language — killpg(1, sig) == kill(-1, sig) == the disaster syscall on Linux/BSD/macOS. **(B)** VALIDATE isinstance(pid, int) and pid > 1 AND isinstance(pgid, int) and pgid > 1 before every process-group signal call (Python os.killpg, Go syscall.Kill(-pgid, sig), Rust nix killpg, Bash kill -<pgid> / pkill -g, C killpg / kill(-pid, sig)). **(C)** TESTS mocking subprocess/proc objects

MUST explicitly set `mock.pid = <int>` (never relying on mock-library defaults), MUST patch the `killpg` call itself, and MUST assert `killpg` was called with `pgid > 1`. **(D)** DEFENSE-IN-DEPTH four-layer coverage per §11.4.4(b) — a pre-build `grep` gate, a post-build CodeGraph query, a runtime kernel audit rule (`syscall=kill a0=0xffffffff a1=9`), and a paired §1.1 mutation regression test. **(E)** honest boundary — this hardens process-group signal APIs; it does NOT prohibit legitimate `start_new_session=True` child-owned groups (the guard remains cheap), and it does NOT prevent the kernel OOM-killer or `systemd-oomd` (§12.6 + §11.4.225 apply there). **(F)** STANDING DEFAULT from the first prompt — every code review scans for `killpg` calls and verifies the guard, every subagent writing subprocess-cleanup code includes it, and every test-writing subagent sets an explicit `mock.pid`. Classification: universal (§11.4.17). Composes §11.4.1 / §11.4.4(b) / §11.4.5 / §11.4.6 / §11.4.85 / §11.4.107(10) / §11.4.115 / §11.4.126 / §11.4.201 / §11.4.225 / §11.4.226. Propagation gate CM-COVENANT-114-263-PROPAGATION (literal 11.4.263) + recommended gates CM-KILLPG-PGID-GUARD / CM-TEST-MOCK-PID-EXPLICIT-INT + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.263 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-killpg-guard`, `--mock-pid-optional`, `--killpg-1-is-ok`, `--kill-minus-1-permitted`, `--suppress-catches-syscall`, `--test-may-not-explicit-pid` flag.

§11.4.264 — Build once, promote ONE immutable content-addressed artifact through every environment; never rebuild per stage (research-derived, 2026-08-20). Cascade reference (short form): exactly ONE artifact is produced per release candidate and THAT artifact is promoted through every environment; rebuilding per stage silently BREAKS THE CHAIN OF EVIDENCE, because every gate that ran against the staging artifact measured a DIFFERENT binary than the one production runs, making each of those green verdicts a §11.4 PASS-bluff about the shipped artifact — evidence correctly captured against the wrong subject; canon’s red-flag form: *“We rebuild for production to be safe.”* ALL hold: **(A)** BUILD ONCE per release candidate under §11.4.246 (reproducible + hermetic + SLSA-L2); determinism is what makes a rebuild DETECTABLY equivalent, never a licence to rebuild instead of promote. **(B)** IDENTITY IS A CONTENT ADDRESS, NOT A TAG — a `sha256:... digest` or platform equivalent, never a mutable tag/branch/version string alone (“tags get re-pushed; digests cannot change without changing the digest”); the attack defended is *“the same version, different bytes”*; composes §11.4.111 and §11.4.200 (identity read BACK from the intended target). **(C)** PROMOTION IS A VERIFICATION, NOT A REBUILD — “verify this exact digest is authentic and approved for the next environment”, not “make a new thing and hope it’s equivalent”; signature, SBOM and provenance TRAVEL WITH the bytes (§11.4.246). **(D)** ENVIRONMENT CONFIG IS INJECTED AT DEPLOY, NEVER BAKED

AT BUILD — the artifact is environment-agnostic; baking per-environment config re-creates the per-stage rebuild it forbids, and §11.4.10 still forbids baking a secret. **(E)** ROLLBACK IS DETERMINISTIC BY CONSEQUENCE — point at the previous known-good digest, “not a rebuild-from-source scramble under incident pressure” — the precondition that makes §11.4.265’s auto-abort a real undo. **(F)** REBUILD-TO-DODGE IS GATE-EVASION, NOT A RETRY — re-running or rebuilding until a failing scan turns green “launders a real failure into a pass”; retries are counted and surfaced (§11.4.248). Honest boundary (§11.4.6): this guarantees the tested bytes are the shipped bytes; it does NOT prove them correct (§11.4.108/ §11.4.40/§11.4.185), does NOT prove the deploy reached the intended target (§11.4.200), and does NOT prove the injected configuration right (§11.4.254) — it closes exactly one link. Classification: universal (§11.4.17) — the consumer supplies its environment ladder, digest scheme and config-injection mechanism as DATA per §11.4.35. Propagation gate CM-COVENANT-114-264-PROPAGATION (literal 11.4.264) + recommended gates CM-BUILD-ONCE-PROMOTE-DIGEST / CM-DEPLOY-CONFIG-INJECTED-NOT-BAKED + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.264 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker regardless of context. No escape hatch — no --rebuild-per-environment, --promote-by-tag, --bake-env-config-at-build, --rebuild-until-green, --skip-digest-pin, --deterministic-build-means-rebuild-is-safe flag.

§11.4.265 — Progressive delivery: traffic-shifted rollout gated by automated analysis on SLO and business metrics, with automatic abort to stable (research-derived, 2026-08-20). Cascade reference (short form): a release reaches users by a bounded-blast-radius strategy, never an ungated cutover, and progressive delivery is the FIRST gate whose oracle is real user traffic — the deploy-layer instantiation of the §11.4 covenant, catching the defect the suite never encoded at a blast radius of a small traffic fraction. ALL hold: **(A)** the consumer declares its strategy per §11.4.35 from canon’s two named forms — CANARY (gradual weighted traffic shift with evaluation pauses; smallest blast radius; old and new run concurrently so schema/compat must tolerate it) or BLUE-GREEN (full parallel stand-up, one-shot cutover, a rollback window; “at the switch, all users hit the new version simultaneously”); for a §11.4.239 critical-invariant surface canon prefers canary paired with blue-green’s instant-undo posture. **(B)** THE PROMOTE/ABORT DECISION IS AUTOMATED, NOT A HUMAN WATCHING A DASHBOARD — a controller compares the canary against the stable baseline over successive analysis windows and auto-promotes or auto-aborts and rolls all traffic back to stable, “no human needed”; canon explicitly rejects the manual approval click (“Manual LGTM is a rubber stamp with no receipt; under agents it looks like oversight while providing none”), composing §11.4.240 and §11.4.262. **(C)** THE LOAD-BEARING CLAUSE — THE ANALYSIS MUST INCLUDE

BUSINESS METRICS, NOT ONLY INFRASTRUCTURE METRICS; canon names infra-only measurement “the classic mistake”: “‘HTTP error rate is fine’ does not tell you the checkout-completion rate quietly dropped 2% ... the failures that matter most are business failures that look technically green”, so an infra-only canary is a §11.4 PASS-bluff wearing a rollout’s uniform. **(D) MULTIPLE CONSECUTIVE ANALYSIS WINDOWS**, never a single sample (§11.4.50/§11.4.107). **(E) POST-DEPLOY SMOKE IS A FAIL-CLOSED PRECONDITION OF TRAFFIC** (§11.4.252/§11.4.254) — it proves wiring and configuration, and a failed smoke gate aborts the promotion before any user is affected. **(F) THE DEPLOY GATE IS INDEPENDENT OF THE MERGE GATE** — “‘got it merged’ is not ‘got it shipped’”. **(G) THRESHOLDS AND WINDOW COUNTS ARE CONSUMER DATA, NEVER CONSTITUTIONAL LITERALS** — canon writes its abort criterion with UNFILLED placeholders (“below X% or p99 exceeds Y ms ... thresholds illustrative — set from your SLOs”) and warns “A release gate that quotes a stale requirement is itself a bluff”. Honest boundary (§11.4.6): progressive delivery bounds blast radius and automates the undo; it does NOT prove the release correct, does NOT replace §11.4.185 manual QA, and does NOT apply where no traffic-shiftable surface exists (an honest §11.4.3 SKIP-with-reason, never a fabricated canary). Canon’s progressive delivery is exclusively traffic-shifting — it is silent on feature flags, dark launch, kill switches and shadow traffic, none of which is mandated, and it contains no “a deploy you cannot roll back is not a deploy” claim. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-265-PROPAGATION (literal 11.4.265) + recommended gates CM-PROGRESSIVE-DELIVERY-STRATEGY-DECLARED / CM-CANARY-ANALYSIS-INCLUDES-BUSINESS-METRIC + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority**: constitution submodule [Constitution.md](#) §11.4.265 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker regardless of context. No escape hatch — no --full-cutover-ok, --infra-metrics-suffice, --manual-approval-is-the-gate, --single-window-analysis, --skip-smoke-before-traffic, --merge-implies-deploy flag.

§11.4.266 — Claim-vs-reality ledger: every ADVERTISED capability is a row, typed from the closed bluff-type vocabulary, and a capability with no passing challenge is a release blocker (research-derived, 2026-08-20). Cascade reference (short form): canon names this the first and cheapest of its portfolio gates — “every advertised capability is a row with a Bluff type and a severity, and a capability with no passing challenge is a release blocker”, making stubbed-core, byte-identical-fork and config-present-but-unwired visible and blocking rather than discovered later. THE DIRECTION OF ENUMERATION IS THE POINT and is what no existing anchor supplies: §11.4.25’s coverage ledger, §11.4.52’s autonomous-path column and §11.4.169’s test-type column are all keyed on FEATURES the project knows it has, so a CLAIM with no feature row is invisible to a feature-keyed ledger by construction — §11.4.266 adds a keying, not a

restatement (§11.4.227). ALL hold: **(A)** ONE LEDGER PER REPOSITORY, enumerated from the CLAIM side; the consumer declares per §11.4.35 which surfaces constitute “advertised” (at minimum the README capability set required reachable by §11.4.212, the §11.4.257 manual/guide/FAQ set, the §11.4.254 capability matrix, and the §11.4.259 badge assertions). **(B)** EACH ROW CARRIES A BLUFF TYPE FROM THE CLOSED SEVEN-MEMBER VOCABULARY plus a severity — green-but-broken (“passes CI, fails on device”), coverage-theater (“the metric lives in prose or a badge, not a failing gate”), rubber-stamp-verified (“‘verified’ is set, never earned”), stubbed-core (“the most-engineered code has no caller”), doc-vs-code-drift (“a countable claim in prose is wrong”), config-present-but-unwired (“the flagship path silently no-ops while the right code exists one dir over”), byte-identical-fork (“the name promises X, the code is Y unchanged”); canon states NO severity scale, so the scale is consumer DATA. **(C)** THE TYPE IS A ROUTING KEY TO A COUNTER-GATE — “it turns ‘another bug’ into ‘another instance of type T’, which is what lets you install one gate that kills the whole type” — each type binding to the anchor that already owns its counter (§11.4.108/§11.4.262, §1.1/§11.4.224(C), §11.4.146(D3)/§11.4.240/§11.4.249, §11.4.27/§11.4.254, §11.4.106/§11.4.186/§11.4.227, §11.4.124/§11.4.196(F), §11.4.251); §11.4.266 makes the ROUTING mandatory, it does not restate the counters. **(D)** THE RELEASE-BLOCKING RULE — a row whose challenge is absent, never executed, or executed only against a stale artifact fingerprint blocks exactly as a FAILING challenge does (§11.4.135/§11.4.236 absence-blocks-as-FAIL applied to the claim side); this is why the ledger is a gate, not a document. **(E)** THE ESCAPE HALF is §11.4.238’s, cited not duplicated. PROVENANCE CORRECTION recorded per §11.4.6: secondary project documentation has called this material an “FP-1 ... FP-14”, “L-1 ... L-16” or “F-1 ... F-31” catalogue; measured against the tracked corpus NO F-n, FP-n or L-n identifier exists in any of the nine modules (control-needle proven), so those labels MUST NOT be cited as corpus wording — the real structures are a 14-row numbered failure-pattern table, a 6-bullet portfolio-gate list, and this 7-member bluff-type vocabulary. Honest boundary (§11.4.6): the ledger proves every ADVERTISED capability has a passing challenge; it does NOT prove the challenge strong (§11.4.245/§11.4.107(10)/§1.1), the capability bug-free (§11.4.118), or enumerate capabilities nobody advertised — claim-keyed and feature-keyed ledgers are complements, and running one leaves an honestly-stated blind side. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-266-PROPAGATION (literal 11.4.266) + recommended gates CM-CLAIM-REALITY-LEDGER-COMPLETE / CM-UNCHALLENGED-CAPABILITY-BLOCKS-RELEASE / CM-LEDGER-ROW-TYPED-FROM-CLOSED-VOCABULARY + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.266 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker regardless of context. No escape hatch — no --ledger-optional, --unchallenged-capability-ok,

--untyped-ledger-row, --invent-bluff-type, --feature-ledger-suffices, --advertise-without-row flag.

§11.4.267 — Shared attempt record + converge-on-evidence / escalate-on-stall: a failed approach is never silently retried, and a non-converging loop is itself a signal (research-derived, 2026-08-20). Cascade reference (short form): canon — “No memory of what was tried → the loop repeats itself. Agents re-attempt discarded approaches. Fix: a shared record of attempts and outcomes (in the task state / PR thread), so a failed approach is not silently retried; and an escalation rule — after N failed iterations, stop and route to a human or a stronger model, rather than burning cycles”; the premise that makes it structural is that “the agent has no memory of prior attempts ... it will re-implement the same wrong fix because it does not remember the last three”, so the record is externalised memory, not a convenience log. WHAT THIS IS NOT (the §11.4.227 boundary, stated first): §11.4.147 governs a CRASHED agent, §11.4.232(C) a HUNG long-op, §11.4.21 the TERMINAL exhaustion audit — none governs a LIVE, HEALTHY, PROGRESSING loop re-walking a dead end its predecessor already proved dead, and §11.4.267 closes exactly that gap. ALL hold: **(A)** a SHARED, DURABLE ATTEMPT RECORD across every actor that may take the next attempt (a successor agent, a §11.4.147 respawn, a §11.4.176 other track); the §11.4.93 DB and §11.4.116 event stream are the substrates and MUST be reused, not duplicated. **(B)** CONSULTED BEFORE ACTING — the record is READ before the next attempt; a recorded-failed approach is not silently retried, and a re-attempt is permitted only on a recorded material difference (a changed precondition, a corrected §11.4.199 sequence, a new §11.4.115(F) RED); a written-but-never-read record is the §11.4.196(F) configured-~~in~~-use failure applied to agent memory. **(C)** CONVERGE ON EVIDENCE — “each loop must monotonically reduce uncertainty”, on a SINGLE SOURCE OF TRUTH (“the verifier’s build exit 1 beats the producer’s fluent paragraph every time”) with CONCRETE FAILURE FEEDBACK (the exact captured error — argv, exit code, failing assertion, offending line — fed back as the next input). **(D)** STALL IS A DETECTED SIGNAL WITH A BOUNDED RETRY AND AN ESCALATION PATH — two named signatures (“the same criterion red after N tries”, “oscillating diffs”), the stall itself tracked and trended, and the forbidden shape explicit: “an unbounded retry loop with no ground-truth feedback and no escalation — that is how agents thrash expensively while appearing busy”; escalation composes §11.4.231 (re-tier), §11.4.101, §11.4.66/§11.4.21. **(E)** N IS CONSUMER DATA, NEVER A CONSTITUTIONAL LITERAL — canon leaves N symbolic in every occurrence and states no numeric, wall-clock, token or cost bound; this anchor mandates that a bound EXISTS and is DECLARED, never a value, and a project enforcing an invented number while attributing it to canon commits the §11.4.6 violation this clause prevents. **(F)** THE LOOP’S DONE-CONDITION is not satisfied while an attempt is in-flight with no outcome, and every escalation reaches a terminal disposition (§11.4.87/§11.4.94/§11.4.97/§11.4.126/§11.4.147/§11.4.197). Honest boundary (§11.4.6): the record prevents the

RECORDED dead end from being re-walked; it does NOT guarantee convergence (an escalation is an honest outcome), does not prove any attempt competent, and does not substitute for §11.4.102. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-267-PROPAGATION (literal 11.4.267) + recommended gates CM-ATTEMPT-RECORD-SHARED-AND-CONSULTED / CM-STALL-BOUNDED-AND-ESCALATED + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.267 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker regardless of context. No escape hatch — no --no-attempt-record, --record-without-reading, --unbounded-retry-loop, --retry-same-failed-approach, --stall-without-escalation, --invent-N-and-cite-corpus flag.

§11.4.268 — Tamper-evident evidence chain + periodic anchor record: deletion, reordering, and tail-truncation of an accepted evidence record are DETECTED, not merely content-mutation (spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26). Cascade reference (short form): every accepted evidence record (the §11.4.115(F) machine-written verdict store, the §11.4.116 event stream, or an equivalent evidence-bearing ledger) MUST be tamper-evident against THREE distinct attacks, not only the content-mutation attack §11.4.115(F)/§11.4.226 already cover — (1) deleting an entry, (2) reordering two entries, (3) truncating the tail — and the mechanism detecting each MUST be named, because a forward-hash-chain alone detects (1) and (2) only when the chain is left internally inconsistent and does NOT detect an actor who deletes an entry and then RECOMPUTES the chain forward from that point; closing that gap requires a SEPARATE periodic anchor record the producer can append to but never rewrite. ALL hold: **(A)** every state-changing call WITHIN the consumer's declared COVERED-CALL SET (§11.4.35) produces a matching chain entry; an UNCITED read MUST NOT be required to produce one (guarding against both volume creep and a gate that refuses correct behaviour), while a read result CITED in support of a claim MUST have an entry — the distinction is CITED-vs-UNCITED, never present-vs-absent-in-storage. **(B)** chain verification walks the FULL forward chain, recomputing every link, and detects a deleted or reordered entry left internally inconsistent — single-entry content mutation detection alone does NOT satisfy this. **(C)** THE ANCHOR IS THE ONLY MECHANISM CATCHING A RECOMPUTED-FORWARD DELETION OR A TAIL TRUNCATION — a periodic ANCHOR entry carrying the chain head digest AND the entry count, written on a declared interval to a location the producer can APPEND to but NOT REWRITE (“the count is what makes wholesale store deletion a detected absence rather than silence”), with its strength recorded honestly as mechanism (cryptographically or append-only enforced) or policy (procedurally enforced only) — mechanism MUST NOT be claimed without evidence that rewriting is mechanically prevented, and absent that evidence the recorded strength is policy; a verdict of tamper-free MUST

name BOTH the chain-walk result AND the anchor cross-check result. **(D)** VERIFICATION THAT CANNOT COMPLETE REFUSES — a chain truncated mid-verification, or an unreachable anchor location, yields an UNVERIFIED status per §11.4.201's conservative-safe default, never a clean pass by default; “could not verify” and “verified clean” are never the same reported state. Honest boundary (§11.4.6): §11.4.268 detects deletion, reordering and tail-truncation of ALREADY-RECORDED entries; it does NOT prove the first entry ever written was truthful (content trustworthiness stays §11.4.115(F) machine-written + §11.4.240 producer=verifier) and makes the evidence tamper-EVIDENT, not CORRECT; where a project already operates a content-addressed atomically-committed store (§11.4.207), §11.4.268 REUSES that chaining primitive rather than standing up a second divergent implementation (§11.4.227 extend-don't-duplicate applied to mechanism reuse). Classification: universal (§11.4.17) — the consumer supplies its chain implementation, anchor location, anchor interval and covered-call set as DATA per §11.4.35. Propagation gate CM-COVENANT-114-268-PROPAGATION (literal 11.4.268) + recommended gates CM-EVIDENCE-CHAIN-DELETE-REORDER-DETECTED / CM-EVIDENCE-CHAIN-ANCHOR-CATCHES-RECOMPUTE-AND-TRUNCATION / CM-EVIDENCE-CHAIN-INCOMPLETE-VERIFICATION-REFUSES + paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.268 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker regardless of context. No escape hatch — no --content-mutation-detection-suffices, --skip-anchor-record, --claim-mechanism-strength-unverified, --chain-truncation-passes-by-default, --reconstructed-chain-is-clean flag.

§11.4.269 — Critic/consensus advisory-only ban AT THE EVIDENCE-ACCEPTANCE SEAM: an ungoverned confidence/consensus signal MAY inform but MUST NEVER substitute for a receipt or adjudicate producer-verifier disagreement; role recorded advisory, never entered into an evidence chain — the CONSTITUTION'S MANDATORY INDEPENDENT-REVIEW-VERDICT FAMILY (§11.4.125/.134/.142/.165/.209/.237/.256) IS UNWEAKENED (spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26). Cascade reference (short form): at the evidence-acceptance / claim-adjudication seam — where a producer's claim of DONE/PASS is accepted or refused and where a producer-verifier DISAGREEMENT is adjudicated — an ungoverned second opinion (a confidence score, an LLM-judge output, or a consensus aggregation over such opinions, none produced under this constitution's own independent-review discipline) is a real, adoptable capability but MUST be admitted in an ADVISORY-ONLY role AT THAT SEAM, never as the mechanism deciding accept/refuse there; the project's own operator decision states it directly — disagreement is adjudicated “on the recorded receipt, never on confidence or argument quality”, with an LLM judge explicitly barred from being the gate. ALL hold: **(A) TWO**

CLOSED TERM DEFINITIONS — a critic is a second/Nth model instance, confidence score or LLM-judge offering an OPINION WITHOUT itself constituting a structurally-separated, receipt-generating review under §11.4.125/§11.4.134/§11.4.142/§11.4.165/§11.4.209/§11.4.237/§11.4.256; consensus aggregates multiple such UNGOVERNED opinions; a verdict produced by the mandatory review family is NOT a “critic” for this anchor’s purposes. **(B)** a claim whose only support is an ungoverned critic/consensus signal is REFUSED, naming the missing receipt — a recorded receipt (a §11.4.115(F) machine-written verdict, a §11.4.268 evidence-chain entry, or a mandatory review’s own recorded GO/NO-GO) remains the only basis for adjudicating disagreement. **(C)** DETERMINISTIC WITH AND WITHOUT THE UNGOVERNED CRITIC — the same claim evaluated once with the ungoverned critic step present and once absent MUST produce the IDENTICAL accept/refuse outcome; a pipeline whose verdict changes when that step is disabled is by definition using it as a gate (this test is scoped to the optional ungoverned critic only — skipping a REQUIRED review is a violation of THOSE anchors). **(D)** ungoverned critic output is recorded with role advisory in a SEPARATE store and MUST NOT appear in any §11.4.268/§11.4.115(F) evidence chain; a mandatory review’s own GO/NO-GO continues to be recorded as its own anchors require and is not retroactively downgraded. **(E)** an ungoverned critic wired into the acceptance gate collapses ORACLE and GATE into one mechanism NOT independent of the producer’s failure surface (correlated model errors, shared blind spots) — the maximum-severity collapse class of §11.4.240/§11.4.249; advisory-only admission avoids re-creating it while retaining the critic’s genuine usefulness (surfacing issues earlier, feeding §11.4.102, prioritising human review). **(F)** THE BAN IS SCOPED TO THE EVIDENCE-ACCEPTANCE / CLAIM-ADJUDICATION SEAM — §11.4.125, §11.4.134, §11.4.142, §11.4.165, §11.4.209, §11.4.237 and §11.4.256 each mandate a structurally-separated verifier whose VERDICT genuinely gates, and NONE is relaxed, narrowed, made advisory or made optional by §11.4.269; citing §11.4.269 to skip, weaken or downgrade any of those seven is itself a §11.4.269(F) violation. Honest boundary (§11.4.6): the anchor does not forbid using a critic to prioritise human review, surface candidate defects, or inform a human/verifier authoring (not executing) a gate — only its use as an automated GATING mechanism substituting for a receipt-backed oracle/gate/verifier chain. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-269-PROPAGATION (literal 11.4.269) + recommended gate CM-CRITIC-CONSENSUS-ADVISORY-ONLY + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.269 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker regardless of context. No escape hatch — no --critic-may-gate, --consensus-score-is-a-receipt, --llm-judge-is-the-gate, --critic-output-in-evidence-chain-OK, --outcome-may-differ-with-critic, --269-overrides-mandatory-review, --downgrade-mandatory-review-to-advisory flag.

§11.4.270 — Dependency-existence-verdict register: every proposed dependency carries a closed-set existence verdict {VERIFIED, AMBIGUOUS, UNVERIFIED} with citable evidence; adoption is gated behind resolution and behind the wiring-sweep result (spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26). Cascade reference (short form): a project adopting an external (or internal, cross-module) dependency to close a coverage gap **MUST** FIRST record for it a closed-set EXISTENCE verdict — VERIFIED, AMBIGUOUS or UNVERIFIED — backed by a citable artifact or an explicit statement that none was found; existence verification is NECESSARY but NOT SUFFICIENT for adoption (“Existing is not the same as working ... several verified entries are at initial release versions, one declares a repository that returns 404, and one’s ‘tamper-evident’ claim is on inspection only ‘append-only’ — which detects nothing if the appender is the party being checked”). ALL hold: **(A) THE REGISTER** — one row per proposed dependency, keyed by a stable identifier (name + version + source per §11.4.111), carrying the verdict and its evidence field. **(B) VERDICT SEMANTICS, EVIDENCE-BASED, NEVER PROMOTED BY OMISSION** — UNVERIFIED is the honest default (a claim with no independent confirmation attempted **MUST** NOT be silently treated as VERIFIED); AMBIGUOUS is assigned when evidence conflicts or a name collision exists (a dependency name colliding with an unrelated project gets AMBIGUOUS, never VERIFIED), with the specific conflicting evidence recorded rather than a vague “unsure”; VERIFIED requires an independently-reproducible check — a real resolve + checksum/hash match, a real successful call or import, or an equivalent control-needle-proven presence per §11.4.201(7)(b). **(C) ADOPTION REFUSED** while the verdict is UNVERIFIED or AMBIGUOUS; adoption is admitted only after the row resolves to VERIFIED (or the AMBIGUOUS conflict is investigated and re-verdicted). **(D) ADOPTION IS ALSO GATED** behind mapping to one of the project’s own recorded UNCOVERED capability gaps (adopting for a capability the project already covers is refused, naming the covering rule — the §11.4.227 duplicate-coverage discipline applied to adoption) AND behind the WIRING-SWEEP RESULT — the adoption step **MUST** NOT be accepted before the wiring step reports its sweep, so a project never decides “adopt” before learning whether the claimed capability is genuinely reachable and wired (§11.4.124/§11.4.196(F) investigate-before-adopt, the mirror of investigate-before-remove). **(E) ZERO ADOPTED WITHOUT VERIFIED** — the count of dependencies adopted without a VERIFIED verdict is ZERO, checked at the same seam that already refuses unwired-gate names per §11.4.227(A). Honest boundary (§11.4.6): the register proves EXISTENCE/RESOLVABILITY was checked and recorded; it does NOT prove the dependency BEHAVES correctly for its adopted purpose (that stays §11.4.244 integration/contract testing) nor that it is SECURE or well-maintained (§11.4.246 supply-chain territory) — it is deliberately narrow, closing the EXISTENCE-CLAIM bluff only. Composes also §11.4.74 (catalogue-first discovery — the VERIFIED-tier check IS that mandated step applied to any dependency), §11.4.31 (an owned-org dependency’s helix-deps.yaml is itself citable evidence) and §11.4.150 (the deep multi-angle research

pass that PRODUCES a row's citable evidence or its explicit none-found statement). Classification: universal (§11.4.17) — the consumer supplies its register location, uncovered-gap set and wiring-sweep mechanism as DATA per §11.4.35. Propagation gate CM-COVENANT-114-270-PROPAGATION (literal 11.4.270) + recommended gate CM-DEPENDENCY-EXISTENCE-VERDICT-REGISTER + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.270 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker regardless of context. No escape hatch — no --unverified-adoption-OK, --ambiguous-treated-as-verified, --skip-wiring-sweep-before-adopt, --adopt-already-covered-capability, --existence-proves-fitness flag.

§11.4.271 — Waiver mechanism: the strict FORMALIZATION of allow-with-tracked-debt (unifying §11.4.234(D)'s recorded-deferral, §11.4.236(4)'s explicit-operator-override, and §11.4.248(A)'s deadline-bound quarantine into ONE roster + expiry + tracked-item schema), distinct from refusal (§11.4.21) and closure (§11.4.90) vocabulary (spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26). Cascade reference (short form): every project needs, on rare occasion, a mechanism to ALLOW a change to proceed despite a normally-blocking missing-evidence condition — never a silent, never a permanent bypass, but a tracked, expiring, human-authorised exception. This is NOT an absent capability being introduced: §11.4.234(D) already permits an explicit, documented deferral flag, §11.4.236(4) already requires an explicit logged operator-set flag for any override of its deploy-readiness seam, and §11.4.248(A) already ships a deadline-bound quarantine for one gate class — three genuine allow-with-known-debt PRECEDENTS this anchor FORMALIZES rather than discovers. What §11.4.271 genuinely adds is the strict, data-bearing, cross-gate SCHEMA those precedents describe only informally and per-mechanism: a ROSTERED NON-PRODUCER authoriser, a MANDATORY UNELAPSED EXPIRY (neither §11.4.234's skip flag nor §11.4.236's override flag currently carries an expiry field at all), and a MANDATORY TRACKED ITEM made mandatory CROSS-GATE. ALL hold: **(A)** the waiver is the ONE formalized schema for an allow-with-known-debt path at a seam that would otherwise refuse; it is invoked explicitly, never inferred, never disables the gate permanently, and a gate already implementing such a path under §11.4.234/§11.4.236/§11.4.248 satisfies §11.4.271 by CONFORMING its existing mechanism to clauses (B)–(D), not by reimplementing one. **(B)** AUTHORISATION MUST RESOLVE TO A ROSTERED IDENTITY, NEVER THE PRODUCER — a free-text authorisation (“looks fine”) is refused exactly as an absent one is, a waiver naming a genuinely rostered identity is accepted (a check that refuses EVERY waiver has replaced a bypass with a deadlock), the roster is consumer DATA per §11.4.35, and the producer of the change MUST NOT authorise their own waiver (§11.4.240). **(C)** EXPIRY IS REQUIRED AND MUST NOT HAVE ELAPSED — a waiver with no expiry, or an

elapsed one, is refused; an absent expiry is NOT read as “valid forever” (the same downgrade-by-omission the satisfaction path already refuses); an elapsed waiver’s gate reverts to blocking automatically, with no grace period and no silent auto-renewal. **(D)** A TRACKED ITEM NAMES THE DEBT — a waiver naming none is refused, “because a waiver is the only ALLOW that leaves the defect unfixed, so it is the one path on which the owed work must stay traceable”; the item is an ordinary §11.4.54-stable-id workable item under the same §11.4.15/§11.4.16/§11.4.33/§11.4.197 lifecycle, and a waiver whose item never resolves is surfaced by the §11.4.261 standing audit. **(E)** COMPOSITION, NOT SUBSTITUTION — a waiver lapsing UNRESOLVED re-enters the §11.4.21 Operator-blocked class for its underlying condition; its tracked item CAN close via §11.4.90 Obsolete if the requirement genuinely changed; neither vocabulary substitutes for the waiver mechanism, and §11.4.234(D) / §11.4.236(4) / §11.4.248(A) each remain the correct narrower mechanism for their own seam class. Honest boundary (§11.4.6): the anchor formalizes rather than discovers the category; the schema is deliberately strict because a waiver is the one path permitted to leave a defect genuinely unfixed while still shipping; it weakens no gate for other candidates, and an expiry repeatedly pushed forward without the debt closing is itself a §11.4.261 finding, not a compliant renewal pattern. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-271-PROPAGATION (literal 11.4.271) + recommended gate CM-WAIVER-ROSTERED-EXPIRY-TRACKED + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.271 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker regardless of context. No escape hatch — no --free-text-authorisation-OK, --waiver-without-expiry, --elapsed-waiver-still-valid, --waiver-without-tracked-item, --producer-may-self-waive, --permanent-waiver-renewal flag.

§11.4.272 — Dynamic, on-demand skill/extension activation: the active capability surface is the MINIMUM needed now, everything else stays DISCOVERABLE and one command away (User mandate, 2026-09-07). Cascade reference (short form): every project whose agent platform exposes a capability surface (Agent Skills, plugins, extensions, tool sets) MUST keep the ACTIVE surface at the minimum genuinely needed, with everything else activatable ON DEMAND — never a static all-on set paid for in every turn (measured 2026-09-07 on Claude Code 2.1.263: a 955-skill pool renders as 29,153 B of skill NAMES per turn, or 101,279 B with descriptions — a 3.5x render-mode cliff the project neither controls nor observes; byte counts exact, the ~7,288 / ~25,319 token figures a labelled bytes/4 ESTIMATE per §11.4.6). ALL hold: **(A)** a deliberately small declared core always active plus the pool it draws from, both consumer DATA per §11.4.35; anything not core is inactive by default, and a core entry is paid for in EVERY turn of EVERY session and MUST earn that. **(B)** DISCOVERABILITY IS NEVER TRADED FOR TOKENS — a deactivated capability MUST remain FINDABLE via an always-

available index listing the whole catalogue with active/inactive state, and §11.4.102(B) continues to bind unconditionally; saving tokens by making a capability unfindable is FORBIDDEN and is a §11.4 bluff at the capability layer. **(C)** FAIL SAFE, NEVER SILENT (§11.4.201) — a failed activation MUST tell the agent explicitly that the capability is NOT available and exit non-zero; an activation that appears to succeed while the capability is unusable leaves the agent believing it holds a tool it does not, and a dangling or unreadable activation MUST be rolled back. **(D)** CONCURRENCY-SAFE BY REFERENCE COUNTING (§11.4.119/§11.4.176) — where several sessions share one checkout, one session's deactivation MUST NOT strip a capability another is using; claims are reference-counted and released only when no other claimant is PROVEN live (§11.4.180 liveness, §11.4.174 ownership), and a claim whose holder cannot be proven live is reported as such, never silently assumed dead or alive. **(E)** HONEST ASYMMETRY — MEASURE WHAT THE PLATFORM ACTUALLY DOES: measured first-hand, activation HOT-LOADS into an already-running session while deactivation does NOT hot-unload, so the saving is a minimal session-start baseline plus on-demand growth and claiming deactivation frees tokens in the live session is a bluff; equally measured, the platform's own permissions.deny Skill(name) rule blocks invocation WITHOUT removing the rendered entry and therefore saves ZERO tokens — a first-party mechanism must be TESTED for the property relied on, never assumed from its documentation (§11.4.74 satisfied by evidence). **(F)** A BUDGET GATE bounds the active surface in BOTH the name-only and description-eager renders, so an upstream switch to eager rendering breaks a gate LOUDLY instead of silently costing tokens every turn. **(G)** BOTH MODES — the engine lives in the constitution submodule (§11.4.28/§11.4.80/§11.4.177, by reference, never copied), yet a project NOT vendoring the submodule MUST still get a working command or an HONEST, LOUD refusal — never a silent no-op; resolution order is explicit (environment override, in-project submodule, ancestor's, operator-configured host checkout, documented host fallback, then honest degradation naming WHAT is unavailable, WHY, WHERE it looked and WHAT still works), the degradation using an exit code distinct from "operation failed" and stating that the existing static capability set is untouched. **(H)** PRUNE-TO-CORE IS THE DEFAULT and is NOT a §11.4.122 removal (operator decision 2026-09-07) — session initialisation DEMOTES every non-core capability rather than merely raising core (raise-only satisfies (A) in letter and voids it in substance as the surface grows monotonically; measured on the reference project: 19 active / 422 B / ~105 tok-per-turn raise-only versus 3 active / 65 B / ~16 tok-per-turn prune-by-default), and it is not the silent removal §11.4.122 forbids because all three reversibility properties hold BY CONSTRUCTION — LISTED (the always-core catalogue of (B) still shows every pruned capability), ACTIVATABLE (one command returns it, hot-loading per (E)), NOT DELETED (a link-style activation loses only the LINK with its target untouched, and a sole-copy capability is MOVED to a preserved store that is itself a searched source — zero bytes destroyed); an unsafe demotion (another live session's claim per (D), a name collision in the preserved store, a move that fails

or lands unreadable, or a capability that would not be re-activatable afterwards) MUST refuse LOUDLY and leave the capability ACTIVE per (C); relocation MUST be relocation-SAFE (path-relative references re-pointed at the identical resolved target before the move, with the post-move readability check as the backstop that refuses and reverts); an EXPLICIT, DOCUMENTED opt-out MUST exist at session scope (an environment override) and project scope (a manifest flag), session scope winning; and where the engine is unreachable, pruning inherits (G)’s honest refusal so a session is NEVER left silently holding fewer capabilities than it believes it has. Honest boundary (§11.4.6): this reduces the per-turn cost of the capability LIST, not of a skill BODY once invoked, is never a licence to hide a capability the task needed, and clause (H) inherits (E)’s measured asymmetry — the saving lands at the next session start for anything already cached, so prune runs AT session initialisation and its in-session effect is never overclaimed. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-272-PROPAGATION (literal 11.4.272) + recommended gates CM-SKILL-SURFACE-BUDGET / CM-SKILL-ACTIVATION-FAIL-SAFE / CM-SKILL-PRUNE-PRESERVES-CONTENT + paired §1.1 mutations (gate-code beyond the shipped scripts = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.272 — the full mandate text is canonical there and is inherited by reference (CONST-059 / §11.4.35). Non-compliance is a release blocker. No escape hatch — no --all-skills-always-on, --skip-activation-engine, --hide-inactive-skills, --silent-activation-failure, --ignore-surface-budget, --assume-skill-loaded flag.

§11.4.273 — Measuring-instrument verification: a census that INFORMS a decision must be control-needed before its result is believed (research-derived, 2026-09-08). Forensic anchor: FIFTEEN instances in one session where an ad-hoc measurement returned a confident, plausible, actionable result that was an artefact of the instrument, not a property of the system — a pattern matching only one of two block-opener forms (manufacturing 24 phantom findings); comm on numerically-sorted input reporting all 34 items missing; \$? read after a pipeline turning exit 1 into 0; a field-name typo reading as “every record lost its key binding”; grep -w matching zai inside zai-coding-plan and corrupting a classification; a summary grep assuming passed, failed when failure prints failed, passed; while read dropping an unterminated final line so a one-element list iterated zero times; a mutation applied with the wrong env knob so the REAL tree was tested and the green meant nothing; a guard probe using a command the guard does not target, proving nothing while reading as a negative; a fixture whose two columns held the same string, so a real column-swap mutation survived at 52/0 under a comment asserting that could not happen; a positive control chosen OUTSIDE the instrument’s scope reporting a working instrument as broken, and another chosen to BE the question under test so it could not discriminate; an awk range whose escaping matched nothing

reporting 838 rows where the system held 134, about to gate a DESTRUCTIVE recreate; **a two-literal denylist searched with ONE literal** — both loaded, both lengths printed, one searched — so a live credential was reported eliminated while still handed over in four copy-pasteable lines per file, with the positive control PASSING because it shared the same single needle; and \$? clobbered by a command substitution in the same argument list, reporting a pandoc failure (exit 64) as exit 0. **The gap:** §11.4.201 governs guards/gates (instruments that BLOCK) and §11.4.107(10) governs analyzers (calibrated golden-good/bad); NEITHER covers the ad-hoc census that INFORMS — a grep, a set difference, an exit-code read, a field lookup — whose output is not a verdict but a FACT a decision is then built on, written inline, used once, trusted implicitly, never itself tested. **Mandate:** any measurement whose result will drive a decision, claim, classification or report MUST be control-neededled BEFORE it is acted on — **(a)** a POSITIVE control (a value known present must be found: catches too-narrow patterns, wrong field names, wrong paths, wrong knobs, silently-empty stages); **(b)** a NEGATIVE control (a fabricated needle must NOT be found: catches too-broad patterns, accidental substring/word-boundary matches, fabricating defaults); **(c)** an EMPTY result is NEVER a finding alone — absent, could-not-look, looked-in-the-wrong-place and output-shape-differed are four distinct states the raw empty distinguishes between none of (§11.4.6); **(d)** a result CONTRADICTING directly-observed evidence is presumed an instrument fault until proven otherwise; **(e)** the instrument is part of the evidence — a captured claim cites the command that produced it (§11.4.5/§11.4.2). **Honest boundary (§11.4.6):** control needles prove an instrument can tell PRESENT from ABSENT; they do NOT prove it measures the RIGHT property, and a correctly-neededled census of the wrong thing is still the wrong answer. Classification: universal (§11.4.17). Composes §11.4.5/.6/.2/.107(10)/.118/.201/§1.1. Propagation gate CM-COVENANT-114-273-PROPAGATION (literal 11.4.273) + recommended gate CM-CENSUS-CONTROL-NEEDED + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.273. Non-compliance is a release blocker. No escape hatch — no --skip-control-needle, --empty-is-absent, --trust-the-grep, --census-needs-no-proof, --obvious-result-exempt flag.

§11.4.274 — Mechanical work belongs in a script, not in an agent's context: extract it, test it, and re-scan for it on a cadence (User mandate, 2026-09-08).

Forensic anchor: in one session five review agents consumed ~400–450k tokens EACH, and the dominant cost was not judgment but MECHANICS — copy a tree to scratch, apply a one-line mutation, run a suite, read the summary, restore, verify byte-identical, compare against the previous round, repeat seven times. Every step is deterministic, fixed-input/fixed-output, decision-free: a shell script, executed instead one tool call at a time by a language model at orders of magnitude the cost, with a fresh chance of transcription error at each step (this session logged ten such instrument errors, §11.4.273). **Rule:** work that is

DETERMINISTIC, REPEATABLE and contains NO JUDGMENT MUST be delegated to a properly written script or small compiled utility and invoked, not performed turn-by-turn — mutation apply/run/restore/verify loops, suite fan-outs and result tabulation, before/after comparisons, scratch setup and teardown, residue and quiescence scans, census and inventory queries, artifact fingerprinting, evidence collection and layout, and any step already written down as a fixed recipe. **(a) EXTRACT:** a mechanical sequence an agent performs more than twice is a defect in the tooling, not a task — name it, script it, invoke it. **(b) THE TOOL IS GOVERNED:** it ships with actionable documentation (§11.4.18), a user guide/manual entry where operator-facing, and tests validating its behaviour AND producing machine evidence deterministically (§11.4.50/§11.4.5) — including failure paths, exit-code semantics (§11.4.201) and the control needles proving its own measurements (§11.4.273); replacing a slow correct process with a fast unproven one is a regression, not an optimisation. **(c) RE-SCAN ON A CADENCE:** every project periodically scans itself for mechanical work still done by agents (recurring command sequences in transcripts/logs, repeated inline recipes, “run these commands in order” docs) and refactors the findings out — a standing obligation, since new mechanical loops accrete continuously. **(d) HONEST BOUNDARY (§11.4.6) — what MUST NOT be extracted:** JUDGMENT. Root-cause determination (§11.4.102), review verdicts (§11.4.125/§11.4.142/§11.4.194), the decision that a fix is correct, the choice of what to measure, the reading of whether evidence supports a claim, and any assessment requiring “is this the right thing to check?” A script that renders a verdict is a bluff gate (§11.4.201), and automating a review into a green checkmark is the exact PASS-bluff this constitution exists to prevent. The division is: the SCRIPT gathers and executes deterministically and cheaply; the AGENT decides what it means. Cost is NEVER a reason to move a decision into a script. Classification: universal (§11.4.17). Composes §11.4.18/.50/.5/.141/.201/.273/.124/§1.1. Propagation gate CM-COVENANT - 114 - 274 - PROPAGATION (literal 11.4.274) + recommended gates CM-MECHANICAL - WORK - EXTRACTED / CM-EXTRACTED - TOOL - DOCUMENTED - AND - TESTED + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped). **Canonical authority:** constitution submodule [Constitution.md](#) §11.4.274. Non-compliance is a release blocker. No escape hatch — no --skip-extraction, --agent-can-do-it, --one-off-is-fine, --script-the-review, --extract-without-tests flag.